Databases and PostgreSQL From the Ground Up

Relations, 8 KB Slotted Pages, B-Link Trees, Write-Ahead Logging, MVCC, and Production Query Execution

An Entry Path into Real Database Internals

November 2026

Preface

Every backend engineer eventually meets the day when the database stops pretending to be an infinite hash map in the sky.

Perhaps a query that ran in four milliseconds on Friday suddenly takes nineteen seconds on Monday morning. Nobody deployed code over the weekend. Perhaps two concurrent requests check that a customer has 100 dollars available, and both pass inside separate `BEGIN ... COMMIT` blocks. Both deduct 80 dollars, and the account settles at −60 dollars. Or perhaps someone adds a harmless-looking `WHERE id NOT IN (SELECT blocked_id FROM blocks)` filter. One row in `blocks` happens to have `blocked_id = NULL`, and the entire query returns zero rows without a warning.

When those moments arrive, guessing at index combinations or copying configuration snippets from blog posts feels terrible. Underneath every surprise sits a deterministic machine built out of concrete data structures. Rows live inside fixed 8,192-byte pages. Indexes form trees whose sibling pointers let readers keep moving while pages split. Crash safety rests on append-only log segments, and concurrency rests on 32-bit transaction IDs. Once you can picture those bytes and trace their rules, PostgreSQL stops feeling mysterious. Every behavior has a physical cause you can inspect, predict, and measure.

I wrote this book because most database books give you only half the picture. Academic textbooks teach relational algebra and serializability graphs cleanly, yet rarely open an actual 8 KB page on disk. They seldom show why CPU alignment pads a row header or why a 4 KB OS write can tear an 8 KB database page in half. Operational manuals list settings like `shared_buffers`, `work_mem`, and `autovacuum_vacuum_scale_factor`, yet seldom derive from first principles what breaks when they are misconfigured. Here we keep the math, the byte-level PostgreSQL layout, and production engineering on the very same page.

What this book is, and who it is for

You know Java 6 (classes, `null`, `synchronized`, `ArrayList`, `HashMap`, and garbage collection) and basic C (structs, pointers, `malloc`, `write`, and `fsync`). You may have written SQL through JDBC or Hibernate, or you may be meeting SQL seriously for the first time. We assume no prior knowledge of database internals. Each new mechanism is first tied to the Java or C construct it resembles: an object graph, a `TreeMap`, a padded C struct, or an unsynced file stream. Then we show where PostgreSQL differs and why.

By the end of these eight chapters, you will be able to trace PostgreSQL from the query parser all the way down to NVMe flash:

- Read a SQL query in the ten-stage order the engine logically evaluates it, from `FROM` through `LIMIT`, and spot `NULL` traps before they ship.
- Inspect a raw 8 KB PostgreSQL table page, trace slot pointers growing downward while rows grow upward, and compute exact row sizes with alignment padding.
- Choose between B-tree, covering (`INCLUDE`), and partial indexes from their physical geometry, and trace how readers traverse splitting pages without blocking. Match specialized workloads to Generalized Inverted Indexes (GIN, for arrays, `JSONB`, and full-text), Generalized Search Trees (GiST, for ranges and geometry), and Block Range Indexes (BRIN, tiny min/max summaries for append-only tables).
- Follow a `COMMIT` through RAM buffers, the Write-Ahead Log (WAL), and `fsync`, and replay crash recovery by hand from a checkpoint.
- Predict which row versions a snapshot can see, why `UPDATE` leaves dead versions behind, when same-page updates avoid index bloat, and why old transaction IDs must be frozen.
- Build two-transaction timelines for lost updates and write skew, pick the right lock or isolation level, and read every counter in `EXPLAIN (ANALYZE, BUFFERS)`.
- Design schemas for financial ledgers—paired with Rust, `Axum`, and `SQLx`—backed by streaming replication and transaction-mode connection pooling.

How to read this book

Every major mechanism in this book follows the same four-step rhythm: a concrete problem, a small paper-and-pencil trace, the exact PostgreSQL data structure, and a hands-on inspection query.

We always start with *the failure or bottleneck that forced the design.* Why not store disk pointers from a customer record directly to their order records? Why not overwrite an updated integer in place inside its 8 KB page? Why not flush modified table pages straight to disk at `COMMIT`? When you feel the physical constraint first, the invention that resolves it reads like common sense.

Next we work through *tiny numbers by hand.* We pack three rows into a miniature page and watch byte offsets move when the middle row is deleted. We split a four-key B-tree leaf node while a concurrent reader is halfway down the tree. We replay crash recovery across six log records and check the log position stamped on each page. Doing the arithmetic on six items gives you complete command over sixty million.

Only after the plain-English picture is clear do we open *PostgreSQL's real C structures and inspection extensions.* Built-in extensions such as `pageinspect`, `pg_walinspect`, `pg_buffercache`, and `pg_stat_statements` let you inspect live pages and log records from an ordinary `psql` prompt. In each chapter we explain what a field does in everyday words first, and then give its real PostgreSQL name so the documentation and source tree feel familiar.

What the recurring callouts mean

Throughout each chapter, shaded callout blocks signal shifts in perspective so you always know what kind of thinking a passage asks of you:

Intuition

The geometric or physical mental picture — the one clear visual model that makes all the details snap into place. If you revisit a chapter months later, reading these boxes will reconstruct the whole story in minutes.

Formal

Exact definitions, invariants, and correctness rules — relational algebra operators, the Write-Ahead Logging (WAL) invariant, or the Multi-Version Concurrency Control (MVCC) snapshot visibility predicate — stated without hand-waving.

Why it works

The engineering and mathematical proof for *why* a mechanism preserves correctness under concurrency or power loss, or why an alternative design fails.

Common trap

Subtle mistakes that bite production systems every week: `NOT IN` against a nullable column, leading range columns in composite B-trees, `TIMESTAMP WITHOUT TIME ZONE`, or `REPEATABLE READ` silently permitting write skew.

Under the hood

Direct looks inside PostgreSQL's C structs (`PageHeaderData`, `HeapTupleHeaderData`, `BufferDesc`) or compact reference code in Rust, C, and Python showing how the mechanism operates at the byte level.

Try it in PostgreSQL

Self-contained SQL sessions you can paste straight into a local PostgreSQL 16+ container to watch raw pages, indexes, locks, and execution plans with your own eyes:

```
-- Start an ephemeral container with:
-- docker run --rm -it -e POSTGRES_PASSWORD=postgres postgres:17 psql -U postgres
SELECT version();
```

One end-to-end trace, so you see how the layers connect

Before we begin Chapter 1, let us trace one tiny SQL statement all the way from text to silicon. Watching all the layers cooperate on a single row gives you a map of the entire book.

Suppose a ledger table `accounts` holds rows with `(id BIGINT, balance_cents BIGINT)`. Account `7` sits on page `0` at slot `1` with a balance of `25000` cents (250.00 USD). A session runs:

```
BEGIN;
UPDATE accounts
```

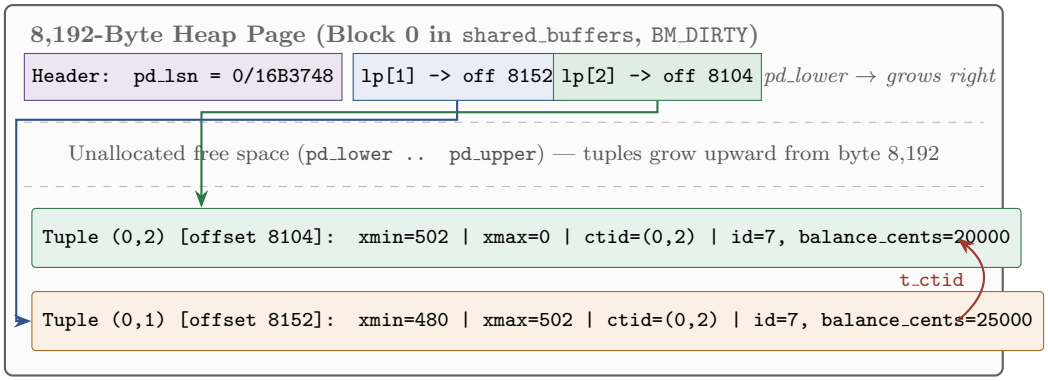
```
   SET balance_cents = balance_cents - 5000
 WHERE id = 7;
COMMIT;
```

A naive mental model imagines PostgreSQL seeking to byte offset 8160 of the table file, overwriting `25000` with `20000`, and returning `UPDATE 1`. PostgreSQL does *none* of that. Overwriting those eight bytes in place would break two promises at once:

1. **Concurrent readers without read locks:** Another session running a report that started ten milliseconds ago must still see `25000`. It should neither block your `UPDATE` nor wait behind it.
2. **Atomic rollback and crash safety:** If your transaction aborts on the next statement—or the server loses power mid-flight—`25000` must still be intact.

Instead, suppose your transaction gets transaction ID `502`, while the original row was inserted earlier by committed transaction `480`. Here is what PostgreSQL actually does in RAM and on disk:

1. **Index lookup and buffer pin (Chapters 2–3):** The executor walks the primary-key B-tree on `id` down to a leaf entry mapping `id = 7` to physical slot `(0, 1)` (page `0`, slot `1`). It finds page `0` in PostgreSQL's shared RAM cache (`shared_buffers`) and pins the 8 KB frame so eviction cannot steal it. Slot `1` points to byte offset `8152` inside that page.
2. **Multi-version row creation in RAM (Chapter 5):** Rather than overwriting `25000`, PostgreSQL marks the old row version as superseded by stamping its expiration field (`t_xmax`) with `502`. Next it carves out a brand-new row version in the page's free space at byte offset `8104`, pointed to by slot `2`. That new version records birth transaction `t_xmin = 502`, expiration `t_xmax = 0`, and `balance_cents = 20000`. Finally, the old row's forward link (`t_ctid`) is updated from `(0, 1)` to `(0, 2)` so readers can follow the chain.
3. **Write-Ahead Logging before page flush (Chapter 4):** Modifying page `0` in RAM marks the buffer dirty, yet PostgreSQL does *not* write that 8 KB table page to disk at commit time. Instead, it appends a short change record to the sequential Write-Ahead Log buffer at byte position (Log Sequence Number, or LSN) `0/16B3748`. It also stamps that position at the start of page `0` (`pd_lsn = 0/16B3748`). When `COMMIT` runs, PostgreSQL appends a tiny commit record at `0/16B3780` and calls `fdatasync` on the sequential log file *only*. Once those few dozen bytes reach flash, your transaction is durable—even while dirty page `0` rests in RAM for minutes until a checkpoint flushes it.



Every piece of that story raises the exact engineering questions the rest of this book answers. What happens when old dead tuples like `(0, 1)` fill up the page? (Page pruning, Heap-Only Tuple (HOT) same-page updates, and `VACUUM` in Chapter 5.) Did every secondary index on `accounts` have to add a new entry pointing to `(0, 2)`? (Not if no indexed column changed and `(0, 2)` fit on the same 8 KB page.) What if power failed after `COMMIT` flushed WAL record `0/16B3780`, before block `0` ever left RAM? (Crash recovery replays the WAL stream forward from the redo point in Chapter 4.)

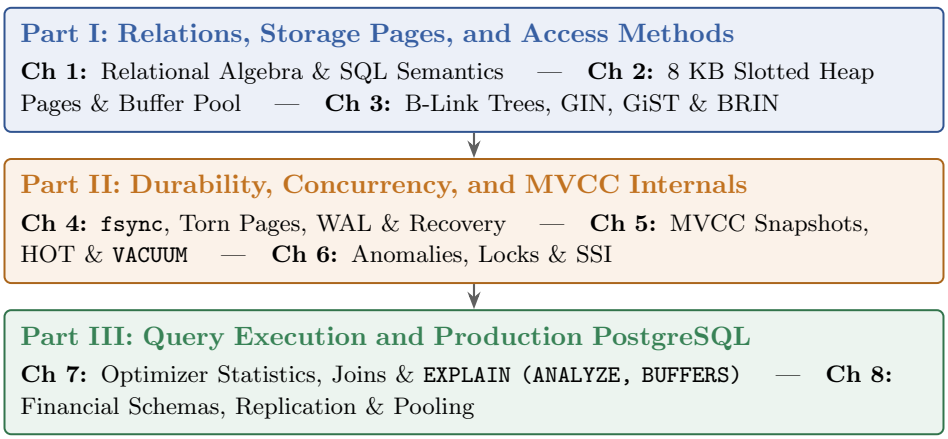
How the book is organized

The eight chapters divide into three parts that climb from query meaning down to storage bytes and back up to production architecture.

Part I — Relations, Storage Pages, and Access Methods (Chapters 1–3). Chapter 1 starts with Codd's relational model and translates relational algebra into SQL's ten-stage logical pipeline. It also unpacks three-valued `NULL` logic, window frames, and recursive queries. Chapter 2 opens the data directory on disk, dissects an 8,192-byte slotted page, aligns columns to hardware boundaries, chunks oversized values out-of-line, and traces clock-sweep eviction in RAM. Chapter 3 builds indexes from B+tree fanout math up to concurrent B-link trees, covering indexes, partial indexes, and Hash, GIN, GiST, and BRIN access methods.

Part II — Durability, Concurrency, and MVCC Internals (Chapters 4–6). Chapter 4 confronts hardware failure: volatile caches, torn 4 KB writes inside 8 KB pages, Write-Ahead Logging, group commit, checkpoints, and crash recovery. Chapter 5 builds Multi-Version Concurrency Control from row birth/expiration IDs, the commit log, and hint bits up to snapshots, same-page chains, `VACUUM`, and 32-bit wraparound. Chapter 6 maps concurrency anomalies—dirty reads, nonrepeatable reads, phantoms, lost updates, and write skew—onto two-transaction timelines. It then contrasts explicit row locks with Serializable Snapshot Isolation.

Part III — Query Execution and Production PostgreSQL (Chapters 7–8). Chapter 7 steps inside the cost-based query planner: column histograms, physical correlation, scan methods, join algorithms, and reading `EXPLAIN (ANALYZE, BUFFERS)` without guesswork. Chapter 8 brings everything together into production financial schemas: integer minor units, UTC timestamps, time-ordered `UUIDv7` keys, balance constraints, partitioning, streaming replication, and connection pooling with `PgBouncer` and Rust's `SQLx`.



Connection

This book pairs directly with two companions in the curriculum. *Algorithms From the Ground Up* develops asymptotic cost analysis, hashing, dynamic programming, and textbook B-trees, which we extend in Chapters 3 and 7 into concurrent B-link trees and relational join planning. *Programming Languages From the Ground Up* builds an asynchronous financial ledger service in Rust with `Axum`, `Tokio`, and `SQLx`; Chapter 8 of this book designs the exact PostgreSQL storage, isolation, replication, and pooling layer that backs that service.

Exercises

Every chapter closes with concrete exercises — calculations, trace tables, and failure scenarios. Work them with a pen before moving on; calculating one page layout or race condition yourself teaches more than skimming ten pages of prose. These five require nothing beyond what we just traced.

Exercise 0.1 (Counting rows per 8 KB page)**.** An empty PostgreSQL heap page is 8,192 bytes long and begins with a 24-byte page header. Each row added to the page consumes one 4-byte slot pointer at the front of the page plus the padded row bytes at the back. Suppose each row in a narrow ledger table occupies 40 bytes (a 24-byte row header plus two 8-byte `BIGINT` columns). What is the maximum number of live rows that fit on a single 8,192-byte page? How many bytes of free space remain unused?

Exercise 0.2 (Why in-place overwrite breaks crash atomicity)**.** Suppose a naive storage engine has no Write-Ahead Log and updates rows by overwriting bytes directly inside table pages on disk. A transaction transfers 50 USD from Account A (stored on page 12) to Account B (stored on page 904). Describe the disk state if power fails halfway between flushing page 12 and flushing page 904. Why can the restarted server not tell whether to undo page 12 or finish page 904?

Exercise 0.3 (Log Sequence Number byte arithmetic)**.** PostgreSQL writes a 64-bit Log Sequence Number as `X/YYYYYYYY`, where `X` is the upper 32 bits in hexadecimal and `YYYYYYYY` is the lower 32 bits in hexadecimal. In our trace above, the heap update record began at `0/16B3748` and the commit record began at `0/16B3780`. How many bytes long was the heap update WAL record?

Exercise 0.4 (Reading through a version chain)**.** Look back at the 8 KB heap-page diagram in this preface, where tuple `(0, 1)` has `xmin = 480, xmax = 502` and tuple `(0, 2)` has `xmin = 502, xmax = 0`. Suppose Transaction `501` started *before* Transaction `502` committed, while Transaction `505` started *after* Transaction `502` committed. Which tuple should Transaction `501` read, which tuple should Transaction `505` read, and why can neither transaction safely delete `(0, 1)` while Transaction `501` is still running?

Exercise 0.5 (Sequential WAL versus random heap I/O)**.** A busy payment checkout inserts one row into an `orders` table that has four secondary B-tree indexes (on `customer_id`, `merchant_id`, `created_at`, and `status`). How many distinct 8 KB pages in `shared_buffers` may be dirtied by this single `INSERT`? Compare the I/O cost of calling `fdatasync` on five scattered files across a 2 TB table space at `COMMIT` versus appending roughly 300 bytes to one sequential `pg_wal` file.

Contents

Preface **iii**

I Relations, Storage Pages, and Access Methods **1**

1 The Relational Model and True SQL Semantics **3**

1.1 Codd's idea: physical data independence 4

1.2 Relations versus SQL tables: sets, bags, and keys 6

1.2.1 Why normalization matters: functional dependencies in one picture 7

1.3 Relational algebra: the engine underneath SQL 8

1.3.1 Why algebraic laws make declarative SQL fast 10

1.4 SQL's true logical evaluation pipeline 10

1.4.1 The `LEFT JOIN`: `ON` predicate versus `WHERE` predicate 12

1.5 `NULL` and Kleene's three-valued logic 13

1.5.1 Why `NOT IN` returns nothing when the subquery contains `NULL` 15

1.6 Window functions: aggregating without collapsing rows 17

1.6.1 The silent `RANGE` versus `ROWS` trap on duplicate timestamps 17

1.7 CTEs and fixpoint iteration with `WITH RECURSIVE` 19

1.7.1 Non-recursive `WITH`, planner inlining, and data-modifying CTEs 19

1.7.2 How `WITH RECURSIVE` actually executes: two tables in a loop 20

1.8 Exercises 22

2 Inside an 8 KB Heap Page and the Buffer Pool **25**

2.1 Where a Table Actually Lives on Disk 26

2.1.1 Why `oid` and `relfilenode` Split: Transactional File Swaps 26

2.1.2 1 GB Segment Splitting 27

2.1.3 Companion Forks: Free Space Map (_fsm) and Visibility Map (_vm) 28
2.2 Anatomy of an 8,192-Byte Slotted Heap Page 29
2.2.1 Byte-Offset Tour of PageHeaderData (Bytes 0..24) 30
2.2.2 The Line Pointer Array: ItemIdData (4 Bytes per Slot) 31
2.2.3 Step-by-Step Trace: Inserting, Deleting, and Compacting Without Breaking Indexes 32
2.3 Inside a Heap Tuple: Headers, Null Bitmaps, and Column Tetris 34
2.3.1 Why a 23-Byte Struct Produces a 24-Byte Header (and Free 8-Column Null Bitmaps) 35
2.3.2 CPU Alignment Padding and "Column Tetris" 36
2.3.3 Decoding Real Disk Bytes with pageinspect 38
2.4 TOAST: The Oversized-Attribute Storage Technique 39
2.4.1 Inside varlena Headers: Saving 3 to 6 Bytes on Every Short String 39
2.4.2 The Four TOAST Strategies and pg_toast Chunk Slicing 40
2.5 Double Buffering, shared_buffers, Pins, and Clock-Sweep 41
2.5.1 Why Not Just Rely on the Linux Page Cache? (Double Buffering) 42
2.5.2 Inside shared_buffers: Hash Partitions and BufferDesc.state 42
2.5.3 Buffer Pins (refcount) vs. Content Locks (LWLock) 43
2.5.4 Clock-Sweep Eviction: Step-by-Step Trace on a 6-Frame Pool 44
2.5.5 Buffer Access Strategies: Why a 500 GB Seq Scan Doesn't Wipe Out RAM . 45
2.6 Exercises 46

3 Indexes: Lehman-Yao B-Link Trees, GIN, GiST, and BRIN 49
3.1 Why Secondary Indexes Exist and B+Tree Branching Math 50
3.1.1 Why Binary Search Trees Lose Badly on Block Devices 50
3.1.2 Inside an 8 KB nbtree Page: IndexTupleData and BTPageOpaqueData 51
3.2 From Textbook B+Trees to PostgreSQL's Lehman–Yao B-Link Tree 52
3.2.1 Why Naive Top-Down Latch Crabbing Limits Multi-Core Scaling 52
3.2.2 The Lehman–Yao Insight: P_HIKEY and Right-Links (btpo_next) 53
3.2.3 Step-by-Step Trace: A Concurrent Reader Crossing an Unfinished Split 54
3.2.4 Modern nbtree Engineering: Tie-Breaking, Deduplication, and Bottom-Up Deletion 55
3.3 Composite Index Lexicographic Geometry and Column Order 57
3.3.1 The Phonebook Ribbon: Equality Pins a Slice, Range Scatters Jumps 57

3.3.2 Sort Elimination and Backward Leaf Scans . . . 59
3.4 Covering Indexes (INCLUDE), the Visibility Map, and True Index-Only Scans . . . 59
3.4.1 Why PostgreSQL Index Tuples Carry Zero MVCC Visibility Metadata . . . 59
3.4.2 How the Visibility Map (_vm) Rescues Index-Only Scans . . . 60
3.4.3 Why INCLUDE Beats Appending Payload Columns to the Key . . . 61
3.5 Partial Indexes (WHERE ...) and Expression Indexes . . . 62
3.5.1 Partial Indexes: Shrinking a 4 GB Queue Index to 2 MB . . . 62
3.5.2 Expression Indexes and Syntactic Predicate Alignment . . . 63
3.6 Beyond B-Trees: Hash, GIN, GiST, and BRIN . . . 63
3.6.1 Hash Indexes (USING hash): 32-Bit Codes Only . . . 64
3.6.2 GIN (Generalized Inverted Index): Multi-Element Containment at Scale . . . 64
3.6.3 GiST (Generalized Search Tree): Bounding Hierarchies and Exclusion Constraints . . . 65
3.6.4 BRIN (Block Range Index): Indexing a Terabyte in 30 Megabytes . . . 66
3.7 Exercises . . . 68

II Durability, Concurrency, and MVCC Internals 71

4 Write-Ahead Logging, Checkpoints, and Crash Recovery 73
4.1 The Physics of write(), fsync(), and Torn 8 KB Pages . . . 73
4.1.1 Why Direct Heap Flushing on COMMIT Fails Twice . . . 75
4.2 Write-Ahead Logging (pg_wal), LSNs, and the Fundamental WAL Invariant . . . 76
4.2.1 Segment Files in PGDATA/pg_wal/ and pg_lsn Byte Arithmetic . . . 77
4.2.2 Inside an XLogRecord and Resource Managers . . . 78
4.2.3 pd_lsn and the Fundamental WAL Invariant . . . 78
4.3 Torn Pages Solved: full_page_writes After a Checkpoint . . . 80
4.3.1 Hole-Stripped Full Page Images (FPIs) vs. Doublewrite Buffers . . . 81
4.4 Commit Durability Tradeoffs, Group Commit, and wal_writer . . . 83
4.4.1 wal_buffers, Parallel Insertion Slots, and Group Commit . . . 84
4.4.2 The synchronous_commit Spectrum and wal_writer . . . 85
4.4.3 UNLOGGED Tables: Trading Crash Survival for Raw RAM-to-Disk Speed . . . 86
4.5 Checkpoints, Background Writer, and Step-by-Step Crash Recovery . . . 87

4.5.1 global/pg_control, Fuzzy Checkpoints, and Why Recovery Rewinds to the Redo Point . . . 87
4.5.2 Three Buffer Writers, Three Distinct Jobs . . . 88
4.5.3 Classic ARIES (3 Passes) vs. PostgreSQL's 1-Pass Redo-Only Recovery . . . 89
4.5.4 Worked LSN-by-LSN Crash Recovery Trace . . . 91
4.6 Exercises . . . 94

5 MVCC, Snapshots, VACUUM, and Transaction ID Wraparound 97
5.1 Two Ways to Build a Time Machine: Undo Logs vs. In-Heap Versions . . . 98
5.2 Inside HeapTupleHeaderData: xmin, xmax, cid, and ctid . . . 99
5.2.1 t_xmin and Lazy Transaction ID Assignment . . . 100
5.2.2 t_xmax: Deletion, Update Chains, and Row Locks . . . 100
5.2.3 t_cid and the Halloween Problem . . . 101
5.3 Commit Status (pg_xact) and Why a SELECT Can Dirty Disk Pages . . . 102
5.3.1 The Commit Log (pg_xact): Two Bits per Transaction . . . 102
5.3.2 Tuple Hint Bits: Caching Commit Status on the Heap Tuple . . . 103
5.4 Snapshots and the Exact Visibility Formula . . . 103
5.5 Write Amplification and HOT (Heap-Only Tuple) Updates . . . 106
5.5.1 How HOT Decouples B-Trees from Unindexed Updates . . . 107
5.5.2 Single-Page Pruning (On-the-Fly Defragmentation Without VACUUM) . . . 108
5.6 Table Bloat, VACUUM, autovacuum, and VACUUM FULL . . . 109
5.6.1 The Three Phases of Concurrent ("Lazy") VACUUM . . . 109
5.6.2 Why VACUUM Does Not Shrink Disk Usage (vs. VACUUM FULL) . . . 109
5.7 32-Bit Transaction ID Wraparound and Tuple Freezing . . . 110
5.7.1 Modular Circle Comparison (TransactionIdPrecedes) . . . 111
5.7.2 The Cure: Tuple Freezing, relfrozenxid, and the Safety Shutdown . . . 112
5.8 Exercises . . . 114

6 Isolation Anomalies, Row Locking, and Serializable Snapshot Isolation 117
6.1 What "Isolation" Actually Promises (and Why ANSI SQL-92 Missed Half the Story) . 118
6.1.1 The ANSI SQL-92 Table vs. PostgreSQL Reality . . . 118
6.2 Read Anomalies: Dirty Reads, Read Skew, and Phantoms . . . 119
6.2.1 Dirty Read (P_1): Reading Uncommitted Bytes . . . 119

6.2.2 Non-Repeatable Read (P_2) and Read Skew Under `READ COMMITTED` . . . 119
6.2.3 Phantom Reads (P_3) . . . 120
6.3 Lost Updates and `EvalPlanQual` Re-Evaluation . . . 121
6.3.1 Case 1: Read-Modify-Write in Application Code Under `READ COMMITTED` . . . 121
6.3.2 Case 2: Atomic SQL `SET bal = bal - 8000` and `EvalPlanQual` (`EPQ`) . . . 121
6.4 Write Skew: Why `REPEATABLE READ` Still Breaks Multi-Row Invariants . . . 122
6.5 Serializable Snapshot Isolation (SSI): True `SERIALIZABLE` Without Read Locks . . . 123
6.5.1 Conflict Edges and the "Dangerous Structure" Theorem . . . 124
6.5.2 How `SIRead` Predicate Locks Work Inside PostgreSQL . . . 124
6.6 Explicit Row Locks, Work Queues, Deadlocks, and Advisory Locks . . . 126
6.6.1 PostgreSQL's Four Row-Level Lock Strengths . . . 126
6.6.2 Non-Blocking Modifiers: `NOWAIT` and `SKIP LOCKED` Job Queues . . . 127
6.6.3 Deadlocks (`deadlock_timeout`) and Deterministic Lock Ordering . . . 127
6.6.4 Transactional Advisory Locks (`pg_advisory_xact_lock`) . . . 128
6.7 Exercises . . . 129

III Query Execution and Production PostgreSQL 131

7 The Query Planner, Join Algorithms, and `EXPLAIN ANALYZE` 133
7.1 From SQL Text to a Running Plan . . . 134
7.2 What the Planner Knows: `ANALYZE` and `pg_stats` . . . 135
7.2.1 Sampling a Table of Any Size . . . 135
7.2.2 Reading One Row of `pg_stats` . . . 135
7.2.3 Turning Statistics into Selectivity . . . 136
7.2.4 When Two Predicates Are Not Independent . . . 137
7.3 The Planner's Currency: Cost Units . . . 138
7.3.1 Pricing a Sequential Scan by Hand . . . 139
7.4 Four Ways to Read One Table . . . 140
7.4.1 Seq Scan . . . 140
7.4.2 Index Scan . . . 140
7.4.3 Bitmap Index Scan plus Bitmap Heap Scan . . . 141
7.4.4 Index-Only Scan . . . 142

7.5 Three Ways to Join Two Tables . . . 143
7.5.1 Nested Loop . . . 143
7.5.2 Hash Join . . . 144
7.5.3 Merge Join . . . 145
7.6 Searching Join Orders . . . 146
7.6.1 Dynamic Programming over Subsets . . . 147
7.6.2 Limits and the Genetic Optimizer . . . 147
7.7 Reading `EXPLAIN (ANALYZE, BUFFERS)` Line by Line . . . 148
7.7.1 A Complete Plan . . . 148
7.7.2 Diagnosing a Bad Plan . . . 150
7.8 Exercises . . . 152

8 Financial Schemas, Replication, and Connection Pooling 155
8.1 Data Types That Never Lie About Money, Time, or Identity . . . 156
8.1.1 Money: Why Binary Floats Fail, and `BIGINT` vs. `NUMERIC` . . . 156
8.1.2 Time: `TIMESTAMPTZ` and the Daylight-Saving Trap . . . 157
8.1.3 Primary Keys at Scale: Why `UUIDv4` Keys Thrash B-Trees and `UUIDv7` Flies . 158
8.2 Encoding Domain Invariants Directly in the Schema . . . 159
8.2.1 `NOT NULL`, `CHECK`, Domains, and Atomic Idempotency Keys . . . 159
8.2.2 Temporal Non-Overlapping Invariants with `EXCLUDE USING gist` . . . 160
8.3 A Double-Entry Financial Ledger in PostgreSQL and Rust . . . 160
8.3.1 Conservation of Value and `DEFERRABLE INITIALLY DEFERRED` Triggers . . . 161
8.3.2 Deadlock-Free Transfers in Rust (`Axum` + `SQLx`) . . . 163
8.4 Declarative Partitioning for Billion-Row Ledgers . . . 167
8.5 High Availability: Physical Streaming and Logical Replication . . . 168
8.5.1 Physical Streaming WAL Replication and Standby Conflicts . . . 169
8.5.2 The `synchronous_commit` Durability and Visibility Ladder . . . 169
8.5.3 Logical Replication for Zero-Downtime Upgrades and Event Streaming . . . 170
8.6 Connection Pooling at Scale: Process Architecture and PgBouncer . . . 171
8.6.1 Why Five Thousand Backend Processes Melt PostgreSQL . . . 171
8.6.2 `PgBouncer` Transaction Multiplexing and the Prepared Statement Trap . . . 172
8.7 Exercises . . . 174

Part I

Relations, Storage Pages, and Access Methods

Chapter 1

The Relational Model and True SQL Semantics

Suppose you are building the storage engine for a bank in 1968. You have magnetic disks divided into fixed-size blocks, a C or COBOL compiler, and three kinds of facts to store: customers, accounts, and ledger postings. How do you organize those records on disk so you can find all postings belonging to a customer?

If you think like a systems programmer accustomed to in-memory structs, the natural answer is *pointers*. Inside each customer record on disk, you store the disk address (cylinder, track, and record offset) of that customer's first account. Inside each account record, you store a pointer to the customer's next account, plus a pointer to that account's first posting. Inside each posting, you store a pointer to the next posting in chronological order, and perhaps a back-pointer to the owning account. To read a customer's monthly statement, your program chases disk pointers around a linked ring, one block seek at a time.

If you have written Java 6, you have written this kind of program. A loop such as

```
for (Account a : customer.getAccounts())
    for (Posting p : a.getPostings())
        total += p.getCents();
```

follows object references, and the JVM stores each reference as a pointer to a heap address. The 1968 databases did the same thing on disk, baking those pointers into the file format. Any query that did not follow the chains built in advance needed new chains, a file reorganization, and new application code.

For twenty years, commercial databases worked that way. IBM's Information Management System (IMS, built for the Apollo program in 1966) arranged records into physical parent–child trees. Charles Bachman's Integrated Data Store (IDS) and the CODASYL standard generalized trees into arbitrary pointer networks. Bachman won the 1973 Turing Award for casting the programmer as a "navigator" steering a cursor through disk addresses.

Then, in June 1970, an IBM mathematician named Edgar F. Codd published an eleven-page paper titled *A Relational Model of Data for Large Shared Data Banks*. He argued that application programs should never follow disk pointers at all.

Focus

This chapter builds Codd's alternative from the ground up and connects it to the SQL that PostgreSQL executes today. By the end, you will be able to answer five practical engineering questions:

- Why did storing direct disk pointers between records make early databases fast for one query and brittle for every new question?
- Why does SQL keep duplicate rows by default, and what physical work does removing them actually cost?
- Why does a column alias written on the first line of `SELECT` fail inside `WHERE`, yet work inside `ORDER BY`?
- How can a single missing value (`NULL`) in a ten-million-row subquery make `NOT IN` return zero rows without a warning?
- How do window functions compute running balances without collapsing rows, and how does `WITH RECURSIVE` walk a tree without a call stack?

1.1 Codd's idea: physical data independence

To see why Codd wanted to ban user-visible pointers from databases, watch what happens to a pointer-based banking system the moment requirements change.

In our 1968 pointer network, every posting chain runs forward from `Account` → `Posting_1` → `Posting_2` → ⋯ → `Posting_k`. Answering "list all postings for Account 102" is fast. You jump to Account 102 and follow `next_posting` links until the chain loops back to the owner.

Now Monday morning arrives, and the fraud desk asks the inverse question: "Which accounts sent a posting of more than 5,000 USD to Merchant 88 yesterday?"

Nothing in your disk layout wires Merchant 88 to its incoming postings, and nothing wires yesterday's date to a starting address. Worse, the application code that reads existing reports is coupled line-by-line to the physical pointer topology:

1. **Asymmetric query cost and brittle code:** Every query must be hand-written as imperative nested loops (`FIND FIRST`, `FIND NEXT WITHIN SET`) tied to existing pointer chains. If an engineer later replaces a linked list with a B-tree or partitions postings by month, every program that called `FIND NEXT` breaks.
2. **Dangling and corrupt pointer chains:** Inserting one posting updates up to four disk pointers across different blocks: previous sibling, next sibling, account head, and merchant ring. If a bug or crash leaves one pointer half-updated, traversing the ring loops forever or lands inside an unrelated account's records.

History

Edgar F. Codd joined IBM after serving as a flight lieutenant in the Royal Air Force and earning a doctorate in mathematics at the University of Michigan. He watched IBM customers

rewrite thousands of lines of code every time a file layout or index changed on System/360 mainframes. His remedy separated the *logical representation of facts* completely from *physical access paths*. Inside IBM, defenders of IMS dismissed Codd's model as an academic toy that could never match hand-tuned pointer navigation. Two prototypes proved otherwise: IBM's System R in San Jose (which created SEQUEL, later SQL, and cost-based query optimization) and Michael Stonebraker's Ingres project at UC Berkeley (whose successor, Postgres, became modern PostgreSQL). Codd received the 1981 Turing Award once automatic optimizers beat hand-navigated pointer rings on both maintainability and throughput.

Codd's design rests on one clean separation, known as **physical data independence**:

- **Logical layer (what your SQL sees):** All information in the database is represented in one way: explicit *values* sitting inside two-dimensional mathematical *relations*. An account holds no disk pointer to its owner. Instead, it holds the value `customer_id = 42`, matching `id = 42` in the `customers` relation.

- **Physical layer (what the storage engine hides):** Underneath that logical interface, PostgreSQL stores rows in unsorted 8 KB pages (Chapter 2) and builds B-tree or block-range indexes (Chapter 3). It can also compress wide columns out-of-line or partition a ledger by month (Chapter 8). You can add or drop five indexes at noon without changing a single character of any `SELECT` query or changing a single answer.

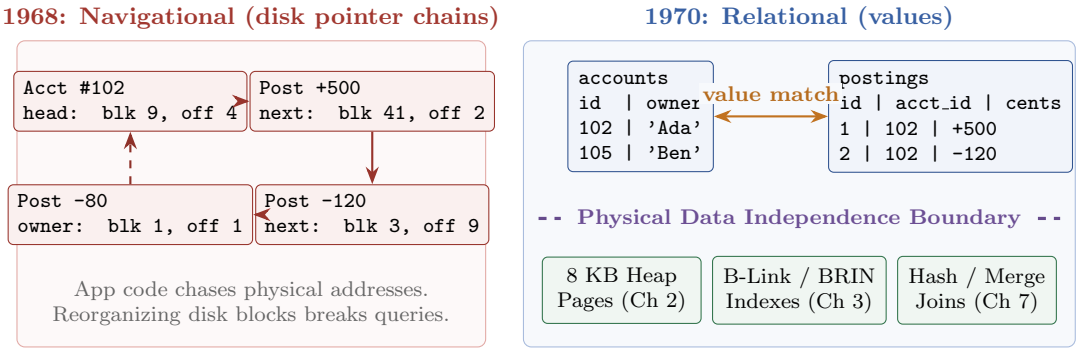


Figure 1.1: Navigational databases expose physical disk pointer rings directly to application code (left). Codd's relational model represents relationships purely through matching data values above a strict data-independence boundary (right), freeing PostgreSQL to swap physical storage layouts and indexes underneath unchanged SQL queries.

Intuition

Pointers still exist inside PostgreSQL — millions of them. Every B-tree leaf entry holds a 6-byte `ctid` pointing to an 8 KB heap block and slot, and every updated heap tuple points forward to its newer version. Codd did not abolish pointers from the *machine*; he moved them *below the abstraction line* where the storage engine maintains them automatically and the query optimizer chooses whether to follow them or scan sequentially.

1.2 Relations versus SQL tables: sets, bags, and keys

To reason sharply about queries, we need both Codd's mathematical definition of a relation and the one pragmatic compromise that real SQL engines make for speed.

Formal

Let $D_1, D_2, \ldots, D_k$ be sets of atomic values called **domains** (for instance, 64-bit integers $\mathbb{Z}_{2^{64}}$, UTF-8 text strings, or UTC microsecond timestamps).

1. A **relation schema** $R(A_1 : D_1, \ldots, A_k : D_k)$ names a relation R together with k distinct **attributes** (columns) A_i, each constrained to domain D_i. The integer k is the **degree** (or **arity**) of R.
2. A **tuple** t conforming to R assigns each attribute A_i an element $t[A_i] \in D_i$ (or, in SQL, the missing-information marker `NULL`).
3. A **pure mathematical relation** (or **relation instance**) over R is a finite subset

$$r \subseteq D_1 \times D_2 \times \cdots \times D_k.$$

   Because r is a *set*, two consequences hold immediately: tuples have **no inherent top-to-bottom order**, and **duplicate tuples cannot exist** in r.

If you open `psql`, create a table without a `PRIMARY KEY`, and run `INSERT INTO logs VALUES ('login')` twice, PostgreSQL stores two identical rows. And if you run `SELECT currency FROM accounts` on a table with ten thousand `'USD'` accounts, PostgreSQL returns `'USD'` ten thousand times rather than collapsing it to a single row. Why does SQL depart from pure set theory here?

The answer is pure algorithmic cost. Suppose a table has $N = 50{,}000{,}000$ rows and you project two non-unique columns: `SELECT merchant_id, amount_cents FROM postings`. Removing duplicates across 50 million pairs is expensive. PostgreSQL would either sort all 50 million rows in $O(N \log N)$ time across temporary disk files, or build a huge RAM hash table with one entry per distinct pair. Most callers streaming rows into a report do not want to pay that tax unless they ask for it.

In Java terms, Codd's relation behaves like a `HashSet<Row>`: adding an equal row twice changes nothing, and iteration order is unspecified. A SQL table without a key behaves like an `ArrayList<Row>` whose order you may not rely on. Duplicates stay, and `SELECT` hands them all back.

An unconstrained SQL table or query result is therefore not a mathematical set. It is a **bag** (a **multiset**), modeled by attaching a non-negative integer count $\mu_R(t) \in \mathbb{N}$ to every possible tuple t:

SQL Operator	Multiset Multiplicity	Physical Consequence in PostgreSQL
`SELECT a, b FROM R`	Sums $\mu_R(t)$ over matches	$O(N)$ streaming pass; zero memory overhead
`SELECT DISTINCT a, b`	$\min(1, \sum \mu_R(t))$	Requires `HashAggregate` or `Sort + Unique`
`R UNION ALL S`	$\mu_R(t) + \mu_S(t)$	$O(1)$-memory `Append` streaming R then S
`R UNION S`	$\min(1, \mu_R(t) + \mu_S(t))$	Concatenates, then hashes or sorts to deduplicate
`R INTERSECT ALL S`	$\min(\mu_R(t), \mu_S(t))$	Pairs up matching multiplicities via hash counters
`R EXCEPT ALL S`	$\max(0, \mu_R(t) - \mu_S(t))$	Subtracts S's count of t from R's count of t

Common trap

Writing `UNION` when you actually know the two branches are disjoint (for example, combining active orders from 2025 with archived orders from 2024) is one of the most common self-inflicted performance wounds in SQL. Bare `UNION` forces PostgreSQL to buffer and hash or sort the combined output just to prove there are no duplicates. Whenever two subqueries are already disjoint — or duplicates are harmless — write `UNION ALL` so PostgreSQL can stream both inputs directly through a zero-overhead `Append` operator.

How do we bring a base SQL table back into Codd's clean set world where every row represents a distinct, addressable fact? By declaring a **key**:

- A **superkey** of schema R is any subset of attributes $K \subseteq \{A_1, \ldots, A_k\}$ such that no two legal tuples in R may agree on all attributes of K: if $t_1[K] = t_2[K]$, then $t_1 = t_2$.
- A **candidate key** is a *minimal* superkey — removing any attribute from K destroys uniqueness. For example, on an `accounts` table, {`id`} might be one candidate key and {`iban`} a second candidate key, whereas {`id`, `currency`} is a superkey that is not minimal.
- A **primary key** (`PRIMARY KEY`) is the candidate key chosen as the canonical identifier of rows in R, with the additional SQL guarantee that none of its columns may ever be `NULL`. PostgreSQL enforces every `PRIMARY KEY` and `UNIQUE` constraint physically by maintaining a unique B-tree index (Chapter 3).
- A **foreign key** (`REFERENCES`) on child relation S asserts *referential integrity* against a candidate key of parent relation R: every non-`NULL` key value appearing in S must currently exist in R.

1.2.1 Why normalization matters: functional dependencies in one picture

Even with a primary key on every table, a poorly factored schema can rot from the inside. Suppose an engineer designs one wide ledger table `postings_denorm` with columns `(posting_id, account_id, owner_email, account_tier, amount_cents)`, where `posting_id` is the primary key. Because `account_id = 102` belongs to `'ada@example.org'` on tier `'pro'`, those two strings are repeated across all 40,000 postings for Account 102.

Mathematically, the table contains a **functional dependency**, written

$$\texttt{account_id} \rightarrow \{\texttt{owner_email}, \texttt{account_tier}\},$$

meaning that knowing `account_id` uniquely determines `owner_email` and `account_tier`, even though `account_id` is *not* a superkey of `postings_denorm`. Whenever a non-superkey attribute determines another attribute, three classic anomalies appear immediately:

1. **Update anomaly (physical write amplification + inconsistency):** When Ada changes her email address, PostgreSQL cannot update one row; it must rewrite 40,000 heap tuples across hundreds of 8 KB pages (Chapter 2). Worse, without a separate `accounts` table, nothing stops a bug from updating 39,999 rows and leaving one row with her old email.
2. **Insertion anomaly:** You cannot open Account 109 for a brand-new customer who has zero postings yet without inventing a fake dummy posting or permitting `posting_id` to be `NULL`.
3. **Deletion anomaly:** If a test account's lone posting is reversed and deleted, the fact that the account ever existed — along with its owner's email and tier — vanishes with it.

Why it works

Boyce–Codd Normal Form (BCNF) in one sentence. A relation schema R is in Boyce–Codd Normal Form if and only if for every non-trivial functional dependency $X \rightarrow Y$ holding on R (meaning $Y \not\subseteq X$), X is a *superkey* of R. Colloquially: *every fact in R depends on the key, the whole key, and nothing but the key.* Splitting `postings_denorm` into `accounts(account_id, owner_email, account_tier)` and `postings(posting_id, account_id, amount_cents)` puts both relations in BCNF: each fact lives in one place, Ada's email update touches exactly one 8 KB page, and no update, insertion, or deletion anomaly can ever occur.

1.3 Relational algebra: the engine underneath SQL

When PostgreSQL receives a SQL query, it does not execute the English words directly. Its parser and analyzer translate your SQL text into a tree of algebraic operators whose inputs are relations and whose outputs are relations. Because every operator produces a relation, operators *compose* cleanly — the output of a join feeds straight into a filter, which feeds into a grouping operator, which feeds into a projection.

Let us fix a concrete miniature dataset that we can trace by hand across the entire chapter: a four-row `accounts` table A and a six-row `postings` table P, with amounts stored in integer cents.

Relation A (`accounts`)

id	owner	tier
101	'Ada'	'pro'
102	'Ben'	'free'
103	'Cara'	'pro'
104	'Dan'	'pro'

Relation P (`postings`)

pid	acct_id	cents	posted_on
1	101	+10000	'2026-10-01'
2	101	−2500	'2026-10-02'
3	102	+4000	'2026-10-02'
4	103	+15000	'2026-10-01'
5	103	−3000	'2026-10-03'
6	103	+2000	'2026-10-03'

Notice Account `104` (`'Dan'`): he opened a `'pro'` account, so he sits in A, yet he has zero rows in P. Watching what happens to Dan across joins and filters will expose half of the bugs people write in production SQL.

Instead of chasing disk pointers by hand, an engineer needs five reusable table operations—plus outer, semi-, and anti-join variants—that compose cleanly into any query:

1. **Filtering rows (Selection, σ):** Most queries want only a subset of rows—say, every `'pro'` account—without altering the columns. Selection writes $\sigma_\theta(R)$ for this horizontal slice, keeping every tuple $t \in R$ that satisfies predicate θ. On our dataset, $\sigma_{\texttt{tier = 'pro'}}(A)$ keeps rows `101`, `103`, and `104`, dropping Ben (`102`).

2. **Picking and computing columns (Projection, π):** Callers rarely want every column on disk, and often want derived values like cents converted to dollars. Projection writes $\pi_{e_1 \to b_1, \ldots, e_m \to b_m}(R)$ for this vertical slice, evaluating $e_1, \ldots, e_m$ on each tuple of R. Pure set algebra deduplicates projected rows automatically, whereas SQL's bag algebra (π^b) keeps duplicates unless `DISTINCT` (δ) is requested.

3. **Matching related tables by value (Cartesian Product $\times$ and Join $\bowtie$):** Because relational rows hold matching keys rather than disk pointers, reconnecting accounts to postings requires pairing rows across tables. The Cartesian product $A \times P$ pairs *every* row of A with *every* row of P, yielding $|A| \cdot |P| = 4 \times 6 = 24$ seven-column tuples. An **inner theta-join**, written $A \bowtie_\theta P$, keeps only the pairs that satisfy condition θ:

   $$A \bowtie_\theta P \;\coloneqq\; \sigma_\theta(A \times P).$$

   When θ tests equality, such as $A.\texttt{id} = P.\texttt{acct_id}$, the join is called an **equi-join**. Out of the 24 pairs in $A \times P$, only the 6 pairs with matching account IDs survive. Those 6 pairs give two rows for Ada, one for Ben, three for Cara, and *zero* for Dan, whose account `104` has no postings in P.

4. **Preserving unmatched rows and testing existence (Outer, Semi-, and Anti-Joins):**

   - **Left Outer Join** (A `LEFT JOIN` P): Reports often must list every customer even when they have zero activity. A left join returns all pairs from $A \bowtie_\theta P$, *plus* each unmatched row of A padded with `NULL` across P's columns. Here Dan (`104`) survives as (`104`, `'Dan'`, `'pro'`, `NULL`, `NULL`, `NULL`, `NULL`).
   - **Semi-Join** ($A \ltimes_\theta P$, SQL `WHERE EXISTS (...)`): Sometimes we only want to check whether an account has *any* posting, without multiplying the account row once per match. Whereas $A \bowtie P$ emits three rows for Cara (`103`), the semi-join $A \ltimes_\theta P$ emits each matched account *once* (`101`, `102`, `103`).
   - **Anti-Join** ($A \triangleright_\theta P$, SQL `WHERE NOT EXISTS (...)`): Finding accounts that have *never* transacted asks the opposite question. The anti-join $A \triangleright_\theta P$ keeps tuples of A with *zero* matches in P—here returning solely (`104`, `'Dan'`, `'pro'`).

5. **Summarizing buckets of rows (Grouping and Aggregation, γ):** Dashboards and ledgers constantly collapse many postings per account into one net balance. Grouping and aggregation, written $\gamma_{G;\, f_1(c_1) \to s_1, \ldots}(R)$, partitions R into buckets sharing equal grouping attributes G. It then collapses each bucket into one output row carrying G and aggregates $f_j(c_j)$ (`COUNT`, `SUM`, `MIN`, `MAX`, `AVG`). Applying $\gamma_{\texttt{acct_id};\, \texttt{SUM(cents)} \to \texttt{bal}}(P)$ collapses P's six rows into three account totals:

   $$(101,\ 7500), \qquad (102,\ 4000), \qquad (103,\ 14000).$$

1.3.1 Why algebraic laws make declarative SQL fast

Why did defining queries in this small algebra defeat hand-coded pointer loops? Because relational operators obey exact **algebraic identities** that let an optimizer transform a slow expression into an equivalent fast one before touching a single disk block.

Suppose a user asks for the posting amounts of all `'pro'` accounts on or after October 2:

$$E_{\text{naive}} \;=\; \pi_{\texttt{owner,cents}}\Big(\sigma_{A.\texttt{tier}=\texttt{'pro'} \,\wedge\, P.\texttt{posted_on}\geq\texttt{'2026-10-02'} \,\wedge\, A.\texttt{id}=P.\texttt{acct_id}}(A \times P)\Big).$$

Executed as written on a production bank with $|A| = 10^6$ accounts and $|P| = 10^8$ postings, forming $A \times P$ first would materialize 10^{14} intermediate tuples — crashing the server long before finishing.

Instead, PostgreSQL's planner applies three equivalence laws automatically:

$$\begin{aligned}
\sigma_{p_1 \wedge p_2}(R) &= \sigma_{p_1}(\sigma_{p_2}(R)) && \text{(cascade of selections)},\\
\sigma_{p_A}(A \bowtie_\theta P) &= \big(\sigma_{p_A}(A)\big) \bowtie_\theta P && \text{(selection pushdown when } p_A \text{ mentions only } A),\\
\sigma_\theta(A \times P) &= A \bowtie_\theta P && \text{(cross product + equality} \to \text{hash/index equi-join)}.
\end{aligned}$$

Pushing $\sigma_{\texttt{tier='pro'}}$ down onto A and $\sigma_{\texttt{posted_on}\geq\texttt{'2026-10-02'}}$ down onto P *before* joining shrinks both inputs dramatically, and replacing $\times$ with an indexed or hash equi-join drops the work from 10^{14} comparisons to linear in the surviving rows.

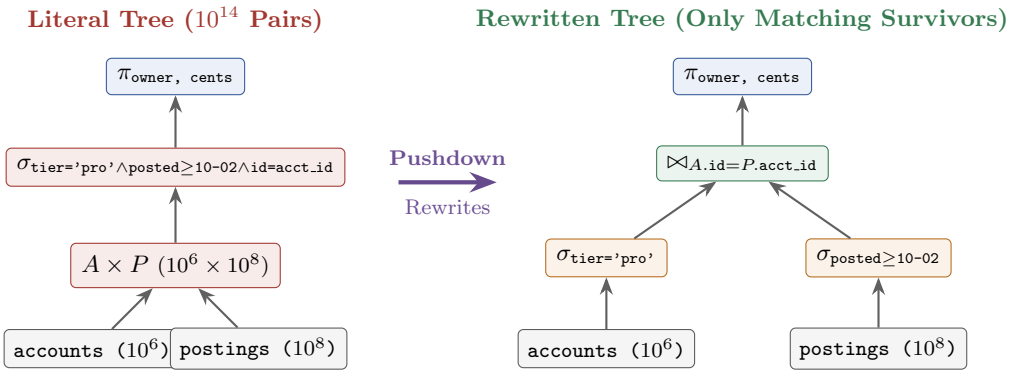


Figure 1.2: Relational algebra optimization in miniature: pushing single-table selections below the join and converting $\sigma_{A.\texttt{id}=P.\texttt{acct_id}}(A \times P)$ into an equi-join produces the exact same output relation while avoiding trillions of useless tuple pairs.

1.4 SQL's true logical evaluation pipeline

If you ask a programmer why the query below fails in PostgreSQL with an unknown-column error (`column "net_cents" does not exist`), they are often momentarily baffled:

```
-- BROKEN: net_cents was written on line 1, so why can't line 3 see it?
SELECT acct_id, SUM(cents) AS net_cents
  FROM postings
 WHERE net_cents > 5000
 GROUP BY acct_id;
```

PostgreSQL rejects it at parse time:

```
ERROR:  column "net_cents" does not exist
LINE 3:  WHERE net_cents > 5000
               ^
```

After all, `net_cents` is defined right there on line 1, two lines *above* `WHERE`. Why can `WHERE` not see it, while `ORDER BY net_cents DESC` at the bottom of the query *can* see it?

Because the *written order* of SQL clauses on the page (chosen in the 1970s to read like English) is **not** the order in which SQL defines a query's meaning. Logically, every `SELECT` block is a ten-stage pipeline. The `SELECT` list itself does not run until **Stage 7**.

Formal

The Ten-Stage Logical Evaluation Pipeline of a SQL Query. Given a single query block, its output bag of rows is defined by ten stages in sequence. Each stage consumes the virtual table produced by the stage before it:

1. `FROM`: Load base tables, views, Common Table Expressions (CTEs, i.e., named `WITH` subqueries, Section 1.7), or `LATERAL` functions into initial virtual relations.
2. `JOIN / ON`: Pair input rows, keep pairs matching `ON`, and (for outer joins) pad unmatched preserved-side rows with `NULL`s.
3. `WHERE`: Filter rows by θ; keep *only* rows where θ is `TRUE`, discarding both `FALSE` and `UNKNOWN`.
4. `GROUP BY`: Partition surviving rows into buckets sharing equal grouping values (putting `NULL`s into one shared bucket), ready for aggregation.
5. `HAVING`: Filter the collapsed groups; keep *only* groups whose aggregate predicate is `TRUE`.
6. `WINDOW`: Evaluate every `OVER (...)` window function across surviving rows without collapsing them.
7. `SELECT`: Compute output expressions and bind column aliases (`AS alias`). Only from Stage 7 onward do those aliases exist.
8. `DISTINCT` (or `DISTINCT ON (...)`): Remove duplicate projected rows.
9. `ORDER BY`: Sort the output rows. This stage can see `SELECT` aliases, and if `DISTINCT` was not used, unprojected input columns too.
10. `LIMIT / OFFSET` (`FETCH FIRST ... ROWS ONLY`): Skip leading `OFFSET` rows and stop after `LIMIT` rows.

Once you know this order, several confusing SQL rules follow directly from the stage numbers. Each comes with the error PostgreSQL actually prints:

1. **Why `WHERE` rejects `SUM(cents) > 5000` and requires `HAVING`:** At Stage 3 (`WHERE`), rows have not been grouped yet. PostgreSQL is looking at one posting row at a time, so `SUM(cents)` does not exist yet. Individual postings are filtered at Stage 3 (`WHERE`), while collapsed account buckets are filtered at Stage 5 (`HAVING`). Writing `WHERE SUM(cents) > 5000` fails with:

```
ERROR:  aggregate functions are not allowed in WHERE
```

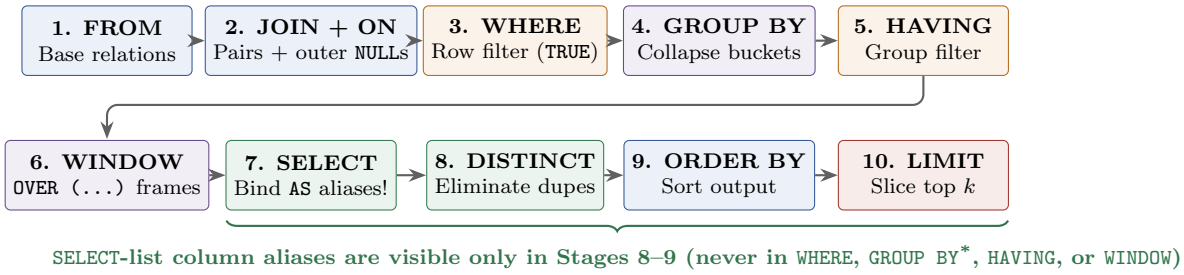


Figure 1.3: SQL's ten-stage logical evaluation pipeline. Notice why `WHERE` cannot reference aggregates (Stage 3 runs before Stage 4), why `WHERE` cannot filter on `ROW_NUMBER()` (Stage 3 runs before Stage 6), and why `SELECT` aliases appear only at Stage 7. (*PostgreSQL extends the SQL standard slightly by letting `GROUP BY` and `ORDER BY` refer to simple `SELECT` output column names when unambiguous, but never inside expressions or `WHERE`/`HAVING`.)

2. **Why `WHERE rn = 1` fails on `ROW_NUMBER() OVER (...) AS rn`:** Window functions are computed at Stage 6, three stages *after* `WHERE`. To keep only the latest posting per account, compute `rn` inside a CTE or subquery first, and then filter `WHERE rn = 1` in the outer query's Stage 3. (PostgreSQL also offers `DISTINCT ON (acct_id)` for this exact pattern.) Referring to `rn` in `WHERE` gives `column "rn" does not exist`, and inlining the window call gives:

```
ERROR:  window functions are not allowed in WHERE
```

3. **Why `ORDER BY owner` after `SELECT DISTINCT tier` errors out:** At Stage 8, `DISTINCT` collapses several input rows into one output row. If Stage 9 (`ORDER BY`) then asks to sort by a column not in the `SELECT` list, which `owner` among Ada, Cara, and Dan represents the single `'pro'` row? The question has no well-defined answer, so PostgreSQL rejects it at parse time:

```
ERROR:  for SELECT DISTINCT, ORDER BY expressions must appear in select list
```

1.4.1 The `LEFT JOIN`: `ON` predicate versus `WHERE` predicate

Nothing shows Stage 2 versus Stage 3 more clearly than filtering the optional side of a `LEFT JOIN`. Suppose you want a report listing *every* `'pro'` account, including zero-activity accounts like Dan (104). Alongside each account, you want the count of postings made on or after `'2026-10-02'`. Compare these two queries, which differ by a single keyword (`AND` versus `WHERE`):

```
-- Query A (WRONG for zero-activity accounts): puts date filter in Stage 3 (WHERE)
SELECT a.id, a.owner, COUNT(p.pid) AS oct2_or_later_cnt
  FROM accounts a
  LEFT JOIN postings p ON p.acct_id = a.id
 WHERE a.tier = 'pro'
   AND p.posted_on >= DATE '2026-10-02'
 GROUP BY a.id, a.owner;

-- Query B (CORRECT): puts right-table date filter in Stage 2 (JOIN ... ON)
SELECT a.id, a.owner, COUNT(p.pid) AS oct2_or_later_cnt
  FROM accounts a
  LEFT JOIN postings p
```

```
    ON p.acct_id = a.id
   AND p.posted_on >= DATE '2026-10-02'
 WHERE a.tier = 'pro'
 GROUP BY a.id, a.owner;
```

Trace Dan (`id = 104`) through both pipelines by hand:

- In **Query A**, Stage 2 (`LEFT JOIN`) finds no postings for Dan and emits the padded row (`104, 'Dan', 'pro', NULL, NULL, NULL, NULL`). Next, Stage 3 (`WHERE`) evaluates `NULL >= DATE '2026-10-02'`, which yields `UNKNOWN`. Because `WHERE` keeps only `TRUE`, **Dan's row is discarded**. Your `LEFT JOIN` silently turned into an `INNER JOIN`.
- In **Query B**, Stage 2 tests `p.acct_id = a.id AND p.posted_on >= '2026-10-02'` *during the join match itself*. Dan matches zero rows under that combined rule, so `LEFT JOIN` emits Dan padded with `NULL`s. Stage 3 checks only `a.tier = 'pro'` (`TRUE`). Stage 4 groups Dan's padded row, and `COUNT(p.pid)` counts non-`NULL` `pid`s to return `0`.

Common trap

Whenever you write `A LEFT JOIN P`, check two things before hitting enter:

1. Put filters on columns of the optional table `P` inside `ON`, never inside `WHERE` (unless you want an anti-join `WHERE p.pid IS NULL` to find unmatched rows).
2. In `SELECT`, write `COUNT(p.pid)`, *never* `COUNT(*)`. On Dan's `NULL`-padded row, `COUNT(*)` counts rows in the bucket and reports `1` posting, whereas `COUNT(p.pid)` counts non-`NULL` `pid`s and reports `0`.

Intuition

Logical order is a specification, not a physical straitjacket. PostgreSQL promises that every query's *answer* matches the ten-stage pipeline. Under the hood, the planner (Chapter 7) reorders work whenever the result cannot change. In Query B, it pushes `WHERE a.tier = 'pro'` down into the scan of `accounts` *before* Stage 2 runs, because filtering the preserved left table early never alters right-side matches. In Query A, it sees that `WHERE p.posted_on >= ...` rejects every `NULL`-padded row anyway, so it simplifies the `LEFT JOIN` into a faster `INNER JOIN`.

1.5 `NULL` and Kleene's three-valued logic

Codd's 1970 paper assumed every tuple cell held a known value from its domain. Real databases quickly ran into messy facts: a pending wire transfer has no `settled_at` timestamp yet, and a cash deposit has no `counterparty_iban`. SQL models missing or inapplicable data with a special marker called `NULL`.

Start with one rule: `NULL` **is not a value**. It is a *marker* meaning "unknown or missing."

Suppose Alice's age is unknown (`NULL`) and Bob's age is also unknown (`NULL`). Are Alice and Bob the same age? *You do not know.* Alice might be 29 and Bob 44, or both might be 31. Asking `NULL = NULL` cannot honestly return `TRUE`, and asking `NULL <> NULL` cannot honestly return `TRUE` either.

Both comparisons return a third truth value: `UNKNOWN` (which PostgreSQL displays in `BOOLEAN` output columns as blank/`NULL`).

Java 6 programmers need to unlearn one habit here. In Java, `null == null` is `true`, and unboxing a `null Boolean` inside `if (flag)` throws `NullPointerException`. SQL does neither. A comparison with `NULL` never throws and never returns `FALSE`. It returns `UNKNOWN`, and the surrounding clause decides what to do with `UNKNOWN`.

Arithmetic and scalar functions propagate unknownness right away: `NULL + 500` is `NULL`, `'USD-' || NULL` is `NULL`, and `ABS(NULL)` is `NULL`. Adding five dollars to an unknown balance produces an unknown balance.

What happens when `UNKNOWN` enters `NOT`, `AND`, and `OR`? SQL uses the three-valued logic formulated by Stephen Cole Kleene in 1938. You do not need to memorize Kleene's tables; read `UNKNOWN` as "maybe `TRUE`, maybe `FALSE`":

- `NOT UNKNOWN` is `UNKNOWN`: flipping an unknown coin is still unknown.
- `FALSE AND UNKNOWN` is `FALSE`: one `FALSE` already dooms an `AND`, no matter what the unknown half hides. By contrast, `TRUE AND UNKNOWN` is `UNKNOWN`, because the outcome still depends on the missing fact.
- Symmetrically, `TRUE OR UNKNOWN` is `TRUE` (one `TRUE` satisfies `OR`), whereas `FALSE OR UNKNOWN` is `UNKNOWN`.

Negation		p AND q				p OR q			
p	NOT p	AND	T	U	F	OR	T	U	F
T	F	T	T	U	F	T	T	T	T
U	U	U	U	U	F	U	T	U	U
F	T	F	F	F	F	F	T	U	F

Kleene's logic breaks the classical law of the excluded middle ($p \vee \neg p$): when `fee_cents` is `NULL`, the tautology-looking filter

```
SELECT * FROM postings WHERE fee_cents = 100 OR fee_cents <> 100;
```

evaluates to `UNKNOWN OR UNKNOWN`, which is `UNKNOWN` — and `WHERE` drops the row. To test for `NULL` explicitly, SQL provides two null-safe predicates that *always* return two-valued `TRUE` or `FALSE` (never `UNKNOWN`):

- `x IS NULL` and `x IS NOT NULL`;
- `x IS DISTINCT FROM y` and `x IS NOT DISTINCT FROM y`, which treat `NULL` as equal to `NULL` and distinct from every non-`NULL` value (`NULL IS NOT DISTINCT FROM NULL` evaluates to `TRUE`).

Common trap

`WHERE` discards `UNKNOWN`, whereas `CHECK` constraints accept `UNKNOWN`. This asymmetry catches experienced engineers off guard. In `WHERE` and `HAVING`, a row survives only if the predicate evaluates to `TRUE`. In a table `CHECK` constraint, however, the SQL standard specifies

that a row is rejected only if the constraint evaluates to `FALSE`. Suppose you write:

```
CREATE TABLE transfers (
  id           BIGINT PRIMARY KEY,
  amount_cents BIGINT NOT NULL,
  fee_cents    BIGINT,                  -- oops: forgot NOT NULL!
  CHECK (fee_cents >= 0 AND fee_cents <= amount_cents)
);
```

A negative fee is rejected as you would expect:

```
INSERT INTO transfers VALUES (1, 5000, -10);
ERROR:  new row for relation "transfers" violates check constraint "transfers_check"
```

But if a client inserts `(1, 5000, NULL)`, the expression `NULL >= 0 AND NULL <= 5000` evaluates to `UNKNOWN`. Because `UNKNOWN` is not `FALSE`, **PostgreSQL accepts the row**. Always mark constrained columns `NOT NULL` unless missingness is genuinely intended.

1.5.1 Why `NOT IN` returns nothing when the subquery contains `NULL`

The `AND` table explains a bug that reaches production regularly. Suppose you maintain a watchlist table `sanctioned_watchlist` with columns `(id, acct_id)`, where `acct_id` is nullable because some entries come from external name-only feeds not yet matched to an internal account ID:

$$\texttt{sanctioned_watchlist.acct_id} = \{102, \texttt{NULL}\}.$$

You want to list every clean account in `accounts` (101, 103, 104) that is *not* on the watchlist. A developer writes:

```
-- BUG: returns zero rows whenever the subquery yields even one NULL
SELECT a.id, a.owner
  FROM accounts a
 WHERE a.id NOT IN (SELECT w.acct_id FROM sanctioned_watchlist w);
```

What should this return? Accounts 101, 103, and 104. What does PostgreSQL actually return? **Zero rows.**

Let us expand SQL's definition of `IN` and `NOT IN` on Ada (`a.id = 101`) to see why:

$$\begin{aligned} \texttt{101 IN (102, NULL)} &\equiv (\texttt{101 = 102}) \texttt{ OR } (\texttt{101 = NULL}) \\ &\equiv \texttt{FALSE OR UNKNOWN} = \texttt{UNKNOWN}. \end{aligned}$$

Now negate that expression with `NOT` (or equivalently apply De Morgan's law):

$$\begin{aligned} \texttt{101 NOT IN (102, NULL)} &\equiv \texttt{NOT}\big(\texttt{101 IN (102, NULL)}\big) = \texttt{NOT UNKNOWN} = \texttt{UNKNOWN} \\ &\equiv (\texttt{101 <> 102}) \texttt{ AND } (\texttt{101 <> NULL}) \\ &\equiv \texttt{TRUE AND UNKNOWN} = \texttt{UNKNOWN}. \end{aligned}$$

Because Stage 3 (`WHERE`) keeps *only* `TRUE`, `UNKNOWN` drops Ada (101), Cara (103), and Dan (104). Meanwhile, for Ben (102), `(102 <> 102) AND (102 <> NULL)` evaluates to `FALSE AND UNKNOWN` =

FALSE, dropping Ben too. One NULL anywhere inside a ten-million-row NOT IN subquery empties the entire output without warning.

Now rewrite the exact same question using NOT EXISTS:

```
-- SAFE & IDIOMATIC ANTI-JOIN: immune to NULLs and plans as a true Hash/Index Anti-Join
SELECT a.id, a.owner
  FROM accounts a
 WHERE NOT EXISTS (
   SELECT 1
     FROM sanctioned_watchlist w
    WHERE w.acct_id = a.id
 );
```

Trace Ada (a.id = 101) through NOT EXISTS:

1. Inside the subquery, WHERE w.acct_id = 101 tests 102 = 101 (FALSE) and NULL = 101 (UNKNOWN). The inner WHERE drops both.
2. The inner subquery therefore returns **zero rows**.
3. EXISTS (empty set) is a two-valued test that always returns boolean FALSE, never UNKNOWN.
4. In the outer WHERE, NOT FALSE becomes TRUE, so Ada (101), Cara (103), and Dan (104) survive.

Better still, PostgreSQL's planner recognizes NOT EXISTS directly as a relational anti-join (Hash Anti Join or Nested Loop Anti Join). By contrast, NOT IN forces slower null-tracking execution paths because of its NULL rules. Make it a reflex: **never write** NOT IN (SELECT ...); **always write** NOT EXISTS **(or** LEFT JOIN ... WHERE right.pk IS NULL**)**.

Why it works

Where SQL quietly switches from comparison equality to "grouping equality." If NULL = NULL is UNKNOWN, what happens when two rows with tier = NULL hit GROUP BY tier or SELECT DISTINCT tier? Under WHERE rules, NULLs could never group together. Instead, SQL uses *grouping distinctness* (IS NOT DISTINCT FROM) for GROUP BY, DISTINCT, UNION, and window PARTITION BY. All NULLs land in a **single bucket**. For UNIQUE indexes, standard SQL goes the opposite way: two NULLs count as distinct so UNIQUE(phone) permits multiple users without phone numbers. (PostgreSQL 15+ lets you override that with UNIQUE NULLS NOT DISTINCT.) Finally, standard SQL aggregates (SUM, AVG, MIN, MAX, COUNT(col)) skip NULL inputs. Watch one edge case: COUNT of zero non-null values returns 0, whereas SUM of zero non-null values returns NULL. Wrap financial sums in COALESCE(SUM(cents), 0).

Pause

Before moving to window functions, check that three rules feel solid:

1. WHERE runs at Stage 3 (on raw joined rows), HAVING runs at Stage 5 (on grouped buckets), and SELECT aliases appear at Stage 7.
2. WHERE keeps only TRUE and drops UNKNOWN; CHECK constraints reject only FALSE and accept UNKNOWN.
3. NOT EXISTS turns an inner UNKNOWN into zero inner rows and then clean outer TRUE,

whereas `NOT IN` lets one `NULL` poison the entire result.

1.6 Window functions: aggregating without collapsing rows

Suppose an auditor asks for Cara's statement (`acct_id = 103`). Each line must show the individual posting amount, the running balance after that posting, and that posting's share of Cara's volume.

Ordinary `GROUP BY` (Stage 4) hits a wall here. `GROUP BY acct_id` crushes Cara's three postings into one summary row, destroying `pid`, `cents`, and `posted_on`. Before SQL:2003 added window functions, engineers had to join `postings` back to a grouped subquery and run a quadratic $O(N^2)$ self-join (`p2.pid <= p1.pid`) for the running sum.

In Java, you would compute a running balance in one sorted pass. You keep a `long balance`, reset it whenever `accountId` changes, and add each posting before printing the row. A window function is that exact loop written declaratively: `PARTITION BY` is the reset rule, `ORDER BY` is the sort, and the frame chooses which rows feed the running total.

Window functions (evaluated at Stage 6 of the logical pipeline) solve this in a single $O(N \log N)$ sort pass (or $O(N)$ streaming pass over an index). Every window call attaches an `OVER (...)` clause with three optional parts:

```
function_name(args) OVER (
  PARTITION BY acct_id                                  -- 1. Independent sub-tables
  ORDER BY posted_on ASC, pid ASC                       -- 2. Sort order inside partition
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW      -- 3. Sliding frame bounds
)
```

1. `PARTITION BY`: Splits Stage 5's surviving rows into independent horizontal groups (just like `GROUP BY`), except *zero rows are collapsed*. Window calculations never cross a partition boundary: Ada's running balance resets cleanly when the scan advances to Ben's partition.
2. `ORDER BY`: Defines the sequence of rows within each partition for ranking functions (`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`), offset navigation (`LAG(col, 1)`, `LEAD(col, 1)`), and cumulative frames.
3. **Frame specification (`ROWS`, `RANGE`, or `GROUPS`)**: For each current row r, defines the exact sliding subset of rows inside r's partition fed into the aggregate.

1.6.1 The silent `RANGE` versus `ROWS` trap on duplicate timestamps

The default frame produces a wrong running balance on Cara's postings (`pid = 4, 5, 6`) in our miniature dataset. Look back at Cara's three rows in P:

- `pid = 4` on `2026-10-01`: +15000 cents;
- `pid = 5` on `2026-10-03`: −3000 cents;
- `pid = 6` on `2026-10-03`: +2000 cents.

Notice that postings `5` and `6` occurred on the *exact same date* (`2026-10-03`). Now suppose an engineer writes a running balance query ordering only by `posted_on`, omitting an explicit frame clause:

```
SELECT pid, posted_on, cents,
       -- Subtle trap: omits ROWS/RANGE, so SQL defaults to RANGE!
       SUM(cents) OVER (
         PARTITION BY acct_id
         ORDER BY posted_on
       ) AS range_running_bal,
       -- Explicit physical row-by-row running balance with deterministic tie-breaker:
       SUM(cents) OVER (
         PARTITION BY acct_id
         ORDER BY posted_on, pid
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS exact_running_bal
  FROM postings
 WHERE acct_id = 103
 ORDER BY posted_on, pid;
```

What is SQL's default window frame when you omit the `ROWS/RANGE` clause?

- If `OVER (...)` has **no** `ORDER BY`, the default frame spans the **entire partition** (`RANGE` from `UNBOUNDED PRECEDING` to `UNBOUNDED FOLLOWING`), so `SUM(cents) OVER (PARTITION BY acct_id)` gives Cara's grand total (14000) on all three rows.
- If `OVER (...)` **includes an** `ORDER BY`, the default frame silently switches to `RANGE` from `UNBOUNDED PRECEDING` to `CURRENT ROW`.

And what does `RANGE ... CURRENT ROW` mean? In `RANGE` mode, `CURRENT ROW` does *not* stop at the physical row currently being visited; it includes **all peer rows tied on the** `ORDER BY` **key**. Because `pid = 5` and `pid = 6` tie on `posted_on = '2026-10-03'`, `range_running_bal` sums $\{+15000, -3000, +2000\} = 14000$ for *both* row 5 and row 6 — Cara's intermediate balance of 12000 after posting 5 completely disappears.

pid	posted_on	cents	Default RANGE	Explicit ROWS
4	2026-10-01	+15000	15000	15000
5	2026-10-03	-3000	14000 (skips 12000!)	12000 (4..5 only)
6	2026-10-03	+2000	14000	14000 (4..6)

Tied `posted_on` peers merged by `RANGE`, separated by `ROWS`

Figure 1.4: Why explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` plus a unique tie-breaker (`pid`) matters for financial running balances: default `RANGE` merges tied timestamps into a single peer step, hiding the intermediate balance of `12000` after posting `5`.

Common trap

The exact same default-frame rule explains why `LAST_VALUE(cents) OVER (PARTITION BY acct_id ORDER BY pid)` surprises most people the first time they run it: instead of returning the final posting of the account, it returns *each row's own* `cents`. Why? Because the default

frame stops at `CURRENT ROW`, so the "last value" inside `UNBOUNDED PRECEDING .. CURRENT ROW` is always the current row itself. Whenever you use `LAST_VALUE` (or `NTH_VALUE`), widen the frame explicitly:

```
LAST_VALUE(cents) OVER (
  PARTITION BY acct_id ORDER BY pid
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
```

1.7 CTEs and fixpoint iteration with `WITH RECURSIVE`

Pure relational algebra $(\sigma, \pi, \times, \cup, -)$ has a celebrated theoretical boundary proved by Alfred Aho and Jeffrey Ullman in 1979: **relational algebra cannot express transitive closure**. If a table `account_hierarchy(id, parent_id, name)` stores an arbitrarily deep tree of corporate sub-accounts (or a graph of wire-transfer hops between accounts), no fixed number of self-joins $H \bowtie H \bowtie \cdots \bowtie H$ can find all descendants when the depth is not known in advance.

SQL bridges that expressiveness gap with **Common Table Expressions (CTEs)**, written with `WITH` for modular non-recursive subqueries and `WITH RECURSIVE` for iterative fixpoint computation.

1.7.1 Non-recursive `WITH`, planner inlining, and data-modifying CTEs

In ordinary queries, `WITH step1 AS (...), step2 AS (...) SELECT ...` lets you write complex pipelines top-to-bottom in the exact order you think about them, rather than burying subqueries four levels deep inside `FROM`.

Under the hood

How PostgreSQL plans `WITH` (`MATERIALIZED` vs. inlined). Before PostgreSQL 12, every CTE acted as a strict *optimization fence*: PostgreSQL materialized the full CTE output into an in-memory/temp-file `Tuplestore` before running the outer query, preventing outer `WHERE` filters from pushing down inside the CTE. Starting in PostgreSQL 12, a non-recursive, side-effect-free CTE referenced at most once is **inlined automatically** into the outer query tree (just like a view) so predicates push straight down to indexes. When you *want* single-evaluation materialization (for instance, calling an expensive function once and joining its result twice), write `WITH summary AS MATERIALIZED (...)`; to force pushdown even when referenced multiple times, write `WITH summary AS NOT MATERIALIZED (...)`.
PostgreSQL also supports **data-modifying CTEs** (`INSERT/UPDATE/DELETE ... RETURNING` inside `WITH`), all executing atomically under a single statement snapshot — ideal for moving archived rows from an active queue into an audit log in one round-trip:

```
WITH archived AS (
  DELETE FROM outbox_events
   WHERE processed_at < NOW() - INTERVAL '30 days'
  RETURNING id, payload, processed_at
)
```

```
INSERT INTO outbox_events_archive (id, payload, processed_at)
SELECT id, payload, processed_at FROM archived;
```

1.7.2 How `WITH RECURSIVE` actually executes: two tables in a loop

The keyword `RECURSIVE` can mislead Java or C programmers: PostgreSQL does *not* push stack frames onto a call stack. Instead, its `RecursiveUnion` operator runs a **bottom-up breadth-first loop** over two temporary buffers: a **Result Table** R and a **Working Table** W. If you have written breadth-first search in Java 6 with an `ArrayDeque<Long>` frontier and a `HashSet<Long>` visited set, you already know this loop. Here W is the frontier, R holds everything found so far, and `UNION` (unlike `UNION ALL`) acts as the visited set.

Every `WITH RECURSIVE` definition has the shape B `UNION [ALL]` $F(W)$, where B is the non-recursive *base query* (anchor) and $F(W)$ is the *recursive step* referencing the CTE's own name:

1. **Step 0 (Seed):** Evaluate the base query B once. Copy its rows into both the Result Table R and the Working Table W.
2. **Loop while W is non-empty:**
   (a) Evaluate the recursive query $F(W)$, substituting the *current Working Table* W (not all of R) wherever the CTE's self-reference appears. Put the newly produced rows into an Intermediate Table I.
   (b) If the connector is `UNION` (rather than `UNION ALL`), discard from I any row already present in R (and deduplicate I itself).
   (c) Append all surviving rows of I to R, replace $W \leftarrow I$, and repeat.
3. **Terminate:** As soon as W becomes empty ($I = \emptyset$), return R. Mathematically, $R = \bigcup_{k \geq 0} F^k(B)$ is the *least fixpoint* of the monotone relational operator $T(X) = B \cup F(X)$.

Let us trace a four-node corporate ledger hierarchy starting from root account `1` (`'Treasury'`). It has children `10` (`'US Ops'`) and `20` (`'EU Ops'`), plus grandchild `101` (`'US Payroll'` under `10`):

```
WITH RECURSIVE acct_tree AS (
  -- Base term B: start at the root Treasury account
  SELECT id, parent_id, name, 0 AS depth, ARRAY[id] AS path
    FROM ledger_accounts
   WHERE id = 1
  UNION ALL
  -- Iterative step F(W): join current Working Table W to child rows
  SELECT c.id, c.parent_id, c.name, w.depth + 1, w.path || c.id
    FROM acct_tree w
    JOIN ledger_accounts c ON c.parent_id = w.id
   WHERE NOT (c.id = ANY(w.path))    -- cycle guard!
)
SELECT id, depth, name, path FROM acct_tree ORDER BY path;
```

Watch the contents of W and R across each iteration:

- **Iteration 0 (Base B):** $W_0 = \{(1, \texttt{NULL}, \texttt{'Treasury'}, 0, [1])\}$. $R = W_0$.

- **Iteration 1 ($F(W_0)$):** Joining W_0 (`id = 1`) to `ledger_accounts.parent_id` finds children `10` and `20`. Now $W_1 = \{(10, \ldots, 1, [1, 10]),\ (20, \ldots, 1, [1, 20])\}$, and R holds $\{1, 10, 20\}$.
- **Iteration 2 ($F(W_1)$):** Joining W_1 (`10` and `20`) finds `101` under `10` and nothing under `20`. Now $W_2 = \{(101, \ldots, 2, [1, 10, 101])\}$, and R holds $\{1, 10, 20, 101\}$.
- **Iteration 3 ($F(W_2)$):** Joining W_2 (`101`) finds zero children. $W_3 = \emptyset$, the loop halts, and PostgreSQL outputs all four rows of R.

Line 11, `WHERE NOT (c.id = ANY(w.path))`, is the cycle guard. What if bad upstream data set `parent_id = 101` on root account `1`, forming a cycle $1 \to 10 \to 101 \to 1$? Because `depth` grows on every turn, each row looks fresh, so W_k would never empty. PostgreSQL would keep appending rows to temporary files until `statement_timeout` canceled the query or the disk filled. Tracking `ARRAY[id] AS path`—or using PostgreSQL 14+'s SQL-standard `CYCLE id SET is_cycle USING path` clause—guarantees termination on any directed graph.

Connection

How Rust's `SQLx` turns SQL's three-valued `NULL`s into compile-time types. In *Programming Languages From the Ground Up*, we connect a Rust financial backend to PostgreSQL with `sqlx::query`. Rather than turning SQL `NULL` into a null reference at runtime, `SQLx` asks PostgreSQL's own analyzer at *Rust compile time* which output columns can ever be `NULL`. Non-null columns become plain `i64` or `String`. Nullable columns—such as the right side of a `LEFT JOIN` or an unguarded `SUM(cents)`—become `Option<i64>`, forcing your Rust code to handle missing values before the binary compiles.

Try it in PostgreSQL

Paste this self-contained session into `psql` to verify all three subtleties from this chapter — `LEFT JOIN` filter placement, the `NOT IN NULL` wipeout versus `NOT EXISTS`, and `RANGE` versus `ROWS` window running balances — on our exact miniature ledger:

```
CREATE TEMP TABLE accounts (id BIGINT PRIMARY KEY, owner TEXT NOT NULL, tier TEXT NOT NULL);
CREATE TEMP TABLE postings (pid BIGINT PRIMARY KEY, acct_id BIGINT REFERENCES accounts(id),
                            cents BIGINT NOT NULL, posted_on DATE NOT NULL);
CREATE TEMP TABLE watchlist (id INT PRIMARY KEY, acct_id BIGINT);

INSERT INTO accounts VALUES (101,'Ada','pro'), (102,'Ben','free'), (103,'Cara','pro'), (104,'Dan','pro');
INSERT INTO postings VALUES
  (1,101, 10000,'2026-10-01'), (2,101,-2500,'2026-10-02'), (3,102,4000,'2026-10-02'),
  (4,103, 15000,'2026-10-01'), (5,103,-3000,'2026-10-03'), (6,103,2000,'2026-10-03');
INSERT INTO watchlist VALUES (1, 102), (2, NULL);

-- 1. Watch NOT IN return 0 rows while NOT EXISTS returns 101, 103, 104:
SELECT id FROM accounts WHERE id NOT IN (SELECT acct_id FROM watchlist);
SELECT id FROM accounts a WHERE NOT EXISTS (SELECT 1 FROM watchlist w WHERE w.acct_id = a.id);
```

```
15
16 -- 2. Compare default RANGE (14000 on pid 5!) vs explicit ROWS (12000 then 14000):
17 SELECT pid, posted_on, cents,
18        SUM(cents) OVER (PARTITION BY acct_id ORDER BY posted_on) AS range_bal,
19        SUM(cents) OVER (PARTITION BY acct_id ORDER BY posted_on, pid
20                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
     rows_bal
21   FROM postings WHERE acct_id = 103 ORDER BY posted_on, pid;
```

1.8 Exercises

Exercise 1.1 (Bag multiplicities by hand)**.** Suppose single-column bag R contains `'USD'`$\times 3$, `'EUR'`$\times 2$, and `'GBP'` $\times 1$, while bag S contains `'USD'` $\times 1$, `'EUR'` $\times 4$, and `'JPY'` $\times 2$. Compute the exact row count and per-currency multiplicity of: (a) `R UNION ALL S`, (b) `R UNION S`, (c) `R INTERSECT ALL S`, (d) `R EXCEPT ALL S`, and (e) `R EXCEPT S`.

Exercise 1.2 (Relational algebra pushdown cost)**.** Relation A has 200,000 account tuples (of which 5,000 have `country = 'NG'`) and relation P has 40,000,000 posting tuples (of which 80,000 occurred today, 2,000 belonging to `'NG'` accounts). Count how many tuple pairs enter the join comparison in:

$$\text{(a)}\quad \sigma_{A.\texttt{country}=\texttt{'NG'} \,\wedge\, P.\texttt{day}=\texttt{today}}(A \times P),$$
$$\text{(b)}\quad \big(\sigma_{\texttt{country}=\texttt{'NG'}}(A)\big) \bowtie_{A.\texttt{id}=P.\texttt{acct_id}} \big(\sigma_{\texttt{day}=\texttt{today}}(P)\big),$$

assuming the equi-join builds an in-memory hash table on the filtered A rows and probes once per filtered P row.

Exercise 1.3 (Diagnosing the ten-stage pipeline)**.** For each SQL query below over our `accounts` and `postings` tables, state whether PostgreSQL executes it or rejects it with an error; if rejected, identify the exact two stage numbers from Figure 1.3 that are out of order, and rewrite the query cleanly:

```
1 -- (1)
2 SELECT acct_id, COUNT(*) AS cnt FROM postings GROUP BY acct_id HAVING cnt >= 2;
3 -- (2)
4 SELECT pid, cents, LAG(cents) OVER (PARTITION BY acct_id ORDER BY pid) AS prev
5   FROM postings WHERE prev < 0;
6 -- (3)
7 SELECT DISTINCT acct_id FROM postings ORDER BY posted_on DESC;
```

Exercise 1.4 (Outer join predicate placement)**.** Using our four-row `accounts` table A and six-row `postings` table P, write down the exact set of rows returned by:

```
1 SELECT a.id, COUNT(*), COUNT(p.pid), COALESCE(SUM(p.cents), 0)
2   FROM accounts a
3   LEFT JOIN postings p ON p.acct_id = a.id AND p.cents < 0
4  GROUP BY a.id
5  ORDER BY a.id;
```

Explain why `COUNT(*)` and `COUNT(p.pid)` disagree on two of the four accounts.

Exercise 1.5 (Three-valued truth table proof)**.** In classical two-valued boolean algebra, De Morgan's law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$ and the distributive law $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ hold identically, as does $p \vee \neg p \equiv$ `TRUE`. Check all three identities across Kleene's three-valued truth tables $\{$`T`, `U`, `F`$\}$: prove that De Morgan's law and distributivity *still hold identically* for every combination of (`T`, `U`, `F`), whereas $p \vee \neg p$ fails specifically when $p =$ `U`.

Exercise 1.6 (Designing a leak-proof `CHECK` constraint)**.** A payments table `refunds` has columns `(id BIGINT PRIMARY KEY, charge_cents BIGINT NOT NULL, refund_cents BIGINT, reason_code TEXT)`. Enforce two rules: (1) when `refund_cents` is present, it satisfies $0 <$ `refund_cents` $\leq$ `charge_cents` and `reason_code` is non-`NULL`; (2) when `refund_cents` is `NULL`, `reason_code` must also be `NULL`. Write a single `CHECK (...)` expression that cannot be bypassed by `UNKNOWN`, and verify all four null/non-null combinations.

Exercise 1.7 (Sliding 7-day rolling volume vs. 7-row rolling volume)**.** An account has postings on October 1 (+100), October 2 (+200), October 2 (+50), and October 10 (+300), with zero postings on October 3 through 9. Contrast what PostgreSQL computes on each of the four rows for:

```
-- Window 1: physical row count frame
SUM(cents) OVER (ORDER BY posted_on, pid ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
-- Window 2: calendar interval range frame (PostgreSQL 11+!)
SUM(cents) OVER (ORDER BY posted_on RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT
    ROW)
```

When do you want `ROWS`, and when is `RANGE BETWEEN INTERVAL ...` genuinely the right tool?

Exercise 1.8 (Tracing `WITH RECURSIVE` on a cyclic transfer graph)**.** Suppose `wires(from_acct, to_acct)` contains edges $(1 \to 2)$, $(2 \to 3)$, $(3 \to 1)$, and $(2 \to 4)$. Trace the contents of the Working Table W_k and Result Table R_k at steps $k = 0, 1, 2, 3$ starting from `from_acct = 1` for: (a) a query using `UNION` projecting only `to_acct`, and (b) a query using `UNION ALL` projecting `(to_acct, depth)` *without* a cycle guard. Why does (a) terminate while (b) runs forever, and how does adding `CYCLE to_acct SET is_cycle USING path` stop (b) cleanly?

In this chapter we stayed deliberately above Codd's data-independence line: relations were sets and bags of abstract tuples, and PostgreSQL promised to evaluate our ten-stage pipeline faithfully. Now it is time to cross below that line, open `$PGDATA/base` on an actual Linux filesystem, and look inside the 8,192-byte slotted pages and `shared_buffers` frames where every tuple physically lives.

Chapter 2

Inside an 8 KB Heap Page and the Buffer Pool

In Chapter 1, a table was a clean mathematical object: a relation (or a SQL bag of rows) governed by set algebra and three-valued logic. When you write `INSERT INTO accounts VALUES (1001, 'Ada', 50000, true)`, SQL lets you picture a neat rectangular spreadsheet gaining one horizontal line.

The Linux kernel knows nothing about relations, columns, or `BIGINT`s. An NVMe drive only reads and writes flat byte arrays at numeric block addresses. C pointers in RAM only know byte offsets and CPU alignment boundaries. Bridging that gap is the job of PostgreSQL's storage and buffer managers. Every mechanism later in this book rests on one physical container: an **8,192-byte (8 KB) slotted heap page**. Indexes point to slots inside those pages, crash recovery stamps log positions at byte 0, and `UPDATE` appends fresh row versions into their free space.

Focus

This chapter opens a table file on disk and follows an 8 KB page into RAM. Along the way, we answer five concrete engineering questions:

- Which actual files on Linux hold a PostgreSQL table, and how can `TRUNCATE` wipe a giant table in milliseconds while still letting `ROLLBACK` restore every row?
- How does an 8 KB page pack variable-length rows and reclaim deleted gaps without breaking external index pointers?
- Why does the order of columns in `CREATE TABLE` silently add or save hundreds of megabytes of CPU alignment padding?
- Where does PostgreSQL put a 400 KB JSON document when no row is allowed to cross an 8 KB page boundary?
- How does PostgreSQL's RAM cache keep hot lookup pages in memory when an analyst runs a sequential scan over a 500 GB historical table?

2.1 Where a Table Actually Lives on Disk

Let us create a small ledger table in `psql` and ask PostgreSQL a direct physical question: *which file on the Linux filesystem holds these rows?*

```
CREATE TABLE accounts (
    account_id    BIGINT  PRIMARY KEY,
    owner_name    TEXT    NOT NULL,
    balance_cents BIGINT  NOT NULL,
    is_active     BOOLEAN NOT NULL
);

SELECT oid, relname, relfilenode
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relname  | relfilenode
-- -------+----------+-------------
--  24576 | accounts |       24576

SELECT pg_relation_filepath('accounts');
--  pg_relation_filepath
-- ----------------------
--  base/16384/24576
```

Relative to your PostgreSQL data directory (`PGDATA`, such as `/var/lib/postgresql/17/main`), the table `accounts` is an ordinary regular file at `PGDATA/base/16384/24576`. Every piece of that path has a clear job:

- `base/` holds all databases in the default tablespace (`pg_default`). Cluster-wide catalogs such as `pg_database` and `pg_authid` live next door in `global/`.
- `16384` is the numeric object ID (`oid`) of the current database in `pg_database`. Each database in the cluster gets its own subdirectory under `base/`.
- `24576` is the table's current physical file number (`pg_class.relfilenode`), not its permanent table ID (`pg_class.oid`). Right after `CREATE TABLE`, those two numbers happen to match. Watch what happens the moment we truncate the table.

2.1.1 Why `oid` and `relfilenode` Split: Transactional File Swaps

Suppose a transaction runs `TRUNCATE accounts;` and then, three statements later, hits an error and runs `ROLLBACK;`. Unlike many older database engines, PostgreSQL makes `TRUNCATE`, `CLUSTER`, and `VACUUM FULL` **fully transactional**.

Think about what would break if `TRUNCATE` simply asked the operating system to chop file `24576` down to zero bytes in place (via Linux `ftruncate(fd, 0)`). The instant the kernel zeroes the file length, the old 8 KB blocks are gone. If the transaction rolls back ten milliseconds later, nothing is left to restore.

Intuition

PostgreSQL solves transactional table rewrites by separating **logical identity** (`pg_class.oid`) from **physical storage file** (`pg_class.relfilenode`):

- Views, foreign keys, triggers, and grants all point to `oid = 24576`. That permanent ID never changes for the life of the table.
- When you run `TRUNCATE accounts` (or rewrite the table with `VACUUM FULL` or `CLUSTER`), PostgreSQL allocates a fresh file number—say `24589`—and creates a new empty file `base/16384/24589`. It then updates the catalog row for `oid = 24576` so its `relfilenode` column points to `24589`.

If your transaction runs `ROLLBACK`, that catalog row update disappears. PostgreSQL deletes the empty file `24589`, while original file `24576` still sits intact on disk. If your transaction runs `COMMIT`, the catalog swap becomes visible atomically, new queries open `24589`, and post-commit cleanup deletes old file `24576`.

Let us verify this split directly in SQL:

```
BEGIN;
TRUNCATE accounts;

SELECT oid, relfilenode, pg_relation_filepath('accounts') AS path
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relfilenode |       path
-- -------+-------------+------------------
--  24576 |       24589 | base/16384/24589

ROLLBACK;

SELECT oid, relfilenode, pg_relation_filepath('accounts') AS path
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relfilenode |       path
-- -------+-------------+------------------
--  24576 |       24576 | base/16384/24576
```

To locate any table file on disk, PostgreSQL combines three integers: the tablespace ID, the database ID, and the current file number. In PostgreSQL 16+ C code (`src/include/storage/relfilelocator.h`), that triple is called `RelFileLocator`: `(spcOid, dbOid, relNumber)`. Whenever you inspect files on disk after maintenance, query `pg_relation_filepath(oid)` rather than assuming the filename still equals `pg_class.oid`.

2.1.2 1 GB Segment Splitting

A busy production table easily grows past 100 GB, yet `ls -lh` inside `PGDATA/base/16384/` never shows a file larger than **1 GiB** (1,073,741,824 bytes).

PostgreSQL slices every table or index file into fixed segments of 131,072 blocks of 8,192 bytes (`RELSEG_SIZE` = 131,072, totaling 1 GiB). Originally added to avoid 2 GB file-size limits on 32-bit Unix systems, 1 GiB segments remain helpful on 64-bit Linux for parallel backups and file copies.

For a 2.6 GiB table with `relfilenode = 24589`, the directory contains three segment files:

- `24589` (blocks 0 … 131,071, bytes 0 … 1 GiB − 1)
- `24589.1` (blocks 131,072 … 262,143, the second GiB)
- `24589.2` (blocks 262,144 … 340,786, the remaining 0.6 GiB)

Locating any 32-bit page number B (`BlockNumber`) on disk takes one integer division and one remainder:

$$\text{segment_index} = \left\lfloor \frac{B}{131{,}072} \right\rfloor, \qquad \text{byte_offset_in_segment} = (B \bmod 131{,}072) \times 8{,}192.$$

For example, block $B = 300{,}000$ sits in segment $\lfloor 300{,}000/131{,}072 \rfloor = 2$ (`24589.2`) at local block 37,856, starting at byte $37{,}856 \times 8{,}192 = 310{,}116{,}352$. Because a block number is a 32-bit unsigned integer (with `0xFFFFFFFF` reserved as invalid), one standard PostgreSQL table holds up to $(2^{32} - 1) \times 8$ KiB ≈ **32** TiB before needing partitioning.

2.1.3 Companion Forks: Free Space Map (_fsm) and Visibility Map (_vm)

Insert a few rows, run `VACUUM accounts;`, and check `PGDATA/base/16384/` again. Two helper files appear beside `24589`: `24589_fsm` and `24589_vm`. Each table can own up to four companion files on disk, which PostgreSQL calls **forks** (`ForkNumber` in `src/include/common/relpath.h`):

1. **Main Fork (`MAIN_FORKNUM = 0`, file `24589`)**: Holds the actual 8 KB table pages (or index pages, for an index).
2. **Free Space Map (FSM, `FSM_FORKNUM = 1`, file `24589_fsm`)**: Suppose `accounts` spans one million pages (≈ 8 GB) with deleted gaps scattered throughout, and a session inserts a 110-byte row. Checking pages one by one for 110 free bytes would turn every `INSERT` into a slow linear scan.

   Instead, the Free Space Map summarizes each table page's free space in **one byte** (`0..255`), measured in 32-byte units ($\lfloor \text{free_bytes}/32 \rfloor$). A leaf byte of `4` promises at least $4 \times 32 = 128$ free bytes on that page. Inside each 8 KB `_fsm` block, roughly 4,000 leaf bytes sit under a **binary max-heap tree** where every parent stores max(left, right):

   - If the root byte at the top of an `_fsm` page reads `100` (3,200 bytes), at least one table page below has 3,200+ free bytes.
   - Finding room for our 110-byte row ($\lceil 110/32 \rceil = 4$ units) walks down from the root in $O(\log N)$ steps, choosing any child ≥ 4. Even on a multi-terabyte table, stacked `_fsm` levels find a target page in at most **three 8 KB buffer lookups**.
   - The Free Space Map is only a hint, not ground truth. If another session fills that page first, the inserter checks the real page header, lowers the `_fsm` byte, and asks the tree for the next candidate.
3. **Visibility Map (VM, `VM_FORKNUM = 2`, file `24589_vm`)**: Stores just **2 bits per table page**:
   - Bit 0 (`VISIBILITYMAP_ALL_VISIBLE = 0x01`): Every row on that page is committed and visible to *all* current and future transactions, with zero dead versions.
   - Bit 1 (`VISIBILITYMAP_ALL_FROZEN = 0x02`): Every row on that page is also frozen against 32-bit transaction ID wraparound (Chapter 5).

Four table pages pack into each byte of the Visibility Map. Subtracting the 24-byte header leaves 8,168 payload bytes, so **one 8 KB `_vm` page tracks 32,672 table pages**—about **255 MiB** of data. Those two bits drive two major speedups. First, **Index-Only Scans** (Chapter 3) check `_vm` in RAM and skip reading the table page whenever `ALL_VISIBLE` is `1`. Second, `VACUUM` (Chapter 5) skips pages already marked clean or frozen. Any write to page B clears its bits.

4. **Initialization Fork (`INIT_FORKNUM = 3`, file `24589_init`)**: Created only for `UNLOGGED` tables and their indexes. It stores an empty template image. After an unclean crash, startup wipes the unlogged main file and copies `_init` over it to reset the table cleanly (Chapter 4).

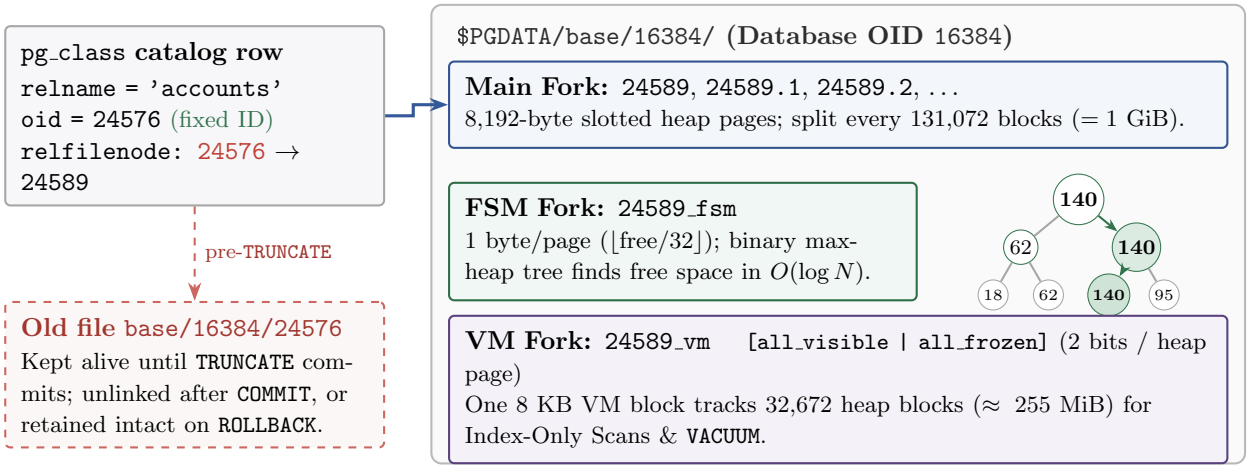


Figure 2.1: Physical file layout under `PGDATA/base/16384/`. Transactional rewrites (`TRUNCATE`, `CLUSTER`, `VACUUM FULL`) preserve `pg_class.oid` while swapping `relfilenode` to a new file family (`24589`, `24589_fsm`, `24589_vm`).

2.2 Anatomy of an 8,192-Byte Slotted Heap Page

Open `base/16384/24576` in a hex editor and slice it every 8,192 bytes (`BLCKSZ` = 8,192). Each 8,192-byte chunk (offsets 0 . . . 8,191) is a self-contained **slotted page**.

Walking question

Try designing the page layout yourself first. You have an 8,192-byte array in RAM, and SQL rows are **variable-length** (`'Ada'` is 3 bytes, `'Grace Hopper'` is 12, and another row carries a 400-byte note). Worse, rows get deleted out of order, leaving holes in the middle of the page while external B-tree indexes hold pointers to live rows.
Suppose a B-tree index stored a row's raw byte offset, say `(block 0, byte 8072)`. The moment you slide rows together to close a deleted 64-byte gap, every moved row's byte offset changes. You would have to hunt down and rewrite entries across every index on the table. How can we pack variable-length rows into 8,192 bytes, merge free space into one gap, and move rows inside the page without breaking a single external index pointer?

The slotted-page layout (pioneered in System R and used across PostgreSQL) solves all three problems with two simple ideas:

1. **Two-way growth toward a central free gap**: Fixed 4-byte **slot pointers** grow *downward* from byte 24 toward higher addresses. Variable-length **rows** (called *heap tuples*) grow *upward* from byte 8,192 toward lower addresses. All unused bytes sit in one contiguous gap in the middle, bounded by a lower-bound byte offset (`pd_lower`) and an upper-bound byte offset (`pd_upper`). Neither side wastes space guessing a fixed boundary in advance.

2. **One level of slot indirection**: External B-tree indexes never store a row's byte offset. Instead, they store a 6-byte tuple ID, `ctid = (block_number, slot_number)`, where `slot_number` $(1, 2, 3, \dots)$ selects a 4-byte slot at the top of the page. When PostgreSQL compacts a page, it slides row bytes upward in RAM and updates the byte offset *inside that page's 4-byte slot*. Every `(block_number, slot_number)` in external indexes stays valid.

2.2.1 Byte-Offset Tour of `PageHeaderData` (Bytes `0..24`)

Before storing any slot pointers, every 8 KB page reserves its first 24 bytes for bookkeeping: the last log position that touched this page, a corruption checksum, free-space bounds, and cleanup hints. In PostgreSQL's C code (`src/include/storage/bufpage.h`), that 24-byte header is `PageHeaderData`:

```
/* Simplified from src/include/storage/bufpage.h */
typedef struct PageHeaderData
{
    /* [0..8]   LSN of last WAL record modifying this page */
    PageXLogRecPtr pd_lsn;

    /* [8..10]  16-bit page checksum (if data_checksums is enabled) */
    uint16         pd_checksum;

    /* [10..12) Flag bits: PD_HAS_FREE_LINES, PD_PAGE_FULL, PD_ALL_VISIBLE */
    uint16         pd_flags;

    /* [12..14) Byte offset to START of free space (end of line pointers) */
    LocationIndex  pd_lower;

    /* [14..16) Byte offset to END of free space (start of lowest tuple) */
    LocationIndex  pd_upper;

    /* [16..18) Byte offset to start of special access-method space */
    LocationIndex  pd_special;

    /* [18..20) Page size (upper 8 bits) + layout version (lower 8 bits) */
    uint16         pd_pagesize_version;

    /* [20..24) Oldest unpruned XMAX on page (hint for in-page pruning) */
    TransactionId  pd_prune_xid;

    /* [24..)   1-indexed array of 4-byte line pointers growing downward */
    ItemIdData     pd_linp[FLEXIBLE_ARRAY_MEMBER];
} PageHeaderData;
```

Let us walk through each field's job and verify that the byte offsets add up to **24 bytes**:

- **Last log position (`[0..8)`, `pd_lsn`, 8 bytes)**: Crash recovery needs to know how up-to-date

this page is, so `pd_lsn` records the 64-bit Write-Ahead Log position of the latest change to this block. (C splits it into two 32-bit halves so it stays 4-byte aligned on 32-bit hardware.) In Chapter 4 we will see the buffer manager check `pd_lsn` before letting any dirty page reach disk.

- **Corruption detector (`[8..10)`, `pd_checksum`, 2 bytes)**: Disks occasionally flip bits silently, so when checksums are enabled (`initdb --data-checksums`, the default in PostgreSQL 18), PostgreSQL hashes all 8,192 bytes plus the block number right before flushing to disk. If storage rot alters a byte later, the next read catches the mismatch and raises `ERROR: invalid page in block`.
- **Page-state hints (`[10..12)`, `pd_flags`, 2 bytes)**: Quick status bits let readers and inserters avoid wasted scans:
  - `PD_HAS_FREE_LINES (0x0001)`: At least one slot is unused, so a future `INSERT` can recycle an old slot number instead of growing `pd_lower`.
  - `PD_PAGE_FULL (0x0002)`: A recent update ran out of room, warning future updates not to retry in-page placement until dead space is pruned.
  - `PD_ALL_VISIBLE (0x0004)`: Page-local mirror of the Visibility Map's `ALL_VISIBLE` bit.
- **Central free-space bounds (`[12..14)` and `[14..16)`, `pd_lower` and `pd_upper`, 2 bytes each)**: Two 16-bit byte offsets frame the central free gap so inserts can check space in $O(1)$ time. On a fresh empty table page, `pd_lower` = 24 and `pd_upper` = 8,192, leaving $8{,}192 - 24 =$ **8,168** free bytes. Testing whether a row fits takes one subtraction: `pd_upper` − `pd_lower`.
- **Index trailer boundary (`[16..18)`, `pd_special`, 2 bytes)**: Index pages reserve a small trailer at the very end of the 8 KB block for sibling pointers (16 bytes on a B-tree page, setting `pd_special` = 8,176 in Chapter 3). Table pages need no trailer, so on every table page `pd_special` = 8,192.
- **Size and format guard (`[18..20)`, `pd_pagesize_version`, 2 bytes)**: Prevents opening pages built with an incompatible block size or format, packing `8192 | 4` = `0x2004` = 8,196 (version 4 since PostgreSQL 8.3).
- **Cleanup trigger stamp (`[20..24)`, `pd_prune_xid`, 4 bytes)**: Stores the oldest deleting or updating transaction ID among dead rows on this page. If that transaction is older than every active snapshot, a reader knows right away that an in-page cleanup pass (Chapter 5) can reclaim space.

2.2.2 The Line Pointer Array: `ItemIdData` (4 Bytes per Slot)

Starting at byte 24 sits an array of 4-byte slot pointers. Each 32-bit slot packs three small integers: a 15-bit byte offset to the row, a 2-bit status flag, and a 15-bit row length. In C (`src/include/storage/itemid.h`), each 4-byte slot is an `ItemIdData` struct, and SQL numbers slots starting at **1** (so slot k lives at byte $24 + 4(k-1)$):

```
/* From src/include/storage/itemid.h */
typedef struct ItemIdData
{
    unsigned lp_off:15,   /* byte offset to tuple (0..32767) */
             lp_flags:2,  /* state of line pointer, see below */
             lp_len:15;   /* byte length of tuple (0..32767) */
} ItemIdData;
```

```
8
9  #define LP_UNUSED    0    /* unused slot (lp_len is always 0) */
10 #define LP_NORMAL    1    /* used slot pointing to a tuple (lp_len > 0) */
11 #define LP_REDIRECT  2    /* HOT redirect (lp_off = target slot, lp_len == 0) */
12 #define LP_DEAD      3    /* dead item, tuple storage already reclaimed */
```

Look how cleanly those 32 bits are budgeted:

- `lp_off` **(15 bits)**: Covers offsets 0 . . . 32,767, addressing every byte of an 8 KB page (and supporting compile-time block sizes up to 32 KB).
- `lp_len` **(15 bits)**: Exact unpadded byte length of the row.
- `lp_flags` **(2 bits)**: Tracks the four-stage lifecycle of a slot. A live or newly deleted row starts as `LP_NORMAL (1)`. A same-page chain root (Chapter 5) can become `LP_REDIRECT (2)`, where `lp_off` points to another *slot number* on the page. Once a dead row's bytes are compacted away—before external indexes are cleaned—the slot turns into `LP_DEAD (3)` (`lp_len` = 0). Keeping that 4-byte gravestone lets index scans arriving at `(0, 2)` skip it safely. Later, `VACUUM` cleans the indexes and resets the slot to `LP_UNUSED (0)` for reuse.

2.2.3 Step-by-Step Trace: Inserting, Deleting, and Compacting Without Breaking Indexes

Let us trace Page 0 of `accounts` from empty initialization through three inserts, one delete, and an in-page compaction (`PageRepairFragmentation`). In Section 2.3 we will verify that these three rows have raw lengths 49, 57, and 49 bytes. Rounding each up to an 8-byte CPU alignment multiple (`MAXALIGN`) carves **56**, **64**, and **56** bytes from `pd_upper`:

1. **Fresh Empty Page 0**:

   `pd_lower` = 24, `pd_upper` = 8,192, free gap = 8,192 − 24 = 8,168 B.

2. **Insert Row 1** (`1001, 'Ada', 50000, true`; raw `lp_len` = 49 B, aligned 56 B):
   - Allocate Slot 1 at bytes `[24..28)`; advance `pd_lower` : 24 → 28.
   - Pull `pd_upper` down by 56 bytes: 8,192 − 56 = **8,136**. Copy the 49 tuple bytes into `[8136..8185)` (leaving `[8185..8192)` as 7 trailing alignment pad bytes).
   - Write Slot 1 = `(lp_off = 8136, lp_flags = LP_NORMAL, lp_len = 49)`.
   - The primary-key B-tree index `accounts_pkey` records `1001` ↦ `ctid = (0, 1)`.
3. **Insert Row 2** (`1002, 'Grace Hopper', 125000, true`; raw `lp_len` = 57 B, aligned 64 B):
   - Allocate Slot 2 at bytes `[28..32)`; advance `pd_lower` : 28 → 32.
   - Pull `pd_upper` down by 64 bytes: 8,136 − 64 = **8,072**. Copy the 57 tuple bytes into `[8072..8129)`.
   - Write Slot 2 = `(lp_off = 8072, lp_flags = LP_NORMAL, lp_len = 57)`.
   - `accounts_pkey` records `1002` ↦ `ctid = (0, 2)`.
4. **Insert Row 3** (`1003, 'Edsger', 90000, false`; raw `lp_len` = 49 B, aligned 56 B):

- Allocate Slot 3 at bytes `[32..36)`; advance `pd_lower` : 32 → 36.
- Pull `pd_upper` down by 56 bytes: 8,072 − 56 = **8,016**. Copy the 49 tuple bytes into `[8016..8065)`.
- Write Slot 3 = `(lp_off = 8016, lp_flags = LP_NORMAL, lp_len = 49)`.
- `accounts_pkey` (and any secondary indexes on `owner_name` or `balance_cents`) records `1003` ↦ `ctid = (0, 3)`. Free gap is now 8,016 − 36 = 7,980 B.

5. **Delete Row 2 (`1002`) and Compact Page 0 (`PageRepairFragmentation`):**
   - Once the `DELETE` commits and no running transaction can see Row 2 anymore, bytes `[8072..8136)` form a **64-byte dead hole** trapped between Tuple 3 (`[8016..8072)`) and Tuple 1 (`[8136..8192)`).
   - Inside RAM, PostgreSQL executes `memmove` to slide Tuple 3 upward by 64 bytes: from `[8016..8072)` into `[8080..8136)`.
   - `pd_upper` jumps upward from 8,016 to **8,080**, merging all 64 reclaimed bytes straight back into the central free gap (8,080 − 36 = 8,044 B).
   - Slot 2 (`[28..32)`) is marked `(lp_off = 0, lp_flags = LP_DEAD, lp_len = 0)`. Notice that **Slot 3 is never renumbered to Slot 2**.
   - Slot 3 (`[32..36)`) simply rewrites its internal offset from `8016` to `8080`: `(lp_off = 8080, lp_flags = LP_NORMAL, lp_len = 49)`.

Why it works

Look at what happened to every external B-tree index pointing to `'Edsger'` at `ctid = (0, 3)`. Not a single index page had to be locked, dirtied, WAL-logged, or rewritten. When an index lookup lands on Page 0 and asks for Slot 3, it reads bytes `[32..36)`, follows the updated `lp_off = 8080`, and lands on Edsger's relocated tuple in one memory dereference. Four bytes of per-row slot indirection decouple *internal page defragmentation* from *global index maintenance*.

Compare this with two mechanisms you already know. Java's compacting garbage collector also slides objects together, and then rewrites every reference to each moved object. It can afford that sweep because it pauses threads and walks RAM. PostgreSQL cannot rewrite external index files on disk every time it tidies one 8 KB page, so indexes store `(block, slot)` instead. Only the 4-byte slot inside the page holds the raw byte offset. In C terms, a raw pointer dangles when `realloc` moves a buffer, whereas an integer array index stays valid.

Pause

Pause and picture the 8,192-byte page from top to bottom:

1. Bytes `0..24` hold the page header, including `pd_lower` (end of slot pointers) and `pd_upper` (start of lowest row).
2. Four-byte slots grow downward from byte 24; variable-length rows grow upward from byte 8,192.
3. External indexes store `(block, slot)`, never a byte offset, so sliding rows inside the page never touches an index.

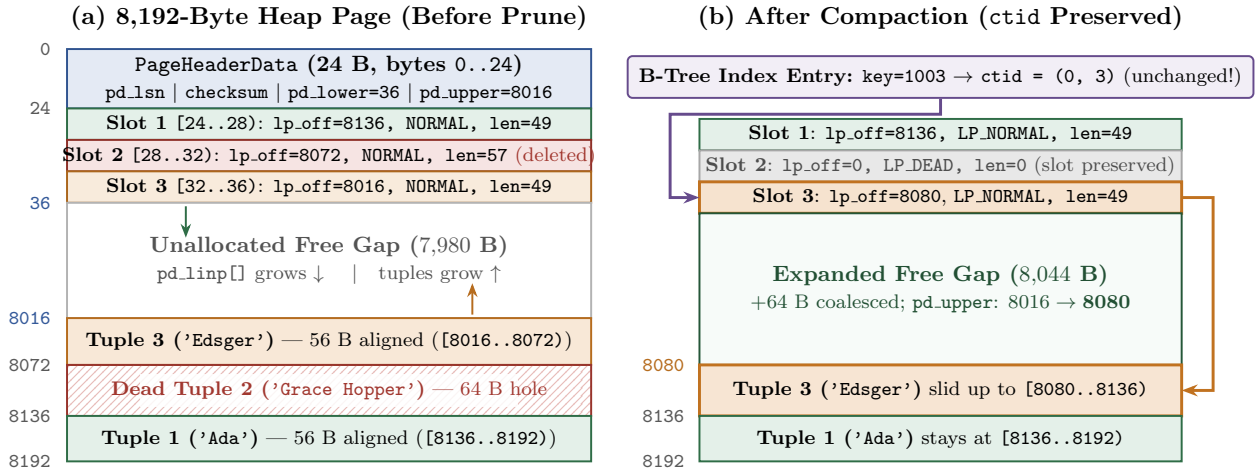


Figure 2.2: Anatomy of an 8,192-byte slotted heap page and in-page defragmentation (`PageRepairFragmentation`). Sliding Tuple 3 upward over Dead Tuple 2 reclaims 64 contiguous bytes into the central gap while updating only Slot 3's internal `lp_off` (8016 → 8080); external index pointers to `ctid = (0, 3)` remain valid with zero index I/O.

2.3 Inside a Heap Tuple: Headers, Null Bitmaps, and Column Tetris

Now let us zoom in on the 49 bytes starting at byte 8,136 of Page 0: Row 1 of `accounts`. Before storing column values, every PostgreSQL row version carries a compact bookkeeping header. It records which transaction created the row, which transaction deleted or superseded it, where any newer version sits, and which columns are `NULL`. In C (`src/include/access/htup_details.h`), that fixed **23-byte** prefix is `HeapTupleHeaderData`, optionally followed by a null bitmap and alignment padding up to byte `t_hoff`:

```
/* Simplified from src/include/access/htup_details.h */
struct HeapTupleHeaderData
{
    union {
        HeapTupleFields t_heap;   /* [0..12) t_xmin (4B), t_xmax (4B), t_cid (4B) */
        DatumTupleFields t_datum; /* used for in-memory composite datums */
    } t_choice;

    ItemPointerData t_ctid;       /* [12..18) current or newer (block, slot) (6B) */

    uint16          t_infomask2;  /* [18..20) lower 11 bits = natts; upper = HOT flags */
    uint16          t_infomask;   /* [20..22) HASNULL, HASVARWIDTH, XMIN/XMAX hint bits */
    uint8           t_hoff;       /* [22..23) byte offset to user data (MAXALIGN'd!) */

    /* [23..) Optional null bitmap (1 bit per column when HEAP_HASNULL is set) */
    bits8           t_bits[FLEXIBLE_ARRAY_MEMBER];

    /* User column bytes start at exact offset t_hoff from start of header! */
};
```

2.3.1 Why a 23-Byte Struct Produces a 24-Byte Header (and Free 8-Column Null Bitmaps)

A Java 6 object on 64-bit HotSpot also starts with a header: an 8-byte mark word and a 4- or 8-byte class pointer. A PostgreSQL row header plays the same role on disk, except its fields track multi-version concurrency rather than garbage collection. Let us link each human job to its C field and count the bytes:

- **Birth and expiration stamps (`[0..4) t_xmin` and `[4..8) t_xmax`, 4 bytes each)**: Concurrent snapshots need to know who created a row version and who retired it, so PostgreSQL stamps these two 32-bit transaction IDs on the front (Chapter 5).
- **Intra-transaction step counter (`[8..12) t_cid`, 4 bytes)**: When one transaction runs several SQL statements in sequence, this command counter tracks which step wrote the version.
- **Version-chain link (`[12..18) t_ctid`, 6 bytes)**: On the latest version of a row, `t_ctid` points to itself, such as `(0, 1)`. When `UPDATE` creates a newer version, the old version's `t_ctid` points forward to the new version's `(block, slot)`.

  Under the hood

  Why is `ItemPointerData` 6 bytes instead of 8 bytes when it holds a 32-bit block number plus a 16-bit slot number? In C, a struct with a `uint32` and a `uint16` gets 4-byte alignment and `sizeof` = 8, wasting 2 tail-padding bytes. PostgreSQL splits the block number into *two 16-bit halves* (`bi_hi` and `bi_lo`). Alignment stays at 2 bytes and `sizeof(ItemPointerData)` stays **6 bytes**, shrinking every row header and every B-tree entry across the database.

- **Column count and chain flags (`[18..20) t_infomask2`, 2 bytes)**: Stores the column count `natts` in its lower 11 bits and same-page update flags in its upper 5 bits.
- **Layout and commit-status flags (`[20..22) t_infomask`, 2 bytes)**: Stores layout flags (null bitmap present, variable-width column present, or an external TOAST pointer for oversized out-of-line values, covered in Section 2.4). It also caches commit/abort **hint bits** directly on the row so readers avoid repeated commit-log lookups.
- **Payload start offset (`[22..23) t_hoff`, 1 byte)**: Records the exact byte offset where user column 1 begins after any optional null bitmap and alignment padding.

That brings us to offset **23**. Why cannot user column 1 (`account_id BIGINT`) start right at byte 23? On 64-bit CPUs, reading an 8-byte integer expects an 8-byte aligned address (`MAXALIGN` = 8). Because every row starts at an 8-byte aligned `pd_upper`, user columns must also start at a multiple of 8 from the row header.

Watch the arithmetic across three cases:

1. **Row has zero `NULL` values (`HEAP_HASNULL = 0`)**: PostgreSQL omits `t_bits[]` completely. Rounding 23 up to the next multiple of 8 yields `t_hoff` = **24** (byte 23 is a single zero-padding byte).
2. **Row has at least one `NULL`, and $1 \leq$ natts ≤ 8 columns (`HEAP_HASNULL = 1`)**: The null bitmap needs $\lceil \text{natts}/8 \rceil = 1$ byte, which drops neatly into **byte 23**—the exact byte previously wasted on padding. Now $23 + 1 = 24$, so `t_hoff` **is still** 24. Even better, any `NULL` column

writes **0 data bytes** into the row payload. (In PostgreSQL's bitmap, bit `1` means *present* and bit `0` means *NULL*.)

3. **Row has at least one `NULL`, and natts = 9 columns**: Now the null bitmap needs $\lceil 9/8 \rceil = 2$ bytes (`[23..25)`), bringing the unaligned header to 25 bytes. Rounding 25 up to a multiple of 8 jumps `t_hoff` from 24 to **32 bytes** (7 padding bytes at `[25..32)`).

Common trap

Adding a 9th column (or a 73rd column, where $23 + 10 = 33 \rightarrow$ `t_hoff` $= 40$) to a wide nullable table adds an invisible **8-byte cliff** to every row with a `NULL`. On the bright side, `NULL` columns in PostgreSQL never reserve their column width on disk. A row with twenty `BIGINT` columns all set to `NULL` takes 23 header bytes + 3 bitmap bytes + 6 pad bytes = **32 bytes total**, not 160 bytes.

2.3.2 CPU Alignment Padding and "Column Tetris"

Once PostgreSQL reaches `t_hoff`, it writes each non-`NULL` column in exact `CREATE TABLE` order. Before writing column i, it advances the offset to the alignment boundary required by that column's type (`pg_type.typalign`):

`typalign`	Boundary	Common PostgreSQL Types	Width (`typlen`)
`'d'` (double)	8 bytes	`BIGINT` (`int8`), `DOUBLE PRECISION`, `TIMESTAMPTZ`, `TIMESTAMP`	8 B (`INTERVAL`: 16 B)
`'i'` (int)	4 bytes	`INTEGER` (`int4`), `REAL` (`float4`), `DATE`, 4B-header `varlena`	4 B (or variable)
`'s'` (short)	2 bytes	`SMALLINT` (`int2`)	2 B
`'c'` (char)	1 byte	`BOOLEAN`, `"char"`, `UUID` (16 B), short 1B-header `varlena`	1 B, 16 B, or var.

Look at `UUID` in the bottom row. Although a `UUID` is 16 bytes wide, its C type is an array of 16 single bytes (`unsigned char data[16]`), so `typalign = 'c'` (1-byte aligned, zero pre-padding). By contrast, `BIGINT` and `TIMESTAMPTZ` require 8-byte alignment (`'d'`). If the previous column ended at offset 25, PostgreSQL inserts **7 zero bytes** (`[25..32)`) so the `BIGINT` starts at offset 32.

Your C compiler applies the exact same rule to structs. On x86-64, `struct { char a; int64_t b; }` has `sizeof` 16, not 9, because `b` must start at offset 8. C pays that padding once in RAM; PostgreSQL pays it on disk, in the Write-Ahead Log, and in `shared_buffers`. Java hides this because HotSpot automatically reorders class fields by size.

PostgreSQL *never* reorders columns behind your back; physical order matches `CREATE TABLE` order. Let us compare two 50-million-row payment-event tables—one alternating narrow and 8-byte columns, and one ordered from widest alignment downward:

```
-- Layout A: Alternating 1B/2B and 8B columns ("Swiss-cheese tuple")
CREATE TABLE events_bad (
    is_settled   BOOLEAN     NOT NULL, -- 1B at [24..25) + 7B pad [25..32)
    account_id   BIGINT      NOT NULL, -- 8B at [32..40)
    region_code  SMALLINT    NOT NULL, -- 2B at [40..42) + 6B pad [42..48)
    created_at   TIMESTAMPTZ NOT NULL, -- 8B at [48..56)
```

```
    retry_flag   BOOLEAN     NOT NULL, -- 1B at [56..57) + 7B pad [57..64)
    amount_cents BIGINT      NOT NULL, -- 8B at [64..72)
    channel_id   INTEGER     NOT NULL, -- 4B at [72..76)
    fee_bps      SMALLINT    NOT NULL  -- 2B at [76..78) -> lp_len=78, MAXALIGN=80B!
);

-- Layout B: Identical 8 columns ordered 8B -> 4B -> 2B -> 1B ("Column Tetris")
CREATE TABLE events_good (
    account_id   BIGINT      NOT NULL, -- 8B at [24..32)
    created_at   TIMESTAMPTZ NOT NULL, -- 8B at [32..40)
    amount_cents BIGINT      NOT NULL, -- 8B at [40..48)
    channel_id   INTEGER     NOT NULL, -- 4B at [48..52)
    region_code  SMALLINT    NOT NULL, -- 2B at [52..54)
    fee_bps      SMALLINT    NOT NULL, -- 2B at [54..56)
    is_settled   BOOLEAN     NOT NULL, -- 1B at [56..57)
    retry_flag   BOOLEAN     NOT NULL  -- 1B at [57..58) -> lp_len=58, MAXALIGN=64B!
);
```

Check the math. Both tables store the exact same $8+8+8+4+2+2+1+1 = \mathbf{34}$ bytes of user payload plus a 24-byte header (58 useful bytes total):

- In `events_bad`, three alignment gaps ($7+6+7 = 20$ B) inflate `lp_len` to 78 B, which `MAXALIGN` rounds to **80 bytes** on the page (+4 B slot = **84 B/row**). Each 8 KB page (8,168 usable bytes) holds $\lfloor 8{,}168/84 \rfloor = \mathbf{97}$ rows. Across 50,000,000 rows, `events_bad` takes 515,464 blocks $\approx \mathbf{4.22}$ GB.
- In `events_good`, every column starts naturally on its boundary with **zero internal padding**: `lp_len` = 58 B, `MAXALIGN` = **64** bytes (+4 B slot = **68 B/row**). Each 8 KB page now holds $\lfloor 8{,}168/68 \rfloor = \mathbf{120}$ rows. Across 50,000,000 rows, `events_good` takes 416,667 blocks $\approx \mathbf{3.41}$ GB.

Reordering eight lines of Data Definition Language (DDL, i.e., `CREATE` or `ALTER TABLE`) saves **16 bytes per row** and **810 MB** of storage, WAL traffic, and RAM cache without changing a single query. If a table also has `TEXT` or `JSONB` columns, put all fixed-width `NOT NULL` columns *first* (8B → 4B → 2B → 1B) and variable-width or nullable columns *last*. That lets PostgreSQL cache fixed byte offsets (`attcacheoff`) and jump straight to `amount_cents` at offset 40 in $O(1)$ instructions.

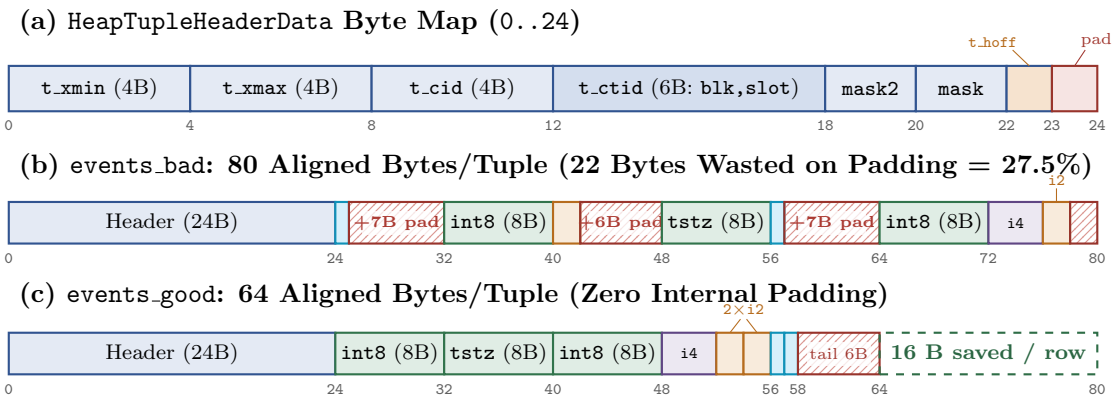


Figure 2.3: Byte-by-byte tuple layout and "Column Tetris." Ordering columns from 8-byte alignment downward eliminates all 20 bytes of internal alignment holes in `events_bad`, shrinking the `MAXALIGN`ed footprint from 80 bytes to 64 bytes.

2.3.3 Decoding Real Disk Bytes with `pageinspect`

Nothing cements this byte arithmetic like inspecting a live PostgreSQL heap page yourself. The built-in `pageinspect` contrib extension exposes raw 8 KB blocks directly as SQL rows.

Try it in PostgreSQL

Run the following session in any local PostgreSQL scratch database (superuser required for `get_raw_page`):

```
CREATE EXTENSION IF NOT EXISTS pageinspect;

INSERT INTO accounts VALUES
    (1001, 'Ada',          50000,  true),
    (1002, 'Grace Hopper', 125000, true),
    (1003, 'Edsger',       90000,  false);

SELECT lower, upper, special, pagesize
FROM page_header(get_raw_page('accounts', 0));
--  lower | upper | special | pagesize
-- -------+-------+---------+----------
--     36 |  8016 |    8192 |     8192

SELECT lp, lp_off, lp_flags, lp_len, t_xmin, t_xmax, t_ctid,
       t_infomask2, t_infomask, t_hoff, t_attrs
FROM heap_page_item_attrs(get_raw_page('accounts', 0), 'accounts'::regclass);
-- lp | lp_off | lp_flags | lp_len | t_xmin | t_ctid | mask2 | mask | hoff
-- ---+--------+----------+--------+--------+--------+-------+------+-----
--  1 |   8136 |        1 |     49 |    742 | (0,1)  |     4 | 2306 |   24
--      t_attrs: {"\\xe903000000000000","\\x09416461","\\x50c3000000000000","\\x01"}
--  2 |   8072 |        1 |     57 |    742 | (0,2)  |     4 | 2306 |   24
--      t_attrs: {"\\xea03000000000000","\\x1b477261636520486f70706572","\\
    x48e8010000000000","\\x01"}
--  3 |   8016 |        1 |     49 |    742 | (0,3)  |     4 | 2306 |   24
--      t_attrs: {"\\xeb03000000000000","\\x0f456473676572","\\x905f010000000000
    ","\\x00"}
```

Every single number from our hand calculation in Section 2.2 appears verbatim. Look closer at Row 1's fields on an x86-64 (little-endian) Linux machine:

- `t_infomask2 = 4`: 4 columns (`natts = 4`), zero HOT flags.
- `t_infomask = 2306 = 0x0902`: Bitwise OR of `0x0002` (`HEAP_HASVARWIDTH`, because `owner_name` is `TEXT`), `0x0100` (`HEAP_XMIN_COMMITTED`), and `0x0800` (`HEAP_XMAX_INVALID`, meaning nobody has deleted Row 1).
- `t_attrs[1] = \xe903000000000000`: Little-endian 8-byte integer `0x03e9` $= 3 \times 256 + 233 =$ **1001**.
- `t_attrs[2] = \x09416461`: The 3-byte string `'Ada'` (`0x41 0x64 0x61`) starts with header byte `0x09`. Total length $1 + 3 = 4$ encoded as a **short 1-byte `varlena` header** is $(4 \ll 1) \mid 1 = 9 =$ `0x09`. Notice the Column Tetris flaw inside `accounts`: putting `owner_name` (ending at byte 36) before `balance_cents BIGINT` forced 4 wasted pad bytes (`[36..40)`), raising `lp_len` to 49 B. Moving `owner_name` last drops `lp_len` to 45 B and

shrinks Row 1's aligned footprint from **56 bytes to 48 bytes**.

2.4 TOAST: The Oversized-Attribute Storage Technique

Stop and ask an uncomfortable question about fixed 8,192-byte pages. Every slot points to a single row inside a single 8 KB block. Subtracting the 24-byte page header plus one 4-byte slot leaves at most 8,160 bytes on an empty page (`MaxHeapTupleSize` = 8,160). Yet PostgreSQL lets you insert a **400 KB** JSON document or a **50 MB** `TEXT` report into one column. Where do those bytes go when no row is allowed to span two 8 KB pages?

First, why *does* PostgreSQL forbid rows from crossing page boundaries? Keeping every row inside one 8 KB frame keeps the storage engine simple and fast:

- Reading a row by `ctid = (block, slot)` pins **one** buffer frame and takes **one** page lock—no multi-page lock deadlocks.
- Every 8 KB page carries a single log position (`pd_lsn`) for crash recovery (Chapter 4).
- In-page compaction is a local `memmove` inside one 8,192-byte C array.

PostgreSQL goes one step further: it wants *at least four rows* to fit on every 8 KB page whenever possible. That target sets the threshold at roughly 2,000 **bytes** (`TOAST_TUPLE_THRESHOLD` = 2,032 B on 64-bit builds). Whenever an inserted or updated row exceeds 2,032 bytes, PostgreSQL runs **TOAST**—*The Oversized-Attribute Storage Technique* (`src/backend/access/common/toast_internals.c`).

2.4.1 Inside `varlena` Headers: Saving 3 to 6 Bytes on Every Short String

Before TOAST touches large strings, PostgreSQL first compacts *short* variable-length values (`TEXT`, `VARCHAR`, `BYTEA`, `NUMERIC`, `JSONB`). Under the hood, all of them share a variable-length header layout called `varlena` (`src/include/varatt.h`).

A naive design prepends a 4-byte length integer to every string. On a 3-letter string like `'USD'` or `'Ada'`, a 4-byte header plus up to 3 alignment bytes would burn **7 overhead bytes** to store 3 characters. Instead, on little-endian CPUs (x86-64 and ARM64), PostgreSQL checks the **lowest two bits of the very first byte** to pick among four compact layouts:

First-Byte Tag	Variant (`varatt.h`)	Align	Payload & Max Capacity
`xxxxxx00` (2b)	4B uncompressed (`va_4byte`)	4 B (`'i'`)	Upper 30 bits = total len up to $2^{30}-1$ (**1 GB**)
`xxxxxx10` (2b)	4B compressed (`va_compressed`)	4 B (`'i'`)	+4B `tcinfo` (raw size + `pglz`/`lz4` codec ID)
`xxxxxxx1` ($\neq$ `0x01`)	**1B short** (`va_1byte`)	**1 B (`'c'`)**	Upper 7 bits = total len 1…127 (**0..126 data B**)
`00000001` (`0x01`)	**18B TOAST pointer**	**1 B (`'c'`)**	Byte 1 = tag (`18`); bytes `2..18` = `varatt_external`

Look at the 1-byte short header (`xxxxxxx1`). Any `TEXT`, `VARCHAR`, or `NUMERIC` value up to **126 bytes** long takes just **1 header byte** and drops to **1-byte alignment** (zero pre-padding). That is why

`VARCHAR(20)` is never smaller on disk than `TEXT` in PostgreSQL: both use the exact same 1-byte header.

Now check the fourth row. When byte 0 equals `0x01` and byte 1 equals `18` (`VARTAG_ONDISK`), the giant value leaves the main table page completely. In its place sits an **18-byte pointer** (`struct varatt_external`) recording the raw size, compressed size, companion table ID, and chunk ID:

```
/* From src/include/varatt.h -- 16-byte payload inside an 18-byte TOAST pointer */
typedef struct varatt_external
{
    int32    va_rawsize;     /* Original uncompressed size (incl. 4B header) */
    uint32   va_extinfo;     /* Lower 30 bits: stored size; upper 2: codec   */
    Oid      va_valueid;     /* Unique chunk_id inside the TOAST table       */
    Oid      va_toastrelid;  /* OID of pg_toast.pg_toast_<table_oid>         */
} varatt_external;
```

2.4.2 The Four TOAST Strategies and `pg_toast` Chunk Slicing

Every column in `pg_attribute` has a 1-character storage policy (`attstorage`) telling TOAST what it may do when a row passes 2,032 bytes:

1. `PLAIN ('p')`: Used by fixed-width types (`BIGINT`, `BOOLEAN`, `TIMESTAMPTZ`, `UUID`). Never compressed and never moved out-of-line.
2. `EXTENDED ('x')`, **default for** `TEXT`, `BYTEA`, `JSONB`, `NUMERIC`): Allows *both* inline compression (`pglz` by default, or faster `lz4` via `default_toast_compression = 'lz4'`) *and* out-of-line chunk storage.
3. `EXTERNAL ('e')`: Moves oversized values out-of-line while **disabling compression**. Two cases benefit from this. First, if a `BYTEA` column already holds pre-compressed PNGs, Zstandard archives, or ciphertext, compressing again wastes CPU for zero savings. Second, if queries read slices via `substring(payload, 50000, 80)`, uncompressed chunks let PostgreSQL jump straight to **one 2 KB chunk** instead of decompressing from byte 0.
4. `MAIN ('m')`: Compresses inline and tries to keep the column **on the main page**. It moves the value out-of-line only when the row still exceeds 8,160 bytes after `EXTENDED` and `EXTERNAL` columns have left.

If no column is allowed to leave the page, an oversized row is rejected rather than split across two pages:

```
CREATE TABLE notes (body TEXT);
ALTER TABLE notes ALTER COLUMN body SET STORAGE PLAIN;
INSERT INTO notes SELECT repeat('x', 9000);
-- ERROR:  row is too big: size 9028, maximum size 8160
```

Here 9,028 is the 24-byte row header + 4-byte length word + 9,000 payload bytes, exceeding the 8,160-byte single-page limit.

Where do evicted values land? Whenever a table has a TOASTable column, PostgreSQL creates a hidden companion table `pg_toast.pg_toast_`⟨oid⟩ with a unique B-tree index on `(chunk_id, chunk_seq)`:

```
-- Conceptual schema of pg_toast.pg_toast_24576:
--   chunk_id   OID    (matches varatt_external.va_valueid)
--   chunk_seq  INT4   (0, 1, 2, ... in byte order)
--   chunk_data BYTEA  (up to TOAST_MAX_CHUNK_SIZE ~= 1,996 bytes each)
--   PRIMARY KEY (chunk_id, chunk_seq)
```

Example 2.1 (Counting TOAST Chunks by Hand)**.** Suppose we add `audit_json JSONB` (`EXTENDED`, `lz4`) to `accounts` (`oid = 24576`) and insert a row whose `audit_json` serializes to 30,720 bytes (30 KiB).

1. Because $30{,}720 + 49 > 2{,}032$, PostgreSQL first compresses `audit_json` with `lz4`, shrinking it to 9,000 **bytes**.
2. $9{,}000 + 49 = 9{,}049$ B still exceeds the 2,032-byte target, so PostgreSQL moves the compressed bytes out-of-line.
3. Each TOAST chunk holds at most 1,996 bytes (`TOAST_MAX_CHUNK_SIZE` = 1,996, sized so four chunks pack cleanly onto one 8 KB TOAST page). Slicing 9,000 bytes yields:

   $$\left\lceil \frac{9{,}000}{1{,}996} \right\rceil = \mathbf{5} \text{ chunks} \quad (\texttt{chunk_seq} = 0, 1, 2, 3 \text{ of } 1{,}996 \text{ B each, plus } \texttt{chunk_seq} = 4 \text{ of } 1{,}016 \text{ B}).$$

4. Those 5 rows go into `pg_toast.pg_toast_24576`. Meanwhile, the main `accounts` row replaces `audit_json` with an **18-byte TOAST pointer**, keeping the main row at $49+18 = 67$ B (72 B aligned).

In production

Understanding that 18-byte pointer explains two everyday production behaviors:

- **Why `SELECT *` on wide tables is slow**: If an object-relational mapper (ORM) runs `SELECT * FROM accounts WHERE is_active`, PostgreSQL must *detoast* every matching row. It probes the TOAST index, reads 5 chunk rows, and decompresses 30 KB of JSON that your code immediately discards. Naming only the columns you need (`SELECT account_id, balance_cents`) scans 110+ compact rows per 8 KB page and **never opens the TOAST table**.
- **Why `UPDATE` on narrow columns stays cheap**: When `UPDATE accounts SET balance_cents = balance_cents + 500` creates a new version of the main row (Chapter 5), `audit_json` is unchanged. PostgreSQL **copies the 18-byte pointer verbatim** into the new row version without rewriting a single byte in `pg_toast_24576`. By contrast, appending one key to a 10 MB `JSONB` value rewrites all 10 MB of TOAST chunks.

2.5 Double Buffering, `shared_buffers`, Pins, and Clock-Sweep

So far we have inspected 8 KB pages on disk. Now let us trace how a running PostgreSQL server moves those 8 KB blocks between NVMe storage and RAM.

PostgreSQL uses one OS process per client connection, and all backend processes attach to a shared RAM cache allocated at startup called `shared_buffers` (`src/backend/storage/buffer/bufmgr.c`). Every read or write of a table, index, FSM, or VM page goes through `shared_buffers` first.

2.5.1 Why Not Just Rely on the Linux Page Cache? (Double Buffering)

When a backend misses in `shared_buffers`, it asks the OS kernel to read 8,192 bytes from the segment file (via the `pread()` system call). Inside `pread()`, Linux first reads the NVMe sectors into its own **OS page cache**, and then copies those 8,192 bytes into PostgreSQL's `shared_buffers` frame. Briefly, the same 8 KB block sits in RAM *twice*—once in the kernel cache and once in `shared_buffers`. This **double buffering** is why database administrators (DBAs) usually set `shared_buffers` to **25%–40% of machine RAM**, leaving the rest for the OS page cache plus query sort/hash memory.

Why not skip `shared_buffers` and memory-map table files directly from the OS page cache with `mmap()`? Two hard requirements forbid it:

1. **Strict Write-Ordering for Crash Safety (The WAL Rule)**: Chapter 4 proves the core rule of durability: *a dirty 8 KB page must never reach disk before the log record describing its change (`pd_lsn`) is synced to disk.* If PostgreSQL edited `mmap`ped kernel pages directly, Linux background threads could flush a half-committed page to NVMe before its log record hit disk. A power cut would then leave unrecoverable corruption. Holding dirty pages in user-space `shared_buffers` ensures the kernel *never sees* modified bytes until PostgreSQL checks `FlushedWAL` $\geq$ `pd_lsn` and calls `pwrite()`.
2. **Zero-Syscall Shared-Memory Reads**: Reading a hot B-tree root page in `shared_buffers` stays entirely in user space: one hash lookup and one atomic CPU instruction, with zero kernel calls or 8 KB copies. (PostgreSQL 17 and 18 are also adding Direct I/O and `io_uring` paths to cut OS-side double buffering.)

2.5.2 Inside `shared_buffers`: Hash Partitions and `BufferDesc.state`

Picture `shared_buffers` with N frames (say `16GB` $\implies$ N = 2,097,152 frames of 8 KB) as three parallel arrays in shared memory (`src/include/storage/buf_internals.h`):

1. `BufferBlocks`: A flat array of N raw 8,192-byte page frames (`buf_id = 0 .. N-1`).
2. **Shared Buffer Lookup Hash Table**: Maps a page's 20-byte disk address (`BufferTag`: file locator, fork, and block number) to its frame index `buf_id`. To keep 64 CPU cores from fighting over one lock, PostgreSQL splits this hash table across **128 independent lock partitions** (`NUM_BUFFER_PARTITIONS = 128`). Two backends looking up different blocks land in different partitions over 99% of the time.
3. `BufferDescriptors`: An array of N **64-byte** frame headers (`BufferDesc`), sized to one CPU cache line so neighboring headers never bounce each other's cache lines.

Inside each 64-byte frame header, PostgreSQL packs the frame's pin count, popularity score, and status flags into a **single 32-bit atomic integer** (`pg_atomic_uint32 state`):

```
/* Simplified from src/include/storage/buf_internals.h */
typedef struct BufferDesc
{
    BufferTag       tag;          /* (RelFileLocator, ForkNumber, BlockNumber) */
    int             buf_id;       /* Frame index 0 .. NBuffers-1               */

    /* Single 32-bit atomic word: [31..22 flags | 21..18 usage | 17..0 refcount] */
```

```
    pg_atomic_uint32 state;

    int             wait_backend_pgprocno; /* For cleanup-lock waiters         */
    int             freeNext;              /* Link in initial free list        */
    LWLock          content_lock;          /* Shared/Exclusive page byte lock  */
} BufferDesc;

/* Exact bit layout of BufferDesc.state:
 *   Bits  0..17 (18 bits): refcount (pin count, up to 262,143 concurrent pins!)
 *   Bits 18..21 ( 4 bits): usage_count (clock-sweep popularity, capped 0..5)
 *   Bits 22..31 (10 bits): BM_LOCKED, BM_DIRTY, BM_VALID, BM_TAG_VALID,
 *                          BM_IO_IN_PROGRESS, BM_PERMANENT, ...
 */
```

Why pack `refcount`, `usage_count`, and `BM_DIRTY` into one 32-bit word? Older PostgreSQL versions took a per-buffer spinlock on every pin and unpin, causing heavy CPU cache-line bouncing on hot index root pages. With one packed 32-bit `state` word, pinning a frame and bumping its popularity runs in **a single lock-free compare-and-swap (CAS) instruction**. Once `usage_count` hits its cap of **5**, `PinBuffer` simply adds +1 to `refcount`.

2.5.3 Buffer Pins (`refcount`) vs. Content Locks (`LWLock`)

Two distinct tools protect a cached page: **pinning** the frame and **locking** its bytes.

Mechanism	Where Stored	What It Protects & How Long It Is Held
Pin (`refcount > 0`)	`state[17:0]`	Prevents *eviction* of the frame. Held across tuple evaluation; **never blocks** other readers or writers.
Shared Lock (`LW_SHARED`)	`content_lock`	Protects `pd_linp[]` / tuple bytes from concurrent `memmove`; held for **microseconds** while locating tuples.
Exclusive Lock (`LW_EXCLUSIVE`)	`content_lock`	Acquired to insert/update tuples or modify header offsets; blocks other `LW_SHARED`/`LW_EXCLUSIVE` holders briefly.
Cleanup Lock	`LW_EXCLUSIVE` + `ref` = 1	Required by `PageRepairFragmentation` / `VACUUM` before sliding tuple bytes so no C pointer goes stale.

Trace one table-scan step to see why PostgreSQL needs both:

1. Your backend hashes `(accounts, MAIN_FORK, block 0)` to `buf_id = 42`, bumps `refcount` ($0 \to 1$, **pinned**), and takes `content_lock` in `LW_SHARED` mode.
2. Under `LW_SHARED`, it checks which slots on Page 0 are visible, records them, and **immediately releases** `content_lock`—while *keeping its pin* (`refcount` = 1).
3. Next, your backend runs `WHERE` filters and sends rows up the plan tree. Because `content_lock` is free, another backend can grab `LW_EXCLUSIVE` on Page 0, `INSERT` Row 4 into the free gap, and commit right away. Meanwhile, your pin (`refcount` ≥ 1) forbids eviction and forbids `VACUUM` from sliding live row bytes underneath your C pointer.

4. When your scan moves on to Page 1, it drops Page 0's `refcount` ($1 \to 0$, **unpinned**).

2.5.4 Clock-Sweep Eviction: Step-by-Step Trace on a 6-Frame Pool

What happens when every frame in `shared_buffers` is full and a backend misses on a block?

Textbooks teach doubly-linked **Least Recently Used (LRU)** queues: every read unlinks a node and moves it to the front of one global list. On a 64-core server doing two million cache hits per second, a global LRU queue stalls because every read mutates shared list pointers. In Java 6, `new LinkedHashMap<K,V>(16, 0.75f, true)` in access-order mode has the exact same trait: every `get()` relinks nodes, so wrapping it in `Collections.synchronizedMap` serializes every read behind one lock.

PostgreSQL avoids that bottleneck with **Clock-Sweep** (`StrategyGetBuffer` in `src/backend/storage/buffer/fr`

- **On a buffer hit (the 99% fast path)**: Touch *nothing* global. Locally bump that frame's 4-bit `usage_count` up to a cap of **5**. Capping at 5 (rather than 15) ensures that if the workload shifts, the clock hand never has to circle the pool 15 times to clear old pages.
- **On a buffer miss (eviction path)**: Advance the shared clock hand `nextVictimBuffer` modulo N, checking each frame:
  1. **If** `refcount` > 0 **(pinned)**: Cannot evict. Leave `usage_count` alone and step `hand` $\leftarrow$ (`hand` $+ 1$) mod N.
  2. **If** `refcount` $= 0$ **and** `usage_count` > 0: Give the frame a second chance by decrementing `usage_count`, and step `hand` forward.
  3. **If** `refcount` $= 0$ **and** `usage_count` $= 0$: **Victim found.** Pin the frame (`refcount` $= 1$) and advance `hand`. If `BM_DIRTY` is set, check `FlushedWAL` $\geq$ `pd_lsn` and write out the old page. (Usually the background writer `bgwriter` cleans low-usage buffers ahead of the clock hand so queries rarely wait on writes.) Finally, swap the hash-table tag and `pread()` the new 8 KB block with `usage_count` $= 1$.

Example 2.2 (Tracing the Clock Hand Across 6 Buffer Frames)**.** Consider a 6-frame pool (`F0..F5`) with `nextVictimBuffer` starting at **Frame 0** and initial states `(refcount, usage)`:

`F0:(1, 3)`, `F1:(0, 2)`, `F2:(0, 1)`, `F3:(2, 0)`, `F4:(0, 1)`, `F5:(0, 2)`.

Watch how `StrategyGetBuffer` finds a victim when a backend misses on `accounts` block 9:

1. **Tick 1 (Frame 0)**: `refcount` $= 1 > 0$ (pinned). Skip immediately; `F0` stays `(1, 3)`. Hand $\to$ `F1`.
2. **Tick 2 (Frame 1)**: `refcount` $= 0$, `usage` $= 2 > 0$. Decrement `usage` : $2 \to 1$. Hand $\to$ `F2`.
3. **Tick 3 (Frame 2)**: `refcount` $= 0$, `usage` $= 1 > 0$. Decrement `usage` : $1 \to$ **0**. Hand $\to$ `F3`.
4. **Tick 4 (Frame 3)**: Even though `usage` $= 0$, `refcount` $= 2 > 0$ (two backends hold pins on `F3`). Pinned frames are immune to eviction; skip `F3`. Hand $\to$ `F4`.
5. **Tick 5 (Frame 4)**: `refcount` $= 0$, `usage` $= 1 > 0$. Decrement `usage` : $1 \to 0$. Hand $\to$ `F5`.
6. **Tick 6 (Frame 5)**: `refcount` $= 0$, `usage` $= 2 > 0$. Decrement `usage` : $2 \to 1$. Hand wraps $\to$ `F0`.

7. **Tick 7 (Frame 0)**: Still pinned (`refcount` = 1). Skip. Hand → `F1`.
8. **Tick 8 (Frame 1)**: `refcount` = 0, `usage` = 1 > 0. Decrement `usage` : 1 → 0. Hand → `F2`.
9. **Tick 9 (Frame 2)**: `refcount` = 0 **and** `usage` = 0. **Frame 2 is evicted.** It loads `accounts` block 9 with state `(refcount=1, usage=1)`, and `nextVictimBuffer` parks at **Frame 3** ready for the next miss.

2.5.5 Buffer Access Strategies: Why a 500 GB `Seq Scan` Doesn't Wipe Out RAM

Clock-Sweep alone has one vulnerability. Suppose your server has `shared_buffers = 16 GB` (2,097,152 frames), holding your hot online transaction processing (OLTP) accounts and B-tree indexes at `usage_count` = 5. At 2:00 PM, an analyst runs an unindexed report:

```
SELECT count(*), sum(amount_cents) FROM historical_ledger; -- 500 GB cold table!
```

Reading 500 GB streams 65,536,000 cold blocks—more than **31 times larger** than the 16 GB pool. If every cold block used the global clock hand, the hand would spin dozens of times, knocking every hot OLTP page from 5 down to 0. One report would **wipe out your entire hot cache** for blocks read once and never needed again.

PostgreSQL stops this cache pollution with **Ring Buffers** (*Buffer Access Strategies*, `GetAccessStrategy` in `freelist.c`):

- `BAS_BULKREAD` **(32-frame = 256 KB ring)**: Whenever a sequential scan reads a table larger than `shared_buffers`/4, the executor allocates a private ring of **32 frames** (256 KB). As the 500 GB scan streams blocks $0, 1, 2, \ldots$, it recycles those same 32 frames over and over. If an OLTP query also touches one of those frames and bumps its `usage_count` > 1 (or pins it), PostgreSQL keeps that popular frame in the main pool and pulls a fresh frame into the ring. Thus a 500 GB scan touches just 256 KB of `shared_buffers`, leaving **99.998%** of hot OLTP RAM untouched.
- `BAS_BULKWRITE` **(2,048-frame = 16 MB ring)**: Used by `COPY FROM`, `CREATE TABLE AS`, and `SELECT INTO` so bulk loads cycle through 16 MB instead of flooding `shared_buffers`.
- `BAS_VACUUM` **(default 2 MB ring via `vacuum_buffer_usage_limit`)**: Keeps autovacuum inside a small ring so routine cleanup never evicts user query caches.

Connection

Synchronized sequential scans (`synchronize_seqscans = on`). Suppose Analyst B starts a second `Seq Scan` on that 500 GB `historical_ledger` halfway through Analyst A's scan. PostgreSQL starts Analyst B *at the exact block Analyst A is currently reading*, and wraps around to block 0 at the end. Both scans share the same 256 KB cache window and cut disk I/O in half. That is also why Chapter 1 warned that `SELECT` without `ORDER BY` never promises to return rows starting from block 0.

Cost summary

Physical Numbers Worth Memorizing:

- **Page size** (`BLCKSZ`): 8,192 B (24 B `PageHeaderData` ⟹ 8,168 B usable; 4 B per

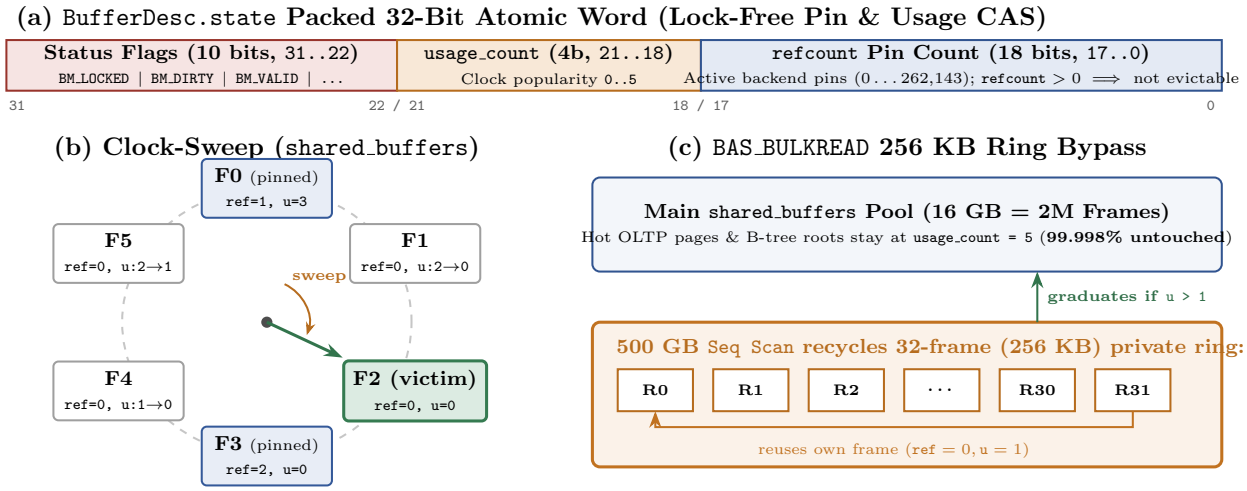


Figure 2.4: Shared buffer management in PostgreSQL. Panel (a) shows the lock-free 32-bit `BufferDesc.state` word. Panel (b) traces Clock-Sweep eviction skipping pinned frames (`F0`, `F3`) and decrementing `usage_count` until `F2` reaches $(0, 0)$. Panel (c) shows the 256 KB `BAS_BULKREAD` ring buffer isolating a 500 GB scan from hot OLTP pages.

`ItemIdData` slot).

- **Segment size (`RELSEG_SIZE`)**: 131,072 blocks = 1 GiB per physical file; max standard table ≈ 32 TiB.
- **Companion forks**: `_fsm` = 1 B/heap page in a binary max-heap tree; `_vm` = 2 bits/heap page (32,672 heap blocks ≈ 255 MiB per 8 KB VM block).
- **Tuple header**: 23 B struct → `t_hoff` = 24 B (with 0 nulls, or with nulls and ≤ 8 columns); +4 B slot + 8-byte `MAXALIGN` means **even a single-boolean row costs** $4 + 32 = 36$ **bytes** on a page.
- **TOAST**: Triggers at ≈ 2,000 B; short `varlena` uses 1 B header up to 126 B; out-of-line pointer is 18 B pointing to ≈ 1,996 B chunks in `pg_toast`.
- `shared_buffers`: 128 hash lock partitions; `usage_count` $\in \{0 \ldots 5\}$; `BAS_BULKREAD` sequential-scan ring = 32 frames = 256 KB.

2.6 Exercises

Exercise 2.1 (Segment File and Byte-Offset Arithmetic)**.** A production ledger table has `pg_class.oid = 41200` and `pg_class.relfilenode = 58910` inside database `oid = 16384`. It currently holds 420,000 heap blocks.

(a) List the exact filenames and byte sizes of all main-fork segment files on disk under `PGDATA/base/16384/`.

(b) An index lookup returns `ctid = (275000, 14)`. Which segment file does PostgreSQL open, and at what byte offset inside that file does the 8 KB page begin? At what byte offset inside

that page does the 4-byte slot for slot 14 sit?

(c) How many 8 KB blocks does the Visibility Map file `58910_vm` need to track all 420,000 heap blocks?

Exercise 2.2 (Tracing Slotted-Page Offsets and In-Page Compaction)**.** Start with a freshly initialized empty 8,192-byte heap page (`pd_lower` = 24, `pd_upper` = 8,192, `pd_special` = 8,192). Four tuples with unpadded lengths `lp_len` = $52, 75, 40, 61$ bytes are inserted in order as Slots 1, 2, 3, and 4.

(a) Accounting for 8-byte `MAXALIGN` when carving tuple space from `pd_upper`, give the exact values of `pd_lower`, `pd_upper`, contiguous free bytes, and each slot's `(lp_off, lp_flags, lp_len)` after all four inserts.

(b) Suppose Tuples 1 and 3 are deleted, their deleting transactions commit behind every active snapshot, and PostgreSQL runs `PageRepairFragmentation` (before index vacuuming). Give the new `pd_lower`, `pd_upper`, and all four slots' `(lp_off, lp_flags, lp_len)`.

(c) Next, `VACUUM` removes the index entries for Slots 1 and 3, marking those slots `LP_UNUSED` and setting `PD_HAS_FREE_LINES` in `pd_flags`. A fifth tuple of length `lp_len` = 46 bytes is now inserted. Which slot number does it receive, and what are the resulting `pd_lower` and `pd_upper`?

Exercise 2.3 (Null Bitmaps and the 9th-Column `t_hoff` Cliff)**.** Consider two schemas where every column is `SMALLINT` (2 bytes, 2-byte aligned): `t8` has 8 `SMALLINT` columns, and `t9` has 9 `SMALLINT` columns. Compute `t_hoff`, raw `lp_len`, and the `MAXALIGN`ed page footprint (including the 4-byte `ItemIdData` slot) for:

(a) A row in `t8` with zero `NULL`s versus a row in `t8` where columns 2 and 7 are `NULL`.

(b) A row in `t9` with zero `NULL`s versus a row in `t9` where column 9 alone is `NULL`. Explain why setting column 9 to `NULL` actually makes the row *8 bytes larger* on the page than storing a concrete integer in column 9.

Exercise 2.4 (Column Tetris on a 100-Million-Row Sensor Table)**.** An IoT team proposes the following schema for a 100-million-row table (all columns `NOT NULL`):

```
CREATE TABLE telemetry (
    online       BOOLEAN,
    device_id    BIGINT,
    firmware_ver SMALLINT,
    recorded_at  TIMESTAMPTZ,
    battery_pct  SMALLINT,
    reading_val  DOUBLE PRECISION,
    sensor_slot  BOOLEAN,
    seq_no       INTEGER,
    trace_uuid   UUID
);
```

(a) Compute the exact offset of every column, total internal + trailing padding bytes, `lp_len`, `MAXALIGN`ed tuple size, rows per 8 KB page, and total table size in GiB for 100,000,000 rows. (Watch `UUID`'s alignment carefully.)

(b) Write an optimal `CREATE TABLE telemetry_packed` ordering that minimizes tuple size without changing any column name or data type. Compute the new bytes saved per row, rows per 8 KB page, and total GiB saved across 100,000,000 rows.

Exercise 2.5 (Decoding Raw Hex Bytes from `pageinspect`)**.** On an x86-64 Linux PostgreSQL server, `heap_page_item_attrs` on a 3-column table `(id INT4, code TEXT, price_cents INT8)` returns a tuple with `t_hoff` = 24, `t_infomask` = `0x0903`, `t_bits` = `'10100000'`, and `t_attrs` = {''\x2a000000'', NULL, ''\x1027000000000000''}.

(a) Decode the exact SQL values of `id`, `code`, and `price_cents`, and explain every set bit of `t_infomask = 0x0903`.

(b) Compute the exact `lp_len` of this tuple, paying close attention to alignment padding before `price_cents` when `code` is `NULL`.

(c) If `code` is updated from `NULL` to `'EUR'`, write the exact hex bytes of `t_attrs[2]` (including its 1-byte `varlena` header) and compute the new `lp_len`.

Exercise 2.6 (TOAST Compression, Chunk Counts, and `EXTERNAL` Slicing)**.** A table `documents` with three `NOT NULL` columns (`doc_id BIGINT`, `title TEXT`, `body TEXT`) stores a row with a 20-byte ASCII `title` and a 120,000-byte `body`.

(a) Under default `EXTENDED` storage with `lz4`, `body` compresses by 3× to 40,000 bytes. How many rows are inserted into the `pg_toast` table (assuming `TOAST_MAX_CHUNK_SIZE` = 1,996 B), how many 8 KB TOAST heap pages do they occupy, and what is the exact `lp_len` of the main heap row in `documents`?

(b) Suppose a hot API endpoint runs `SELECT substring(body, 110000, 64) FROM documents WHERE doc_id = 42` ten thousand times per second. Compare how many TOAST chunks are fetched and decompressed per query under `EXTENDED` versus `EXTERNAL` storage. When does `EXTERNAL` win despite using 3× more disk space?

Exercise 2.7 (Clock-Sweep Hand Simulation)**.** A 5-frame toy buffer pool (`F0..F4`) starts with `nextVictimBuffer = F1` and states `(refcount, usage_count)`:

`F0:(0, 3)`, `F1:(1, 1)`, `F2:(0, 2)`, `F3:(0, 0)`, `F4:(0, 4)`.

(a) Backend A misses on block 10 and calls `StrategyGetBuffer`. Trace every tick of the clock hand, give the victim frame, the new state of all 5 frames (assuming Backend A keeps block 10 pinned for a moment), and the new position of `nextVictimBuffer`.

(b) Immediately afterward, Backend A unpins its buffer (`refcount` : $1 \to 0$) while Backend B unpins `F1` (`refcount` : $1 \to 0$), and Backend C misses on block 11. Trace `StrategyGetBuffer` again from the current `nextVictimBuffer` until the second victim is chosen, listing the final `(refcount, usage_count)` of every frame.

Exercise 2.8 (Sizing Ring Buffers vs. Cache Pollution)**.** A server has `shared_buffers` = 32 GiB. Two workloads run concurrently: 4,000 OLTP point lookups per second hitting a 20 GiB hot index/heap working set, and a batch extract-transform-load (ETL) pipeline running `Seq Scan` over a 2 GiB staging table `raw_imports` ten times per minute.

(a) Using PostgreSQL's `shared_buffers`/4 threshold for `BAS_BULKREAD`, determine whether `Seq Scan` on `raw_imports` uses the 256 KB ring buffer or allocates straight from the main pool, and explain why.

(b) Now suppose `raw_imports` grows from 2 GiB to 12 GiB. How does `Seq Scan`'s buffer allocation behavior suddenly change, and what happens to `EXPLAIN (ANALYZE, BUFFERS) shared hit` vs. `shared read` counts across consecutive scans of `raw_imports`?

Chapter 3

Indexes: Lehman-Yao B-Link Trees, GIN, GiST, and BRIN

At the end of Chapter 2, we left our rows inside 8,192-byte slotted heap pages. Every row lives at a 6-byte physical address `ctid` = (block, slot): a 32-bit page number paired with a 16-bit slot number. Hand PostgreSQL `ctid` = (8412, 9), and it pins page 8,412 in RAM, reads slot 9, and returns the row in microseconds.

The trouble is that SQL queries never hand you a `ctid`. An API request asks for every posting of `account_id = 4291804` since October 1, or the user matching `email = 'ada@lovelace.org'`. Because heap pages place new rows wherever the Free Space Map finds room, rows have no physical order on disk. Without a separate lookup structure mapping column values to `ctid`s, PostgreSQL has only one option: read page 0, check every row, read page 1, and keep scanning to the end of the file.

Focus

This chapter builds PostgreSQL's indexes from the ground up and shows how to pick the right index for each query shape:

- Why does a binary search tree stall on disk, and how does packing roughly 350 keys into one 8 KB B+tree page reach 100 million rows in three hops?
- How can hundreds of readers search a B-tree at full speed while writers are splitting pages right underneath them, without deadlocking?
- In a multi-column index, why does putting an equality column before a range column read one tight slice, while flipping the order scans thousands of cold pages?
- How can PostgreSQL answer a query from the index alone (*Index-Only Scan*) when index entries carry zero transaction visibility stamps?
- When does a B-tree hit a wall, and when should you reach for a partial index, expression index, Hash, GIN, GiST, or BRIN instead?

3.1 Why Secondary Indexes Exist and B+Tree Branching Math

Put concrete numbers on that full-table scan first. Suppose `postings` holds $N = 100{,}000{,}000$ (10^8) rows. Each row stores an ID, account ID, amount, timestamp, currency, and status—72 payload bytes plus the 24-byte row header, or 96 bytes. Adding the 4-byte slot pointer brings each row to 100 bytes. An 8,192-byte page has 8,168 usable bytes after its 24-byte page header, so each page packs $\lfloor 8168/100 \rfloor = 81$ rows.

Dividing 10^8 total rows by 81 rows per page gives the full footprint of the table on disk:

$$\left\lceil \frac{100{,}000{,}000}{81} \right\rceil = 1{,}234{,}568 \text{ pages} \times 8{,}192 \text{ bytes} \approx 10.11 \text{ GB}.$$

Even on a fast NVMe drive streaming at 2.5 GB/s, reading 10.11 GB for one lookup takes four seconds. At 500 lookups per second, the server would need 2,000 seconds of disk work every second.

3.1.1 Why Binary Search Trees Lose Badly on Block Devices

Your algorithms instinct says: build a balanced binary search tree over the keys. For $N = 10^8$, a red-black tree has height $\lceil \log_2(10^8) \rceil = 27$. Twenty-seven register comparisons take less than a microsecond. That is how Java's `TreeMap` works in RAM, where chasing an object reference costs nanoseconds.

On disk, comparisons are nearly free and *page misses* dominate everything. Storage drives and `shared_buffers` move data in 8,192-byte pages, not 32-byte nodes. As binary-tree nodes are allocated over months of random writes, parent and child nodes land on scattered pages. Even if the top 10 levels sit in RAM, walking the remaining 17 levels triggers 17 random 8 KB NVMe reads in sequence. At 70 μs per read, that stalls for 1.2 ms—hauling 8,160 wasted bytes across the bus just to read one 8-byte key and two child pointers.

Rudolf Bayer and Edward McCreight solved this in 1970 by sizing each tree node to match a full disk page. In a **B+tree**, each 8 KB internal page packs *hundreds* of separator keys and child page numbers, while all row pointers (`ctid`s) live at the bottom leaf level. Every step down the tree divides the remaining search space by roughly 350 instead of 2.

Example 3.1 (Exact Fanout Arithmetic on an 8 KB `nbtree` Page)**.** Let us count the bytes inside an 8,192-byte PostgreSQL B-tree (`nbtree`) internal page indexing an 8-byte `BIGINT` column such as `account_id`:

- **Usable space per page:** Subtracting the 24-byte page header at the top and a 16-byte sibling-link footer at the bottom (`pd_special` = 8176) leaves $8{,}192 - 40 = 8{,}152$ bytes.
- **Bytes per index entry:** Each entry takes a 4-byte slot pointer plus a 16-byte index record: an 8-byte entry header (6 bytes for the target block/slot plus 2 bytes of length/null flags) followed by the 8-byte `BIGINT` key. Because 16 is already a multiple of 8, no alignment padding is added. Each entry costs $4 + 16 = 20$ bytes.
- **Branching factor B:** Reserving one 20-byte slot for the page's upper-bound key leaves $\lfloor 8132/20 \rfloor = 406$ child links on a packed page. After splits (70–95% fill) or with slightly wider keys, an everyday internal page averages roughly $B \approx 350$ children.

Watch how fast powers of 350 grow as we step down from the root:

Tree Level	Pages	Size (8 KB/pg)	Where It Lives in Production
Level 2 (Root page)	1	8 KB	Permanently hot in `shared_buffers`
Level 1 (Internal)	350	2.73 MB	Permanently hot in `shared_buffers`
Level 0 (Leaves, 350^2)	122,500	957.03 MB	Hot subset in RAM; cold on NVMe

By default, PostgreSQL leaves 10% free room on new leaf pages (`fillfactor = 90`) to absorb future inserts. Each leaf page holds $\lfloor 0.90 \times 8132/20 \rfloor = 365$ `(account_id, ctid)` pairs. Multiply 122,500 leaves by 365 pairs: **three index levels cover** 44.7 **million rows.** Add just one more internal level (350^3 leaves, with all non-leaf pages totaling $\approx$ 960 MB in RAM) and four levels cover **15.6 billion rows**.

Pause on the RAM column of Example 3.1. To find one row out of 44.7 million:

1. Pin the root page (in RAM, 0 disk reads) and binary-search 350 keys (9 comparisons) to pick 1 of 350 level-1 pages.
2. Pin that level-1 page (part of a 2.73 MB slice that stays in RAM, 0 disk reads) and run 9 comparisons to pick 1 of 122,500 leaf pages.
3. Pin the leaf page (at most 1 NVMe read if cold) and binary-search 365 entries to get `ctid` = $(8412, 9)$.
4. Pin table page 8,412 (at most 1 NVMe read if cold) and read slot 9.

Instead of reading 1,234,568 pages (10.11 GB), PostgreSQL pins **4 buffers** and does **0 to 2 NVMe reads** (16 KB worst case)—a 630,000× drop in I/O.

3.1.2 Inside an 8 KB `nbtree` Page: `IndexTupleData` and `BTPageOpaqueData`

PostgreSQL reuses Chapter 2's 8,192-byte slotted page layout for indexes too: the 24-byte page header sits at byte 0, and 4-byte slot pointers grow downward. Only two parts differ from a table page. First, each index entry starts with a tiny 8-byte header instead of a 23-byte row header. Second, the last 16 bytes of the page (`8176..8192`) hold left/right sibling page numbers and the page's tree level. In C (`src/include/access/itup.h` and `nbtree.h`), those two structs are `IndexTupleData` and `BTPageOpaqueData`:

```
/* 8-byte header prepended to every index entry (src/include/access/itup.h) */
typedef struct IndexTupleData {
    ItemPointerData t_tid;   /* 6 bytes: heap ctid on leaf, or child blkno on internal */
    unsigned short  t_info;  /* 2 bytes: bit 15 = has nulls, bit 14 = has varlena,
                                         bits 12..0 = tuple byte length (max 8191) */
} IndexTupleData;            /* Key column bytes follow immediately after t_info */

/* 16-byte trailer at offset 8176..8192 of every nbtree page (nbtree.h) */
typedef struct BTPageOpaqueData {
    BlockNumber btpo_prev;   /* 4 bytes: left sibling block (P_NONE = 0 if leftmost) */
    BlockNumber btpo_next;   /* 4 bytes: right sibling block (P_NONE = 0 if rightmost) */
    uint32      btpo_level;  /* 4 bytes: tree level (0 = leaf, >= 1 = internal;
                                         reused for xact epoch on deleted pages) */
    uint16      btpo_flags;  /* 2 bytes: BTP_LEAF, BTP_ROOT, BTP_META,
                                         BTP_DELETED, BTP_HALF_DEAD, BTP_INCOMPLETE_SPLIT
    */
```

```
    BTCycleId   btpo_cycleid;/* 2 bytes: vacuum cycle ID preventing split-scan races */
} BTPageOpaqueData;
```

Three details in those 24 bytes of C declarations matter:

- **Dual use of `t_tid`:** On a leaf page (`btpo_level` $== 0$), `t_tid` holds the 6-byte table `ctid` (block, slot). On an internal branch page (`btpo_level` ≥ 1), its 4-byte block field holds the *child index page number*, while its 2-byte offset field packs status bits and column counts.
- **Dynamic root via Block 0's metapage:** Block 0 of every `nbtree` file is a small metadata page (`BTP_META`) storing the current root block number (`btm_root`) and tree height. When a single-leaf index splits, PostgreSQL allocates a new root block and updates `btm_root`. Sessions cache that root block number locally so normal lookups skip Block 0 completely.
- **Doubly-linked sibling chains (`btpo_prev` and `btpo_next`):** Every level—especially leaf level 0—forms a doubly-linked horizontal ribbon of sorted pages. Once a range scan lands on its starting key, it *never climbs back up to parent pages*. Forward scans (`ASC`) walk right via `btpo_next`; backward scans (`DESC`) walk left via `btpo_prev`.

Intuition

Why B+trees keep table pointers strictly at the leaves. In Bayer's original 1970 B-tree, internal nodes carried row pointers too. A B+tree turns internal nodes into pure *road signs* (separator keys + child block numbers), putting every key-to-`ctid` pair in the leaf chain. That pays off twice: internal nodes stay narrow so fanout B stays huge, and a 5,000-row range scan walks straight across neighboring leaves instead of zigzagging up and down the tree.

3.2 From Textbook B+Trees to PostgreSQL's Lehman–Yao B-Link Tree

A textbook B+tree flies on one thread. Now run 128 concurrent worker processes inserting orders and reading balances at 80,000 transactions per second. Under a textbook locking rule, all 128 cores quickly pile up waiting on a single 64-byte cache line: the page lock on the root node.

3.2.1 Why Naive Top-Down Latch Crabbing Limits Multi-Core Scaling

First see what breaks if two sessions touch a B+tree with no coordination at all:

1. **Reader vs. concurrent leaf split:** Reader Q checks parent R, sees that keys in $(40, 100]$ live on leaf P (block 17), and pauses 2 μs before pinning block 17. In those 2 μs, Writer W fills block 17 and splits it into block 17 covering $(40, 60]$ and new block 41 covering $(60, 100]$, moving $\{71, 82, 85, 94\}$ to block 41. When Reader Q wakes up and checks block 17 for $k = 85$, key 85 is gone. A naive reader falsely reports that committed account 85 does not exist.
2. **Cascading splits upward:** When Writer W inserts into a full leaf P, splitting P adds a separator key to parent R. If R is also full, R splits too, cascading all the way up to the root.

Textbooks prevent this with **latch crabbing** (hand-over-hand locking):

- A *reader* locks parent R, locks child P, and *only then* unlocks R.
- A *writer* holds exclusive locks down its ancestor chain until it reaches a node with spare room (so a lower split cannot cascade past it). Even optimistic variants force writers to re-walk from the root under exclusive locks whenever internal nodes fill up.

Holding two locks at every step hurts multi-core scaling. Even shared read locks execute atomic increments on the root buffer's header, bouncing that cache line across CPU sockets twice per lookup. Worse, holding a parent lock while waiting on an NVMe read stalls every writer under that parent.

In Java 6 terms, latch crabbing is nested `synchronized (parent) { synchronized (child) { ... } }`. Java 6's `ConcurrentSkipListMap` takes the smarter path: readers walk without locks, and writers publish new links atomically so an in-flight reader can always follow the chain rightward. Philip Lehman and S. Bing Yao brought that exact idea to disk B-trees in 1981.

3.2.2 The Lehman–Yao Insight: `P_HIKEY` and Right-Links (`btpo_next`)

Lehman and Yao's **B-link tree** (extended to page deletion by Lanin and Shasha in 1986) powers PostgreSQL's `nbtree` (`src/backend/access/nbtree/README`).

They asked one question: *why did Reader Q get lost when Leaf P split?* Block 17 was missing two local clues:

1. Looking only at block 17, Q could not tell that key 85 *used* to belong here and had moved right.
2. Even if Q knew 85 moved right, Q had no direct pointer from block 17 to new block 41 while parent R was still un-updated.

The B-link tree fixes both by storing an explicit upper-bound key and a right-neighbor link on every page:

Definition 3.1 (Lehman–Yao B-Link Page Invariants in PostgreSQL)**.** Every live `nbtree` page P (leaf or internal) maintains two local fields inside its 8 KB block:

1. **Right-sibling link (`btpo_next`):** Every non-rightmost page stores the block number of its right neighbor at the same level. When P splits into P and P', P.`btpo_next` points to P' *before* P is unlocked.
2. **Explicit High Key (`P_HIKEY`, slot 1):** On every non-rightmost page, slot 1 stores an upper-bound key K_{high} rather than data. Every key on P satisfies $k \le K_{\text{high}}$. Any lookup arriving at P with $k > K_{\text{high}}$ knows P split and steps right via `btpo_next`. Real data entries start at slot 2 (or slot 1 on the rightmost page, whose upper bound is $+\infty$).

History

From Berkeley Postgres to Lehman–Yao. Early university Postgres used simpler B-tree locking that bottlenecked under multi-user load. Paul Aoki and Mike Olson replaced it with the Lehman–Yao `nbtree` ("New B-Tree") in the early 1990s. Tom Lane later added Lanin–Shasha page deletion, and Peter Geoghegan added `ctid` tie-breaking, suffix truncation, deduplication, and bottom-up deletion across PostgreSQL 12–14.

3.2.3 Step-by-Step Trace: A Concurrent Reader Crossing an Unfinished Split

Let us trace every lock and pointer step with small numbers so you see how a reader crosses an unfinished split safely.

Suppose parent R (block 4) routes keys in $(40, 100]$ to leaf P (block 17) and keys > 100 to block 29. Leaf P has `P_HIKEY` $= 100$ at slot 1, data keys $\{42, 49, 55, 60, 71, 82, 94\}$ in slots 2..8, and `btpo_next` $=$ 29.

Writer W arrives to insert $k = 85$ (`ctid` $= (912, 6)$), and Leaf P is full:

1. **Phase 1 of Split (Leaf Only):** Writer W locks P (block 17) exclusively and allocates a fresh page P' (block 41), locking block 41 exclusively too. Notice W holds **zero locks on parent** R. Splitting at 60, W rewrites both leaves in RAM:
   - Right sibling P' (block 41) gets the old high key (`P_HIKEY` $= 100$), upper keys $\{71, 82, 85, 94\}$, `btpo_prev` $= 17$, and `btpo_next` $= 29$.
   - Original leaf P (block 17) keeps $\{42, 49, 55, 60\}$, lowers its high key to `P_HIKEY` $= 60$, sets `btpo_next` $= 41$, and sets `BTP_INCOMPLETE_SPLIT` as a reminder that parent R still needs updating.

   W logs the split to WAL and **unlocks both block 17 and block 41**.
2. **The Half-Split Window (Reader Q Descends):** At this exact microsecond, the OS pauses W before it touches parent R. Reader Q runs `SELECT * FROM accounts WHERE account_id = 85`:
   - Q takes a shared lock on parent R (block 4), sees $40 < 85 \leq 100 \implies$ block 17, and **unlocks block 4 completely** before locking block 17. Between those two steps, Q holds **zero page locks anywhere in the tree**.
3. **Stepping Right (`_bt_moveright`):** Reader Q checks block 17's slot 1 (`P_HIKEY`) before searching data slots:

   $$k = 85 \ > \ P.\texttt{P_HIKEY} = 60 \quad \implies \quad \text{Block 17 split after we read its parent.}$$

   Without touching parent R, Q follows `btpo_next` $= 41$ rightward (locking block 41 shared, then unlocking block 17). On block 41, $85 \leq 100$ holds, so binary search finds $(85, \texttt{ctid} = (912, 6))$ at slot 4.
4. **Phase 2 of Split (Parent Update):** Writer W wakes up, locks parent R (and briefly block 17 to clear `BTP_INCOMPLETE_SPLIT`), inserts downlink ($60 \rightarrow$ blk 41) into R, and unlocks. Future readers seeking 85 jump straight from block 4 to block 41.

Why it works

Why Lehman–Yao cannot deadlock (even if the writer crashes mid-split). Notice the lock order across the entire tree:

- Descending readers hold *at most one page latch at a time*: they release the parent latch before locking the child. A thread holding zero locks while requesting one cannot join a wait cycle.
- Horizontal `_bt_moveright` steps lock pages strictly *left-to-right* (Blk 17 $\rightarrow$ Blk 41). Split-

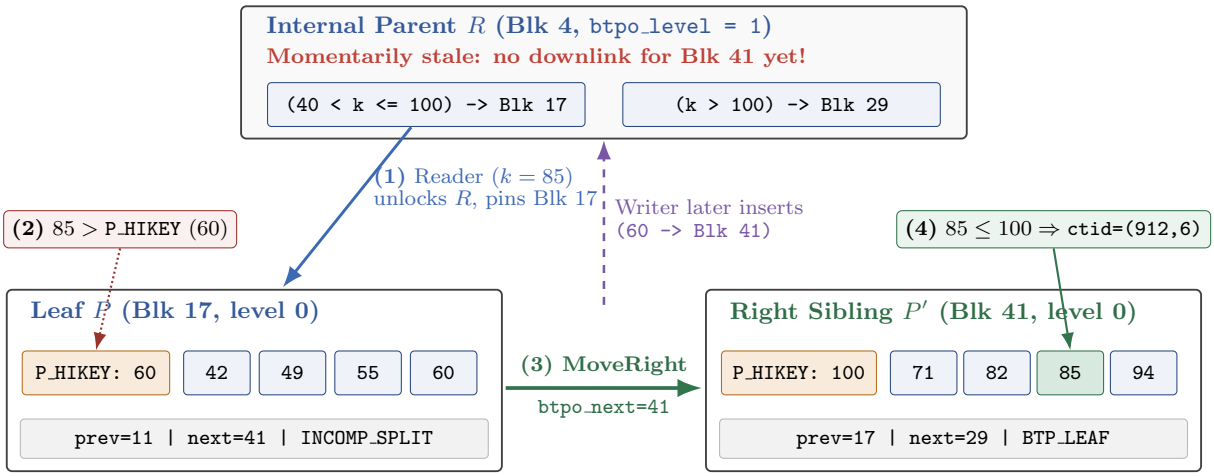


Figure 3.1: Lehman–Yao B-link split in action. Writer splits Leaf P (Block 17) into P and P' (Block 41), installs lowered `P_HIKEY = 60` and `btpo_next = 41`, and unlocks both leaves *before* updating Parent R. A concurrent reader seeking $k = 85$ follows the stale parent pointer to Block 17 without holding two locks during descent, sees $85 > 60$, and steps right via `_bt_moveright` to find 85 on Block 41.

ting writers also lock siblings left-to-right and climb strictly *bottom-up*: they release leaf locks before locking the parent. Because held latches form a strict directed acyclic graph, page-latch deadlocks cannot occur.

- Even if power fails right before the parent receives Block 41's pointer, the index stays 100% accurate. Lookups for $(60, 100]$ simply take one extra `_bt_moveright` hop from Block 17 to Block 41. Later, the next split on that parent sees `BTP_INCOMPLETE_SPLIT` and inserts the missing downlink.

Pause

Pause and check the two split tricks. Every non-rightmost index page stores its own upper-bound key (`P_HIKEY`) plus a right-sibling link (`btpo_next`). If a concurrent split moved keys away while you were stepping down from the parent, `key > P_HIKEY` tells you immediately, and `btpo_next` walks you right to the moved keys.

3.2.4 Modern `nbtree` Engineering: Tie-Breaking, Deduplication, and Bottom-Up Deletion

Classical B-link trees struggled with three common OLTP patterns until PostgreSQL 12, 13, and 14 fixed each one:

1. **Unique keys via heap `ctid` tie-breaking (PG 12):** What happens when 2,000,000 rows share `status = 'PENDING'`? Older versions stored duplicates in arbitrary physical order across thousands of leaf pages, turning inserts into slow scans. Since PostgreSQL 12, `nbtree` uses the 6-byte row `ctid` as an implicit final tie-breaker. Every entry $(k, \texttt{ctid})$ is now globally unique, so binary search lands straight on the right leaf page in $O(\log N)$ time.

2. **Posting-list deduplication (PG 13):** Fast duplicates were still wide: storing `'USD'` 365 times on one page repeated the key and header 365 times. With `deduplicate_items = on` (default since PG 13), a full leaf page runs a compression pass before splitting. Identical keys merge into one **posting list tuple** (`BT_IS_POSTING`): one copy of `'USD'` followed by a packed array of 6-byte `ctid`s. That cuts status and foreign-key indexes by 3× to 4×.

3. **Bottom-up index deletion (PG 14):** Remember from Chapter 2 that updating a row writes a new version at a new `ctid`. Whenever *any* indexed column changes, PostgreSQL cannot use a same-page HOT chain; it must insert a new entry into *every* index on the table. Before PG 14, updating `last_seen_at` 50 times pushed 50 dead duplicate entries into `idx_accounts_email`, splitting leaf pages before `autovacuum` arrived. PostgreSQL 14 added **bottom-up index deletion**: right before a duplicate-driven split, `nbtree` checks the heap pages for those duplicates, reclaims dead slots in place, and cancels the split.

Try it in PostgreSQL

Inspect a real `nbtree` metapage, `P_HIKEY` slot, and PG 13+ deduplicated posting list tuple using PostgreSQL's built-in `pageinspect` extension:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;

CREATE TABLE ledger_demo (
    posting_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    currency   CHAR(3) NOT NULL,
    amount     BIGINT  NOT NULL
);

INSERT INTO ledger_demo (currency, amount)
SELECT CASE WHEN mod(g, 5) = 0 THEN 'EUR' ELSE 'USD' END, g * 100
FROM generate_series(1, 5000) AS g;

CREATE INDEX idx_ledger_currency ON ledger_demo (currency);

-- 1. Inspect Block 0 (the metapage) to find root block & tree height
SELECT version, root, level, fastroot, fastlevel
FROM bt_metap('idx_ledger_currency');
--  version | root | level | fastroot | fastlevel
-- ---------+------+-------+----------+-----------
--        4 |    3 |     1 |        3 |         1

-- 2. Inspect Leaf Block 1: slot 1 is P_HIKEY; slots 2+ are posting lists!
SELECT itemoffset AS off, ctid, itemlen, htid, tids[1:2] AS sample_tids, data
FROM bt_page_items('idx_ledger_currency', 1)
LIMIT 4;
--  off |  ctid   | itemlen |  htid  |     sample_tids     |    data
-- -----+---------+---------+--------+---------------------+------------
--    1 | (0,1)   |      16 | (5,42) | NULL                | 55 53 44 20
--    2 | (16,42) |     808 | (0,5)  | {"(0,5)","(0,10)"}  | 45 55 52 20
--    3 | (16,42) |     808 | (1,28) | {"(1,28)","(1,33)"} | 45 55 52 20
--    4 | (16,22) |     224 | (3,11) | {"(3,11)","(3,16)"} | 45 55 52 20
```

Look at `off = 1`: `tids` is `NULL` because slot 1 on non-rightmost leaf Block 1 is `P_HIKEY` (`'USD'` with tie-breaker `htid` = (5, 42), ASCII hex `55 53 44`). Look at `off = 2`: a single 808-byte

posting list tuple packs **132 heap ctids** for `'EUR'` (`45 55 52`) at just $808/132 \approx 6.12$ bytes per row.

3.3 Composite Index Lexicographic Geometry and Column Order

Single-column B-trees are straightforward because numbers and timestamps already live on a 1D line. Real production queries, however, almost always filter on two or three dimensions at once:

```
SELECT posting_id, amount_cents, created_at
FROM postings
WHERE account_id = 42
  AND created_at >= '2026-10-01 00:00:00+00'
ORDER BY created_at DESC
LIMIT 20;
```

A B-tree leaf chain is strictly one-dimensional: every entry sits before or after its neighbor along a single ribbon of pages linked by `btpo_next`. How do we map a 2D plane of (`account_id`, `created_at`) pairs onto that 1D ribbon? And why does `(account_id, created_at DESC)` finish in 0.08 ms while `(created_at DESC, account_id)` on the same two columns takes 450 ms?

3.3.1 The Phonebook Ribbon: Equality Pins a Slice, Range Scatters Jumps

Suppose we build:

```
CREATE INDEX idx_postings_acct_created
    ON postings (account_id ASC, created_at DESC);
```

PostgreSQL compares two composite keys (a_1, t_1) and (a_2, t_2) in **lexicographic (dictionary) order**, just like sorting a phonebook by (Last Name, First Name):

$$(a_1, t_1) \prec (a_2, t_2) \quad \Longleftrightarrow \quad (a_1 < a_2) \ \vee \ \big(a_1 = a_2 \ \wedge \ t_1 > t_2\big),$$

where the second inequality flips to $t_1 > t_2$ because we specified `created_at DESC`. In Java 6 this is exactly the `Comparator` you would hand to `Collections.sort` or to a `TreeMap`:

```
public int compare(Posting x, Posting y) {
    if (x.accountId != y.accountId)
        return x.accountId < y.accountId ? -1 : 1;  // ascending
    return y.createdAt.compareTo(x.createdAt);      // descending
}
```

The B-tree keeps its leaf entries in this order, so everything you know about a sorted `TreeMap` (cheap range scans with `subMap`, no help for a lookup on the second field alone) carries over directly.

Picture the resulting leaf pages stretched left-to-right across the floor. First come *all* postings for `account_id = 1`, sorted newest-to-oldest from 2026 back to 2019. Immediately after account 1's oldest 2019 posting sits account 2's newest 2026 posting, and so on.

Now trace two queries across that exact same leaf ribbon (illustrated side by side in Figure 3.2):

1. **Equality first, range second (`account_id = 42 AND created_at >= '2026-10-01'`):** Fixing `account_id` $= 42$ narrows the outer dimension to one contiguous slice of the ribbon. Inside that slice, `created_at` runs strictly newest to oldest. PostgreSQL seeks straight to $(42, +\infty)$ and reads consecutive entries while `created_at` $\geq$ '2026-10-01'. The instant it hits the first entry older than October 1, it **stops**. Every index entry it touched matched both conditions.

2. **Range first, filter second (`account_id BETWEEN 41 AND 43 AND created_at >= '2026-10-25'`):** Because the leading column spans a range, October 2026 timestamps are no longer adjacent on the ribbon. Account 41's late-October entries appear first, followed by *seven years of older postings for account 41* (`2026-10-24` down to `2019-01-02`), before account 42's late-October entries finally start. A range scan from $(41, +\infty)$ to $(43, \text{'2026-10-25'})$ must walk across thousands of cold historical leaf pages in between, checking `created_at` $\geq$ '2026-10-25' on every skipped entry.

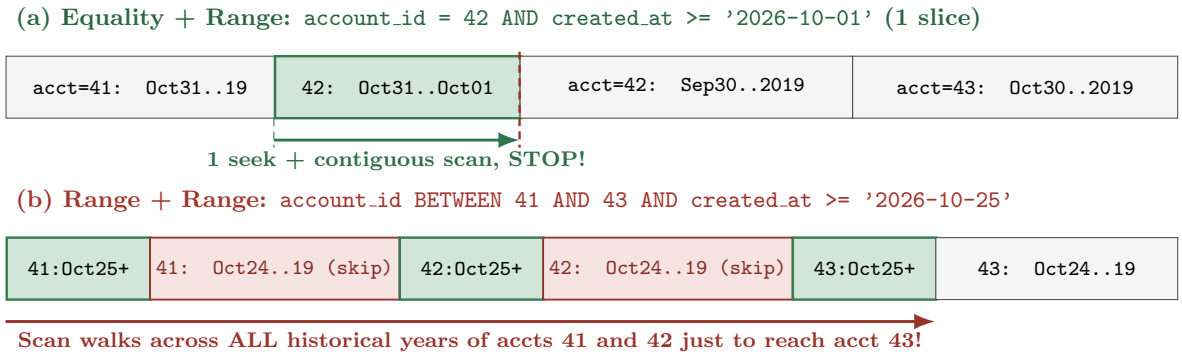


Figure 3.2: Lexicographic ribbon geometry on `(account_id ASC, created_at DESC)`. In (a), fixing `account_id = 42` isolates one sub-ribbon where `created_at` is contiguous. In (b), a leading range scatters recent timestamps into disconnected islands separated by years of cold index entries.

Common trap

`IN (...)` lists vs. true range inequalities. PostgreSQL's B-tree executor has a shortcut for small discrete lists like `WHERE account_id IN (41, 42, 43) AND created_at >= '2026-10-25'`. Instead of trudging through seven years of account 41's history, `_bt_advance_array_keys()` sees the end of $(41, \text{'2026-10-25'})$ and jumps straight to $(42, +\infty)$. Once your leading filter is a continuous inequality (`amount_cents > 50000` or `created_at >= ...`), however, there is no short list of keys to jump across. Follow this column-ordering rule for composite B-trees:

1. Equality (=) $\longrightarrow$ 2. Range / `ORDER BY` $\longrightarrow$ 3. `INCLUDE` payload.

Ignore the folk rule "always put the highest-cardinality column first." If your query writes `WHERE tenant_id = p_tenant AND status = 'OPEN' AND created_at >= p_since`, both `tenant_id` and `status` are equality filters. Both belong ahead of the range column `created_at`, even if `status` has only four distinct values.

3.3.2 Sort Elimination and Backward Leaf Scans

Column order also decides whether PostgreSQL can stream `ORDER BY ... LIMIT` k in $O(k)$ page reads without allocating a `Sort` buffer. Look at our opening query:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT posting_id, amount_cents, created_at
FROM postings
WHERE account_id = 42
  AND created_at >= '2026-10-01 00:00:00+00'
ORDER BY created_at DESC
LIMIT 20;
-- Limit  (cost=0.56..28.41 rows=20 width=24) (actual time=0.031..0.068 rows=20 loops=1)
--   Buffers: shared hit=22
--   -> Index Scan using idx_postings_acct_created on postings
--        Index Cond: ((account_id = 42) AND (created_at >= '2026-10-01 00:00:00+00'::
    timestamptz))
```

Notice what is *missing* from that plan: there is **no `Sort` node**. Even if account 42 has 5,000,000 postings in October, PostgreSQL seeks to $(42, +\infty)$, emits the first 20 live rows from the ribbon, and stops after 22 buffer hits in 68 μs.

What if an analyst asks for `ORDER BY account_id DESC, created_at ASC` against our `(account_id ASC, created_at DESC)` index? Because *every* column's direction flipped at once, the requested order is the exact mirror image of our leaf ribbon. PostgreSQL runs an `Index Scan Backward`, walking `btpo_prev` links right-to-left with zero sorting. By contrast, asking for `ORDER BY account_id ASC, created_at ASC` flips only the second column. Neither forward nor backward walks produce that order, so the planner must add a `Sort` (or PG 13+'s `Incremental Sort`, which sorts one `account_id` group at a time).

3.4 Covering Indexes (`INCLUDE`), the Visibility Map, and True Index-Only Scans

Look again at the `EXPLAIN (ANALYZE, BUFFERS)` plan at the end of Section 3.3: returning 20 rows from a 3-level B-tree touched **22 buffers**. Where did those 22 hits come from? Three were index pages (root, level 1, leaf), and 19 were separate 8 KB table pages. Even when 20 index entries sit side by side on one leaf page, their row `ctid`s point to 19 historical blocks scattered across a 10 GB table. If those blocks are cold on NVMe, summing `SUM(amount_cents)` across 10,000 postings triggers up to 10,000 random 8 KB disk reads ($\approx$ 700 ms).

That invites a natural question: *if we store `amount_cents` and `currency` right inside the leaf entry next to `(account_id, created_at)`, can PostgreSQL answer the query from the index alone?*

3.4.1 Why PostgreSQL Index Tuples Carry Zero MVCC Visibility Metadata

Engines built around **clustered primary indexes** (like MySQL's InnoDB or SQL Server) make the primary-key B+tree *be* the table. Leaf pages hold full rows plus transaction IDs, while secondary indexes point to the primary key.

PostgreSQL made the opposite bet: tables are unclustered slotted heaps addressed by 6-byte `ctid`s. By design, **every index entry (`IndexTupleData`) carries zero transaction visibility fields (`t_xmin`, `t_xmax`, `t_cid`)**. As we counted in Section 3.1, an index entry stores only a 6-byte target pointer (`t_tid`), 2 bytes of length and null flags (`t_info`), and the indexed column bytes.

Intuition

Why omit `xmin` and `xmax` from every index? Count the write cost if indexes stored transaction headers. Suppose your `orders` table has 6 secondary indexes. When a session runs `DELETE FROM orders WHERE order_id = 991`, heap-only MVCC stamps `t_xmax` on *one* 8 KB table page—zero index pages are dirtied at `DELETE` time. If every index entry carried its own `t_xmax`, every `DELETE` and update would have to descend and dirty all 6 indexes synchronously, multiplying random write I/O and WAL volume by 6×.

Omitting `xmin` and `xmax` from indexes keeps writes fast, but it creates a read puzzle. When an index scan sees `(account_id=42, amount_cents=12500, ctid=(8412, 9))`, how does it know whether row `(8412, 9)` committed five days ago? What if it was inserted two microseconds ago by an uncommitted transaction, or already deleted by another session?

Before PostgreSQL 9.2, the answer was blunt: *it didn't*, so PostgreSQL had to visit the table page on every index match without exception.

3.4.2 How the Visibility Map (`_vm`) Rescues Index-Only Scans

PostgreSQL 9.2 solved that puzzle without adding a single byte to `IndexTupleData`. Instead, it enlisted the **Visibility Map (`_vm`)** fork we met in Chapter 2.

Inside `PGDATA/base/<dbOid>/<relfilenode>_vm`, PostgreSQL keeps just **2 bits per 8 KB table page**:

- Bit 0 (`VISIBILITYMAP_ALL_VISIBLE`): when set to 1, `VACUUM` has verified that *every live row on that 8 KB block is committed and visible to all current and future snapshots.* Any subsequent insert, update, or delete on that block clears bit 0 back to 0.
- Bit 1 (`VISIBILITYMAP_ALL_FROZEN`): tells aggressive `VACUUM` it can skip already-frozen pages during transaction-ID wraparound maintenance (Chapter 5).

Check the compression ratio of that bitmap. Two bits per 8,192 bytes means 1 byte of `_vm` covers 4 table pages (32 KB), and one 8 KB `_vm` page tracks ≈ 32,640 table blocks (255 MB). For our entire 10.11 GB `postings` table (1,234,568 pages), the whole `_vm` file takes just $\lceil 1{,}234{,}568/4 \rceil \approx$ **308** KB—38 pages pinned permanently in `shared_buffers`.

When every column a query needs sits inside the index leaf entry, PostgreSQL's `IndexOnlyScan` runs a two-path check for each match `ctid` = (blk, off) (Figure 3.3):

1. Check bit 0 for `blk` inside the cached `_vm` page.
2. **Fast path (`ALL_VISIBLE` == 1):** Every row on block `blk` is guaranteed visible. Emit the columns straight from the leaf entry and move on. **Zero table page pins, zero NVMe reads.**
3. **Fallback path (`ALL_VISIBLE` == 0):** A recent write touched block `blk` and cleared its bit.

Pin table block `blk`, check slot `off`'s visibility headers against the query snapshot, emit the row if visible, and bump the `Heap Fetches` counter in `EXPLAIN (ANALYZE, BUFFERS)`.

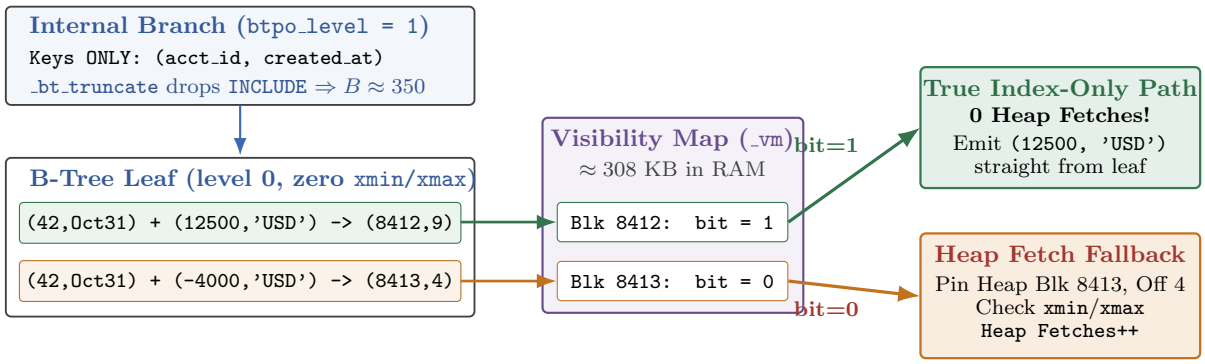


Figure 3.3: Covering index (`INCLUDE`) and Visibility Map (`_vm`) execution pipeline. Internal branch nodes drop `INCLUDE` attributes via `_bt_truncate` to stay narrow. At the leaf level, the executor probes the cached 2-bit-per-page `_vm` bitmap: bit 1 emits payload straight from the index with zero heap I/O; bit 0 falls back to pinning the heap block to verify MVCC visibility.

3.4.3 Why `INCLUDE` Beats Appending Payload Columns to the Key

Since PostgreSQL 11, you create a covering index using the `INCLUDE` clause:

```
CREATE INDEX idx_postings_acct_created_cov
    ON postings (account_id ASC, created_at DESC)
    INCLUDE (amount_cents, currency);
```

Why write `INCLUDE (amount_cents, currency)` instead of putting all four columns inside the key list `(account_id, created_at, amount_cents, currency)`? Two concrete wins separate them:

1. **Suffix truncation keeps internal pages narrow (`_bt_truncate`):** When a leaf splits, PostgreSQL promotes a separator key into the parent page. On an `INCLUDE` index, `_bt_truncate()` strips *all* non-key `INCLUDE` columns (plus any trailing key columns not needed to separate the two halves) before writing the parent entry. Upper branch pages store only `(account_id, created_at)`, keeping $B \approx 350$ even if you `INCLUDE` a 64-byte description.

2. **Clean uniqueness constraints:** Suppose `users.email` must be unique and login runs `SELECT user_id, pw_hash FROM users WHERE email = p_email`. Writing `CREATE UNIQUE INDEX ON users (email, user_id, pw_hash)` breaks your constraint: two users could register the same `email` with different `user_id`s. Writing `CREATE UNIQUE INDEX ON users (email) INCLUDE (user_id, pw_hash)` enforces uniqueness strictly on `email` while making login a single-leaf Index-Only Scan.

Common trap

When "Index Only Scan" in `EXPLAIN` is a mirage. Never stop reading `EXPLAIN` at the title `Index Only Scan`. Always run `EXPLAIN (ANALYZE, BUFFERS)` and check the line right below it: `Heap Fetches`. By default, `autovacuum_vacuum_scale_factor = 0.20` waits until 20% of a table changes before running `VACUUM`. On a 100-million-row table, millions

of recently written blocks sit with `ALL_VISIBLE = 0`. Your plan prints `Index Only Scan`, yet `Heap Fetches: 48219` reveals it read 48,219 table blocks anyway. Lowering per-table `autovacuum_vacuum_scale_factor = 0.01` (or `autovacuum_vacuum_insert_scale_factor = 0.005` on append-heavy ledgers) keeps `_vm` bits set and drives `Heap Fetches` back to 0.

3.5 Partial Indexes (`WHERE ...`) and Expression Indexes

Every index we have built so far paid storage, `shared_buffers` RAM, and write-ahead log tax for 100% of the rows in the table. Yet many of the hottest queries in production care about a tiny, predictable fraction of rows.

3.5.1 Partial Indexes: Shrinking a 4 GB Queue Index to 2 MB

Consider a transactional outbox or webhook job table `jobs` holding 100,000,000 historical delivery rows. At any moment, 99,950,000 rows (99.95%) have `status = 'processed'`, while only 50,000 rows (0.05%) have `status = 'pending'`. Every millisecond, worker threads poll for the oldest ready jobs:

```
SELECT job_id, payload
FROM jobs
WHERE status = 'pending' AND run_after <= NOW()
ORDER BY run_after ASC
LIMIT 50
FOR UPDATE SKIP LOCKED;
```

Compare a full composite index against a **partial index** carrying a `WHERE` filter:

```
-- Naive full index: indexes all 100,000,000 rows (~3.8 GB on disk!)
CREATE INDEX idx_jobs_full ON jobs (status, run_after);

-- Partial index: indexes ONLY the 50,000 pending rows (~1.9 MB on disk!)
CREATE INDEX idx_jobs_pending ON jobs (run_after ASC)
    WHERE status = 'pending';
```

Count the wins of `idx_jobs_pending`:

- 2,000× **smaller RAM footprint:** 50,000 rows at ∼ 28 bytes each (plus churn slack) fit in ≈ 240 pages (∼ 1.9 MB). That whole index stays warm in L3 cache and `shared_buffers` instead of evicting 3.8 GB of table pages. Notice we omitted `status` from the key list `(run_after ASC)`: every entry in the index already satisfies `status = 'pending'`.
- **Zero maintenance on excluded rows:** When an audit job updates a `'processed'` row, neither old nor new row matches `status = 'pending'`. PostgreSQL skips `idx_jobs_pending` completely and preserves same-page HOT updates.
- **Conditional uniqueness:** Partial unique indexes express rules that plain SQL `UNIQUE` cannot. The index below lets a customer save many cards (`is_primary = FALSE`) while guaranteeing at most one primary card:

```
CREATE UNIQUE INDEX uq_one_primary_card
    ON payment_methods (customer_id)
 WHERE is_primary = TRUE;
-- A second primary card for customer 42 fails with:
-- ERROR:  duplicate key value violates unique constraint "uq_one_primary_card"
-- DETAIL:  Key (customer_id)=(42) already exists.
```

One rule governs planner matching: **the query's `WHERE` clause must logically imply the index's `WHERE` filter** at plan time. A query writing `WHERE status = 'pending'` matches immediately. A generic prepared plan for `WHERE status = p_status` cannot use `idx_jobs_pending` because `p_status` could be `'processed'` at runtime; PostgreSQL 12+'s custom-plan mode can use it once the literal `'pending'` is bound.

3.5.2 Expression Indexes and Syntactic Predicate Alignment

What if login runs case-insensitive lookups or filters a scalar field inside a `JSONB` webhook payload?

```
-- A plain index on (email) CANNOT satisfy LOWER(email) = ...!
CREATE UNIQUE INDEX idx_users_email_lower ON users (LOWER(email));

-- Indexing a scalar TEXT value extracted from a JSONB column
-- Note the mandatory double parentheses around non-function expressions!
CREATE INDEX idx_webhooks_merchant ON webhooks ((payload->>'merchant_id'));
```

In an **expression index**, PostgreSQL evaluates the function during `INSERT`/`UPDATE` and stores the *result* inside the index entry. Thus `idx_users_email_lower` stores `'ada@lovelace.org'` even when the table row holds `'Ada@Lovelace.org'`. Better yet, `ANALYZE` collects dedicated histograms on the expression itself in `pg_stats`, giving the planner accurate row estimates for `(payload->>'merchant_id')`.

Two strict rules bite teams who deploy expression indexes carelessly:

1. **Expressions must be `IMMUTABLE`:** Given identical input bytes, the function must return the exact same output forever, independent of session settings or wall-clock time. Indexing `DATE_TRUNC('day', created_at)` on a `TIMESTAMPTZ` column fails with `ERROR: functions in index expression must be marked IMMUTABLE`, because "midnight" depends on the session's `TimeZone` setting. Pin the zone explicitly: `DATE_TRUNC('day', created_at AT TIME ZONE 'UTC')` is `IMMUTABLE`.
2. **Query predicates must match syntactically:** The planner does not prove arbitrary string-function equivalences. If your index is on `LOWER(email)`, writing `WHERE email ILIKE 'ada@lovelace.org'` or `WHERE UPPER(email) = 'ADA@LOVELACE.ORG'` walks right past the index into a `Seq Scan`. Write `WHERE LOWER(email) = LOWER(p_email)`.

3.6 Beyond B-Trees: Hash, GIN, GiST, and BRIN

A Lehman–Yao B-link tree assumes one foundational property about your search predicate: *every row maps to a single scalar (or tuple) key drawn from a totally ordered 1D domain* ($<, \leq, =, \geq, >$).

The moment your data violates that assumption, B-trees hit a geometric wall:

- What if keys are 250-byte URLs queried *only* for exact equality `=`?
- What if a single row contains *50 distinct searchable tokens*—an array `tags TEXT[]`, a nested `JSONB` document, or a 300-word full-text `tsvector`?
- What if keys live in 2D space (polygons, GPS coordinates) or represent intervals (`tstzrange`) tested for *overlap* (`&&`), where no 1D total order preserves containment?
- What if an append-only ledger holds 10 billion rows (1 TB) and you cannot afford 215 GB of B-tree pages just to slice time windows?

Rather than hard-wiring B-trees into the executor, Stonebraker and Rowe designed Postgres around an extensible **Access Method API** (`pg_am` and `IndexAmRoutine`). Let us open the hood on the four companions sitting beside `nbtree` in `src/backend/access/`.

3.6.1 Hash Indexes (`USING hash`): 32-Bit Codes Only

A PostgreSQL Hash index (`src/backend/access/hash/`) arranges 8 KB pages into dynamic linear-hashing buckets, chaining overflow pages only when collisions fill a bucket. Its key space-saving trick is what it omits: **a Hash index never stores the raw column bytes.** Each entry stores only a **4-byte (32-bit) hash code** plus the 6-byte row `ctid` (16 bytes aligned, sorted by hash code inside each bucket page for binary search).

Put numbers on that. Indexing a 240-byte `webhook_url` across 50,000,000 rows builds a $\sim$ 13.5 GB B-tree, whereas `CREATE INDEX ON deliveries USING hash (webhook_url)` takes just $\sim$ 1.1 GB. Because two distinct strings can share a 32-bit hash, a Hash scan *always* visits the table page to recheck `webhook_url = p_url`. Consequently, Hash indexes cannot run Index-Only Scans, enforce `UNIQUE`, sort `ORDER BY`, or span multiple columns.

> **Reading note**
>
> Before PostgreSQL 10, manuals warned against Hash indexes because bucket splits were not WAL-logged—a crash required a manual `REINDEX`. Since PostgreSQL 10, Hash indexes are 100% WAL-logged, crash-safe, and replicated. Reach for them when you have wide `TEXT` or `BYTEA` keys queried strictly with `=`.

3.6.2 GIN (Generalized Inverted Index): Multi-Element Containment at Scale

Suppose a `payments` table stores gateway metadata in a `JSONB` column `meta`, and risk engineers filter with `meta @>` on `"card_brand": "visa"` and `"three_ds": "failed"`. Or suppose support searches ticket text via `WHERE body_tsv @@ plainto_tsquery('english', 'chargeback reversal')`. A B-tree on the whole `JSONB` document cannot help: `'three_ds'` sits buried mid-document, not at byte 0.

A **GIN index** (`src/backend/access/gin/`) works like the subject index at the back of a book. Instead of mapping one row to one key, GIN calls `extractValue` to split each row into *dozens of atomic tokens*—words like `'chargeback'` or JSONB fields. It then maps each distinct token to the sorted list of row `ctid`s containing it (Figure 3.4, top):

1. **The Entry Tree:** A standard B-tree over *distinct tokens*. Across 50 million documents, English stems or JSONB keys often number only $\sim$ 200,000, keeping the Entry Tree compact.

2. **Posting List vs. Posting Tree:** When a rare token like `'chargeback'` appears in 40 rows, GIN packs those 40 `ctid`s inline as a **varbyte delta-compressed list** (2–3 bytes per `ctid`). When a frequent token like `'visa'` outgrows the inline limit ($\sim$ `GinMaxItemSize`), GIN spills its list into a dedicated **Posting Tree** keyed by `(block, offset)`. Keeping `ctid`s sorted lets multi-word queries (`'visa' AND 'failed'`) intersect bitmaps in one linear pass and read table blocks sequentially.

3. **The `fastupdate` Pending List:** Notice GIN's write cost: inserting one 60-field JSONB row would otherwise dirty up to 60 separate Entry Tree leaves. To avoid 60× foreground write amplification, PostgreSQL enables `fastupdate = on` by default. New token/`ctid` pairs append sequentially to an unsorted **pending list** (up to `gin_pending_list_limit`, default 4 MB). Readers check both the Entry Tree and the tiny pending list until `VACUUM` (or an insert crossing 4 MB) bulk-merges the pending list in one sorted batch.

In production

`jsonb_ops` vs. `jsonb_path_ops` and OLTP latency spikes. PostgreSQL ships two GIN operator classes for `JSONB`:

```
-- Default jsonb_ops: indexes every key AND value separately (supports @>, ?, ?|, ?&)
CREATE INDEX idx_pay_meta_default ON payments USING gin (meta);

-- jsonb_path_ops: hashes each full root-to-leaf path (e.g. hash('card_brand','visa'))
-- Supports @> containment ONLY, but is 2x-3x smaller and much faster when values repeat!
CREATE INDEX idx_pay_meta_paths   ON payments USING gin (meta jsonb_path_ops);

-- Full-text search via generated stored tsvector column + GIN
ALTER TABLE support_tickets
  ADD COLUMN search_tsv TSVECTOR
  GENERATED ALWAYS AS (to_tsvector('english', coalesce(subject,'') || ' ' ||
    coalesce(body,''))) STORED;
CREATE INDEX idx_tickets_fts ON support_tickets USING gin (search_tsv);
```

Watch out for one classic p99 tail-latency spike. When an `INSERT` pushes the `fastupdate` list past 4 MB, *that user's foreground session* synchronously flushes all 4 MB into the Entry Tree, stalling one API call for 50–300 ms. On strict p99 tables, lower `gin_pending_list_limit = '1MB'` with aggressive `autovacuum`, or set `WITH (fastupdate = off)` when rows emit only 2–4 tokens.

3.6.3 GiST (Generalized Search Tree): Bounding Hierarchies and Exclusion Constraints

In 1995, Joseph Hellerstein, Jeffrey Naughton, and Avi Pfeffer posed a bold design question. Could one balanced-tree engine power 2D R-trees for maps, interval trees for bookings, and nearest-neighbor k-NN search without rewriting buffer locking and crash recovery each time? Their answer, built in

`src/backend/access/gist/`, is **GiST**.

A B-tree slices a one-dimensional line into strictly non-overlapping key ranges. By contrast, an internal GiST entry stores a **bounding box** P enclosing every item in its child subtree—say, a 2D rectangle around 300 delivery zones or the start-to-end span of 300 `tstzrange` bookings. **Sibling bounding boxes in GiST may overlap.** A GiST search calls `consistent(entry, query)` on each entry of an internal page and visits *every* child whose box overlaps the query. Inserts pick the child needing the smallest box expansion (`penalty`) and split full pages via `picksplit`.

Beyond PostGIS maps (`&&`, `ST_Contains`, `<->`), GiST powers an integrity rule B-trees cannot express: **Exclusion Constraints**. Suppose no two bookings for the same `room_id` may have overlapping `during` intervals. A B-tree `UNIQUE` index only forbids *identical* intervals (`=`), and running `SELECT COUNT(*) ... WHERE during && new_slot` in app code races when two sessions insert concurrently under `READ COMMITTED`. With the built-in `btree_gist` extension, PostgreSQL blocks overlaps directly inside the GiST index:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE room_bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    INT       NOT NULL,
    during     TSTZRANGE NOT NULL,
    -- Forbid any two rows where (room_id = room_id) AND (during && during)!
    CONSTRAINT no_double_booking
        EXCLUDE USING gist (room_id WITH =, during WITH &&)
);
```

Like `UNIQUE` in `nbtree`, `EXCLUDE USING gist` checks physical index pages across both committed and uncommitted in-flight rows, blocking the second overlapping writer until the first commits or aborts. An overlapping insert fails cleanly:

```
ERROR:  conflicting key value violates exclusion constraint "no_double_booking"
DETAIL:  Key (room_id, during)=(7, ["2026-11-02 10:00:00+00","2026-11-02 11:00:00+00"))
         conflicts with existing key (room_id, during)=(7, ["2026-11-02 09:30:00+00",
         "2026-11-02 10:30:00+00")).
```

3.6.4 BRIN (Block Range Index): Indexing a Terabyte in 30 Megabytes

Finally, picture an append-only `ledger_events` table holding 10,000,000,000 (10^{10}) immutable rows at $\sim$ 100 bytes each—134,217,728 pages, or 1.0 **TB of table data**. Analysts query 15-minute windows by `recorded_at`. A B-tree on `recorded_at` costs $\sim$ 20 bytes $\times 10^{10}$ rows $\approx$ **215** GB on disk.

Pause and notice how those rows land on disk. Because events arrive in chronological order, blocks 0..127 hold 00:00:00–00:00:04 on January 1, blocks 128..255 hold 00:00:04–00:00:09, and so on. Why store ten billion individual row pointers when the table is *already physically sorted by arrival time*?

Introduced in PostgreSQL 9.5 (`src/backend/access/brin/`), a **Block Range Index (BRIN)** drops per-row `ctid`s completely. It slices the table into consecutive chunks of `pages_per_range` blocks (default 128 pages = 1 MB). For each 128-page chunk, BRIN stores **one tiny summary**

holding only (`min_val`, `max_val`) across all rows in those 128 pages (Figure 3.4, bottom):

$$\frac{134{,}217{,}728 \text{ heap pages}}{128 \text{ pages/range}} = 1{,}048{,}576 \text{ ranges} \times \sim 28 \text{ bytes/summary} \approx \mathbf{29.4} \text{ MB total.}$$

That is 215 **GB for a B-tree versus** $\sim$ 30 **MB for BRIN**—more than 7,000× smaller. When a query filters `recorded_at` to a 15-minute window, PostgreSQL scans the 30 MB BRIN summary in milliseconds and skips 99.98% of the 1 MB chunks whose [min, max] misses the window. It then runs a `Bitmap Heap Scan` with `Recheck Cond` across only the ≈ 150 matching 1 MB chunks.

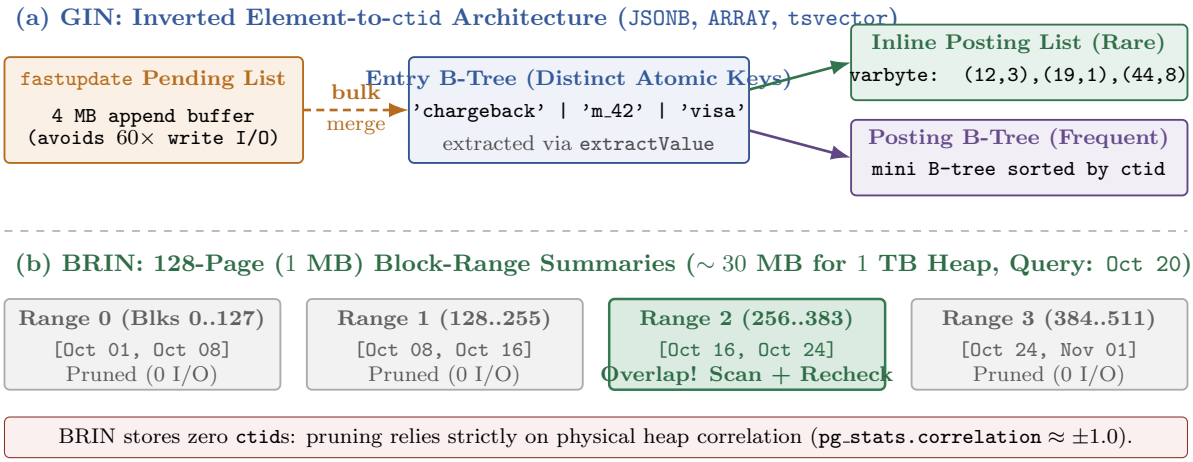


Figure 3.4: Two opposite poles of non-B-tree indexing. In (a), **GIN** inverts multi-element rows into exact `ctid` posting lists/trees cushioned by a `fastupdate` buffer. In (b), **BRIN** summarizes 128-page (1 MB) heap chunks by [min, max] to prune append-only streams at 0.003% space overhead.

Failure mode

The BRIN trap: physical correlation decay. BRIN stores no row pointers; its pruning power depends entirely on how tightly each 128-page chunk clusters values on disk. Check `SELECT correlation FROM pg_stats WHERE tablename = 'ledger_events' AND attname = 'recorded_at';`:

- When `correlation` sits near +1.0 or −1.0 (append-only monotonic timestamps or sequences, never updated or deleted), each 128-page chunk spans a narrow [min, max] window and BRIN skips > 99.9% of the table.
- Now suppose you build BRIN on a random `UUIDv4` (`correlation` ≈ 0.00). Or suppose a retention job deletes January rows, `VACUUM` marks free space in blocks 0..127 inside `_fsm`, and November inserts reuse those slots. **A single November row landing in block 12 stretches Range 0's summary from `[Jan 01, Jan 01]` to `[Jan 01, Nov 01]`.** Worse, BRIN never shrinks [min, max] automatically when extreme rows are deleted; shrinking requires `brin_desummarize_range()` or `REINDEX`. Once scattered updates or `_fsm` reuses mix old and new timestamps across chunks, every range overlaps every query, and your 30 MB BRIN index silently scans 100% of the 1 TB table.

Cost summary

PostgreSQL Access Method Decision Matrix:

Method	Key Structure	Operators	Typical Size	Sweet-Spot Workload
`btree`	1D total order + `ctid`	`< <= = >= >`	15–30% heap	OLTP point/range, `ORDER BY`, `UNIQUE`, `INCLUDE`
`hash`	32-bit hash + `ctid`	`=` only	5–12% heap	Exact lookups on wide URLs, tokens, `BYTEA`
`gin`	Inverted key → `ctids`	`@> <@ ? @@ &&`	20–80% heap	`JSONB`, `ARRAY`, `tsvector` full-text, trigrams
`gist`	Bounding hierarchy	`&& @> <->`	20–50% heap	PostGIS 2D shapes, `tstzrange`, `EXCLUDE`
`brin`	128-page [min, max]	`< <= = >= >`	$\sim$ 0.003% heap	Multi-TB append-only streams ($\lvert r\rvert \approx 1$)

Across Chapters 2 and 3, we treated RAM edits—inserting a row, lowering `P_HIKEY`, linking `btpo_next` across a split—as if writing into `shared_buffers` made them permanent. What happens when power fails after Leaf P reaches NVMe while Leaf P' still sits in volatile RAM? Or when Linux tears an 8,192-byte page in half across two 4,096-byte disk sectors? That question brings us to Part II and Chapter 4: Write-Ahead Logging, checkpoints, and crash recovery.

3.7 Exercises

Exercise 3.1 (Exact B+Tree Fanout: `BIGINT` vs. `UUID` Keys)**.** Use the 8,192-byte `nbtree` page layout from Section 3.1: 24-byte `PageHeaderData`, 16-byte `BTPageOpaqueData`, 4-byte `ItemIdData` per slot, 8-byte `IndexTupleData` header, and 8-byte `MAXALIGN` padding.

1. Compute the exact per-entry byte cost (slot pointer plus aligned `IndexTupleData`) for a single 16-byte `UUID` column, and compare it against the 20-byte cost of an 8-byte `BIGINT`.
2. Assume internal pages average 80% occupancy after splits and leaf pages use `fillfactor = 90` (each reserving one slot for `P_HIKEY`). Compute the internal branching factor B and leaf capacity L for both `BIGINT` and `UUID`.
3. For 500,000,000 (5×10^8) rows, find the exact number of tree levels and the RAM needed to pin all non-leaf pages in `shared_buffers` for both key types.

Exercise 3.2 (Tracing a Concurrent Lehman–Yao Split by Hand)**.** Leaf Block 22 covers $(100, 200]$ (`P_HIKEY` = 200, `btpo_prev` = 9, `btpo_next` = 35) and holds $\{108, 115, 124, 130, 152, 168, 191\}$. Parent Block 5 routes $(100, 200] \to$ Block 22. Writer W inserts 175, splits Block 22 at separator 130 into Block 22 and new Block 59, unlocks both leaves, and pauses *before* locking Parent Block 5.

1. Write down `P_HIKEY`, the data keys, `btpo_prev`, `btpo_next`, and `BTP_INCOMPLETE_SPLIT` on Block 22 and Block 59 during this pause.
2. Trace the page pins, shared latches, and `P_HIKEY` checks for Reader A ($k = 124$) and Reader B ($k = 175$). How many latches does Reader B hold at once during descent versus during `_bt_moveright`?
3. Why must `_bt_moveright` lock pages left-to-right (Block 22 → Block 59)? What deadlock risk appears during *backward* scans (`ORDER BY k DESC` following `btpo_prev` right-to-left) if a left neighbor splits concurrently, and how does releasing and re-checking avoid a latch cycle?

Exercise 3.3 (Deduplication Math and Bottom-Up Deletion)**.** An `orders` table holds 20,000,000 rows split evenly across 5 values of `status CHAR(4)` (`'OPEN'`, `'PAID'`, `'SHIP'`, `'DONE'`, `'VOID'`).

1. With `deduplicate_items = off`, compute the byte size of one leaf entry: an 8-byte `IndexTupleData` header plus a 1-byte `varlena` header and 4 ASCII bytes padded to 8 bytes, plus a 4-byte slot pointer. Estimate total leaf size in MB at `fillfactor = 90`.

2. With `deduplicate_items = on`, suppose each posting list packs 130 6-byte `ctid`s (16 B header+key + 130×6 B $= 796$ B, padded to 800 B plus a 4-byte slot pointer). Recompute total leaf size in MB and the compression ratio.

3. Suppose `orders` also indexes `updated_at`, and a worker updates `updated_at = NOW()` ten times on an `'OPEN'` row without changing `status`. Explain why HOT is blocked, why 10 `'OPEN'` entries land on the leaf page, and how PG 14's bottom-up deletion prevents a split.

Exercise 3.4 (Composite Ribbon Geometry and Sort Elimination)**.** A multi-tenant ledger `entries(entry_id, tenant_id, account_id, status, posted_at, amount_cents)` runs three hot queries:

- Q_1: `WHERE tenant_id = p_tenant AND account_id = p_acct AND posted_at >= p_start ORDER BY posted_at DESC LIMIT 25`
- Q_2: `WHERE tenant_id = p_tenant AND status = 'SETTLED' AND posted_at BETWEEN p_start AND p_end`
- Q_3: `WHERE tenant_id = p_tenant AND amount_cents > 100000 AND posted_at >= p_start`

1. Design the smallest set of composite B-trees so Q_1 runs with **zero `Sort` node** (touching ~ 28 entries) and Q_2 reads one contiguous slice. Give exact column orders and `ASC`/`DESC` directions.

2. Using Figure 3.2, explain why no single 1D B-tree makes both `amount_cents > 100000` and `posted_at >= p_start` contiguous in Q_3. If 0.1% of a tenant's rows match `posted_at >= p_start` while 15% match `amount_cents > 100000`, which column belongs second after `tenant_id`?

Exercise 3.5 (Visibility Map Churn and `Heap Fetches` Latency)**.** A 50,000,000-row `transactions` table spans 600,000 heap pages ($\approx$ 4.69 GB) with covering index `(merchant_id, created_at) INCLUDE (net_cents)`.

1. Compute the exact byte size and 8 KB page count of its `_vm` fork at 2 bits per table page.

2. Right after `VACUUM`, 100% of pages have `ALL_VISIBLE = 1`. Over the next 20 minutes, 90,000 random single-row updates hit the 600,000 pages. Using $1 - (1 - 1/M)^K \approx 1 - e^{-K/M}$, find the fraction of pages whose `_vm` bit is now 0.

3. A query runs `SELECT SUM(net_cents) FROM transactions WHERE merchant_id = 77`, matching 20,000 index entries scattered across cold table pages (80 μs per random NVMe read; index and `_vm` warm in RAM). Compute expected `Heap Fetches` and wall-clock time right after `VACUUM` versus 20 minutes later.

Exercise 3.6 (Partial and Expression Index Debugging)**.** A teammate proposes three indexes on a 150,000,000-row `invoices` table (99.7% `'paid'`, 0.2% `'open'`, 0.1% `'overdue'`):

```sql
-- Index 1
CREATE INDEX idx_inv_day ON invoices (DATE_TRUNC('day', issued_at));
-- Index 2
CREATE INDEX idx_inv_code ON invoices (LOWER(reference_code));
-- Index 3
CREATE INDEX idx_inv_unpaid ON invoices (due_at) WHERE status IN ('open', 'overdue');
```

1. `issued_at` is `TIMESTAMPTZ`. Explain why Index 1 fails with an immutability error, and fix both `CREATE INDEX` and a UTC-day query for `2026-10-31`. Then show how a half-open range on bare `issued_at` avoids an expression index altogether.
2. App code runs `WHERE LOWER(TRIM(reference_code)) = 'inv-901'`. Explain why PostgreSQL skips Index 2, and fix the contract cleanly.
3. Estimate the size ratio of Index 3 versus a full index on `(status, due_at)`. Can `WHERE status = 'overdue' AND due_at < NOW()` use Index 3? Justify via predicate implication.

Exercise 3.7 (GIN Internals: `jsonb_ops`, `jsonb_path_ops`, and Pending Flushes)**.** Consider inserting this `JSONB` row into `charges(charge_id, meta)`:

```sql
INSERT INTO charges (meta) VALUES
('{"gateway":"stripe","risk":{"score":92,"flags":["avs_mismatch","velocity"]}}');
```

1. List all Entry Tree tokens extracted under default `jsonb_ops` versus the root-to-leaf path hashes under `jsonb_path_ops`.
2. Which queries can `jsonb_path_ops` accelerate, and which force a sequential scan: (a) `meta @>` matching `velocity` inside `risk.flags`; (b) `meta ? 'gateway'`; (c) `meta->'risk'->>'score' = '92'`?
3. Suppose each insert appends ~ 256 bytes to the GIN pending list (`gin_pending_list_limit = 4MB`). After roughly how many inserts does a foreground session stall on a synchronous merge, and what two settings remove that p99 spike?

Exercise 3.8 (Quantifying BRIN Scan Amplification Under `_fsm` Poisoning)**.** An append-only `telemetry` table packs 102,400,000 rows at 80 rows per page across 1,280,000 pages ($\approx$ 10 GB). A BRIN index on `ts` with `pages_per_range = 128` creates $R = 10{,}000$ block ranges across 100 days (100 ranges per day).

1. Initially rows are strictly chronological: querying the latest hour of Day 100 overlaps $\lceil 100/24 \rceil = 5$ ranges. How many MB of heap does PostgreSQL read?
2. A job deletes 0.1% of rows uniformly across Days 1..99 and runs `VACUUM`. On Day 100, `_fsm` routes $K = 30{,}000$ new rows uniformly at random into the 9,900 ranges of Days 1..99. What is the probability $1 - (1 - 1/9900)^{30000}$ that a historical range receives at least one Day-100 row?
3. What happens to that historical range's $[\min, \max]$ summary when one Day-100 row lands inside it, and how many MB of heap must the "latest hour of Day 100" query now scan?

Part II

Durability, Concurrency, and MVCC Internals

Chapter 4

Write-Ahead Logging, Checkpoints, and Crash Recovery

In Chapters 2 and 3 we built fast 8,192-byte table pages and B-tree index blocks inside PostgreSQL's RAM cache (`shared_buffers`). As long as power flows steadily into DRAM, updates fly: inserting a row updates the page in memory, tweaks each index leaf, and marks those buffers dirty in microseconds. Now pull the server's power cord out of the wall. Every byte in `shared_buffers` vanishes. Every byte in the Linux kernel page cache vanishes too. Worse, any 8 KB page caught mid-flight to the SSD lands split into two mismatched 4 KB halves. How does PostgreSQL return `COMMIT` in under a millisecond while guaranteeing that a power cut never loses a committed row or leaves a broken index?

Focus

By the end of this chapter, you will be able to answer five questions from first principles:

- Why does an ordinary `write()` call leave your data in volatile RAM, and how can power loss slice an 8 KB database page into two mismatched 4 KB disk sectors?
- How does appending tiny change records to one sequential Write-Ahead Log (WAL) let PostgreSQL commit transactions without waiting on scattered table and index writes?
- Why does replaying a tiny change record fail on a half-written disk page, and how does saving one full page snapshot after each checkpoint heal torn writes?
- When forty clients hit `COMMIT` at the same instant, how does one leader thread sync all forty commits in a single disk flush?
- When power returns after a crash, how does PostgreSQL recover in one forward pass—with zero undo rollback work, even if a crashed batch job modified ten million rows?

4.1 The Physics of `write()`, `fsync()`, and Torn 8 KB Pages

Before reading any PostgreSQL recovery code, picture what happens physically when a Linux program writes to a file. Engineers casually say "we wrote the buffer to disk" when `write(fd, buf,`

`8192)` returns `8192`. Physically, nothing reached the drive. Those 8,192 bytes merely copied from one region of volatile RAM into another region of volatile RAM.

Trace a modified 8 KB page from PostgreSQL's RAM pool all the way into non-volatile NAND flash cells:

1. **PostgreSQL `shared_buffers` (user-space RAM):** When an `INSERT` or `UPDATE` changes block 7, it edits an 8,192-byte frame in shared memory and flips its dirty bit (`BM_DIRTY`). If `postgres` crashes with `SIGSEGV` or `kill -9` and resets shared memory, every dirty frame here disappears.

2. **Linux kernel page cache (OS RAM):** To hand a page to the operating system, PostgreSQL calls the positional write syscall `pwrite(fd, page, 8192, offset)`. The kernel copies those 8,192 bytes into two 4,096-byte kernel memory buffers (`struct folio`), marks them dirty, and returns `8192` right away. Notice the gap between process death and power loss. If `postgres` dies via `kill -9` right after `pwrite()`, Linux is still running and flushes those kernel buffers seconds later. If the kernel panics or building power fails, the entire OS page cache evaporates.

3. **NVMe controller DRAM cache (on-drive RAM):** Even when Linux transfers 4 KB blocks over PCIe to an SSD, the drive does not program slow NAND flash right away. Most SSDs carry 512 MB to 4 GB of onboard DDR4 DRAM to absorb bursts and hold Flash Translation Layer (FTL) maps. Without backup power on the drive, bytes sitting in controller DRAM still vanish when voltage drops.

4. **Non-volatile NAND flash (`fsync` / `fdatasync`):** To demand true persistence, a program calls `fdatasync(fd)` (or opens its log file with `O_DSYNC`, which syncs every write automatically). Linux pushes dirty kernel buffers to the drive *and* sends an NVMe `Flush` command (or sets the hardware Force Unit Access bit, `FUA`, which bypasses volatile drive cache). A compliant SSD does not complete `fdatasync()` until every byte sits in NAND flash or behind backup capacitors.

Java 6 hides the same tiers behind `FileOutputStream.write()`, which returns as soon as the bytes reach tier 2, the kernel page cache. A Java program that needs durability must call `fos.getFD().sync()` or `FileChannel.force(true)`, and both end in `fsync(2)`. In C the pair is explicit:

```
ssize_t n = pwrite(fd, page, 8192, off); /* returns once bytes sit in the page cache */
if (n != 8192)
    report_error();
if (fdatasync(fd) != 0)                  /* returns once the device reports durable */
    abort();                             /* PostgreSQL PANICs here instead          */
```

Why abort instead of retrying? In 2018 PostgreSQL engineers discovered a nasty Linux kernel trap: after a failed `fsync`, Linux can mark the dirty kernel pages clean and discard them. A second `fsync` then returns success even though the bytes never reached disk. Since February 2019, any data-file `fsync` error during a checkpoint halts PostgreSQL immediately so restart replays cleanly from the Write-Ahead Log:

```
PANIC:  could not fsync file "base/16384/16391": Input/output error
```

Failure mode

Lying consumer drives vs. enterprise Power-Loss Protection (PLP). Enterprise NVMe SSDs carry a row of tantalum or supercapacitors on their circuit boards. When 12 V power drops, those capacitors power the drive for a few milliseconds to drain onboard DRAM into NAND flash, letting the drive honor `NVMe Flush` fast and honestly. Cheap consumer SSDs omit those capacitors to save two dollars. Worse, some consumer firmware lies and reports `Flush` complete before programming NAND just to win benchmarks. Pull the plug on a lying drive mid-load and bytes that `fdatasync()` promised were safe vanish, corrupting any database engine.

4.1.1 Why Direct Heap Flushing on `COMMIT` Fails Twice

Suppose we had no write-ahead log and tried to make `COMMIT` durable the direct way. On every `COMMIT`, we would find each 8 KB table and index page dirtied by that transaction, call `pwrite()`, and run `fdatasync()` on each file before replying to the client.

That design fails twice: first on *speed*, then on *correctness*.

First, count the random I/O cost. Inserting one row into `orders` with four B-tree indexes dirties five separate 8 KB blocks across five files in `PGDATA/base/16384/`. If an index leaf splits, that single index dirties three more blocks (left leaf, new right leaf, parent). Syncing five to eight random 8 KB blocks on every one-row commit turns sequential flash bandwidth into random-write thrashing and wears out SSD erase blocks. It also stops you from keeping hot pages in RAM across thousands of updates.

Second, and worse: even if you paid that 100× latency tax, **a power cut mid-flush destroys the page**. Compare the block sizes across the stack:

PostgreSQL page = 8,192 bytes vs. Linux VM page / NVMe 4Kn sector = 4,096 bytes.

Standard filesystems (`ext4`, `XFS`) and NVMe drives write atomically in 4,096-byte sectors, never 8,192-byte pages. Every 8 KB PostgreSQL page spans two adjacent 4 KB sectors: bytes `0..4095` in sector $2k$ and bytes `4096..8191` in sector $2k + 1$. Recall Chapter 2's slotted page layout. The 24-byte page header and 4-byte slot pointers grow *downward* from byte `0` in sector $2k$, while row bytes grow *upward* from byte `8192` in sector $2k + 1$.

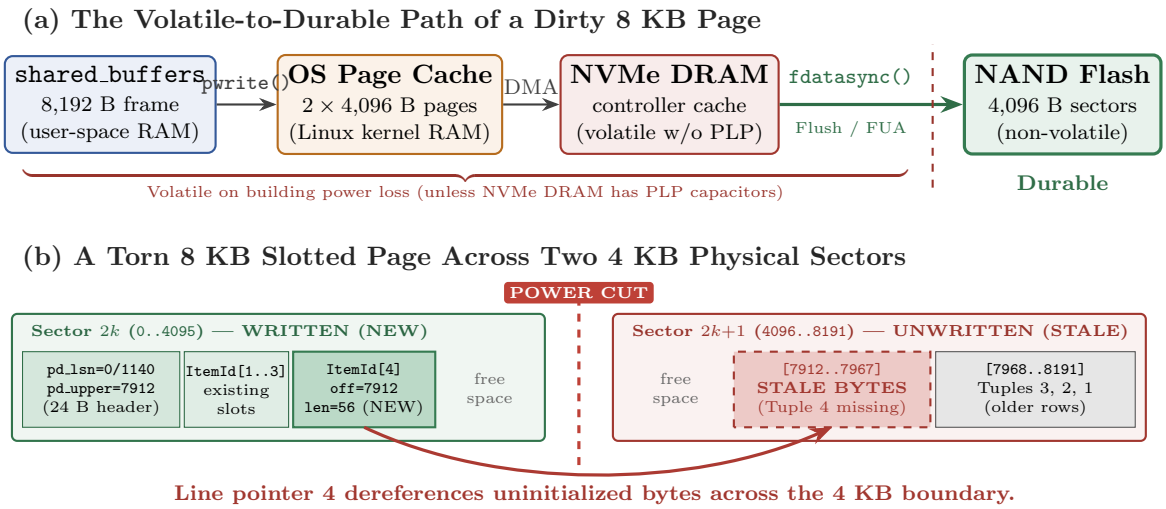


Figure 4.1: Top: the four memory tiers between a PostgreSQL backend and non-volatile NAND flash. Bottom: because an 8,192-byte PostgreSQL page spans two 4,096-byte hardware sectors, power loss mid-write produces a *torn page* whose updated header and line pointers in Sector $2k$ point into stale bytes in Sector $2k + 1$.

Figure 4.1(b) shows this *torn page* (or *fractured write*) in action. Before our insert, block 7 held three rows in bytes `7968..8191` (`pd_upper = 7968`). Inserting a 56-byte fourth row lowers `pd_upper` to `7912`, writes the row into bytes `7912..7967` (Sector $2k + 1$), and writes `ItemId[4] = (off=7912, len=56)` at byte `36` (Sector $2k$). Now suppose power dies right after Sector $2k$ lands in NAND flash, before Sector $2k + 1$ arrives. On reboot, block 7 on disk is half new and half old: `ItemId[4]` points to offset `7912`, where stale bytes or zeroes sit. With data checksums enabled (`initdb --data-checksums`), the 16-bit `pd_checksum` catches the mismatch and aborts the query with `ERROR: invalid page in block 7`—detecting corruption, though unable to heal it. Without checksums, PostgreSQL decodes garbage bytes as customer balances.

Solving both problems at once—avoiding scattered `fdatasync()` calls at `COMMIT` *and* healing any torn 8 KB page—is the job of Write-Ahead Logging.

4.2 Write-Ahead Logging (`pg_wal`), LSNs, and the Fundamental WAL Invariant

Intuition

Separate *recording* a change from *applying* it. Recording is one sequential append to a single file. Applying touches scattered 8 KB pages in many files. `COMMIT` waits only for the append to become durable; the scattered pages are written later, in batches. If power fails before they are written, restart reads the appended records and applies them again. One rule makes this safe: **a data page may not reach disk before the log records describing its changes are durable.**

PostgreSQL's Write-Ahead Log (implemented in `src/backend/access/transam/xlog.c`) turns that idea into one append-only byte stream. That logical stream starts at byte `0` when `initdb` creates a

cluster and grows upward for the life of the database.

4.2.1 Segment Files in PGDATA/pg_wal/ and pg_lsn Byte Arithmetic

Keeping a multi-terabyte log in one giant file would make deleting old history painful. Instead, PostgreSQL slices the byte stream into fixed **16 MB** (16,777,216 = 0x01000000 byte) segment files inside PGDATA/pg_wal/. Internally, each 16 MB segment is split into 2,048 pages of 8,192 bytes, each starting with a small header (XLogPageHeaderData) so recovery can verify alignment every 8 KB.

Every byte position in this stream has a 64-bit address called a **Log Sequence Number** (XLogRecPtr in C, or pg_lsn in SQL). PostgreSQL prints a 64-bit LSN as two hex numbers separated by a slash:

$$\texttt{X/YYYYYYYY} \quad \Longleftrightarrow \quad (\texttt{0xX} \ll 32) \mid \texttt{0xYYYYYYYY}.$$

Read that format in plain terms: X counts how many 4 GB (2^{32}-byte) chunks have passed, and YYYYYYYY gives the hex byte offset inside the current 4 GB chunk (dropping leading zeros, so 0/016B3748 prints as 0/16B3748). To compare two LSNs or measure the WAL bytes between them, treat both as plain 64-bit integers and subtract.

Definition 4.1 (Log Sequence Number and Segment Mapping)**.** A *Log Sequence Number* $L \in [0, 2^{64})$ is a byte offset into PostgreSQL's WAL stream. With segment size $S = 2^{24} = 16{,}777{,}216$ bytes (0x01000000) and a 32-bit timeline ID $T \geq 1$ (starting at $T = 1$ and incremented on point-in-time recovery promotion), L maps to:

$$\text{segno}(L) = \lfloor L/S \rfloor = L \gg 24, \qquad \text{offset}(L) = L \bmod S = L \,\&\, \texttt{0x00FFFFFF}.$$

Because each 4 GB chunk (2^{32} bytes) holds $2^{32}/2^{24} = 256 = \texttt{0x100}$ segments of 16 MB, the 24-character hex filename in PGDATA/pg_wal/ concatenates three 8-digit hex fields:

$$\underbrace{\texttt{TTTTTTTT}}_{\text{timeline } T} \quad \underbrace{\texttt{XXXXXXXX}}_{\lfloor \text{segno}/256 \rfloor = (L \gg 32)} \quad \underbrace{\texttt{000000SS}}_{\text{segno mod } 256}.$$

That third group 000000SS explains a classic surprise when listing PGDATA/pg_wal/: file 000000010000000000000FF rolls straight over to 000000010000000100000000 instead of ...00000100. Because 256 × 16 MB = 4 GB, the middle eight digits count 4 GB chunks, while the last eight digits only ever run from 00 to FF.

Example 4.1 (Concrete pg_lsn Arithmetic by Hand)**.** Suppose pg_current_wal_lsn() returns $L_1 =$ 0/16B3748 before a batch insert and $L_2 =$ 0/16B3A90 right after, on timeline $T = 1$:

1. **Which 16 MB file holds L_1, and at what byte offset?** Pad the right half to eight hex digits: $L_1 =$ 0x00000000_016B3748. Dropping the last six hex digits ($\gg 24$) gives segno(L_1) = 0x01 = 1. Keeping the last six hex digits gives offset(L_1) = 0x6B3748 = 7,026,504 bytes (≈ 6.70 MB into file 000000010000000000000001).

2. **How many WAL bytes did the batch generate?** Subtract directly in base 16:

$$L_2 - L_1 = \texttt{0x016B3A90} - \texttt{0x016B3748} = \texttt{0x348} = 3 \times 256 + 4 \times 16 + 8 = 840 \text{ bytes}.$$

3. **Cross-epoch subtraction:** Suppose $L_3 =$ 0/FFFFFC00 and $L_4 =$ 1/400 (short for 1/00000400, or 0x00000001_00000400). Then

$$L_4 - L_3 = \texttt{0x100000400} - \texttt{0x0FFFFFC00} = \texttt{0x800} = 2{,}048 \text{ bytes},$$

matching SELECT pg_wal_lsn_diff('1/400', '0/FFFFFC00'); → 2048.

4.2.2 Inside an `XLogRecord` and Resource Managers

What actually sits at offset `0/16B3748` inside that file? Every logged event—a row insert, a B-tree split, a `COMMIT`, or a checkpoint—starts with a fixed 24-byte envelope recording its length, 32-bit transaction ID (XID), previous record address, subsystem owner, and checksum. In C (`src/include/access/xlogrecord.h`), that envelope is `XLogRecord`:

```
/* Fixed 24-byte header at the front of every WAL record (xlogrecord.h) */
typedef struct XLogRecord
{
    uint32          xl_tot_len;   /* total byte length of entire record       */
    TransactionId   xl_xid;       /* 32-bit XID that wrote record (or 0)      */
    XLogRecPtr      xl_prev;      /* start LSN of previous record in WAL      */
    uint8           xl_info;      /* opcode flags specific to resource mgr    */
    RmgrId          xl_rmid;      /* resource manager ID (heap, btree, xact)  */
    /* 2 bytes of struct alignment padding here -> 20 bytes so far          */
    pg_crc32c       xl_crc;       /* CRC32C over payload + bytes 0..19        */
} XLogRecord;                     /* sizeof(XLogRecord) == 24 bytes           */
```

Right after those 24 bytes come zero or more **block headers** (`XLogRecordBlockHeader`). Each block header names a touched page by its physical file triple (`RelFileLocator`), fork, and 32-bit block number, followed by per-block change bytes (or a Full Page Image) and the record's main payload.

Three fields in `XLogRecord` keep recovery modular and safe:

- **`xl_rmid` and `xl_info` (Resource Managers):** Instead of one giant recovery `switch`, PostgreSQL splits WAL replay across two dozen **Resource Managers** (`RmgrTable` in `rmgr.c`), each supplying an `rm_redo` replay routine:
  - `RM_XLOG_ID` (`XLOG`): checkpoints, settings changes, standalone full-page writes.
  - `RM_XACT_ID` (`Transaction`): `COMMIT`, `ABORT`, and two-phase records.
  - `RM_HEAP_ID` / `RM_HEAP2_ID`: `INSERT`, `UPDATE`, `HOT_UPDATE`, `DELETE`, in-page pruning, and freeze records.
  - `RM_BTREE_ID` (`Btree`): leaf inserts, page splits, and page deletions.
- **`xl_prev` (Back-pointer check):** Every record points to the start LSN of the record right before it. To save I/O, PostgreSQL recycles old 16 MB WAL files by renaming them to future segment numbers without zeroing their old bytes. As a result, bytes *past* the crash point in a recycled file often hold old `XLogRecord` headers from days ago. Checking the page header LSN and `xl_prev` stops recovery from replaying stale leftovers.
- **`xl_crc` (Hardware CRC32C):** Computed with CPU instructions (`crc32q` on x86, CRC32C on ARM) over the whole record. If power dies halfway through writing a 200-byte record, `xl_crc` fails to match, marking the exact end of valid WAL.

4.2.3 `pd_lsn` and the Fundamental WAL Invariant

How does an 8 KB page on disk remember how much of the WAL stream it already includes? Every table, index, and visibility-map page stores an 8-byte LSN stamp right at bytes `0..7` of its header (`pd_lsn` in `PageHeaderData`):

```c
typedef struct PageHeaderData
{
    PageXLogRecPtr pd_lsn;        /* bytes 0..7: LSN of last modifying record */
    uint16         pd_checksum;   /* bytes 8..9: 16-bit FNV-1a page checksum  */
    uint16         pd_flags;      /* bytes 10..11: header flag bits           */
    LocationIndex  pd_lower;      /* bytes 12..13: start of free space        */
    LocationIndex  pd_upper;      /* bytes 14..15: end of free space          */
    /* ... */
} PageHeaderData;
```

Whenever a backend edits buffer P in `shared_buffers` under an exclusive page lock, it builds an `XLogRecord` and appends it into `wal_buffers`. That append returns the record's *ending* LSN L. The backend stamps `pd_lsn = L` into bytes `0..7` of P and marks P dirty.

At that moment, both `wal_buffers` and P still sit in volatile RAM. Neither has touched NAND flash yet. One rule governs which one must reach flash first.

Theorem 4.1 (The Write-Ahead Logging Invariant)**.** *Let P be a dirty buffer in `shared_buffers` stamped with `pd_lsn`$(P) = L$, and let* FlushLSN *be the highest WAL byte offset synced (`fdatasync`) to disk inside `PGDATA/pg_wal/`. PostgreSQL's buffer manager (`FlushBuffer` in `bufmgr.c`) enforces:*

$$P \text{ may be written to the OS via } \texttt{pwrite(2)} \quad \Longrightarrow \quad \text{FlushLSN} \geq \texttt{pd_lsn}(P) = L.$$

Whenever FlushLSN $< L$, `FlushBuffer` *blocks and calls `XLogFlush(L)` before calling `pwrite(2)`. Together with torn-page protection (Section 4.3) and commit flushing, this rule guarantees that every committed write survives crashes and no disk page holds an unlogged change.*

Proof. In Haerder and Reuter's buffer taxonomy, PostgreSQL is a **Steal / No-Force** engine:

- **No-Force:** When transaction T_1 commits at L_{commit}, PostgreSQL syncs only the sequential WAL stream up to FlushLSN $\geq L_{\text{commit}}$. Dirty 8 KB pages touched by T_1 stay in RAM. Because all of T_1's edits sit at LSNs $L_i < L_{\text{commit}} \leq$ FlushLSN, every log record needed to rebuild T_1's work is already safe on flash before the client hears `COMMIT`.

- **Steal:** Conversely, if `shared_buffers` needs room, PostgreSQL may *steal* frame P dirtied by an uncommitted transaction T_2 at LSN L and write P to its table file early. Because `FlushBuffer` forces FlushLSN $\geq L$ before `pwrite(2)`, the log record at L (plus its full-page image if P tears) is already safe on flash. Recovery can rebuild a valid page P and let transaction commit status govern row visibility. □

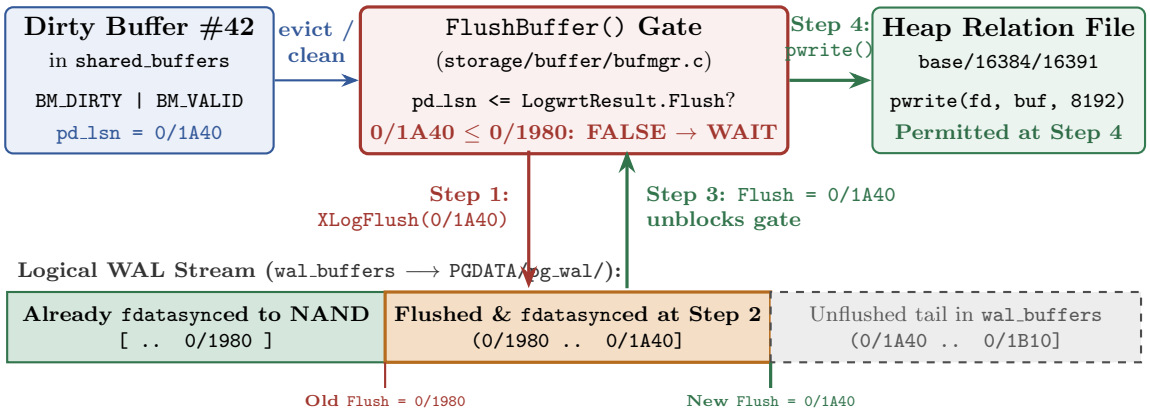


Figure 4.2: How `FlushBuffer()` enforces the WAL Invariant in shared memory. Dirty Buffer #42 carries `pd_lsn = 0/1A40`, while durable WAL (`XLogCtl->LogwrtResult.Flush`) has only reached `0/1980`. The gate blocks `pwrite()`, forces `wal_buffers` out through `fdatasync()` up to `0/1A40`, and only then lets the 8 KB heap page leave user-space RAM.

Pause

Pause and check the WAL rule. Every 8 KB page carries the LSN of its latest edit (`pd_lsn`). Before that page can leave `shared_buffers` for disk, the WAL file must be `fdatasync`ed at least as far as `pd_lsn`. Log bytes always hit non-volatile flash ahead of the data pages they describe.

Failure mode

What breaks if `bgwriter` violates the WAL Invariant? Trace a concrete violation. Transaction T_1 splits B-tree leaf block 19: it allocates right-sibling block 44 and moves half of block 19's keys there at LSN `0/1A40`. Because T_1 has not committed yet, `0/1A40` sits only in volatile `wal_buffers` (`FlushLSN = 0/1980`). Now suppose a buggy writer pushes block 19 to disk without calling `XLogFlush(0/1A40)`, and power fails before `0/1A40` or block 44 reaches flash. On reboot, recovery stops at `0/1980`. Block 44 does not exist on disk, while block 19 has permanently lost half its index entries *for already committed rows* and points to a missing page.

4.3 Torn Pages Solved: `full_page_writes` After a Checkpoint

Test your intuition with one question before reading on.

Walking question

Suppose PostgreSQL obeys the WAL Invariant (Theorem 4.1): before dirty block 7 (`pd_lsn = 0/1140`) writes to its table file, its WAL record at `0/1140` is already `fdatasync`ed in `pg_wal`. A millisecond later, `bgwriter` calls `pwrite()` on block 7, and power fails mid-write, tearing block 7 across its two 4 KB sectors as in Figure 4.1(b). Since record `0/1140` is safe on disk, why can't recovery simply replay `0/1140` onto block 7 to fix the tear?

Trace the exact byte offsets and you will see naive delta replay fail in *two separate ways*.

Normally, WAL records use **physiological logging**: physical as to *which* 8 KB block changed ("table 16384, block 7"), logical inside that block's layout ("lower `pd_upper` by 56, write these 56 row bytes there, and point `ItemId[4]` at `pd_upper`"). Those compact deltas take only 55 to 80 bytes instead of 8,192 bytes. To avoid inserting the same row twice during replay, recovery compares the disk page's `pd_lsn` against the record's end LSN:

$$\texttt{page.pd_lsn} \geq \texttt{record.end_lsn} \implies \text{skip replay (}\texttt{BLK_DONE}\text{);}$$
$$\texttt{page.pd_lsn} < \texttt{record.end_lsn} \implies \text{apply delta and set } \texttt{page.pd_lsn} \leftarrow \texttt{record.end_lsn}.$$

Now watch both halves of a torn 8 KB write break that check:

1. **Failure Case A (Sector $2k$ lands, Sector $2k+1$ is lost):** Where does `pd_lsn` sit inside an 8,192-byte page? At bytes `0..7`, right at the start of Sector $2k$. When Sector $2k$ lands on flash and power cuts before Sector $2k+1$ arrives, the torn page carries the *new* `pd_lsn = 0/1140` in Sector $2k$, while Sector $2k+1$ still misses the 56-byte row. On reboot, recovery reads block 7, sees `page.pd_lsn` = `0/1140` $\geq$ `0/1140`, and concludes block 7 was already fully written. It **skips record** `0/1140`, leaving the page permanently broken.
2. **Failure Case B (Sector $2k+1$ lands first, Sector $2k$ is lost):** NVMe queues freely reorder unbarriered 4 KB sector writes. Suppose Sector $2k+1$ (row bytes) hits flash while Sector $2k$ (header and slot pointers) stays at old `pd_lsn = 0/1080`. Worse, suppose record `0/1140` compacted the page (`PageRepairFragmentation`), shifting twenty rows upward in Sector $2k+1$ and updating twenty slot offsets in Sector $2k$. Replaying physiological deltas onto a half-shifted, half-unshifted 8 KB page reads misaligned offsets and scrambles the block.

Why it works

Physiological delta replay works only when the 8 KB page on disk is a **clean snapshot from one real instant in time**—either wholly before a WAL record or wholly after it. A torn 8 KB page never existed in RAM; no chain of deltas can build safely on two mismatched 4 KB halves.

4.3.1 Hole-Stripped Full Page Images (FPIs) vs. Doublewrite Buffers

Storage engines solve torn pages in two main ways. MySQL's InnoDB uses a **doublewrite buffer**: before scattering dirty 16 KB pages to table files, InnoDB writes full 16 KB copies sequentially into a staging area and calls `fsync()`, doubling every background page write.

PostgreSQL takes a lighter path governed by `full_page_writes = on` (enabled by default). Instead of writing every evicted page twice, PostgreSQL ties torn-page protection to checkpoints. Whenever a checkpoint starts, PostgreSQL records that starting WAL offset in a shared pointer called RedoRecPtr:

Formal

The `full_page_writes` Rule (`XLogCheckBufferNeedsBackup`). Let RedoRecPtr be the WAL LSN where the latest checkpoint *began*. When a backend edits buffer P and logs a

WAL record, it checks P's *previous* `pd_lsn` (before stamping the new LSN):

- If $\texttt{pd_lsn}_{\text{prior}}(P) \leq \text{RedoRecPtr}$, this is the **first edit to page P since the checkpoint began**. PostgreSQL embeds a complete **Full Page Image (FPI)** of the 8 KB page inside this WAL record (`BKPBLOCK_HAS_IMAGE | BKPIMAGE_APPLY`).
- If $\texttt{pd_lsn}_{\text{prior}}(P) > \text{RedoRecPtr}$, an earlier record after RedoRecPtr already saved an FPI for page P. Every 2nd, 3rd, ..., 10,000th edit to page P until the next checkpoint writes **only a compact 55–80 byte delta**.

Trace how this heals both Case A and Case B at once. A checkpoint starts at RedoRecPtr = `0/1000`. Block 7 (previously at `0/0E20`) is touched first at `0/1080` (which saves an FPI), updated by deltas at `0/1140` and `0/1190`, and then torn on disk when power fails.

On reboot, recovery scans forward from RedoRecPtr = `0/1000`. The first record touching block 7 is `0/1080`, which carries an FPI. When recovery (`XLogReadBufferForRedoExtended`) sees an FPI, it **never reads or trusts the torn disk page's** `pd_lsn`. It zeroes an 8 KB buffer in RAM, copies the clean FPI straight from WAL, sets `pd_lsn = 0/1080`, and returns `BLK_RESTORED`. Even if Sector $2k$ on disk claimed `0/1190`, the FPI overwrites all 8 KB in RAM back to the intact `0/1080` state. Next, recovery applies the 64-byte delta at `0/1140` and the 58-byte delta at `0/1190` cleanly in RAM.

Better still, an FPI rarely writes all 8,192 bytes to WAL. Between `pd_lower` (end of slot pointers) and `pd_upper` (start of rows) sits an empty **free-space hole**. PostgreSQL strips that hole (`BKPIMAGE_HAS_HOLE`), storing only its start offset and length in a 4-byte header:

$$\text{Raw FPI bytes} = 8{,}192 - (\texttt{pd_upper} - \texttt{pd_lower}).$$

If block 7 holds ten rows totaling 2,000 bytes (`pd_lower = 64`, `pd_upper = 6192`), the hole is 6,128 bytes, so the raw FPI takes only $64 + 2{,}000 = 2{,}064$ bytes. Turning on `wal_compression = lz4` (or `zstd` in PG 15+) shrinks those 2,064 bytes further to **650–900 bytes**.

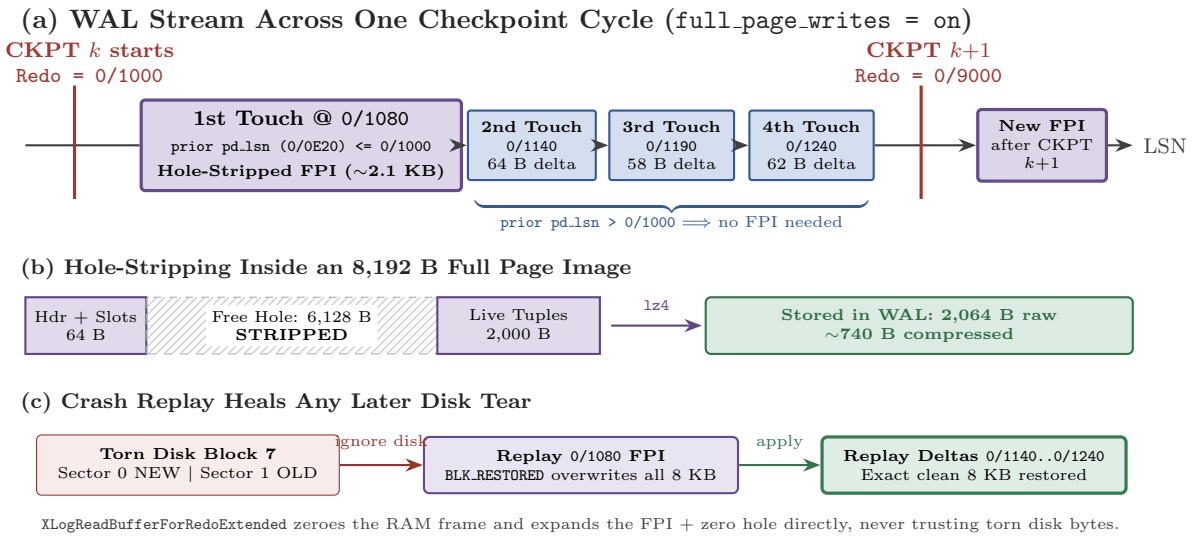


Figure 4.3: How `full_page_writes` eliminates torn pages without a doublewrite buffer. Only the *first* edit to Block 7 after each checkpoint writes a hole-stripped Full Page Image (`0/1080`); subsequent edits before the next checkpoint write tiny ~60-byte deltas. On crash recovery, the FPI unconditionally overwrites the entire 8 KB block in memory (`BLK_RESTORED`), healing any mid-write tear on disk before deltas replay.

In production

Diagnosing and taming the "Checkpoint WAL Storm" in production. Starting a checkpoint advances RedoRecPtr. Right away, *every hot page in* ***`shared_buffers`*** satisfies `pd_lsn` ≤ RedoRecPtr again. Over the next 30 to 90 seconds, the first update to each active page emits a Full Page Image. That burst spikes WAL volume (`wal_fpi` and `wal_bytes` in `pg_stat_wal`), disk sync load, and replication traffic by 3× to 10×. Two settings flatten that sawtooth wave on busy OLTP clusters:

1. Raise `checkpoint_timeout` from `5min` to `15min`–`30min` (raising `max_wal_size` to `16GB`–`64GB` to match), so hot buffers pay the FPI cost three to six times less often.
2. Set `wal_compression = lz4` (or `zstd`), which cuts FPI bytes by 60%–75% and often *raises* commit throughput on cloud block volumes.

Never set `full_page_writes = off` on `ext4` or `XFS`. It is safe only on Copy-on-Write filesystems like ZFS with `recordsize=8k`, where the filesystem never overwrites an 8 KB block in place.

4.4 Commit Durability Tradeoffs, Group Commit, and `wal_writer`

Appending a 70-byte WAL record is sequential, avoiding scattered table seeks. Still, a sharp engineer should ask a latency question right here. Even sequential `fdatasync()` takes 80 μs on local NVMe and 800 μs on a cloud block volume. If one thread holds a lock for 800 μs on every `COMMIT`, isn't a 64-core server stuck at 1/0.0008 s = 1,250 commits per second?

If every session ran its own `fdatasync()` one by one, that ceiling of 1,250 transactions per second

(TPS) would be unbreakable. Breaking past it requires separating *copying WAL bytes into shared RAM* from *flushing shared RAM to flash*, then batching concurrent committers via **group commit**.

4.4.1 `wal_buffers`, Parallel Insertion Slots, and Group Commit

In shared memory, PostgreSQL allocates a ring buffer of 8 KB WAL frames sized by `wal_buffers` (1/32 of `shared_buffers`, clamped between 64 KB and 16 MB). Two separate lock layers coordinate that ring:

1. **Parallel RAM insertion (`NUM_XLOGINSERT_LOCKS = 8`):** When forty sessions run `INSERT` or `UPDATE` at once, each computes its `XLogRecord` checksum in private memory. Each session then grabs one of eight `WALInsertLocks`, reserves its exact byte slice $[L_{\text{start}}, L_{\text{end}})$ via an atomic fetch-add, and copies its bytes into `wal_buffers` alongside seven other CPU cores.
2. **Leader-follower disk flush (`WALWriteLock`):** When session B_1 runs `COMMIT`, it appends a tiny commit record ending at $L_1 =$ `0/1500` and calls `XLogFlush(0/1500)`. Only one process at a time holds `WALWriteLock` to call `write()` and `fdatasync()` on the WAL file. Watch what happens during the 800 μs while B_1 waits inside the kernel for flash programming:
   - Sessions $B_2, B_3, \ldots, B_{40}$ do *not* stall on copying. Using the eight insertion locks, all thirty-nine append their commit records into `wal_buffers`, pushing the RAM tail from `0/1500` to $L_{40} =$ `0/4B20`. They then queue behind `WALWriteLock`.
   - Once B_1's `fdatasync()` finishes, B_1 publishes `Flush = 0/1500` in shared memory and drops `WALWriteLock`.
   - Session B_2 wakes up as the new **group leader**. Instead of flushing only to its own $L_2 =$ `0/1590`, B_2 checks how far `wal_buffers` has filled—**all the way to** $L_{40} =$ `0/4B20`. Leader B_2 writes the whole slice `(0/1500 .. 0/4B20]` in one sequential `write()` and runs **one** `fdatasync()` for all thirty-nine transactions.
   - When B_2 publishes `Flush = 0/4B20` and drops `WALWriteLock`, followers $B_3 \ldots B_{40}$ see `Flush` $\geq L_i$ in shared memory and return `COMMIT` with **zero** system calls.

Under the hood

Here is the core leader-follower double-check pattern distilled from `XLogFlush()` in `src/backend/access/transam/xlog.c`:

```
void XLogFlush(XLogRecPtr record_end_lsn)
{
    /* Fast path: did an earlier group leader already sync past our LSN? */
    if (record_end_lsn <= LogwrtResult.Flush)
        return;

    /* Wait our turn (or wake up after the current leader finishes) */
    LWLockAcquire(WALWriteLock, LW_EXCLUSIVE);

    /* Refresh shared Flush pointer; if the previous leader swept our
     * commit record into its fdatasync batch, exit with ZERO syscalls! */
    RefreshLogwrtResult();
    if (record_end_lsn <= LogwrtResult.Flush)
    {
```

```
        LWLockRelease(WALWriteLock);
        return;
    }

    /* We are the new group leader: flush ALL ready WAL bytes in wal_buffers,
     * sweeping in every follower that appended while we waited! */
    XLogRecPtr insert_pos = WaitXLogInsertionsToFinish(GetXLogInsertRecPtr());
    XLogWrite(insert_pos);   /* sequential write() + single fdatasync()  */
    LWLockRelease(WALWriteLock);
}
```

Now forty clients on a 0.8 ms cloud disk are no longer capped at 1,250 TPS: batching thirty-nine commits per 0.8 ms flush lifts durable throughput past 45,000 TPS on the same drive. PostgreSQL also offers `commit_delay` (a brief leader pause when at least `commit_siblings` sessions are active), though real concurrency already builds full batches naturally while the previous `fdatasync()` runs.

4.4.2 The `synchronous_commit` Spectrum and `wal_writer`

Suppose a table ingests millions of telemetry readings or click events per minute. Losing the last half-second of clicks during a rare power cut is fine, provided the database **never corrupts**.

Faced with that goal, developers sometimes reach for the wrong knob and destroy a cluster by setting `fsync = off`. Separate those two settings permanently:

Common trap

`synchronous_commit = off` is safe engineering; `fsync = off` is Russian roulette.

- `fsync = off` strips `fdatasync()` out of PostgreSQL everywhere—commits, checkpoints, buffer evictions, and control-file updates. If power drops, table pages, B-tree pages, commit logs, and `pg_control` hit disk out of order. That breaks the WAL Invariant and leaves the cluster unrecoverable. Use `fsync = off` *only* on disposable test containers.
- `synchronous_commit = off` (even per-transaction via `SET LOCAL synchronous_commit = off;`) skips *only* the foreground wait at `COMMIT`: the backend appends `XACT_COMMIT` into `wal_buffers` and replies right away without calling `XLogFlush()`. Even so, **`full_page_writes` and the WAL Invariant in `FlushBuffer()` stay 100% active.** No dirty page can ever reach disk ahead of its synced WAL record.

Who flushes `wal_buffers` when sessions use `synchronous_commit = off`? A background daemon named `wal_writer` wakes every `wal_writer_delay` (default `200ms`, or once `wal_writer_flush_after = 1MB` builds up) and flushes unwritten WAL. It writes full 8 KB WAL pages on normal ticks and flushes any partial tail page once a cycle passes, bounding unsynced commits on power loss to at most $3 \times$ `wal_writer_delay` $\approx$ **600** ms. After a power outage, PostgreSQL recovers cleanly to a 100% valid state from ≤ 600 ms earlier, with zero torn pages and zero broken indexes.

Better still, `synchronous_commit` works per transaction. On the same connection pool, a payment transfer uses `synchronous_commit = on`, while a click logger runs:

```
BEGIN;
```

```
SET LOCAL synchronous_commit = off;
INSERT INTO page_views (user_id, url, viewed_at) VALUES (4812, '/checkout', now());
COMMIT;  -- returns in ~15 microseconds without waiting on NVMe fdatasync!
```

Once you add streaming standbys (Chapter 8), `synchronous_commit` expands to five levels. Each transaction picks how many failure domains its commit record must cross before replying `COMMIT`:

Setting	Primary WAL `fdatasync`?	Sync standby guarantee before client ACK	Power loss on primary host	Primary disk destroyed
`off`	Async ($\leq 3d$)	None	Lose $\leq$ 600 ms; 0 corruption	Lose unshipped WAL
`local`	**Yes**	None (local only)	**0 committed rows lost**	Lose unshipped WAL
`remote_write`	**Yes**	Standby OS page cache (`write`)	**0 committed rows lost**	0 lost unless dual power cut
`on` *(default)*	**Yes**	Standby NAND (`fdatasync`)*	**0 committed rows lost**	**0 committed rows lost**
`remote_apply`	**Yes**	Standby NAND + replayed in RAM	**0 committed rows lost**	**0 lost + read-your-writes**

Table 4.1: Exact durability guarantees across all five `synchronous_commit` modes (d = `wal_writer_delay` = 200 ms). *When `synchronous_standby_names` is empty, `on` behaves identically to `local`.

4.4.3 `UNLOGGED` Tables: Trading Crash Survival for Raw RAM-to-Disk Speed

For scratch data—nightly ETL staging tables, rebuildable caches, or temporary session state—even compact WAL records are wasted work. Declaring a table `UNLOGGED`:

```
CREATE UNLOGGED TABLE etl_staging_events (
    batch_id    bigint NOT NULL,
    payload     jsonb  NOT NULL
);
```

tells PostgreSQL (`relpersistence = 'u'` in `pg_class`) to **skip WAL completely** on writes and index updates. Sessions edit 8 KB pages in `shared_buffers` without WAL locks, Full Page Images, or replication traffic, often doubling or tripling bulk load speed.

How does PostgreSQL stop half-flushed `UNLOGGED` pages from corrupting queries after a crash? During `CREATE UNLOGGED TABLE`, PostgreSQL writes one tiny WAL-logged template file next to the main file (say `base/16384/24580`): an empty initialization fork named `24580_init` (plus an empty root `_init` fork for each index).

- On a **clean shutdown** (`pg_ctl stop -m fast`), the shutdown checkpoint flushes every dirty `UNLOGGED` buffer cleanly, so all rows survive normal restarts.
- On **any crash recovery**, `ResetUnloggedRelations()` finds every relation in `PGDATA/base/` with an `_init` fork, deletes the potentially torn main files, and copies `24580_init` over `24580`. Every `UNLOGGED` table resets cleanly to zero rows before clients connect.

4.5 Checkpoints, Background Writer, and Step-by-Step Crash Recovery

One puzzle remains. If we append to `PGDATA/pg_wal/` forever without flushing dirty pages to table files, `pg_wal` fills the SSD, and a crash after six months of uptime has to replay six months of WAL from `0/0`.

A **checkpoint** establishes a safe floor on disk so PostgreSQL can recycle old 16 MB WAL files and keep crash recovery down to seconds. How an online checkpoint runs without freezing user traffic hinges on two separate pointers inside `pg_control`.

4.5.1 `global/pg_control`, Fuzzy Checkpoints, and Why Recovery Rewinds to the Redo Point

When a crashed server boots up, how does `StartupXLOG()` know where in `pg_wal` to start reading? It opens a tiny metadata file at `PGDATA/global/pg_control`. That file holds one checksum-protected struct (`ControlFileData`) small enough to fit inside a single 512-byte sector for atomic updates:

```
/* Simplified excerpt from src/include/catalog/pg_control.h */
typedef struct CheckPoint
{
    XLogRecPtr      redo;           /* Redo start LSN when ckpt BEGAN!   */
    TimeLineID      ThisTimeLineID; /* current WAL timeline              */
    FullTransactionId nextXid;      /* next free 64-bit XID              */
    /* ... oldestActiveXid, oldestMulti, timestamp ... */
} CheckPoint;

typedef struct ControlFileData
{
    uint64          system_identifier;
    DBState         state;          /* DB_SHUTDOWNED vs DB_IN_PRODUCTION */
    XLogRecPtr      checkPoint;     /* LSN of latest Checkpoint Record   */
    CheckPoint      checkPointCopy; /* embedded copy including .redo!    */
    uint32          data_checksum_version;
    pg_crc32c       crc;
} ControlFileData;
```

Look at those two LSN fields in `ControlFileData`: `checkPoint` (say `0/1230`) and `checkPointCopy.redo` (say `0/1000`). Why are they different?

Picture a server with 64 GB of `shared_buffers` trying to take an instant "stop-the-world" checkpoint. It would have to freeze all writes, blast 20 GB of dirty pages to disk, `fsync` three hundred files, log one checkpoint record, and unfreeze clients. Every five minutes, user requests would stall for 25 seconds.

Instead, PostgreSQL runs **spread (fuzzy) online checkpoints** controlled by three settings:

- `checkpoint_timeout` (default `5min`, often tuned to `15min`–`30min`): max time between checkpoints.
- `max_wal_size` (default `1GB`, often `16GB`–`64GB`): soft WAL budget before starting an early checkpoint.

- `checkpoint_completion_target` (default `0.9`): fraction of the interval spent pacing disk writes (0.9×300 s $= 270$ s).

Trace what happens from $t = 0$ s to $t = 270$ s while normal traffic keeps running:

1. **At $t = 0$ s (LSN `0/1000`):** `checkpointer` wakes up and records the current WAL offset as `RedoRecPtr = 0/1000` (triggering Full Page Images for pages touched from `0/1000` onward). It then flags every buffer *already dirty at $t = 0$ s* with `BM_CHECKPOINT_NEEDED`.
2. **Across $t \in (0\text{ s}, 270\text{ s})$:** `checkpointer` trickles those flagged buffers to disk with gentle pauses and finally calls `fsync()` on each touched file. Meanwhile, clients never stop writing. At $t = 40$ s (LSN `0/1080`), client T_1 edits buffer B_7 (which was clean at $t = 0$ s, or already flushed at $t = 10$ s). Buffer B_7 now sits dirty in RAM with `pd_lsn = 0/1080`, and `checkpointer` does **not** have to flush B_7 during this cycle.
3. **At $t = 270$ s (LSN `0/1230`):** Once every pre-`0/1000` dirty buffer is synced to flash, `checkpointer` writes an `XLOG_CHECKPOINT_ONLINE` record at `0/1230` storing `redo = 0/1000`. It syncs WAL to `0/1230`, updates `global/pg_control`, and recycles WAL segments strictly older than `0/1000`.

Pause

Pause and check the two checkpoint LSNs. The checkpoint record lands at `0/1230` when flushing *finishes*, whereas `redo = 0/1000` marks when flushing *started*. Only edits before `0/1000` are guaranteed to sit in table files on disk, so crash recovery always rewinds to `0/1000` and replays forward from there.

Why it works

Rewinding to `redo = 0/1000` protects every edit made while the checkpoint was running, such as T_1's change to B_7 at `0/1080`. Starting replay at `0/1230` would lose T_1's committed row. That overlap also explains PostgreSQL's WAL sizing formula (`CalculateCheckpointSegments`). While Checkpoint 2 runs for $c \cdot T$ seconds (c = `checkpoint_completion_target` = 0.9), PostgreSQL must still keep WAL back to Checkpoint 1's Redo point. Right before Checkpoint 2 finishes, live WAL spans $(1 + c)$ intervals:

$$\text{Peak live WAL} \approx (1 + \texttt{checkpoint_completion_target}) \times R_{\text{wal}} \times \texttt{checkpoint_timeout}.$$

At $R_{\text{wal}} = 20$ MB/s and `checkpoint_timeout = 15min` (18 GB per interval), `max_wal_size` needs at least 1.9×18 GB $\approx$ **35** GB to avoid early forced checkpoints.

4.5.2 Three Buffer Writers, Three Distinct Jobs

Dirty 8 KB buffers leave `shared_buffers` through three different processes, tracked in `pg_stat_checkpointer`, `pg_stat_bgwriter`, and PG 16+'s `pg_stat_io`:

- **`checkpointer` (recovery floor & WAL recycling):** Flushes every buffer dirtied before RedoRecPtr once per checkpoint and calls `fsync()`. On a well-tuned server, `checkpointer` performs 70%–95% of all data writes, collapsing 500 in-RAM updates to a hot block into a

single disk write.

- `bgwriter` **(clean-buffer headroom):** Wakes every `bgwriter_delay` (`200ms`) and sweeps just ahead of the clock-sweep hand, cleaning low-usage dirty buffers (up to `bgwriter_lru_maxpages`) so user queries find clean frames ready. It never writes checkpoints.
- **Client backend writes (the warning light):** When a user query needs a buffer frame and hits a dirty victim before `bgwriter` cleaned it, *that user's `SELECT` must call `FlushBuffer()` inline.* Rising client-backend writes in `pg_stat_io` mean `shared_buffers` or `bgwriter` is falling behind.

4.5.3 Classic ARIES (3 Passes) vs. PostgreSQL's 1-Pass Redo-Only Recovery

Most textbooks teach IBM's **ARIES** recovery algorithm (Mohan et al., 1992), designed for engines that overwrite rows in place. On startup, ARIES runs **three passes** over the log:

1. **Analysis pass (forward):** Scans from the checkpoint to rebuild the Dirty Page Table and list uncommitted "loser" transactions.
2. **Redo pass (forward, "repeating history"):** Replays all logged physical edits—winners and losers alike—restoring pages to their exact pre-crash state.
3. **Undo pass (backward):** Walks backward along each loser transaction's log chain, physically undoing uncommitted edits and writing Compensation Log Records. If a batch job updated 50 million rows and power died at 99%, eager ARIES Undo must roll back 50 million rows.

Now open `src/backend/access/transam/xlogrecovery.c` in PostgreSQL and look for the Undo pass. **There isn't one.** PostgreSQL writes zero Undo records and never scans WAL backward.

Connection

How MVCC (Chapter 5) eliminates the Undo phase completely. Recall from Chapter 2 that a PostgreSQL `UPDATE` never overwrites a row in place: it marks `t_xmax = 702` on the old version and inserts a new version marked `t_xmin = 702`. During crash recovery, PostgreSQL runs **one forward Redo pass** from RedoRecPtr to the end of valid WAL, replaying both committed T_1 (`xid = 701`) *and* crashed T_2 (`xid = 702`).
Why doesn't T_2's uncommitted row corrupt queries when clients reconnect? Commit status lives in PostgreSQL's Commit Log (CLOG), stored under `PGDATA/pg_xact/` with two bits per 32-bit transaction ID (XID): `00 = IN_PROGRESS`, `01 = COMMITTED`, `10 = ABORTED`, and `11 = SUB_COMMITTED`. Transaction T_1 logged `XACT_COMMIT`, so Redo sets `pg_xact[701] = 01`. Crashed T_2 never logged `COMMIT`, so `pg_xact[702]` stays `00` while recovery advances `nextXid` past `702`. Any past XID still marked `00` is **instantly treated as aborted by every snapshot**. Readers ignore T_2's new version, keep seeing the old version, and let background `VACUUM` reclaim dead bytes later. Whether T_2 touched ten rows or a hundred million, startup Undo time is **0** ms.

Inside `xlogutils.h`, every block reference inspected during that single forward Redo pass resolves to one of four explicit actions returned by `XLogReadBufferForRedoExtended()`:

```
typedef enum
{
    BLK_NEEDS_REDO, /* page.pd_lsn < record.end_lsn -> apply delta & set LSN */
```

```
    BLK_DONE,       /* page.pd_lsn >= record.end_lsn -> already applied, skip */
    BLK_RESTORED,   /* record carried an FPI -> 8 KB block restored from WAL  */
    BLK_NOTFOUND    /* relation/block truncated & dropped later -> skip       */
} XLogRedoAction;
```

Algorithm 1 puts the entire `StartupXLOG()` recovery state machine onto one page.

Algorithm 1: PostgresCrashRecovery: Single-Pass Redo in `xlogrecovery.c`

Input: `global/pg_control` and segment files in `PGDATA/pg_wal/`.
Output: Consistent `shared_buffers`, `pg_xact`, and relation files ready for clients.

Read `ControlFileData` from `global/pg_control` and verify CRC32C;
; `// Bootstrap control file.`
if state = *DB_SHUTDOWNED* **then**
    return ; `// Clean shutdown; zero recovery needed.`
Read Checkpoint Record at `ControlFile.checkPoint` and set RedoLSN ← `ckpt.redo`;
; `// Find Redo start.`
Initialize WAL reader at RedoLSN;
while ReadNextXLogRecord*() returns header + payload with valid* ***xl_prev*** *and CRC32C* **do**
    L_{end} ← ending byte offset of current `XLogRecord`;
    ; `// Record end LSN.`
    nextXid ← max(nextXid, `record.xl_xid` + 1);
    if *record.xl_rmid* = *RM_XACT_ID* **then**
        Update `pg_xact[record.xl_xid]` to `01 (COMMITTED)` or `10 (ABORTED)`;
        ; `// Update commit log.`
    else
        foreach *Block B referenced in* *record* **do**
            if *Block B carries a Full Page Image (BKPIMAGE_APPLY)* **then**
                Zero frame P in `shared_buffers` and expand the FPI into P.;
                Set `P.pd_lsn` ← L_{end} and mark P dirty;
                ; `// BLK_RESTORED: heals any tear.`
            else
                Read disk block B into buffer frame P.;
                if *P.pd_lsn* $< L_{end}$ **then**
                    Call `RmgrTable[record.xl_rmid].rm_redo(record, P);`
                    ; `// BLK_NEEDS_REDO.`
                    Set `P.pd_lsn` ← L_{end} and mark P dirty.;
                `// Otherwise P.pd_lsn` $\geq L_{\text{end}} \implies$ `BLK_DONE (idempotent skip).`
`// First invalid CRC32C or header marks exact end-of-WAL boundary.`
ResetUnloggedRelations() ; `// Copy _init forks over all UNLOGGED tables.`
Write end-of-recovery checkpoint (or promote); set state ← `DB_IN_PRODUCTION`;

4.5.4 Worked LSN-by-LSN Crash Recovery Trace

Let us trace Algorithm 1 on concrete hex LSNs so every step is crystal clear. Heap Block 7 of `accounts` starts on disk with `pd_lsn` = `0/0E20` and one row `(id=1, bal=500)`. Watch six events run right before power fails:

1. `0/1000` **(Checkpoint starts):** `checkpointer` starts an online checkpoint and sets RedoRecPtr = `0/1000`.
2. `0/1080` **(T_1, `xid=701`, first touch):** T_1 inserts `(id=2, bal=300)` on Block 7. Because prior `pd_lsn` = `0/0E20` $\leq$ `0/1000`, PostgreSQL logs `INSERT` with a **Full Page Image** ending at `0/1080`, stamping `pd_lsn` = `0/1080` in RAM.
3. `0/1140` **(T_1, `xid=701`, delta update):** T_1 updates `id = 1` to `bal = 400` via a same-page `HOT_UPDATE`. Because prior `pd_lsn` = `0/1080` > `0/1000`, PostgreSQL logs only a 64-byte delta ending at `0/1140`.
4. `0/1190` **(T_2, `xid=702`, uncommitted insert):** Concurrent session T_2 inserts `(id=3, bal=999)` on Block 7 via a 58-byte delta ending at `0/1190`. Session T_2 has *not* committed yet.
5. `0/1210` **(T_1 commits):** T_1 runs `COMMIT`, logs `COMMIT` for `xid=701`, and calls `XLogFlush(0/1210)`. Client T_1 receives `COMMIT`.
6. `0/1230` **(Checkpoint ends) &** `0/1260` **(Torn write + power cut):** `checkpointer` logs `XLOG_CHECKPOINT_ONLINE` (`redo = 0/1000`) at `0/1230` and updates `pg_control`. Moments later, `bgwriter` flushes Block 7 (`pd_lsn = 0/1190`)—allowed since WAL is synced past `0/1230`—and **power cuts mid-write**:
   - On disk, **Block 7 is torn**: Sector 0 has `pd_lsn = 0/1190` and three slot pointers, while Sector 1 still holds stale pre-`0/1000` bytes.
   - In `pg_wal`, T_2 was writing another record at `0/1260`; only 40 bytes reached flash, leaving a broken `xl_crc`.

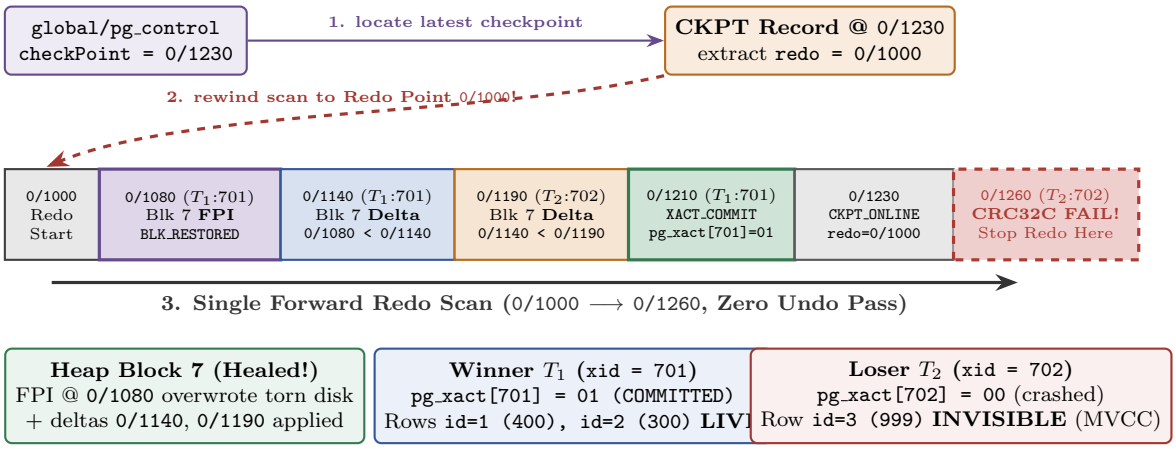


Figure 4.4: Crash-recovery trace from `global/pg_control` to readiness. Recovery reads `0/1230`, rewinds to `redo = 0/1000`, heals torn Block 7 via the `0/1080` FPI, and replays T_1 and T_2 forward. It stops cleanly at the `0/1260` checksum mismatch, while `pg_xact` and MVCC hide T_2's uncommitted row in $O(1)$ time.

Now step through Table 4.2 as electricity returns and `StartupXLOG()` boots:

WAL LSN	Record Type	Buffer `pd_lsn` Check	Action	Resulting State in Memory
`pg_control`	Bootstrap	`state = IN_PRODUCTION`	Crash detected	Jump to `0/1230`, extract `redo = 0/1000`.
`0/1080`	`HEAP INSERT+FPI`	Ignored (has FPI)	`BLK_RESTORED`	Torn disk `pd_lsn=0/1190` discarded. Clean 8 KB Block 7 restored at `pd_lsn = 0/1080`.
`0/1140`	`HEAP HOT_UPDATE`	`0/1080 < 0/1140`	`BLK_NEEDS_REDO`	T_1 updates `id=1` to `bal=400`; Block 7 `pd_lsn` ← `0/1140`.
`0/1190`	`HEAP INSERT`	`0/1140 < 0/1190`	`BLK_NEEDS_REDO`	T_2 inserts `(id=3, bal=999)` (`t_xmin=702`); Block 7 `pd_lsn` ← `0/1190`.
`0/1210`	`XACT COMMIT`	—	Update CLOG (`pg_xact`)	`pg_xact[701]` ← `01` (`COMMITTED`).
`0/1230`	`CKPT_ONLINE`	—	Metadata sync	Update `nextXid` ≥ `703`; keep scanning.
`0/1260`	Truncated tail	`xl_crc` mismatch	**End of WAL**	Stop scan; `pg_xact[702]` stays `00` (never committed, so every later snapshot treats T_2 as aborted).

Table 4.2: Exact LSN-by-LSN state transitions executed by `StartupXLOG()` on our crashed cluster.

The server log for this restart shows the same steps:

```
LOG:  database system was interrupted; last known up at 2026-11-01 09:12:03 UTC
LOG:  database system was not properly shut down; automatic recovery in progress
LOG:  redo starts at 0/1000
LOG:  incorrect resource manager data checksum in record at 0/1260
LOG:  redo done at 0/1230
LOG:  checkpoint starting: end-of-recovery immediate wait
LOG:  database system is ready to accept connections
```

The checksum line is not an error: it is how recovery finds the end of valid WAL.

Look closely at row 2 (`0/1080`) of Table 4.2. Had `0/1080` been a delta instead of an FPI, recovery would have read torn Block 7, seen Sector 0's `pd_lsn = 0/1190`, skipped all three edits, and left Sector 1 full of garbage. Instead, `BLK_RESTORED` replaced the torn disk block in RAM at `0/1080` and replayed `0/1140` and `0/1190` cleanly on top. Meanwhile, T_2's uncommitted row `(id=3, bal=999)` stays harmlessly hidden behind `t_xmin = 702`.

Try it in PostgreSQL

In PostgreSQL 15 and later, the built-in `pg_walinspect` extension lets you watch this exact FPI-then-delta transition live inside SQL without even leaving `psql`:

```
CREATE EXTENSION IF NOT EXISTS pg_walinspect;

CREATE TABLE wal_demo (id int PRIMARY KEY, bal int);
INSERT INTO wal_demo VALUES (1, 500);

-- Force a checkpoint so RedoRecPtr advances past wal_demo's current pd_lsn:
```

```
CHECKPOINT;

-- Capture LSN right after the checkpoint, then touch Block 0 TWICE:
SELECT pg_current_wal_lsn() AS start_lsn \gset
BEGIN;
INSERT INTO wal_demo VALUES (2, 300);            -- 1st touch -> logs an FPI!
UPDATE wal_demo SET bal = 400 WHERE id = 1;      -- 2nd touch -> tiny delta!
COMMIT;
SELECT pg_current_wal_lsn() AS end_lsn \gset

SELECT start_lsn, end_lsn, xid, resource_manager AS rmgr,
       record_type, record_length AS rec_len, fpi_length AS fpi_len
FROM pg_get_wal_records_info(:'start_lsn', :'end_lsn')
WHERE resource_manager IN ('Heap', 'Transaction');
```

Running this snippet produces output structured right along our theoretical trace:

```
--  start_lsn |  end_lsn  | xid |    rmgr     | record_type | rec_len | fpi_len
-- -----------+-----------+-----+-------------+-------------+---------+---------
--  0/19420A8 | 0/1942150 | 745 | Heap        | INSERT      |     168 |     120
--  0/19421A0 | 0/19421E8 | 745 | Heap        | HOT_UPDATE  |      72 |       0
--  0/19421E8 | 0/1942210 | 745 | Transaction | COMMIT      |      40 |       0
```

Check the `fpi_len` column. On the first edit after `CHECKPOINT`, `fpi_len` is `120` bytes: the 24-byte header, two 4-byte slot pointers, and two 44-byte rows, with the free-space hole stripped out. Total record size is 168 bytes once you add the 24-byte `XLogRecord` header and block metadata. Twenty microseconds later, `HOT_UPDATE` touches that same page and logs `fpi_len = 0`—just a 72-byte delta, followed by a 40-byte `COMMIT`. Offline, you can inspect raw segment files with `pg_waldump -p PGDATA/pg_wal 000000010000000000000001`.

Cost summary

Cost Summary of PostgreSQL WAL & Crash Recovery:

- `COMMIT` **I/O (`synchronous_commit = on`):** 1 sequential `write()` + 1 `fdatasync()` shared across K concurrent committers ($O(1/K)$ sync calls per transaction; zero random heap/index `fsyncs`).
- **Per-row WAL size:** $\sim$ 55–90 bytes per heap/B-tree delta; $+(8{,}192 - \text{hole})$ bytes (or $\sim 0.3\times$ compressed with `lz4`) on *only the first touch* per 8 KB block per checkpoint.
- **Crash recovery time:** O(WAL bytes since RedoRecPtr) in a single forward scan, plus $O(1) = 0$ **Undo work** regardless of how many rows crashed transactions modified.

Everything in this chapter gives PostgreSQL a rock-solid physical floor: 8 KB pages never tear, committed rows never vanish, and crashed uncommitted writes sit behind `pg_xact` status `00`. That sets up our next big question. When twenty queries read Block 7 at the same time, how does each query decide in nanoseconds which row versions belong to its snapshot? And who eventually reclaims dead rows like T_2's aborted `(id=3, bal=999)` without blocking readers? That is the story of Multi-Version Concurrency Control, snapshots, and `VACUUM` in Chapter 5.

4.6 Exercises

Exercise 4.1 (`pg_lsn` hex arithmetic and 24-character segment filenames)**.** Consider two WAL positions L_a = `2/FE800028` and L_b = `3/24001A8` on timeline $T = 2$ with the default 16 MB segment size (`0x01000000` bytes):

1. Write L_a and L_b as 64-bit hex integers and compute `pg_wal_lsn_diff`(L_b, L_a) in hex and decimal. How many 16 MB segment boundaries does $[L_a, L_b]$ cross?
2. Using Definition 4.1, give the 24-character filename in `PGDATA/pg_wal/` and the decimal byte offset inside that file for L_a and L_b.
3. Repeat part (2) for `initdb --wal-segsize=64` (64 MB segments = `0x04000000` bytes, or 64 = `0x40` segments per 4 GB chunk).

Exercise 4.2 (Hole-stripped FPI byte budget and checkpoint WAL amplification)**.** An 8,192-byte heap page holds 20 live rows of 120 bytes each and zero dead slots. Recall that `PageHeaderData` is 24 bytes and each slot pointer is 4 bytes.

1. Compute `pd_lower`, `pd_upper`, the free-space hole length, and the uncompressed size of the hole-stripped FPI (plus 28 bytes of record and image headers).
2. Suppose an app runs 50 single-row `HOT_UPDATE` commits on this page. In-page pruning keeps live row bytes at 2,400 bytes, and each non-FPI `HOT_UPDATE` + `COMMIT` pair logs 72+40 = 112 bytes. Compare total WAL bytes when (a) zero checkpoints start during the burst versus (b) a tiny `max_wal_size` starts a checkpoint every 10 updates. How much does `wal_compression = lz4` (3: 1 on FPIs) save in case (b)?

Exercise 4.3 (Proving both failure cases of delta replay without `full_page_writes`)**.** Suppose an admin sets `full_page_writes = off` on `ext4` over a 4Kn NVMe SSD. Block 4 sits cleanly on disk at `pd_lsn` = `0/5000` with `pd_upper` = 8000.

1. Build a two-record WAL sequence: `0/5060` compacts rows via `PageRepairFragmentation`, and `0/50C0` inserts a 64-byte row. Prove step-by-step why replay corrupts Block 4 both when Sector 0 (`0..4095`) lands without Sector 1 (`4096..8191`) and when Sector 1 lands without Sector 0.
2. With `initdb --data-checksums` enabled, does `pd_checksum` let PostgreSQL *repair* Block 4 without an FPI, or only stop with an error? Explain why.

Exercise 4.4 (Violating the WAL Invariant)**.** Suppose you delete the `XLogFlush(lsn)` check inside `FlushBuffer()`. Uncommitted transaction T_9 (`xid = 880`) splits B-tree leaf Block 10 into Block 10 and new Block 25 at LSN `0/7400` while `Flush` sits at `0/7100`. Next, `bgwriter` writes Block 10 (not Block 25) cleanly to flash, and power fails before `wal_buffers` flushes past `0/7100`.

1. Trace `StartupXLOG()` on reboot. What is the highest LSN replayed?
2. Even though Block 10 did not tear and T_9 never committed, prove that index scans for rows committed *hours earlier* can now miss rows or fail with missing-block errors. Why didn't MVCC save you?

Exercise 4.5 (Hand-tracing a multi-page crash recovery table)**.** After a power outage, `global/pg_control` points to a checkpoint at `0/9200` with `redo = 0/9000` and `nextXid = 500`. On disk right after the crash:

- Heap Block 3 is intact at `pd_lsn` = `0/8F00` with `(k='a', v=10)`.
- Heap Block 8 is **torn**: Sector 0 has `pd_lsn` = `0/9180`, while Sector 1 holds stale bytes from `0/8D00`.

The WAL stream from `0/9000` holds eight records:

1. `0/9060` (`xid=501`): `HEAP INSERT` + **FPI** on Block 8 (restores `(k='b', v=20)`).
2. `0/90C0` (`xid=502`): `HEAP INSERT` + **FPI** on Block 3 (adds `(k='c', v=30)`).
3. `0/9120` (`xid=501`): `HEAP HOT_UPDATE` delta on Block 8 (`k='b'` updated to `v=25`).
4. `0/9160` (`xid=502`): `XACT_ABORT`.
5. `0/9180` (`xid=503`): `HEAP INSERT` delta on Block 8 (adds `(k='d', v=40)`).
6. `0/91C0` (`xid=501`): `XACT_COMMIT`.
7. `0/9200` (`xid=0`): `XLOG_CHECKPOINT_ONLINE` (`redo = 0/9000`).
8. `0/9250` (`xid=503`): `XACT_COMMIT` — **truncated mid-record (`xl_crc` mismatch)**.

Build a trace table like Table 4.2 showing the `XLogRedoAction` for each record, the final `pd_lsn` of Blocks 3 and 8, and the `pg_xact` bits for XIDs `501`, `502`, and `503`. Which `(k, v)` rows does `SELECT * FROM t ORDER BY k` return after recovery? Under `synchronous_commit = on`, could client `503` have received a `COMMIT` reply before power died?

Exercise 4.6 (Group commit throughput and `synchronous_commit` failure windows)**.** A primary runs on a cloud volume where `fdatasync()` takes $L_{\text{sync}} = 1.25$ ms. Copying a transaction's WAL bytes into `wal_buffers` takes 25 μs of CPU across 8 `WALInsertLocks`.

1. With one sequential client ($C = 1$) and `synchronous_commit = on`, what is the max `COMMIT/s`?
2. With $C = 64$ concurrent clients running one-row transactions under `synchronous_commit = on`, find (a) `fdatasync()` calls per second, (b) average commits per leader flush, and (c) the new throughput ceiling (comparing `fdatasync` against the 8-lock insertion limit).
3. Suppose a synchronous standby sits 2.0 ms away (round-trip). For each `synchronous_commit` mode in Table 4.1, give client commit latency and state whether acknowledged commits can be lost on (i) primary kernel panic, (ii) primary power loss, or (iii) permanent primary SSD loss.

Exercise 4.7 (Sizing `max_wal_size` and checkpointer I/O bandwidth)**.** An OLTP cluster has `shared_buffers = 32GB`, emits WAL at $R_{\text{wal}} = 45$ MB/s, and dirties 14.4 GB of buffers per checkpoint. Settings are `checkpoint_timeout = 20min` and `checkpoint_completion_target = 0.9`.

1. Find the WAL written per 20-minute interval. Using the (1+`checkpoint_completion_target`) rule, compute the minimum `max_wal_size` (plus 20% burst headroom) so checkpoints trigger by time rather than WAL volume.
2. Compute the average write rate—in MB/s and in 8 KB I/O operations per second (IOPS)—that `checkpointer` sends to the OS across its spread window ($0.9 \times 1{,}200$ s).
3. Suppose power fails one second *before* Checkpoint $k + 1$ writes its checkpoint record. How many gigabytes of WAL must `StartupXLOG()` scan on reboot, and how long does recovery take at 600 MB/s?

Chapter 5

MVCC, Snapshots, `VACUUM`, and Transaction ID Wraparound

In Chapter 4, Write-Ahead Logging solved crash survival: if power fails mid-transaction, the redo log guarantees that committed bytes survive and torn 8 KB pages heal on restart. Now suppose the server stays up, the SSDs are healthy, and five hundred clients connect at once.

Picture a payment platform at 4:00 PM. Four hundred checkout workers are running small transfers—subtracting 50 USD from one row of `accounts` and adding 50 USD to another—while an analyst audits all ten million accounts:

```
SELECT SUM(balance_minor) FROM accounts;
```

Suppose a transfer moves 5,000 cents from Account A (on block 12) to Account Z (on block 98,400) while the scan sits at block 40,000. If the scan reads Account A *before* the debit and reaches block 98,400 *after* the credit commits, it counts those 5,000 cents twice. Flip the block order and 5,000 cents vanish from the audit, even though every single transfer balanced.

Early databases prevented that miscount with two-phase read/write locks. The `SELECT` locked every page it read, blocking writers until all ten million rows finished scanning, while write locks stalled the scan right back. Throughput collapsed whenever analytics and live payments shared a table.

PostgreSQL takes a different path: *Multi-Version Concurrency Control* (*MVCC*). Under MVCC, **readers never block writers, and writers never block readers**. Instead of overwriting a row in place while another query might still be reading it, every update writes a fresh version alongside the old one. Transaction IDs stamped on each version tell every reader which copy belongs to its moment in time.

If you have used Java's `CopyOnWriteArrayList`, you already know this pattern. An iterator keeps walking the array it started with, even while a writer builds a modified copy and swaps the list pointer. Readers take zero locks, and the garbage collector frees the old array once no iterator needs it. PostgreSQL applies that same idea one row at a time, with background `VACUUM` acting as the garbage collector.

Focus

By the end of this chapter, you will be able to answer five questions from first principles:

- Why does PostgreSQL write new row versions right into table pages instead of using a separate undo log, and why does that make a ten-million-row `ROLLBACK` instant?
- How can a 100% read-only `SELECT` query dirty thousands of buffer pages and emit megabytes of Write-Ahead Log?
- How does a three-number snapshot string like `740:748:742,745` let an audit query read a frozen point in time while payments commit around it?
- Why does indexing a frequently updated column slow down *every* other index on the table, and how do same-page (HOT) updates avoid that trap?
- How can 32-bit transaction counter wraparound make committed rows vanish without touching a byte on disk—and how does tuple freezing prevent it?

5.1 Two Ways to Build a Time Machine: Undo Logs vs. In-Heap Versions

Once you decide that an `UPDATE` must not destroy the bytes an older reader still needs, there are only two places in the storage engine where the multiple versions of a row can live. Every major relational database picks one of these two designs, and nearly every operational quirk of Oracle, MySQL/InnoDB, and PostgreSQL flows from that single choice.

Design A: Overwrite in place and push old bytes to an undo log (Oracle, InnoDB). In InnoDB and Oracle, the main table page holds *only the newest version* of each row. When transaction T_2 changes a balance from `10000` to `5000`, the engine writes `5000` straight into the table page. First, though, it copies the displaced value ("balance was `10000`") into a separate *undo log* and links the row header backward to that undo record.

Look at what Design A optimizes and what it penalizes:

- **Fast in-place updates for unindexed columns:** If an `UPDATE` leaves secondary index columns untouched, secondary indexes need no new entries because the row's slot or primary key stayed put.
- **Slow `ROLLBACK` on big writes:** If a batch `UPDATE` modifies five million rows and aborts at the end, the engine must walk five million undo records and copy old bytes back into table pages.
- **Undo-chain walks for older snapshots:** A five-minute-old report query cannot read table pages at face value. Whenever it hits a recently changed row, it must chase undo links backward to rebuild an older version. If the undo log recycles space first, Oracle raises `ORA-01555: snapshot too old`.

Design B: Append new tuple versions directly into the heap (PostgreSQL). Michael Stonebraker originally built Postgres around an archival idea: never overwrite rows at all, so queries

could ask for a table "as of last Tuesday." Modern PostgreSQL replaced infinite history with garbage collection, yet kept that physical layout. When transaction T_2 updates a row, the old version stays right where it sits on its 8 KB page. PostgreSQL stamps an expiration ID onto the old version, writes the complete new version into a free slot (on the same page if room permits), and links the old version forward to the new one.

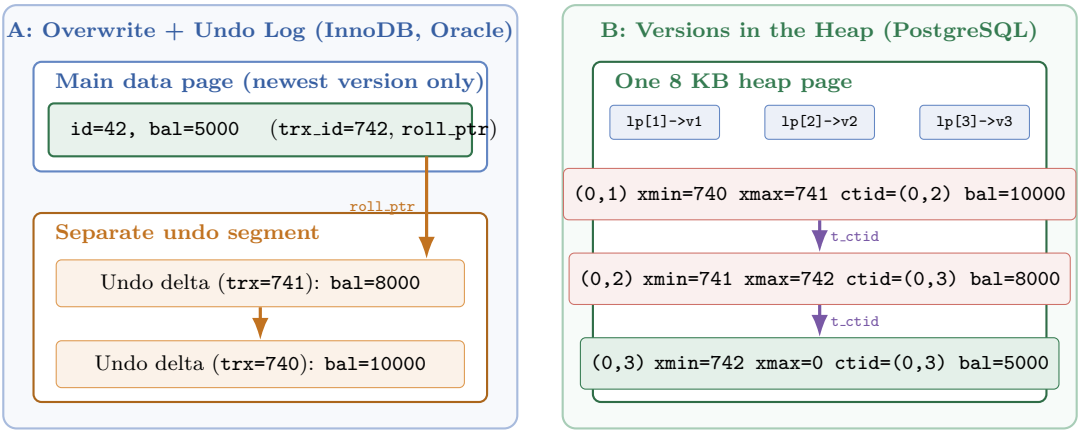


Figure 5.1: Two physical layouts for MVCC. Left: InnoDB and Oracle overwrite the main page with the newest value (`bal=5000`) and push older values into an undo segment. Older readers chase `roll_ptr` backward, and `ROLLBACK` copies undo deltas back onto data pages. Right: PostgreSQL keeps older versions inside the 8 KB heap page, stamps `xmax`, appends a new version, and links versions forward via `t_ctid`.

Intuition

In PostgreSQL, an `UPDATE` is physically a `DELETE` (stamping an expiration transaction ID onto the existing tuple) paired with an `INSERT` (appending a brand-new tuple version). Because the old bytes were never overwritten, a `ROLLBACK` of a ten-million-row update takes less than a millisecond: PostgreSQL flips two status bits in the commit log to `ABORTED` and walks away. Future readers see the aborted transaction ID on the new tuples and ignore them on sight.

Every strength and operational quirk of PostgreSQL comes back to Figure 5.1. Rollbacks run in $O(1)$ time, historical reads inspect table pages directly, and queries never abort over a full undo segment. The price is *dead tuples*. Once every active query moves past transaction `742`, versions `(0,1)` and `(0,2)` sit on the page taking up space until pruning or `VACUUM` reclaims them.

5.2 Inside `HeapTupleHeaderData`: `xmin`, `xmax`, `cid`, and `ctid`

To tell row versions apart without an undo log, every physical row version carries four small stamps on its front. Two record which transaction created the version and which transaction retired or locked it. Two more record which step inside a multi-command transaction wrote it and where any newer version lives. Chapter 2 showed that every row begins with a 23-byte header (`HeapTupleHeaderData` in `src/include/access/htup_details.h`), padded to 24 bytes. Here are those exact fields:

```
/* Simplified from src/include/access/htup_details.h */
typedef struct HeapTupleFields {
```

```
    TransactionId t_xmin;        /* 4 bytes: inserting XID */
    TransactionId t_xmax;        /* 4 bytes: deleting / locking XID, or 0 */
    union {
        CommandId t_cid;         /* 4 bytes: command ID within transaction */
        TransactionId t_xvac;    /* legacy VACUUM FULL xid */
    } t_field3;
} HeapTupleFields;               /* 12 bytes */

typedef struct HeapTupleHeaderData {
    union {
        HeapTupleFields t_heap;  /* 12 bytes: MVCC transaction fields */
        DatumTupleFields t_datum;/* used only for in-memory composite datums */
    } t_choice;
    ItemPointerData t_ctid;      /* 6 bytes: (block_hi, block_lo, offset) */
    uint16 t_infomask2;          /* 2 bytes: natts + HOT / keys-updated flags */
    uint16 t_infomask;           /* 2 bytes: hint bits, null-bitmap, lock flags */
    uint8  t_hoff;               /* 1 byte: offset to user data */
    /* optional null bitmap follows, then aligned user columns */
} HeapTupleHeaderData;           /* 23 bytes fixed */
```

Let us trace each of the four MVCC fields, because PostgreSQL packs a lot of state into these 20 bytes.

5.2.1 `t_xmin` and Lazy Transaction ID Assignment

Every writing transaction receives an increasing 32-bit transaction ID (XID, C type `TransactionId`), such as `740`. When transaction `740` runs an `INSERT`, it stamps `t_xmin = 740` and `t_xmax = 0` (signaling "no deleter yet") into the new tuple's header.

Notice the qualifier: *every writing transaction*. What if your app runs five thousand read-only `SELECT` queries per second? Spending a 32-bit `xid` on every read would burn through four billion IDs in days. Instead, PostgreSQL assigns 32-bit XIDs *lazily*. At `BEGIN`, a session gets only a cheap, backend-local virtual XID. PostgreSQL allocates a permanent 32-bit `xid` from shared memory only when the session runs its first `INSERT`, `UPDATE`, `DELETE`, `SELECT FOR UPDATE`, or `pg_current_xact_id()`.

5.2.2 `t_xmax`: Deletion, Update Chains, and Row Locks

On a freshly inserted row, `t_xmax = 0`. Three events write a non-zero ID into `t_xmax`:

1. **A `DELETE`:** Transaction `741` deletes the row by setting `t_xmax = 741`. Its `t_ctid` still points to its own slot, say `(0,1)`, meaning "no newer version follows me."
2. **An `UPDATE`:** Transaction `741` sets `t_xmax = 741` on old version `(0,1)`, inserts a new version at `(0,2)` with `t_xmin = 741, t_xmax = 0`, and points the old version's `t_ctid` to `(0,2)`.
3. **A row lock (`SELECT ... FOR SHARE / FOR UPDATE`):** Suppose transaction `741` locks a row without editing it. Keeping a shared-memory lock entry per row in a million-row job would exhaust RAM. Instead, PostgreSQL writes `t_xmax = 741` right onto the row and sets `HEAP_XMAX_LOCK_ONLY` (`0x0080`) in `t_infomask`, telling readers the row was *not* deleted. If transactions `741` and `743` both share-lock the same row, PostgreSQL allocates one *MultiXactId* for $\{741, 743\}$, stores it in `t_xmax`, and sets `HEAP_XMAX_IS_MULTI` (`0x1000`).

5.2.3 `t_cid` and the Halloween Problem

Why spend 4 header bytes on `t_cid` (Command ID)? Suppose one transaction runs two SQL statements back to back:

```
BEGIN;
-- Command 0: give every underpaid account a 1,000-cent bonus
UPDATE accounts SET balance_minor = balance_minor + 1000
 WHERE balance_minor < 50000;

-- Command 1: inspect our own uncommitted change
SELECT balance_minor FROM accounts WHERE id = 42;
COMMIT;
```

Both commands share the *same* `xid`, say `740`. Look at the two opposite visibility needs inside transaction `740`:

- In Command 1 (`SELECT`), transaction `740` *must* see the new version written by Command 0, and *must not* see the retired old version.
- During Command 0 (`UPDATE`) itself, as the scan walks blocks 0 to 500, a new version with `balance_minor + 1000` might land on block 420 ahead of the scan. If Command 0 could see its own new versions, it would visit that row again, add *another* 1,000 cents, and loop until every account hit 50,000 cents. Engineers call this the **Halloween Problem**. IBM's System R team hit it on Halloween 1976, when a 10% raise query for employees under 25,000 USD kept re-updating rows until everyone earned at least 25,000 USD.

PostgreSQL prevents the Halloween Problem with a 32-bit `CommandId` counter inside each transaction. It starts at `0` and increments before each row-modifying command. Command 0 stamps `cmin = 0` on every version it writes. While Command 0 runs, its snapshot requires `cmin < 0` for its own `xid`, hiding rows created during Command 0 from Command 0's own scan. Once Command 0 finishes, the counter ticks to `1`, so Command 1 checks `cmin < 1` and sees Command 0's updates cleanly. (If one transaction both inserts and deletes the same tuple, a backend-local *combo CID* packs both `cmin` and `cmax` into the 4-byte `t_cid` slot.)

Try it in PostgreSQL

With PostgreSQL's `pageinspect` extension you can inspect the live 8 KB page and check every claim in this section yourself. Run this sequence on a scratch database:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE demo_mvcc (id INT PRIMARY KEY, bal BIGINT);

INSERT INTO demo_mvcc VALUES (1, 10000);
UPDATE demo_mvcc SET bal = 8000 WHERE id = 1;

BEGIN;
UPDATE demo_mvcc SET bal = 5000 WHERE id = 1;
ROLLBACK;

SELECT lp, t_xmin, t_xmax, t_ctid,
       t_infomask::bit(16) AS infomask
```

```
13  FROM heap_page_items(get_raw_page('demo_mvcc', 0));
```

You will see *three* physical tuples on page 0 (the XID numbers on your machine will differ; we use 740, 741, 742):

- `lp = 1`: `t_xmin = 740`, `t_xmax = 741`, `t_ctid = (0,2)` (retired by the committed `UPDATE`).
- `lp = 2`: `t_xmin = 741`, `t_xmax = 742`, `t_ctid = (0,3)` (still the live row. Its `t_xmax = 742` belongs to an *aborted* transaction, so readers ignore `t_xmax` and keep `bal = 8000`).
- `lp = 3`: `t_xmin = 742`, `t_xmax = 0`, `t_ctid = (0,3)` (dead on arrival because transaction `742` rolled back).

5.3 Commit Status (pg_xact) and Why a SELECT Can Dirty Disk Pages

Pause on `lp = 2` in the lab above: its header says `t_xmin = 741` and `t_xmax = 742`. Nowhere inside those 23 bytes did transaction `742` come back during `ROLLBACK` to erase `t_xmax = 742`; doing so across a million touched pages would make `ROLLBACK` slow again. So how does a subsequent `SELECT` discover that `741` committed whereas `742` aborted?

5.3.1 The Commit Log (pg_xact): Two Bits per Transaction

To answer that question in $O(1)$ time, PostgreSQL keeps a compact commit-status bitmap on disk under `PGDATA/pg_xact/` (formerly `pg_clog`), cached in shared memory via a Simple LRU (*SLRU*) pool. Because every transaction ends in one of four states, `pg_xact` needs just **2 bits per** `TransactionId`:

Bits	C constant	Meaning
00	STATUS_IN_PROGRESS	Started (or unassigned); neither committed nor aborted
01	STATUS_COMMITTED	Committed (either synchronously or asynchronously)
10	STATUS_ABORTED	Rolled back explicitly or crashed before commit
11	STATUS_SUB_COMMITTED	Subtransaction (SAVEPOINT) committed; check parent XID

Check the packing math. Four transactions fit in 1 byte ($4 \times 2 = 8$ bits), so one 8,192-byte page of `pg_xact` records:

$$8{,}192 \text{ bytes} \times 4\frac{\text{transactions}}{\text{byte}} = 32{,}768 \text{ transactions}.$$

Ten million transactions take barely 2.5 MB of cache. Finding the exact bit-pair for `xid` needs zero tree lookups: divide by 32,768 for the page, take (`xid` mod 32,768)/4 for the byte, and shift by (`xid` mod 4) $\times$ 2. At `COMMIT` or `ROLLBACK`, PostgreSQL logs the outcome to WAL and flips those 2 bits.

5.3.2 Tuple Hint Bits: Caching Commit Status on the Heap Tuple

If `pg_xact` is tiny and $O(1)$, why not check it every time a query visits a row?

Because `pg_xact` sits in shared memory shared across every CPU core. If sixty-four cores scan 100 million rows per second, hammering shared SLRU locks two hundred million times per second would stall the CPU interconnect.

PostgreSQL skips repeat lookups by caching four **hint bits** right inside each row's 16-bit `t_infomask` flag word:

- `HEAP_XMIN_COMMITTED` (`0x0100`): `t_xmin` committed.
- `HEAP_XMIN_INVALID` (`0x0200`): `t_xmin` aborted (the version never existed).
- `HEAP_XMAX_COMMITTED` (`0x0400`): `t_xmax` committed.
- `HEAP_XMAX_INVALID` (`0x0800`): `t_xmax` is `0` or aborted.

Watch how a newly inserted row gets its hint bit:

1. Transaction `740` inserts the row on page 0. Because `740` has *not* committed yet, neither `HEAP_XMIN_COMMITTED` nor `HEAP_XMIN_INVALID` is set.
2. Transaction `740` runs `COMMIT`. PostgreSQL syncs the commit WAL record and sets `pg_xact[740] = 01`. It does *not* go back to page 0.
3. Ten seconds later, the first reader (even a plain `SELECT`) visits page 0 and sees `t_xmin = 740` with no hint bit. That reader checks `pg_xact`, confirms `740`'s commit WAL record is already flushed to disk, sets `HEAP_XMIN_COMMITTED` inside the 8 KB buffer, and **marks the buffer dirty**.
4. Every later `SELECT` checks `t_infomask & HEAP_XMIN_COMMITTED` in one CPU instruction and never touches `pg_xact`.

Common trap

Operators are often surprised when a read-only `SELECT COUNT(*) FROM ledger_events;` right after a bulk load triggers gigabytes of disk writes and `EXPLAIN (ANALYZE, BUFFERS)` prints `Buffers: shared hit=120000 dirtied=120000`.
Nothing is broken: your `SELECT` is the first visitor after the bulk commit, stamping `HEAP_XMIN_COMMITTED` onto every page. With data checksums (`initdb --data-checksums`) or `wal_log_hints = on`, the first hint-bit update on a page after a checkpoint also writes a full-page WAL image (`FPI_FOR_HINT`) so a torn write cannot break the checksum. After any bulk `COPY`, run `VACUUM ANALYZE table_name;` right away to set hint bits, Visibility Map bits, and statistics in one predictable pass.

5.4 Snapshots and the Exact Visibility Formula

Hint bits tell us *whether* transaction `740` committed, yet that alone is not enough. Recall our 4:00 PM audit query: if the audit started at 4:00:00 PM and transfer `746` committed at 4:00:01 PM, `746` is committed, yet the 4:00:00 PM audit *must ignore it*.

To freeze a consistent view of the database, a query takes a **snapshot** (`SnapshotData` in `src/include/utils/snapshot.h`) by scanning `ProcArray`—the shared-memory array listing each connected backend's active `xid`. Run `SELECT pg_current_snapshot();` at any time to see your own snapshot string:

`xmin:xmax:xip_list`, for example: `740:748:742,745`.

Watch out for one naming quirk in PostgreSQL's C code: `xmin` and `xmax` inside a *snapshot* describe active transaction bounds, not row birth and death:

Definition 5.1 (MVCC Snapshot Triple)**.** A PostgreSQL MVCC snapshot $S = (\text{xmin}_S,\ \text{xmax}_S,\ \text{xip}_S)$ captured from `ProcArray` consists of:

1. xmin_S: the **lowest still-active** transaction ID across the system at snapshot creation (or xmax_S if none are active). Every transaction $x < \text{xmin}_S$ has already finished—it either committed before S was taken or aborted.

2. xmax_S: **one past the highest completed** (or allocated) transaction ID at snapshot creation. Every transaction $x \geq \text{xmax}_S$ had not even finished (or started) when S was taken, so its effects are unconditionally *invisible* to S.

3. xip_S (*transactions in progress*): the sorted list of transaction IDs in the half-open interval $[\text{xmin}_S, \text{xmax}_S)$ that were still actively running at the instant S was captured. Even if $x \in \text{xip}_S$ commits one microsecond later, x remains *invisible* to S.

Look how cleanly $(\text{xmin}_S, \text{xmax}_S, \text{xip}_S) = (740, 748, \{742, 745\})$ partitions the entire timeline of transactions (Figure 5.2):

- Any $x < 740$ (such as `738`, `739`): finished before our snapshot. Visible if committed; invisible if aborted.

- $x \in [740, 748)$:

  - $x \in \{742, 745\}$: was still running when our snapshot snapped the shutter. **Invisible**, even if it commits later.

  - $x \in \{740, 741, 743, 744, 746, 747\}$: already finished before our snapshot. Visible if committed; invisible if aborted.

- Any $x \geq 748$ (such as `748`, `749`): started or finished after our snapshot. **Invisible**, even if already committed.

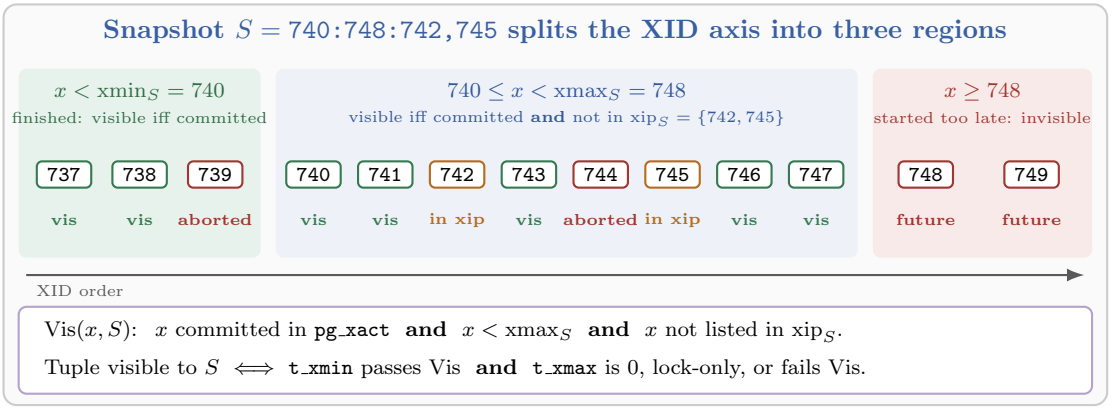


Figure 5.2: Geometry of a PostgreSQL MVCC snapshot $S =$ 740:748:742,745. Transactions 742 and 745 were running when S was captured (xip$_S$), so even if 742 commits moments later, Vis$(742, S)$ stays false for the lifetime of snapshot S. Transaction 748 had not finished when S was taken, so it is invisible even after it commits.

Before writing out the exact logic, state the visibility test in two plain questions. First, *was this row version already born when my snapshot started?* Its creator (t_xmin) must have committed before xmax$_S$ and must not appear in the still-running list xip$_S$ (or must come from an earlier command of my own transaction). Second, *was this row version still alive when my snapshot started?* Either nobody has deleted it (t_xmax $= 0$ or lock-only), or its deleter (t_xmax) aborted or committed *after* my snapshot took effect. Inside src/backend/access/heap/heapam_visibility.c, HeapTupleSatisfiesMVCC() encodes both checks (with x_{self} as our own XID and c_{cur} as our current command ID):

Formal

Exact MVCC Tuple Visibility Rule (HeapTupleSatisfiesMVCC). First define when another transaction $x \neq x_{\text{self}}$ is visible to snapshot $S = (\text{xmin}_S, \text{xmax}_S, \text{xip}_S)$:

- Transaction x's writes are visible to S when:

$$\text{Vis}(x, S) \iff \big(\text{Status}(x) = \texttt{COMMITTED}\big) \wedge \big(x < \text{xmax}_S\big) \wedge \big(x \notin \text{xip}_S\big).$$

(Whenever $x <$ xmin$_S$, $x \notin$ xip$_S$ holds automatically without searching xip$_S$.)

A physical row version T with header (t_xmin, t_xmax, $c_{\min}$, $c_{\max}$, t_infomask) is **visible** to snapshot S if and only if *both* its birth condition (**B**) and its non-death condition (**D**) hold:

1. **Birth condition (B):** Either (t_xmin $= x_{\text{self}}$ and $c_{\min} < c_{\text{cur}}$), or (t_xmin $\neq x_{\text{self}}$ and Vis(t_xmin, S) holds, or HEAP_XMIN_FROZEN is set).
2. **Non-death condition (D):** At least one of the following holds:
   (a) t_xmax $= 0$, or HEAP_XMAX_INVALID is set (deleter aborted), or HEAP_XMAX_LOCK_ONLY is set (locked, not deleted);
   (b) t_xmax $= x_{\text{self}}$ and $c_{\max} \ge c_{\text{cur}}$ (deleted by our own current or later command, so this command still sees it);
   (c) t_xmax $\neq x_{\text{self}}$ and $\neg$ Vis(t_xmax, S) (deleter is still running, aborted, listed in xip$_S$, or has t_xmax $\ge$ xmax$_S$).

Example 5.1 (Evaluating Tuple Visibility by Hand)**.** Suppose a read-only audit query captures snapshot $S =$ `740:748:742,745` (x_{self} unassigned). Every completed transaction committed except `744` (which rolled back), and `742` commits *after* our snapshot was taken. Which of these five versions on page 0 does our query emit?

Slot	t_xmin	t_xmax	Note	Birth (B)?	Alive (D)? $\implies$ Emit?
(0,1)	735	741	both committed	Yes ($735 < 740$)	No ($\text{Vis}(741, S)$) $\implies$ **Hide**
(0,2)	741	742	742 commits later	Yes ($741 \notin \text{xip}_S$)	Yes ($742 \in \text{xip}_S$) $\implies$ **Emit**
(0,3)	742	0	742 commits later	No ($742 \in \text{xip}_S$)	— $\implies$ **Hide**
(0,4)	743	744	744 aborted	Yes ($743 \notin \text{xip}_S$)	Yes (744 aborted) $\implies$ **Emit**
(0,5)	746	749	749 lock-only	Yes ($746 < 748$)	Yes ($749 \geq 748$) $\implies$ **Emit**

Look closely at `(0,1)`, `(0,2)`, and `(0,3)`. They are three physical versions of *one* logical row updated by `741` and then `742`. Version `(0,1)` died before our snapshot, and `(0,3)` was born in `742`, which was still running when our snapshot froze. **Exactly one version**—`(0,2)`—passes both (B) and (D). That single-visible-version rule lets our 4:00 PM audit scan ten million rows while live payments fly past.

Pause

Pause and check the two-question rule. A row version is visible to snapshot S when (1) its creator `t_xmin` committed before S froze, and (2) its deleter `t_xmax` had *not* visibly committed when S froze. Every version chain produces either zero or one visible version per snapshot.

Walking question

When is a snapshot taken? That single timing choice is the sole difference between PostgreSQL's first two transaction isolation levels (Chapter 6):

- Under `READ COMMITTED` (PostgreSQL's default), PostgreSQL takes a **fresh snapshot at the start of every individual SQL statement** inside the transaction.
- Under `REPEATABLE READ` (and `SERIALIZABLE`), PostgreSQL takes **one snapshot at the start of the first non-control SQL statement** in the transaction and reuses that identical $(\text{xmin}_S, \text{xmax}_S, \text{xip}_S)$ triple until `COMMIT` or `ROLLBACK`.

5.5 Write Amplification and HOT (Heap-Only Tuple) Updates

Now flip from the reader's perspective to the B-tree indexes we studied in Chapter 3, and watch a painful consequence of Design B emerge.

Suppose your `accounts` table has four B-tree indexes:

1. `accounts_pkey` on `(id)`,
2. `idx_accounts_owner` on `(owner_id)`,
3. `idx_accounts_currency` on `(currency)`,

4. `idx_accounts_updated` on `(updated_at)`.

Remember from Chapter 3 that PostgreSQL secondary indexes do *not* point to primary keys (unlike InnoDB); every PostgreSQL B-tree leaf entry points directly to a physical heap `ctid = (block, offset)`. Now run a tiny balance update that also bumps `updated_at`:

```
UPDATE accounts
   SET balance_minor = balance_minor - 5000,
       updated_at    = now()
 WHERE id = 42;
```

Old version `(0,1)` stays put; new version `(0,2)` lands on the table. Because `updated_at` is indexed, `idx_accounts_updated` obviously needs a new entry pointing to `(0,2)`. What about the other three indexes—`id`, `owner_id`, and `currency`—whose values *did not change*? Those three indexes currently point only to `(0,1)`. Without an index update, a future lookup on `owner_id = 99` would land on dead `(0,1)` and miss the row. Consequently, naive in-heap MVCC must insert a new entry into **every index on the table** pointing to `(0,2)`.

Updating one 8-byte integer just dirtied one table page *plus four B-tree leaf pages*, emitted five WAL records, and bloated all four indexes. Before PostgreSQL 8.3, that write amplification punished every write-heavy table.

5.5.1 How `HOT` Decouples B-Trees from Unindexed Updates

PostgreSQL 8.3 solved that common case with **Heap-Only Tuples** (**HOT**). Suppose we drop `idx_accounts_updated` (updating *only* unindexed `balance_minor`), and suppose page 0 still has free space for new version `(0,2)`.

Look at what already sits inside page 0: a forward link `t_ctid = (0,2)` from `(0,1)` to `(0,2)`. Chasing `t_ctid` across *different* blocks during an index scan would cost extra random disk reads. Chasing `t_ctid` **inside the same 8 KB page already pinned in** `shared_buffers`, by contrast, costs zero extra I/O and just a few nanoseconds.

PostgreSQL therefore runs a **HOT update** whenever *both* rules hold:

1. **Zero indexed columns change** (checked against all columns used by any index, including expression and partial indexes); and
2. **The new version fits on the exact same 8 KB page** as the old version.

When a HOT update fires:

- **Zero B-tree indexes are touched.** Every index keeps its single leaf entry pointing to root slot `(0,1)`.
- Old version `(0,1)` gets `HEAP_HOT_UPDATED` (`0x4000`) in `t_infomask2`.
- New version `(0,2)` gets `HEAP_ONLY_TUPLE` (`0x8000`) in `t_infomask2`, meaning no index points straight to `(0,2)`.
- When an index scan visits `(0,1)`, it sees `HEAP_HOT_UPDATED` and walks `(0,1)` → `(0,2)` → `(0,3)` inside the pinned buffer to find the version matching its snapshot.

5.5.2 Single-Page Pruning (On-the-Fly Defragmentation Without `VACUUM`)

HOT brings a second big win alongside skipping index writes: **single-page HOT pruning**.

Think about why full `VACUUM` normally has to scan every index before freeing a dead slot `(0,1)`. If an index still points to `(0,1)` and you reuse slot 1 for Customer B, a lookup for Customer A returns Customer B's account.

Now look at `(0,2)` inside a HOT chain `(0,1)` → `(0,2)` → `(0,3)` once `(0,1)` and `(0,2)` are older than every active snapshot (`OldestXmin`):

- *No index anywhere* points to `(0,2)` or `(0,3)`.
- Every index points *only* to root slot `ItemId[1]`.

Thus the very next query touching this 8 KB page—even a plain `SELECT` or `UPDATE` when free space runs low—can clean up the chain **locally on this one page in microseconds, touching zero indexes**:

1. It deletes the dead bytes of `v1` and `v2`, sliding live `v3` upward to compact free space.
2. It turns root slot `ItemId[1]` into a 4-byte redirect (`LP_REDIRECT`) pointing straight to slot 3.
3. It marks `ItemId[2]` as `LP_UNUSED`, ready for the very next `UPDATE`.

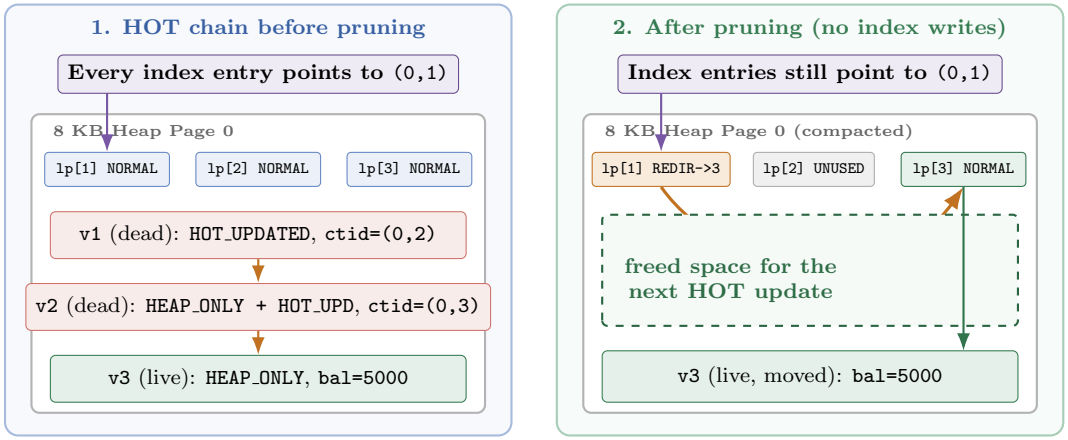


Figure 5.3: Single-page HOT pruning inside an 8 KB page. Left: Two HOT updates left dead versions `v1` and `v2` on Page 0 while every B-tree index kept pointing to root slot `lp[1]`. Right: Once `v1` and `v2` pass `OldestXmin`, the next query compacts Page 0 in place, redirects `lp[1]` to `lp[3]`, and frees `lp[2]` with zero index writes.

In production

On a busy `accounts` table handling 15,000 balance updates per second, two small DDL rules prevent runaway bloat:

1. **Keep fast-changing counters out of secondary indexes** unless queries truly filter or sort on them. Indexing `updated_at` or `balance_minor` disables HOT on every balance change. Check your HOT ratio in `pg_stat_user_tables` (`n_tup_hot_upd` vs. `n_tup_upd`); on a healthy balance table, $> 95\%$ of updates are HOT.

2. **Reserve same-page headroom with `fillfactor`:** Heap tables default to `fillfactor = 100`, packing new pages 100% full so the *first* `UPDATE` spills to another block. Running `ALTER TABLE accounts SET (fillfactor = 85);` leaves 15% (≈ 1.2 KB) free during inserts so HOT chains fit and self-prune on the same page.

5.6 Table Bloat, `VACUUM`, `autovacuum`, and `VACUUM FULL`

When a row is `DELETE`d, or when an `UPDATE` cannot use HOT, single-page pruning can free the dead row bytes, yet it **must leave a 4-byte `LP_DEAD` stub in the slot array** and **must leave dead entries inside every B-tree index**. Clearing both requires `VACUUM`.

5.6.1 The Three Phases of Concurrent ("Lazy") `VACUUM`

Plain `VACUUM` ("lazy vacuum" in `src/backend/access/heap/vacuumlazy.c`) takes only a `SHARE UPDATE EXCLUSIVE` lock—**never blocking `SELECT`, `INSERT`, `UPDATE`, or `DELETE`**. It runs in three phases:

1. **Phase 1 — Heap Scan:** `VACUUM` finds `OldestXmin`, the oldest active snapshot across all sessions, replication slots, and prepared transactions. It scans the table sequentially while *skipping* pages marked `ALL_VISIBLE` in the **Visibility Map** (`_vm`). Along the way, it prunes dead row payloads and collects dead 6-byte tuple identifiers (TIDs, each a `(block, slot)` address) in RAM up to `autovacuum_work_mem`. PostgreSQL 17 replaced the old 1 GB flat TID array with a compact radix tree that holds far more TIDs per megabyte.

2. **Phase 2 — Index Vacuuming:** Whenever the dead-TID buffer fills (or Phase 1 ends), `VACUUM` scans every index on the table and deletes entries matching the dead-TID set. Memory sizing matters hugely here. If your dead-TID buffer holds only one-tenth of a bloated table's dead TIDs across 8 indexes, `VACUUM` must scan all 8 indexes *ten times*. Sizing memory to fit all dead TIDs at once collapses ten passes into one.

3. **Phase 3 — Slot Cleanup, FSM, and VM:** Once no index points to those TIDs, `VACUUM` revisits the table pages and flips `LP_DEAD` slots to reusable `LP_UNUSED`. It then logs the freed space in the **Free Space Map** (`_fsm`) for future writes and sets `ALL_VISIBLE` / `ALL_FROZEN` bits in the Visibility Map.

5.6.2 Why `VACUUM` Does Not Shrink Disk Usage (vs. `VACUUM FULL`)

Suppose you insert 50 million test rows (8 GB on disk), run `DELETE FROM ledger_test;`, and run `VACUUM ledger_test;`. Checking `pg_relation_size('ledger_test')` can still show 8 GB if even one live row sits near the end of the file.

Why? Plain `VACUUM` frees space *inside* existing 8 KB pages and lists it in the Free Space Map for future inserts. It truncates OS file blocks only at the *trailing end* of the file when those tail pages are 100% empty. Moving a live row from block 999,999 down to block 0 would change its `ctid`, breaking open scans and indexes unless the whole table were locked.

`VACUUM FULL`, by contrast, rewrites the entire table and rebuilds every index into brand-new files before deleting the old ones. It shrinks an 8 GB bloated file with 1,000 live rows down to 64 KB, at two steep costs:

- `VACUUM FULL` holds an `ACCESS EXCLUSIVE` **lock** across the entire rewrite, blocking *every* ***SELECT*** *and write* on the table.
- Because old and new files coexist until commit, `VACUUM FULL` needs enough free disk space to hold a full second copy of the table and its indexes.

On 24/7 systems, engineers avoid `VACUUM FULL` during live traffic and use online tools like `pg_repack`. Those build a clean shadow table in the background, replay concurrent writes from a log table, and take `ACCESS EXCLUSIVE` only for a brief catalog swap at the end.

Common trap

Two production traps that silently paralyze `autovacuum`:

1. **One `idle in transaction` session:** Suppose someone opens `psql`, runs `BEGIN; SELECT * FROM accounts LIMIT 1;`, and leaves for a four-hour lunch. If that session holds an open snapshot or uncommitted write, global `OldestXmin` stays pinned at noon. Even if `autovacuum` runs every minute, it **cannot reclaim a single dead version created anywhere in the database after noon**. Always set `idle_in_transaction_session_timeout = '60s'` and alert on abandoned replication slots in `pg_replication_slots`.
2. **The 20% default scale factor on big tables:** By default, `autovacuum` wakes when dead rows pass $\tau = 50$ plus $s = 0.20$ times table size:

$$\texttt{n_dead_tup} > \tau + s \times \texttt{reltuples}.$$

   On a 200-million-row `postings` table, $50 + 0.20 \times N$ waits for **40,000,050 dead rows** before starting. Lower the scale factor per large hot table:

```
ALTER TABLE accounts SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_cost_delay   = 2,
    autovacuum_vacuum_cost_limit   = 2000
);
```

5.7 32-Bit Transaction ID Wraparound and Tuple Freezing

The final consequence of in-heap MVCC can halt writes across an entire cluster. Check `HeapTupleHeaderData` in Section 5.2: `t_xmin` and `t_xmax` are 32-bit unsigned integers (`uint32`). Why hasn't PostgreSQL widened them to 64 bits on disk?

Storage cost. Widening `t_xmin` and `t_xmax` to 8 bytes each adds **8 bytes of fixed header to every row in every table**. On a 10-billion-row postings table with 48-byte rows, 8 extra bytes per row adds 80 GB of disk and RAM ($\approx 17\%$). PostgreSQL tracks a 64-bit `FullTransactionId` in memory (returned by `pg_current_xact_id()`), while on-disk row headers store only the low 32 bits.

With 32 bits there are $2^{32} = 4{,}294{,}967{,}296$ distinct values. PostgreSQL reserves the first three:

- `0` = `InvalidTransactionId` (used for empty `t_xmax`),
- `1` = `BootstrapTransactionId` (used during `initdb` catalog creation; always visible),
- `2` = `FrozenTransactionId` (legacy sentinel for frozen tuples; always visible),
- `3..`$(2^{32} - 1)$ = normal circular `TransactionId` space (`FirstNormalTransactionId = 3`).

Whenever the 32-bit counter increments past 4,294,967,295 (`0xFFFFFFFF`), it wraps right around to `3` (skipping `0`, `1`, `2`).

5.7.1 Modular Circle Comparison (`TransactionIdPrecedes`)

If `xid` wraps around a finite circle of size $\approx 2^{32}$, you cannot compare two transaction IDs a and b with ordinary integer inequality `a < b`. Right after wraparound from `4,294,967,295` to `3`, transaction `3` happened *after* `4,294,967,295`, even though $3 < 4{,}294{,}967{,}295$ as unsigned integers.

Instead, PostgreSQL defines ordering on the modular circle $\mathbb{Z}/2^{32}\mathbb{Z}$ using signed 32-bit modular subtraction (`src/backend/access/transam/transam.c`):

```
/* Normal XID comparison in PostgreSQL: true if id1 logically precedes id2 */
bool NormalTransactionIdPrecedes(TransactionId id1, TransactionId id2) {
    int32 diff = (int32) (id1 - id2);
    return (diff < 0);
}
```

Look at the geometry of `(int32)(id1 - id2) < 0` on the 2^{32} clock dial (Figure 5.4):

- Stand at the current transaction counter x_{now}.
- Looking counterclockwise up to $2^{31} \approx 2.147$ billion steps behind x_{now}, `(int32)(t_xmin - x_now) < 0`: all of those 2^{31} XIDs lie in the **visible Past**.
- Looking clockwise up to $2^{31} - 1 \approx 2.147$ billion steps ahead of x_{now}, `(int32)(t_xmin - x_now) > 0`: all of those 2^{31} XIDs lie in the **invisible Future**.

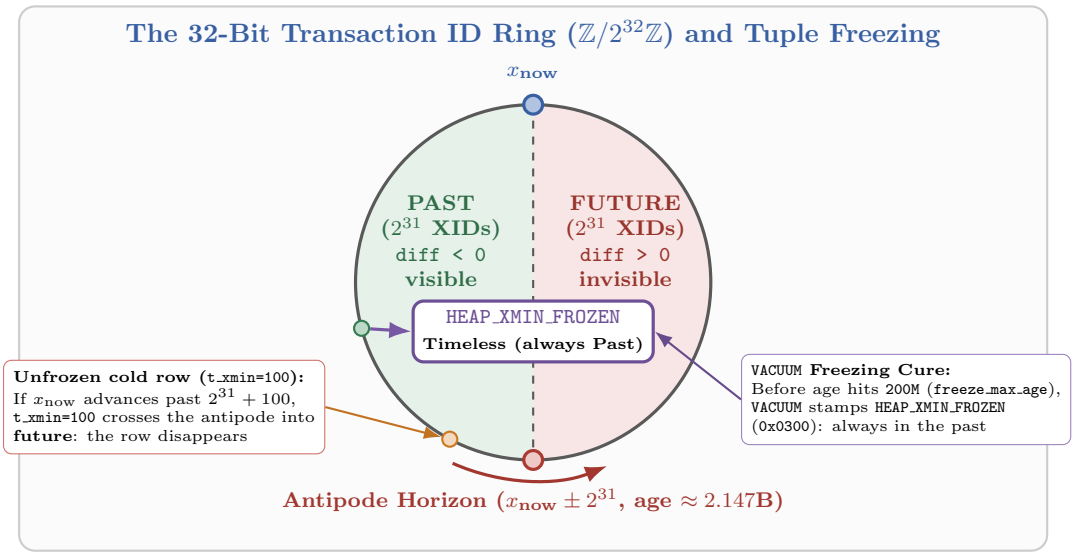


Figure 5.4: Modular 32-bit `TransactionId` arithmetic. Half the 2^{32} circle behind x_{now} is the visible Past; the half ahead is the invisible Future. As x_{now} marches clockwise, an untouched row's `t_xmin` drifts relatively toward the antipode (2^{31} transactions old). Freezing stamps `HEAP_XMIN_FROZEN` onto the tuple header, detaching the row from modular comparison forever.

Here is what would happen if a row were never frozen:

1. In January, transaction `100` inserts a founding-capital row into `journal_entries`, so the tuple has `t_xmin = 100`. It is an immutable historical record; nobody ever updates or deletes it again.
2. Over the next eight months or so, your payment service runs 100 write transactions per second, advancing x_{now} past 2,147,483,748 ($= 2^{31} + 100$).
3. Now an auditor runs `SELECT * FROM journal_entries WHERE id = 1;`. PostgreSQL compares `t_xmin = 100` against $x_{\text{now}} = 2{,}147{,}483{,}749$:

$$\texttt{(int32)(100 - 2147483749)} = +2{,}147{,}483{,}647 > 0 \quad \text{(so 100 is in the future).}$$

4. No byte on disk has changed, yet **that committed row disappears from every query**, because `t_xmin = 100` moved across the antipode from "2.1 billion transactions ago" to "2.1 billion transactions in the future."

5.7.2 The Cure: Tuple Freezing, `relfrozenxid`, and the Safety Shutdown

PostgreSQL prevents wraparound loss through **tuple freezing**. Once a committed tuple's `t_xmin` is older than `OldestXmin` and older than `vacuum_freeze_min_age` (default 50 million XIDs), `VACUUM` marks it frozen by setting *both* `xmin` hint bits at once:

$$\texttt{HEAP_XMIN_FROZEN} = \texttt{HEAP_XMIN_COMMITTED} \mid \texttt{HEAP_XMIN_INVALID} = \texttt{0x0300}.$$

(Since PostgreSQL 16, `VACUUM` also freezes younger tuples opportunistically when it is already writing a full-page image of that page, because the extra freezing then costs almost nothing.) Before

PostgreSQL 9.4, freezing overwrote `t_xmin` with `2`, which destroyed forensic history. Setting `0x0300` instead keeps the original `t_xmin` readable in `pageinspect` while telling every visibility check to treat the tuple as committed infinitely long ago.

Once every tuple on an 8 KB page is frozen, `VACUUM` also sets the page's **all-frozen bit** in the Visibility Map (`_vm` fork). Later anti-wraparound vacuums skip those pages entirely, so a 50 TB archive table whose old pages are all frozen costs almost nothing to keep safe.

PostgreSQL tracks each table's oldest unfrozen XID in `pg_class.relfrozenxid` and the database-wide oldest in `pg_database.datfrozenxid`. Three guardrails protect your cluster:

1. **Routine freeze threshold (`autovacuum_freeze_max_age = 200,000,000`, $\approx 9.3\%$ of 2^{31}):** Once `age(relfrozenxid)` passes 200 million XIDs—even if `autovacuum = off`—PostgreSQL launches a mandatory anti-wraparound `autovacuum`. It uses all-frozen bits in `_vm` to skip frozen pages and then deletes old `pg_xact` segments.
2. **Client warning zone ($\approx$ 40 million XIDs before 2^{31}):** Every XID-allocating command emits `WARNING: database "prod" must be vacuumed within N transactions`.
3. **Hard safety stop (3 million XIDs before 2^{31}):** Before wraparound can hide committed rows, PostgreSQL **refuses new XIDs**, failing writes with:

```
ERROR: database is not accepting commands that assign new
       transaction IDs to avoid wraparound data loss in
       database "prod"
HINT:  Execute a database-wide VACUUM in that database.
```

   Read queries still work, and `VACUUM` needs no XID, so a superuser can connect normally and run `VACUUM (FREEZE, INDEX_CLEANUP OFF, TRUNCATE OFF) worst_table;`. Skipping index cleanup advances `relfrozenxid` fastest.

Try it in PostgreSQL

Monitor XID wraparound health in every production database before it ever reaches 50% of the 2^{31} ceiling. Add this exact query to your operational runbook and Prometheus exporter:

```
SELECT c.oid::regclass                        AS table_name,
       age(c.relfrozenxid)                    AS xid_age,
       ROUND(100.0 * age(c.relfrozenxid)
             / 2147483647.0, 2)               AS pct_toward_wraparound,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
  FROM pg_class c
  JOIN pg_namespace n ON n.oid = c.relnamespace
 WHERE c.relkind IN ('r', 'm', 't')
   AND n.nspname NOT IN ('pg_catalog', 'information_schema')
 ORDER BY age(c.relfrozenxid) DESC
 LIMIT 10;
```

Notice `'t'` in `relkind IN ('r', 'm', 't')`. TOAST tables (Chapter 2) track their *own* `relfrozenxid` separate from their parent table, and a slow vacuum on a bloated TOAST table has surprised many experienced teams.

MVCC snapshots let readers and writers run concurrently without read locks. Still, one big concurrency question remains. What happens when two sessions both read a consistent snapshot and

write back based on what they saw—either colliding on the same row, or modifying *two different rows* whose combined balance must stay positive? That brings us to isolation levels, concurrency anomalies, row locks, and Serializable Snapshot Isolation in Chapter 6.

5.8 Exercises

Exercise 5.1 (Storage design tradeoffs)**.** A batch job runs `UPDATE ledger_staging SET fee_minor = fee_minor + 10;` across 8,000,000 rows, hits a constraint error on the last row, and runs `ROLLBACK`. Compare step by step what InnoDB (in-place update + undo log) and PostgreSQL (in-heap versions + `pg_xact`) do during (a) the 7,999,999 updates, (b) the `ROLLBACK` itself, and (c) the next hour.

Exercise 5.2 (Commit log byte and bit offsets)**.** Recall that `pg_xact` packs 2 status bits per `TransactionId` into 8,192-byte SLRU pages. (a) For `xid = 1,000,005`, compute the 0-indexed `pg_xact` page number, the byte offset inside that page, and the bit-shift (`0`, `2`, `4`, or `6`) inside that byte. (b) At 5,000 write transactions per second for 24 hours, how many megabytes of `pg_xact` do those transactions occupy before truncation?

Exercise 5.3 (Tracing snapshot visibility by hand)**.** A `REPEATABLE READ` transaction has $x_{\text{self}} = 912$, $c_{\text{cur}} = 2$, and snapshot $S =$ `905:915:907,910`. Every other transaction < 915 committed except `909`, which aborted. Meanwhile, `907` committed two seconds *after* S froze, `910` is still running, and `916` committed five seconds after S froze. State whether each row version below is **visible** or **invisible**, naming the exact clause of `HeapTupleSatisfiesMVCC`:

1. `t_xmin = 902, t_xmax = 907` (normal delete).
2. `t_xmin = 907, t_xmax = 0`.
3. `t_xmin = 908, t_xmax = 909` (normal delete).
4. `t_xmin = 908, t_xmax = 911, infomask = HEAP_XMAX_LOCK_ONLY`.
5. `t_xmin = 912, cmin = 1, t_xmax = 0`.
6. `t_xmin = 912, cmin = 2, t_xmax = 0`.

Exercise 5.4 (The Halloween Problem)**.** Create `nums(x INT)` with rows `1`, `2`, `3` and an index on `(x)`. Suppose PostgreSQL skipped checking `cmin < curcid` during `UPDATE nums SET x = x + 1 WHERE x > 0;`. Trace an ascending index scan step by step, and explain how `t_cid` stops the infinite loop without hiding the updated rows from the *next* statement in the same transaction.

Exercise 5.5 (Why a read-only query dirties buffers and emits WAL)**.** Right after a checkpoint finishes, a loader inserts 500,000 rows into `events` across 4,000 pages and runs `COMMIT`. Next, on a server with data checksums enabled, an analyst runs `SELECT COUNT(*) FROM events;`. (a) Why does `EXPLAIN (ANALYZE, BUFFERS)` on that `SELECT` report roughly `dirtied=4000`? (b) Why does that first `SELECT` emit ≈ 32 MB of WAL, whereas running the same `SELECT` one second later reports `dirtied=0` and emits zero WAL?

Exercise 5.6 (Designing for HOT updates)**.** A `sessions` table has columns `id`, `user_id`, `seen_at`, and `hits`, with secondary B-tree indexes on `(user_id)` and `(seen_at)`. Every HTTP request runs:

```
UPDATE sessions
   SET seen_at = now(), hits = hits + 1
 WHERE id = $1;
```

(a) What fraction of these updates can use HOT today, and how many index leaf entries are inserted per 1,000 requests? (b) Product reads `seen_at` only once an hour to purge sessions older than 30 days. Redesign the index or update threshold (plus `fillfactor`) so $> 98\%$ of requests become HOT updates while keeping the hourly purge fast.

Exercise 5.7 (Autovacuum trigger math)**.** A `ledger_postings` table holds $N = 150{,}000{,}000$ rows and receives 2,000 non-HOT updates per second. (a) Under default threshold `50` and scale factor `0.20`, how many hours pass between autovacuums, and how many dead rows build up first? (b) Pick per-table `autovacuum` settings so cleanup starts every $\approx 500{,}000$ dead rows (≈ 4 minutes).

Exercise 5.8 (Wraparound modular arithmetic)**.** Test PostgreSQL's modular XID check $(\text{int32})(\text{id}_1 - \text{id}_2) < 0$ on each pair below:

1. `id1 = 4294967290` (`0xFFFFFFFA`) and `id2 = 10` (`0x0000000A`).
2. `id1 = 100` and `id2 = 2147483747` ($= 100 + 2^{31} - 1$).
3. `id1 = 100` and `id2 = 2147483749` ($= 100 + 2^{31} + 1$). What happens to an unfrozen committed row with `t_xmin = 100` between (2) and (3)?

Exercise 5.9 (Emergency wraparound triage)**.** At 2:00 AM, a database has 8 million XIDs left before wraparound shutdown (`age(datfrozenxid) = 2,139,000,000`) and burns 1,500 XIDs/s (≈ 88 minutes left). (a) Write queries on `pg_stat_activity`, `pg_prepared_xacts`, and `pg_replication_slots` to find what is pinning `OldestXmin`. (b) Once cleared, why must you run `VACUUM (FREEZE, INDEX_CLEANUP OFF, TRUNCATE OFF)` on the oldest table instead of `VACUUM FULL`?

Chapter 6

Isolation Anomalies, Row Locking, and Serializable Snapshot Isolation

In Chapter 5 we built PostgreSQL's snapshot engine: every query receives a frozen list of completed transactions, so readers and writers glide past each other without read locks. At first glance, snapshots look like the whole answer to concurrency. Every query sees a self-consistent photograph of the database, nobody reads half-written bytes, and rollbacks vanish cleanly.

Now try running a bank on snapshots alone.

Suppose Alice holds two sub-accounts under one margin agreement: `Checking = 80 USD` and `Savings = 80 USD`. Firm policy says *her combined balance across both accounts may never drop below 0 USD*. At 10:00:00.000 AM, Alice submits two 100 USD withdrawals from two browser tabs at the same millisecond—one from Checking, one from Savings. Both workers run at `REPEATABLE READ`:

- Worker 1 reads its snapshot: `Checking = 80, Savings = 80` (total 160 USD). Withdrawing 100 USD leaves $60 \geq 0$, so Worker 1 sets `Checking` to -20 and commits.
- Worker 2 reads its identical 10:00:00.000 AM snapshot: `Checking = 80, Savings = 80` (total 160 USD). Withdrawing 100 USD leaves $60 \geq 0$, so Worker 2 sets `Savings` to -20 and commits.

Neither worker read an uncommitted byte. Neither worker touched the row the other modified. Both finished under `REPEATABLE READ` without a single error or lock wait—yet Alice just withdrew 200 USD against 160 USD of collateral, leaving a -40 USD balance.

Focus

By the end of this chapter, you will be able to answer five questions from first principles:

- Why does wrapping two `SELECT` queries in `BEGIN ... COMMIT` at PostgreSQL's default isolation level still allow inconsistent totals?
- Why does subtracting a balance in application code lose money under concurrency, while `SET bal = bal - 8000 WHERE bal >= 8000` stays safe?
- How can two `REPEATABLE READ` transactions break a multi-row business rule without ever touching the same row?

- How does PostgreSQL's `SERIALIZABLE` mode catch every multi-row race condition without placing blocking locks on readers?
- Why does PostgreSQL split row locks into four strengths, and how do `SKIP LOCKED` and `ORDER BY id ASC` eliminate queue bottlenecks and deadlocks?

6.1 What "Isolation" Actually Promises (and Why ANSI SQL-92 Missed Half the Story)

The "I" in ACID stands for *Isolation*: even when hundreds of transactions overlap across CPU cores, each should behave *as if it had the database to itself.* Database theory calls that gold standard **serializability**.

Definition 6.1 (Serializable Execution)**.** A concurrent execution of committed transactions $T_1, T_2, \ldots, T_k$ is *serializable* if its final state and query results match running those same transactions **strictly one after another with zero overlap** in some order $T_{\pi(1)} \to T_{\pi(2)} \to \cdots \to T_{\pi(k)}$.

Check Alice's two withdrawals against Definition 6.1. Only two one-at-a-time orders exist for T_1 (withdraw 100 USD from Checking) and T_2 (withdraw 100 USD from Savings):

- T_1 **then** T_2**:** T_1 sees $80 + 80 = 160$, sets Checking to -20, and commits. Next, T_2 sees $-20 + 80 = 60 < 100$ and *rejects* the second withdrawal. Final total: +60 USD.
- T_2 **then** T_1**:** Symmetrically, T_2 sets Savings to -20, and T_1 sees $80 + (-20) = 60 < 100$ and *rejects* the first withdrawal. Final total: +60 USD.

In *every* serial execution, at most one withdrawal commits and +60 USD remains. Any schedule leaving −40 USD violates serializability: no one-at-a-time history could ever produce that state.

6.1.1 The ANSI SQL-92 Table vs. PostgreSQL Reality

When ANSI SQL-92 defined isolation levels (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`), the standard had two famous blind spots exposed in 1995 by Berenson, Bernstein, Gray, Melton, and the O'Neils:

1. It assumed 1980s lock-based engines rather than MVCC snapshots; and
2. It defined levels using only *three read phenomena* (Dirty Reads, Non-Repeatable Reads, Phantom Reads), omitting **Lost Updates** and **Write Skew**.

In PostgreSQL, MVCC snapshots reshape that matrix in ways every backend engineer must know:

PostgreSQL Level	Dirty	Non-Repeatable	Phantom	Lost Update / Write Skew
`READ UNCOMMITTED`[†]	Impossible	Possible	Possible	Possible / Possible
`READ COMMITTED` (default)	Impossible	Possible	Possible	**Possible** / Possible
`REPEATABLE READ` (SI)	Impossible	Impossible	**Impossible**	Impossible[‡] / **Possible**
`SERIALIZABLE` (SSI)	Impossible	Impossible	Impossible	Impossible / Impossible

[†]PostgreSQL accepts `READ UNCOMMITTED` for SQL compatibility and maps it directly to `READ COMMITTED`. [‡]Blocked via automatic `40001` conflict abort. SI = Snapshot Isolation; SSI = Serializable Snapshot Isolation (Section 6.5).

Two facts jump right out of that table:

- **Dirty reads never happen in PostgreSQL** at any level, because Chapter 5's visibility check rejects uncommitted `t_xmin` values unconditionally.
- **PostgreSQL's `REPEATABLE READ` already blocks phantom reads**, yet it is *not* serializable: it implements pure *Snapshot Isolation* (*SI*), which still allows **Write Skew**, whereas `SERIALIZABLE` implements *Serializable Snapshot Isolation* (*SSI*).

Let us trace each anomaly on concrete two-session timelines.

6.2 Read Anomalies: Dirty Reads, Read Skew, and Phantoms

6.2.1 Dirty Read (P_1): Reading Uncommitted Bytes

A *dirty read* happens when session T_2 reads bytes written by concurrent session T_1 *before* T_1 commits. If T_1 then runs `ROLLBACK`, T_2 acted on money that never existed:

```
-- T1                                    -- T2
BEGIN;                                   BEGIN;
UPDATE accounts SET bal = 99999900
 WHERE id = 1;
                                         -- Under true READ UNCOMMITTED:
                                         SELECT bal FROM accounts WHERE id = 1;
                                         -- Sees 99999900 and wires cash out!
ROLLBACK; -- T1 aborts!
```

In PostgreSQL, when T_1 updates the row, old version `(0,1)` gets `t_xmax = xid(T1)` and new version `(0,2)` gets `t_xmin = xid(T1)`. While `xid(T1)` is still running in `pg_xact`, T_2 sees that `(0,1)`'s deleter has not committed (so `(0,1)` stays visible) and `(0,2)`'s creator has not committed (so `(0,2)` stays hidden). Dirty reads cannot happen.

6.2.2 Non-Repeatable Read (P_2) and Read Skew Under `READ COMMITTED`

Recall how PostgreSQL's default level—`READ COMMITTED`—takes snapshots: **it grabs a fresh snapshot at the start of every SQL statement**.

Suppose Alice has Checking (`id = 1`, `5000` cents) and Savings (`id = 2`, `5000` cents). Session T_1 moves `4000` cents from Checking to Savings while Session T_2 (a dashboard running at default `READ COMMITTED`) reads Alice's two accounts in two separate `SELECT` queries (Figure 6.1):

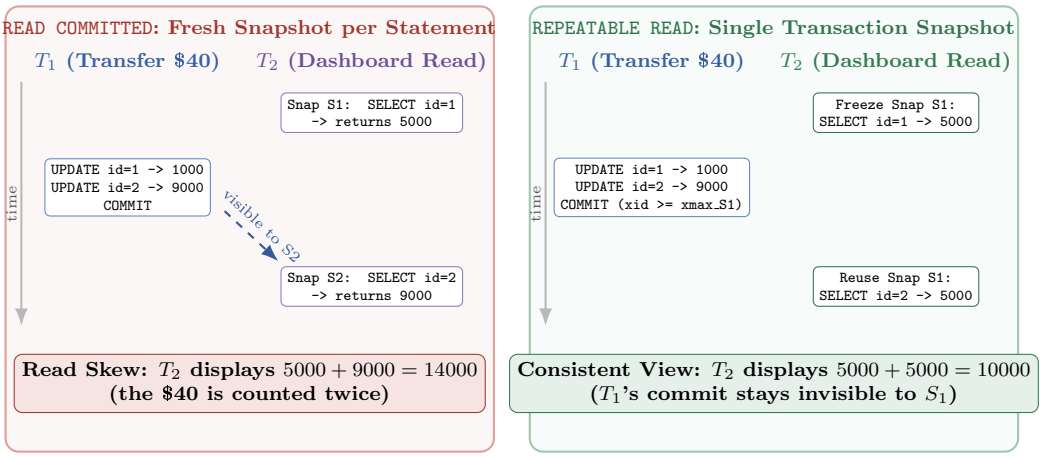


Figure 6.1: Read skew across two queries in one transaction. Left: Under default `READ COMMITTED`, T_2's second `SELECT` takes a new snapshot S_2 after T_1 commits, reading Checking *before* the transfer (`5000`) and Savings *after* it (`9000`). Right: Under `REPEATABLE READ`, T_2 reuses S_1 for every query, reading a consistent `10000`.

If T_2 instead re-read `id = 1` in its second query, it would see `5000` first and `1000` second—a *non-repeatable read*. Seeing an inconsistent cut across two related rows is its multi-row twin, **read skew**.

6.2.3 Phantom Reads (P_3)

A *phantom read* is the set-level twin of a non-repeatable read: concurrent session T_1 **`INSERT`s a new row** (or `DELETE`s one) matching T_2's `WHERE` filter between two queries in T_2. Under `READ COMMITTED`:

1. T_2 runs `SELECT COUNT(*) FROM postings WHERE entry_id = 500;` and gets `2`.
2. T_1 inserts a third fee leg into `entry_id = 500` and commits.
3. T_2 runs `SELECT amount_minor FROM postings WHERE entry_id = 500;` expecting two rows—and gets **three** rows.

Switch T_2 to `BEGIN ISOLATION LEVEL REPEATABLE READ;` and both non-repeatable reads and phantom reads disappear. The third posting's `t_xmin` belongs to T_1 ($\geq \text{xmax}_{S_1}$ or in xip_{S_1}), so S_1 hides the inserted row cleanly.

Common trap

Many developers assume wrapping two `SELECT` queries in `BEGIN; ... COMMIT;` freezes a shared snapshot automatically. **At default `READ COMMITTED`, `BEGIN ... COMMIT` around plain `SELECT`s does not freeze your snapshot across statements.** Every `SELECT` still grabs a new snapshot. Multi-query reports or reconciliations *must* run `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY;` (or combine both reads into one SQL query).

6.3 Lost Updates and `EvalPlanQual` Re-Evaluation

Now let both concurrent sessions *write* to the same account row `id = 1`, starting at `bal = 10000` (100.00 USD). Two payment workers simultaneously process two 80.00 USD (`8000`-cent) ATM withdrawals. What happens depends on *how* you write the SQL and *which* isolation level you pick.

6.3.1 Case 1: Read-Modify-Write in Application Code Under `READ COMMITTED`

In ORM-heavy codebases, developers often fetch a row into app memory, subtract in code, and write back the literal result:

```
-- Worker 1 (READ COMMITTED)            -- Worker 2 (READ COMMITTED)
BEGIN;                                  BEGIN;
SELECT bal FROM accounts WHERE id = 1; SELECT bal FROM accounts WHERE id = 1;
-- App computes: 10000 - 8000 = 2000    -- App computes: 10000 - 8000 = 2000

UPDATE accounts SET bal = 2000
 WHERE id = 1;
                                        UPDATE accounts SET bal = 2000
                                         WHERE id = 1;
                                        -- Blocks waiting on Worker 1's XID!
COMMIT;
                                        -- Unblocks, writes 2000, and COMMITS!
```

The ATM handed out 80.00 USD *twice* (160.00 USD total), yet `accounts.bal` ends at `2000` (20.00 USD). Worker 2 overwrote Worker 1's debit using a stale read: a **Lost Update**. It is the exact same bug as two Java threads running `balance = balance - 80` without synchronization.

6.3.2 Case 2: Atomic SQL `SET bal = bal - 8000` and `EvalPlanQual` (EPQ)

Now push the subtraction into a single atomic SQL `UPDATE` with a balance guard under `READ COMMITTED`:

```
UPDATE accounts
   SET bal = bal - 8000
 WHERE id = 1 AND bal >= 8000
RETURNING bal;
```

Trace what PostgreSQL's update executor does when Worker 1 and Worker 2 run this statement at the same microsecond under `READ COMMITTED`:

1. Both workers take a snapshot seeing version `(0,1)` with `bal = 10000`, where `10000 >= 8000` is `true`.
2. Worker 1 reaches the row first, stamps its XID into `(0,1)`'s `t_xmax`, inserts `(0,2)` with `bal = 2000`, and commits.
3. Worker 2 arrives a microsecond behind, sees Worker 1's active `t_xmax` on `(0,1)`, and **waits for Worker 1 to finish**.

4. Once Worker 1 commits, Worker 2 wakes up facing a dilemma. Using old `(0,1)` (`10000`) would lose Worker 1's debit; blindly subtracting `8000` from new `(0,2)` (`2000`) would write `-6000` and violate `WHERE bal >= 8000`.

To handle concurrent row updates safely under `READ COMMITTED`, PostgreSQL walks forward to the newest committed row version and re-runs both the `WHERE` filter and the `SET` expression against it. Internally, this re-check step is called `EvalPlanQual` (`EPQ`):

- Worker 2 follows `(0,1)`'s `t_ctid` link to committed version `(0,2)` (`bal = 2000`) and locks it.
- `EvalPlanQual` plugs `(0,2)` into the query plan and **re-checks `WHERE` and `SET` on `(0,2)`**.
- On `bal = 2000`, `bal >= 8000` is **false**. Worker 2 skips the row, returns `UPDATE 0`, and declines the second ATM withdrawal.

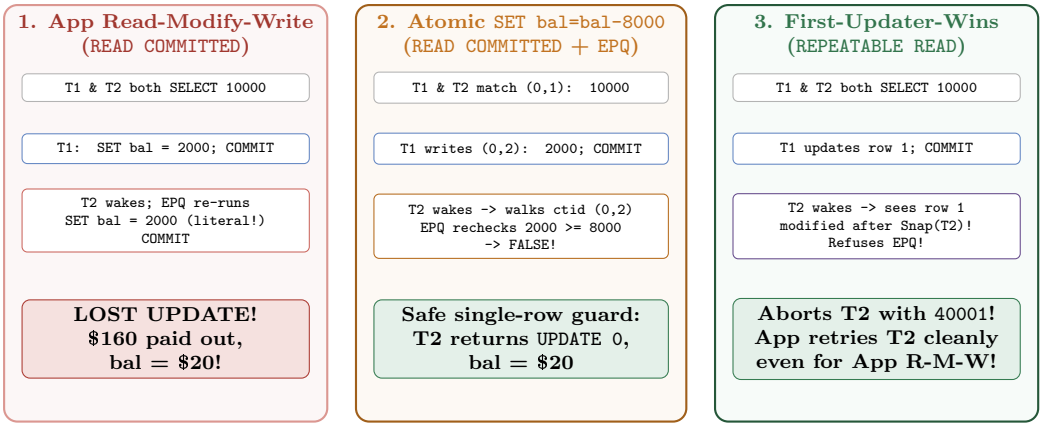


Figure 6.2: Three outcomes when T_1 and T_2 each withdraw 80 USD from a 100 USD row. (1) App read-modify-write under `READ COMMITTED` loses T_1's debit because `SET bal = 2000` writes a literal constant. (2) Atomic `SET bal = bal - 8000 WHERE bal >= 8000` runs `EvalPlanQual` on `(0,2)` and declines the second debit. (3) `REPEATABLE READ` aborts any concurrent edit to the same row with `SQLSTATE 40001`.

Common trap

Even `EvalPlanQual` has a sharp edge on multi-table `UPDATE ... FROM` queries. Under `READ COMMITTED`, `EvalPlanQual` reloads only *locked target rows*; unlocked joined tables in `FROM` or subqueries keep the rows from the statement's original snapshot. For single-row debits, `SET bal = bal - p_amt WHERE bal >= p_amt` is safe. For multi-statement read-modify-write code without explicit locks, `REPEATABLE READ` catches every same-row collision automatically via **First-Updater-Wins** (`SQLSTATE 40001`).

6.4 Write Skew: Why `REPEATABLE READ` Still Breaks Multi-Row Invariants

Look back at Panel 3 of Figure 6.2. Why did `REPEATABLE READ` catch that lost update? Because T_1 and T_2 modified **the exact same row** (`id = 1`). When T_2 tried to stamp `t_xmax` on `id = 1`, it saw T_1's committed `t_xmax` and raised `40001`.

Now return to Alice's two sub-accounts from our opening page, or the classic hospital on-call rule ("at least one night-shift doctor must stay `on_call = true`"). Watch Worker 1 and Worker 2 run under `REPEATABLE READ`:

```
-- Worker 1 (REPEATABLE READ)            -- Worker 2 (REPEATABLE READ)
BEGIN ISOLATION LEVEL REPEATABLE READ;  BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT SUM(bal) FROM sub_accounts       SELECT SUM(bal) FROM sub_accounts
 WHERE user_id = 7;  -- returns 16000    WHERE user_id = 7;  -- returns 16000

-- 16000 - 10000 = 6000 >= 0 (OK!)      -- 16000 - 10000 = 6000 >= 0 (OK!)
UPDATE sub_accounts                     UPDATE sub_accounts
   SET bal = bal - 10000                   SET bal = bal - 10000
 WHERE user_id = 7 AND kind = 'chk';     WHERE user_id = 7 AND kind = 'sav';

COMMIT; -- Succeeds!                    COMMIT; -- ALSO SUCCEEDS!
-- Combined balance is now -2000 + (-2000) = -4000 (< 0)!
```

Database researchers call this bug **Write Skew**:

1. T_1 and T_2 read *overlapping* rows ($R(T_1) = \{\texttt{chk}, \texttt{sav}\}$ and $R(T_2) = \{\texttt{chk}, \texttt{sav}\}$).
2. Using what they read, T_1 and T_2 write to *disjoint* rows ($W(T_1) = \{\texttt{chk}\}$, $W(T_2) = \{\texttt{sav}\}$, so $W(T_1) \cap W(T_2) = \emptyset$).
3. Because their write sets never touch the same row, First-Updater-Wins never fires, and the multi-row rule ($\texttt{chk} + \texttt{sav} \geq 0$) breaks silently.

Write skew strikes `INSERT`s too (*insert write skew*):

- **Room double-booking** (without an exclusion constraint): Worker 1 checks `COUNT(*) = 0` for Room 4 from 2–3 PM and inserts booking A; Worker 2 sees `COUNT(*) = 0` too and inserts booking B. Neither updated an existing row, so both commit under `REPEATABLE READ`.
- **Daily payout cap**: Two concurrent payouts check that Merchant 99's daily wires total under 50,000 USD, and each inserts a 40,000 USD wire. Both commit under `REPEATABLE READ`, sending 80,000 USD.

6.5 Serializable Snapshot Isolation (SSI): True `SERIALIZABLE` Without Read Locks

For thirty years after System R, textbooks taught a painful tradeoff: preventing write skew required **Two-Phase Locking (2PL)**—specifically **Strict Two-Phase Locking (S2PL)**, where every `SELECT` took shared read and index-gap locks that blocked writers until commit. Because read locks hurt throughput, PostgreSQL before version 9.1 mapped `SERIALIZABLE` silently to `REPEATABLE READ`.

In 2008, Michael Cahill, Uwe Röhm, and Alan Fekete proved a breakthrough result, shipped in PostgreSQL 9.1 by Kevin Grittner and Dan Ports. You can keep **100% non-blocking MVCC reads** and still guarantee full serializability by watching lightweight read-before-write conflict links in RAM. PostgreSQL calls this engine **Serializable Snapshot Isolation** (**SSI**).

6.5.1 Conflict Edges and the "Dangerous Structure" Theorem

When two transactions T_1 and T_2 touch the same row or index range x, three ordering links can form:

1. **Write-Read ($T_1 \xrightarrow{\text{wr}} T_2$):** T_1 writes x and commits; T_2's snapshot reads that version. Thus T_1 comes before T_2.
2. **Write-Write ($T_1 \xrightarrow{\text{ww}} T_2$):** T_1 writes x and commits; T_2 overwrites T_1's version. Again T_1 comes before T_2.
3. **Read-Write Antidependency ($T_1 \xrightarrow{\text{rw}} T_2$):** T_1 reads an *older* version of x, while concurrent T_2 writes a *newer* version of x (or inserts a row matching T_1's `WHERE` filter). In any matching one-at-a-time history, T_1 **must come before** T_2: had T_2 run first, T_1 would have seen T_2's write.

Now trace those links on Alice's Checking/Savings overdraft:

- T_1 read old `sav` (`8000`) while concurrent T_2 updated `sav` to `-2000`: $T_1 \xrightarrow{\text{rw}} T_2$.
- T_2 read old `chk` (`8000`) while concurrent T_1 updated `chk` to `-2000`: $T_2 \xrightarrow{\text{rw}} T_1$.

Together they form a cycle:

$$T_1 \xrightarrow{\text{rw}} T_2 \xrightarrow{\text{rw}} T_1.$$

T_1 must run before T_2, and T_2 must run before T_1—no valid serial order exists. Fekete et al. proved that *every* Snapshot Isolation anomaly leaves one telltale two-step footprint:

Theorem 6.1 (Dangerous Structure Under Snapshot Isolation)**.** *In any non-serializable Snapshot Isolation schedule, the dependency graph has a cycle, and* ***every minimal cycle contains two consecutive*** rw***-antidependency edges****:*

$$T_{\text{in}} \xrightarrow{\text{rw}} T_{\text{pivot}} \xrightarrow{\text{rw}} T_{\text{out}},$$

where each adjacent pair overlapped in time and T_{out} commits first ($T_{\text{in}} = T_{\text{out}}$ in 2-party write skew).

Intuition

Why does Theorem 6.1 keep SSI fast? Searching a huge graph for arbitrary cycles on every query would be slow. Theorem 6.1 says PostgreSQL never needs full cycle search. It merely watches for any **pivot transaction** T_{pivot} with both an incoming rw-link and an outgoing rw-link whose target T_{out} commits first. One local two-hop check in shared memory catches every anomaly.

6.5.2 How `SIRead` Predicate Locks Work Inside PostgreSQL

To spot when T_2 writes data that T_1 already read—even an empty gap where no row existed yet—PostgreSQL records passive read markers in shared memory called **`SIRead` predicate locks** (`predicate.c`):

1. **`SIRead` markers never block anyone:** Despite the name, an `SIRead` lock is just a passive sticky note in RAM saying "serializable session T_1 read this row, index page, or table." Readers never wait on writers, and writers never wait on `SIRead` markers.

2. **Index leaf tags catch phantom inserts:** When T_1 runs an index scan `WHERE room = 4 AND ...` and finds zero rows, PostgreSQL tags the **B-tree leaf page** where `room = 4` belongs. When concurrent T_2 inserts a booking onto that leaf page, it trips T_1's marker and records $T_1 \xrightarrow{\text{rw}} T_2$. (Without an index, a `Seq Scan` must tag the whole table—so always index columns filtered inside `SERIALIZABLE` transactions to avoid false-positive retries.)

3. **Bounded RAM via promotion:** If a query reads thousands of rows on one page, PostgreSQL merges their row markers into one page-level (or table-level) marker governed by `max_pred_locks_per_transaction`. Merging never misses a real conflict; at worst it causes an extra `40001` retry.

4. **Retained briefly past `COMMIT`:** In Alice's example, T_1 commits *first*, and the cycle completes only when T_2 commits second. PostgreSQL therefore keeps a committed transaction's `SIRead` summary until all overlapping serializable transactions finish.

Pause

Pause and compare the three MVCC isolation levels. `READ COMMITTED` takes a new snapshot per statement. `REPEATABLE READ` keeps one snapshot per transaction and aborts same-row write collisions. `SERIALIZABLE` adds non-blocking `SIRead` markers to abort multi-row write-skew cycles with `40001` so your app can retry cleanly.

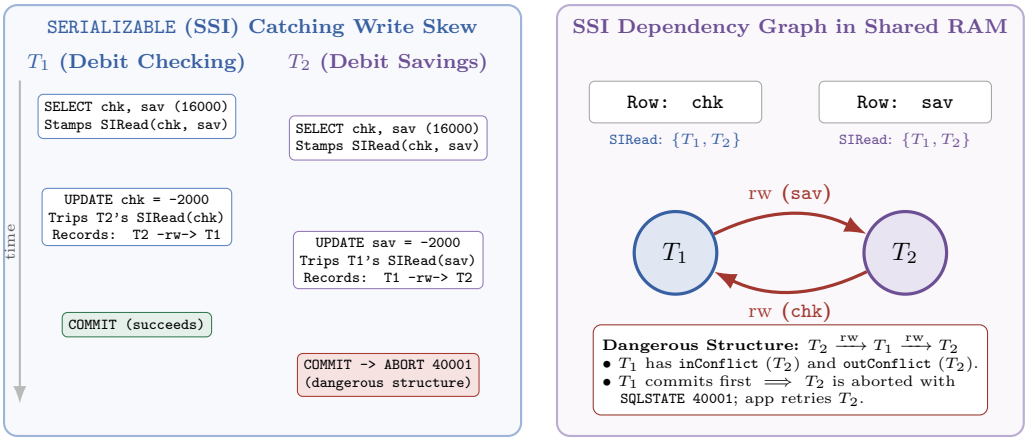


Figure 6.3: How Serializable Snapshot Isolation (SSI) catches write skew without blocking readers. Left: T_1 and T_2 stamp passive `SIRead` markers on `chk` and `sav`. Right: Writing `chk` and `sav` creates two rw-edges forming a cycle; once T_1 commits, T_2 aborts with `SQLSTATE 40001` and retries cleanly against the updated `6000` balance.

In production

Two rules every `SERIALIZABLE` production service lives by:

1. **Wrap every `REPEATABLE READ` or `SERIALIZABLE` transaction in a retry loop** catching `SQLSTATE 40001` and `40P01`, rolling back, and re-running from `BEGIN` with brief jitter. Never fire irreversible side effects (like charging a card or sending an email) inside a retryable transaction; write a transactional `outbox` row instead.

2. **Mark long read-only reports `SERIALIZABLE READ ONLY DEFERRABLE`:** Plain

`SERIALIZABLE` on a 30-second report places `SIRead` markers that can trigger `40001` retries. Starting with:

```
BEGIN ISOLATION LEVEL SERIALIZABLE READ ONLY DEFERRABLE;
```

waits briefly at `BEGIN` for a safe snapshot isolated from active writers, then runs the entire 30-second report with **zero `SIRead` markers and zero `40001` errors**.

6.6 Explicit Row Locks, Work Queues, Deadlocks, and Advisory Locks

SSI is optimistic: sessions run without waiting and retry if `40001` fires. Under heavy contention on one hot row—say, 200 checkout workers debiting the same flash-sale SKU—re-running 199 aborted transactions wastes CPU. On hot rows, **pessimistic locking** (queuing writers cleanly on the first pass) wins.

6.6.1 PostgreSQL's Four Row-Level Lock Strengths

Why does PostgreSQL offer *four* `SELECT ... FOR ...` row-lock modes instead of just "shared" and "exclusive"? Because of **foreign keys**.

Suppose `postings.account_id` references `accounts(id)`. Inserting a posting for Account 42 must stop anyone from deleting Account 42 or changing its `id` before commit. Before PostgreSQL 9.3, that child insert took `FOR SHARE` on `accounts(id = 42)`, **blocking every concurrent balance `UPDATE` on Account 42** even though `id` never changed.

PostgreSQL fixed that bottleneck by splitting row locks into four strengths:

1. `FOR KEY SHARE`**:** Weakest lock, taken automatically on parent rows during foreign-key checks. Conflicts *only* with `DELETE` and key-changing updates (`FOR UPDATE`).
2. `FOR SHARE`**:** Shared lock freezing the whole row until commit. Blocks `FOR NO KEY UPDATE` and `FOR UPDATE`, while allowing other `FOR SHARE` readers.
3. `FOR NO KEY UPDATE`**:** Taken automatically by any `UPDATE` that leaves `PRIMARY KEY` / `UNIQUE` columns untouched. **Compatible with `FOR KEY SHARE`**, so child inserts and parent balance updates run side by side without blocking.
4. `FOR UPDATE`**:** Strongest row lock (taken by `DELETE` or key-changing updates). Blocks all four row-lock modes, while still never blocking plain MVCC `SELECT`s.

To prevent Alice's Checking/Savings overdraft pessimistically under default `READ COMMITTED`, lock **both** rows in ID order before summing:

```
BEGIN;
-- Locks BOTH sub-accounts in deterministic ID order before summing!
SELECT id, kind, bal
  FROM sub_accounts
 WHERE user_id = 7
```

```
 ORDER BY id ASC
   FOR NO KEY UPDATE;
-- Worker 2 sleeps right here until Worker 1 commits, then sees bal = 6000!
```

6.6.2 Non-Blocking Modifiers: `NOWAIT` and `SKIP LOCKED` Job Queues

By default, `FOR UPDATE` waits until the lock holder finishes. Two clauses change that wait behavior:

- `FOR UPDATE NOWAIT`: Fails *immediately* with `SQLSTATE 55P03` if the row is locked instead of parking an HTTP thread. In user-facing APIs, always use `NOWAIT` or `SET LOCAL lock_timeout = '2s';` so one stuck lock cannot drain your connection pool.

- `FOR UPDATE SKIP LOCKED`: Quietly skips rows already locked by other workers. Six lines of SQL turn an ordinary table into an ACID work queue where fifty workers claim disjoint batches with zero lock waits:

```
WITH claimed AS (
    SELECT id
      FROM webhook_outbox
     WHERE status = 'pending' AND next_attempt_at <= now()
     ORDER BY next_attempt_at ASC
     LIMIT 25
       FOR UPDATE SKIP LOCKED
)
UPDATE webhook_outbox w
   SET status = 'processing', attempts = attempts + 1
  FROM claimed c
 WHERE w.id = c.id
RETURNING w.id, w.endpoint_url, w.payload_json;
```

6.6.3 Deadlocks (`deadlock_timeout`) and Deterministic Lock Ordering

Whenever transactions lock multiple rows one by one, opposite lock orders can form a wait cycle:

- T_1 transfers 10 USD from Account 10 to Account 20: locks row `10`, then waits on row `20`.

- Simultaneously, T_2 transfers 15 USD from Account 20 to Account 10: locks row `20`, then waits on row `10`.

Now T_1 waits on T_2 and T_2 waits on T_1; neither can move forward.

PostgreSQL checks for deadlocks lazily. Building a global wait graph on every microsecond lock wait would waste CPU when 99.99% of row locks clear in under a millisecond. Instead, a waiting backend sets a timer for `deadlock_timeout` (default `1s`). Only if it is still waiting after 1,000 ms does it scan the wait graph, spot $T_1 \to T_2 \to T_1$, and abort one victim with `SQLSTATE 40P01: deadlock detected`.

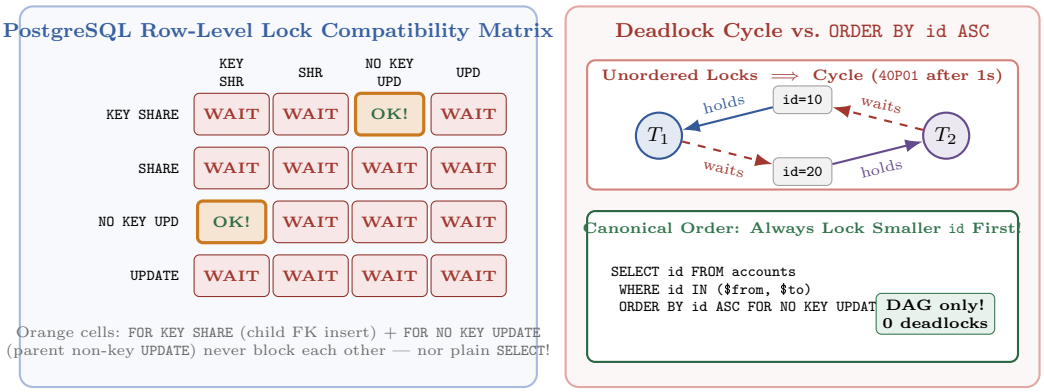


Figure 6.4: Left: Compatibility across PostgreSQL's four row-lock modes. Highlighted orange cells show why child foreign-key inserts (`FOR KEY SHARE`) never block parent balance updates (`FOR NO KEY UPDATE`). Right: Arbitrary lock order allows a 2-cycle, whereas locking in ascending key order (`ORDER BY id ASC`) makes every wait edge point upward ($u < v$), ruling out deadlocks.

The proof behind Figure 6.4 takes two lines. If every session locks accounts in strictly increasing `id` order, any session holding k_1 while waiting for k_2 satisfies $k_1 < k_2$. A cycle $k_1 < k_2 < \cdots < k_m < k_1$ would imply $k_1 < k_1$, which is impossible. Sorting account IDs before locking eliminates transfer deadlocks completely.

6.6.4 Transactional Advisory Locks (`pg_advisory_xact_lock`)

One puzzle remains: how do you lock a resource **whose row does not exist yet**? Suppose two requests concurrently onboard merchant `tax_id = 'US-98-7654321'` and run validation checks before inserting the first row. `SELECT ... FOR UPDATE WHERE tax_id = ...` finds zero rows and locks nothing.

PostgreSQL solves this with **advisory locks**—application-defined 64-bit integer keys held purely in shared memory, generating zero heap writes, zero dead rows, and zero WAL:

```
BEGIN;
-- Hash a namespace + business key to a 64-bit BIGINT lock ID;
-- automatically released at COMMIT or ROLLBACK!
SELECT pg_advisory_xact_lock(
    hashtextextended('onboard_tax_id:' || $1, 0)
);
-- Now strictly one worker per tax_id executes this critical section at a time!
COMMIT;
```

Always prefer transaction-scoped `pg_advisory_xact_lock` over session-scoped `pg_advisory_lock`. Behind a transaction-mode pooler like `PgBouncer` (Chapter 8), an unreleased session lock stays stuck on the pooled backend connection and blocks unrelated tenants.

We now know how PostgreSQL keeps concurrent transactions fast and isolated. Yet before any query can lock or read a row, PostgreSQL must choose *how* to find it across billions of rows and dozens of tables—sequential scan, index scan, bitmap scan, nested loop, hash join, or merge join. Chapter 7 opens the query planner and reads `EXPLAIN (ANALYZE, BUFFERS)` line by line.

6.7 Exercises

Exercise 6.1 (Testing serializability by hand)**.** Accounts start at `A = 100` and `B = 100`. Consider two committed transactions:

- T_1: reads `A`, then sets `B = A + 50`.
- T_2: reads `B`, then sets `A = B * 2`.

(a) Find the final (`A`, `B`) pair for both serial orders ($T_1 \to T_2$ and $T_2 \to T_1$). (b) Find the final (`A`, `B`) pair when T_1 and T_2 run concurrently from the same initial snapshot under `REPEATABLE READ`. Does `REPEATABLE READ` raise an error, and is the result serializable? (c) Trace what changes under `ISOLATION LEVEL SERIALIZABLE`.

Exercise 6.2 (Read skew in pagination and reporting)**.** At default `READ COMMITTED`, an API wraps two queries in `BEGIN; ... COMMIT;`. First it runs `SELECT COUNT(*) FROM invoices WHERE user_id = 99 AND status = 'unpaid';` for a badge, and 5 ms later it fetches `SELECT id, amount_minor FROM invoices` with the same filter. Show an interleaving with a concurrent payment where the badge says "1 unpaid invoice" while the list is empty. Give two ways to fix the mismatch without blocking the writer.

Exercise 6.3 (Tracing `EvalPlanQual` step by step)**.** Table `wallets(id INT PRIMARY KEY, bal BIGINT)` holds one row `(1, 10000)`. Sessions T_1 and T_2 run at `READ COMMITTED` and start simultaneously:

- T_1: `UPDATE wallets SET bal = bal - 7000 WHERE id = 1 AND bal >= 7000;`
- T_2: `UPDATE wallets SET bal = bal - 5000 WHERE id = 1 AND bal >= 5000;`

Suppose T_1 locks `(0,1)` first. (a) If T_1 commits after 20 ms, which version does `EvalPlanQual` inspect for T_2, what does `bal >= 5000` return, and what is the final balance? (b) Repeat (a) if T_2 subtracts `2000` with `WHERE id = 1 AND bal >= 2000`. (c) Contrast (b) with T_2 running at `REPEATABLE READ`.

Exercise 6.4 (Insert write skew and SSI index gaps)**.** Table `on_call_shifts(shift_date DATE, doctor_id INT, PRIMARY KEY (shift_date, doctor_id))` has two doctors on `'2026-11-05'`. At most *three* doctors may claim overtime per date. Doctors 3 and 4 concurrently check `COUNT(*) = 2 < 3` for `'2026-11-05'` and each `INSERT`s a row. (a) Why do both `INSERT`s commit under `REPEATABLE READ`? (b) Which B-tree page gets an `SIRead` marker under `SERIALIZABLE`, and why does the second committer fail with `40001`? (c) If `on_call_shifts` had no index on `shift_date` (forcing a `Seq Scan`), what happens under `SERIALIZABLE` when Doctor 5 books `'2026-12-25'`?

Exercise 6.5 (Drawing the dangerous structure)**.** Three concurrent transactions run under `REPEATABLE READ` over $x = 0, y = 0, z = 0$:

- T_1 reads y (0), writes $x \leftarrow 1$, and commits at $t = 3$.
- T_2 reads z (0), writes $y \leftarrow 1$, and commits at $t = 2$.
- T_3 reads x (0), writes $z \leftarrow 1$, and commits at $t = 1$.

Draw all rw-edges among $\{T_1, T_2, T_3\}$ and verify that no two transactions wrote the same variable. Prove $(x, y, z) = (1, 1, 1)$ cannot come from any of the $3! = 6$ serial orders, and name $T_{\text{in}}, T_{\text{pivot}}, T_{\text{out}}$ in Theorem 6.1.

Exercise 6.6 (`FOR UPDATE` vs. `FOR NO KEY UPDATE`)**.** Parent table `accounts(id, bal)` is referenced by child foreign key `transfers(account_id)`. What lock waits occur when Connection A holds `SELECT bal FROM accounts WHERE id = 42 FOR UPDATE;` open for 50 ms while 20 sessions insert into `transfers` for `account_id = 42`? What changes as soon as Connection A uses `FOR NO KEY UPDATE`?

Exercise 6.7 (Transactional outbox edge cases with `SKIP LOCKED`)**.** Review the `FOR UPDATE SKIP LOCKED` queue query in Section 6.6. (a) Worker A claims 25 rows, marks them `'processing'`, commits, and crashes with `SIGKILL` after sending 10 webhooks. What happens to the other 15 rows without a `leased_until TIMESTAMPTZ` column? (b) Write a crash-safe claim query using `leased_until` so abandoned items re-enter the pool after 60 seconds without double-claiming active work.

Exercise 6.8 (Multi-row deadlock construction and prevention)**.** A payroll endpoint credits k accounts $r_1, \ldots, r_k$ by running k separate `UPDATE accounts SET bal = bal + p_amt WHERE id = p_id;` statements in a loop, then debiting funding account f. (a) Build a two-session deadlock between two payroll runs sharing $f = 1$ and $r = 50$. (b) Rewrite the lock step as a single `SELECT ... WHERE id = ANY(p_ids) ORDER BY id ASC FOR NO KEY UPDATE;` query and prove why deadlocks vanish.

Exercise 6.9 (Advisory lock collision probability and pooling)**.** (a) Compare 32-bit `hashtext('tenant:' || id)` against 64-bit `hashtextextended('tenant:' || id, 0)` inside `pg_advisory_xact_lock`. Using $p \approx k^2/(2M)$ for $k = 1{,}000$ active locks, estimate the chance that two unrelated tenants collide under 32-bit versus 64-bit hashes. (b) What breaks when code calls session-level `pg_advisory_lock(42)` behind a transaction-mode pooler and skips `pg_advisory_unlock(42)` on error?

Part III

Query Execution and Production PostgreSQL

Chapter 7

The Query Planner, Join Algorithms, and `EXPLAIN ANALYZE`

Here is a query that a payments dashboard runs every few seconds:

```
SELECT a.id, a.owner_name, SUM(p.amount_minor) AS total
  FROM accounts a
  JOIN postings p ON p.account_id = a.id
 WHERE a.region = 'EU'
   AND p.created_at >= '2026-10-01'
 GROUP BY a.id, a.owner_name
 ORDER BY total DESC
 LIMIT 10;
```

The query says *what* answer we want. It says nothing about *how* to get it. Should PostgreSQL read all 20 million postings and throw most of them away? Should it find the EU accounts first through an index, and then look up each account's postings through a second index? Should it build a hash table of EU accounts in memory and stream the postings past it? Each of those is a correct way to compute the same ten rows. On one real table, the first plan took 45 seconds and the second took 0.2 seconds. Same SQL, same data, same hardware: a factor of two hundred.

The piece of PostgreSQL that chooses among those plans is the *query planner* (also called the optimizer). It never runs the candidates. It *predicts* what each would cost using statistics about your data and a small set of cost constants, and then hands the cheapest prediction to the executor. When the prediction is right, you never think about it. When it is wrong, a query that ran in 3 ms yesterday runs for 40 seconds today, and the only way to see why is to read `EXPLAIN (ANALYZE, BUFFERS)` output.

This chapter builds the planner's reasoning from the ground up. We start with the column statistics it collects and the cost units it uses as currency. Then we trace the four ways it reads a single table, the three ways it joins two tables, and how it searches join orders. Finally, we read `EXPLAIN (ANALYZE, BUFFERS)` output line by line to spot exactly where a slow plan went wrong.

Focus

By the end of this chapter you will be able to answer five practical engineering questions:

- How can sampling just 30,000 random rows let PostgreSQL estimate filter matches on a five-billion-row table?
- Why does an index scan on a timestamp column run thirty times faster than the exact same 1% index scan on a random amount column?
- When does PostgreSQL build an in-memory page bitmap instead of using a plain index scan or reading the whole table?
- How does the planner choose between a Nested Loop, a Hash Join, and a Merge Join—and what happens when a Hash Join runs out of RAM?
- When a query suddenly slows from 3 ms to 40 seconds, which numbers in `EXPLAIN (ANALYZE, BUFFERS)` reveal the root cause?

7.1 From SQL Text to a Running Plan

Every statement passes through four stages inside the backend process that owns your connection:

1. **Parse.** The SQL text becomes a parse tree. Syntax errors are caught here. Nothing about tables has been checked yet.
2. **Analyze and rewrite.** Names are resolved against the system catalogs (`a.region` becomes "column 4 of relation object identifier (OID) 16402"), types are checked, and views are expanded into their underlying queries. The result is a *query tree*.
3. **Plan.** The planner turns the query tree into a *plan tree*: a tree of physical operators such as `Seq Scan`, `Index Scan`, `Hash Join`, and `Sort`. This is the only stage that makes choices, and it is the subject of this chapter.
4. **Execute.** The executor runs the plan tree. Each node is an iterator: a parent asks its child for the next row, the child asks *its* child, and rows flow upward one at a time. (This is the "Volcano" iterator model; PostgreSQL also batches some work, such as building hash tables, but the pull-one-row interface is the mental model to keep.)

The relational algebra from Chapter 1 lives in stage 3. A join is commutative and associative, a selection σ can be pushed below a join, and a projection π can drop columns early. Those algebraic laws generate many logically equivalent trees. The planner's job is to attach a physical algorithm to every node of every reasonable tree, estimate the cost, and keep the cheapest.

Intuition

Think of the planner as a travel agent who has never visited the city. It cannot drive each route to time it. Instead it owns a map (table and index sizes), a census (statistics about the values in each column), and a price list (cost constants per page read and per row processed). Every route it proposes is only as good as that map and census. Most bad plans are not caused by a stupid search; they are caused by a census that is stale or blind to some pattern

in your data.

7.2 What the Planner Knows: `ANALYZE` and `pg_stats`

7.2.1 Sampling a Table of Any Size

The `ANALYZE` command (run automatically by autovacuum after enough rows change) reads a *random sample* of each table and stores a summary per column in the catalog `pg_statistic`. The human-readable view of that catalog is `pg_stats`.

How big is the sample? PostgreSQL samples $300 \times$ `default_statistics_target` rows. With the default target of 100, that is **30,000 rows**, whether the table holds 50,000 rows or 5 billion. This can feel wrong at first. Why would 30,000 rows describe 5 billion?

Because the questions the planner asks are about *fractions*, not totals. "What fraction of postings have `status = 'settled'`?" is a proportion, and the error of an estimated proportion from a random sample of size n shrinks like $1/\sqrt{n}$, regardless of population size. With $n = 30{,}000$, a value that occurs in 10% of rows is estimated to within about ± 0.35 percentage points (two standard errors of $\sqrt{0.1 \times 0.9/30000} \approx 0.0017$). That is plenty for choosing between a sequential scan and an index scan.

What a small sample *cannot* do well is count rare things. If a column has 40 million distinct values in 5 billion rows, the sample sees most values exactly once, and the estimate of `n_distinct` is a statistical extrapolation that can be off by a factor of several. We will see how to fix that when it matters.

7.2.2 Reading One Row of `pg_stats`

Here is what `pg_stats` reports for two columns of a 20-million-row `postings` table:

```
SELECT attname, null_frac, avg_width, n_distinct,
       most_common_vals, most_common_freqs,
       histogram_bounds, correlation
  FROM pg_stats
 WHERE tablename = 'postings'
   AND attname IN ('status', 'amount_minor');
```

```
-[ RECORD 1 ]-----+-----------------------------------
attname           | status
null_frac         | 0
avg_width         | 8
n_distinct        | 5
most_common_vals  | {settled,pending,failed}
most_common_freqs | {0.82,0.12,0.04}
histogram_bounds  |
correlation       | 0.31
-[ RECORD 2 ]-----+-----------------------------------
attname           | amount_minor
null_frac         | 0.002
```

```
avg_width         | 8
n_distinct        | -0.21
most_common_vals  |
most_common_freqs |
histogram_bounds  | {0,1000,2000,5000,20000}
correlation       | 0.02
```

(The real histogram has 101 bounds, giving 100 buckets; we use five bounds and four buckets so we can do the arithmetic by hand.) Each field answers one question the planner will ask:

- `null_frac`: the fraction of rows where the column is `NULL`. A predicate like `amount_minor > 0` can never match those rows, so the planner removes them first.
- `avg_width`: average stored bytes per value. The planner uses it to estimate how much memory a sort or hash table will need.
- `n_distinct`: the number of distinct non-null values. A *positive* number is an absolute count: `status` has 5 distinct values. A *negative* number is a fraction of the row count: -0.21 means "about 21% of rows have a distinct value," so the planner computes $0.21 \times 20{,}000{,}000 = 4.2$ million distinct amounts. The negative form is used when the number of distinct values seems to grow with the table, so the estimate stays sensible as the table grows. A value of -1 means every row is distinct, as for a primary key.
- `most_common_vals` and `most_common_freqs` (the most-common-values, or MCV, list): the most frequent values and their frequencies, as observed in the sample. Only values that are noticeably more common than average make the list.
- `histogram_bounds`: boundaries of an *equi-depth* histogram over the values that are *not* in the MCV list. Equi-depth means each bucket holds the same number of rows, so buckets are narrow where values are dense and wide where they are sparse.
- `correlation`: how closely the physical order of rows on disk follows the sorted order of this column, from -1 (perfectly reversed) through 0 (random) to $+1$ (perfectly sorted). We will see in Section 7.4 why this one number can swing an index scan's cost by a factor of forty.

7.2.3 Turning Statistics into Selectivity

The planner's central quantity is *selectivity*: the fraction of a table's rows that a predicate keeps. Multiply selectivity by the row count and you get the estimated `rows` you see in `EXPLAIN`. Let us compute several by hand from the statistics above.

Example 7.1 (Equality on a value in the MCV list)**.** For `WHERE status = 'pending'`, the value is in the MCV list with frequency 0.12. The selectivity is simply 0.12, and the estimate is $0.12 \times 20{,}000{,}000 = 2{,}400{,}000$ rows.

Example 7.2 (Equality on a value not in the MCV list)**.** For `WHERE status = 'reversed'`, the value is not in the MCV list. The planner reasons: the MCVs account for $0.82 + 0.12 + 0.04 = 0.98$ of rows, and there are no `NULL`s, so $1 - 0.98 = 0.02$ of rows hold the *other* values. There are $5 - 3 = 2$ other distinct values. Assume they split the remainder evenly:

$$\text{sel} \;=\; \frac{1 - \text{null_frac} - \sum \text{mcv_freqs}}{\text{n_distinct} - |\text{MCV}|} \;=\; \frac{1 - 0 - 0.98}{5 - 3} \;=\; 0.01.$$

Estimate: $0.01 \times 20{,}000{,}000 = 200{,}000$ rows.

Example 7.3 (A range predicate using the histogram)**.** For `WHERE amount_minor < 3500`, there is no MCV list for this column, so every non-null row is covered by the histogram with bounds $\{0, 1000, 2000, 5000, 20000\}$. Each of the four buckets holds $\frac{1}{4}$ of the non-null rows.

The value 3500 falls inside the third bucket $[2000, 5000)$. Buckets one and two lie entirely below 3500: that is $\frac{2}{4}$. Within bucket three, PostgreSQL assumes values are spread uniformly and interpolates linearly: 3500 sits $\frac{3500-2000}{5000-2000} = 0.5$ of the way across, contributing $0.5 \times \frac{1}{4} = \frac{1}{8}$. So the histogram fraction is $\frac{2}{4} + \frac{1}{8} = 0.625$. Finally, remove the 0.2% of `NULL`s:

$$\text{sel} = 0.625 \times (1 - 0.002) \approx 0.624, \qquad \text{rows} \approx 12{,}475{,}000.$$

A `BETWEEN 1500 AND 3500` predicate works the same way: compute the fraction below 3500 (0.625), subtract the fraction below 1500 ($\frac{1}{4} + 0.5 \times \frac{1}{4} = 0.375$), and get 0.25 of non-null rows.

Figure 7.1 shows these three pictures together, along with what `correlation` measures.

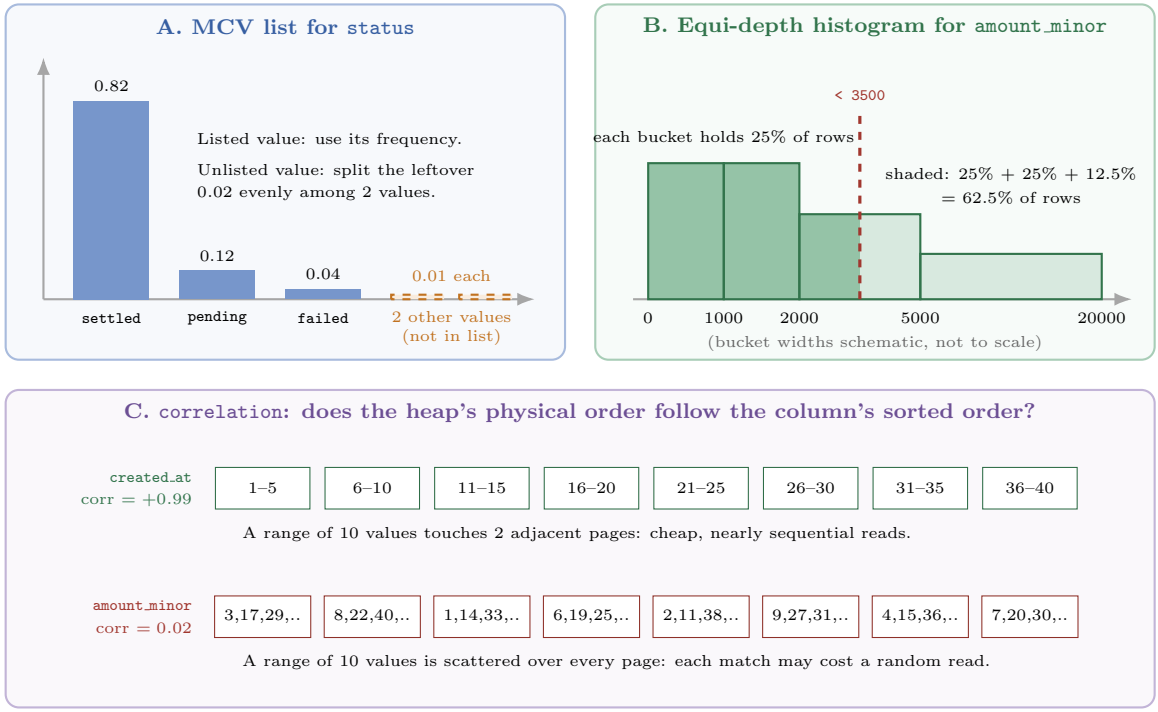


Figure 7.1: The three statistics that drive most row estimates. A: the MCV list gives exact frequencies for common values and an even split of the leftover for rare ones. B: an equi-depth histogram puts the same number of rows in every bucket, and the planner interpolates linearly inside the bucket containing the boundary value. C: `correlation` measures whether rows that are adjacent in sorted order are also adjacent on disk.

7.2.4 When Two Predicates Are Not Independent

With two predicates joined by `AND`, PostgreSQL by default assumes they are statistically independent and multiplies their selectivities:

$$\text{sel}(A \wedge B) = \text{sel}(A) \times \text{sel}(B).$$

That is often fine. It is badly wrong when the columns are related. Consider a `transfers` table where 60% of rows have `currency = 'USD'` and 55% have `country = 'US'`. Independence predicts

$$0.60 \times 0.55 = 0.33,$$

but almost every US transfer is in dollars, so the true fraction with both is about 0.54. The planner underestimates by about 40%. That is survivable. Now take `city = 'Lyon' AND postal_code = '69001'`: if each has selectivity 0.001, independence predicts 10^{-6}, yet every row in that postal code is in Lyon, so the truth is close to 10^{-3}. The estimate is *a thousand times* too small, and a thousand-times-too-small row count is exactly what makes the planner pick a nested loop that runs for minutes (we will see why in Section 7.5).

PostgreSQL cannot discover such relationships on its own, because `pg_stats` describes each column separately. You can tell it to look at a group of columns together with *extended statistics*:

```
CREATE STATISTICS addr_stats (dependencies, ndistinct, mcv)
    ON city, postal_code FROM customers;
ANALYZE customers;
```

The three kinds each fix a different mistake:

- `dependencies` measures how strongly one column determines another. If `postal_code` determines `city` with degree f (from 0 to 1), the planner estimates $\text{sel}(A \wedge B) = \text{sel}(A) \times \big(f + (1 - f)\,\text{sel}(B)\big)$. With $f = 1$, the second predicate costs nothing extra, which is right: knowing the postal code already tells you the city.
- `ndistinct` records the number of distinct *combinations*, which fixes `GROUP BY city, postal_code` estimates (otherwise the planner multiplies the two distinct counts and predicts far too many groups).
- `mcv` keeps a most-common-values list over the *pairs*, which gives exact frequencies for common combinations, including ones where the columns are anti-correlated.

Common trap

Statistics only exist for plain columns, and for expressions that have an index or an expression statistic built on them. If you write `WHERE date_trunc('day', created_at) = '2026-10-31'`, the planner has no histogram for that expression and falls back to a hard-coded default selectivity (0.5% for equality). On a 20-million-row table that predicts 100,000 rows no matter whether the true answer is 60 or 6 million. Either rewrite the predicate as a range on the raw column (`created_at >= '2026-10-31' AND created_at < '2026-11-01'`), which also lets an ordinary index on `created_at` be used, or create an expression index or `CREATE STATISTICS ... ON (date_trunc('day', created_at))` so `ANALYZE` collects a histogram for it.

7.3 The Planner's Currency: Cost Units

Once the planner knows how many rows each step touches, it needs a common unit to compare a plan that reads many pages sequentially against a plan that reads few pages randomly but does more CPU work. PostgreSQL measures everything in arbitrary *cost units*, anchored so that reading one 8 KB page sequentially costs 1.0:

Parameter	Default	What it prices
`seq_page_cost`	1.0	Reading one page as part of a sequential run
`random_page_cost`	4.0	Reading one page at an arbitrary location
`cpu_tuple_cost`	0.01	Processing one heap row
`cpu_index_tuple_cost`	0.005	Processing one index entry
`cpu_operator_cost`	0.0025	Evaluating one operator or function call
`effective_cache_size`	4GB	How much of the data the OS and `shared_buffers` likely cache

Two of these deserve a comment.

`random_page_cost = 4.0` dates from spinning disks, where a random read cost a seek of several milliseconds while sequential reads streamed. The default is already a compromise: on a real hard drive a random read is closer to 40 times the cost of a sequential one, but most random reads hit cache. On NVMe SSDs, random and sequential reads cost nearly the same, and most teams set `random_page_cost = 1.1`. That single change makes the planner far more willing to use indexes.

`effective_cache_size` allocates nothing. It is only a hint: "assume about this much data is cached somewhere." The planner uses it in the Mackert–Lohman formula (below) to predict how many repeated page fetches will be cache hits.

7.3.1 Pricing a Sequential Scan by Hand

Take a `ledger_lines` table that occupies 10,000 pages and holds 500,000 rows (50 rows per 8 KB page). The cost of reading every row and testing one `WHERE` predicate is

$$\underbrace{10{,}000 \times 1.0}_{\text{pages}} + \underbrace{500{,}000 \times 0.01}_{\text{rows}} + \underbrace{500{,}000 \times 0.0025}_{\text{one predicate per row}} = 10{,}000 + 5{,}000 + 1{,}250 = 16{,}250.$$

You can check this against PostgreSQL directly:

```
EXPLAIN SELECT * FROM ledger_lines WHERE amount_minor > 900000;
```

```
Seq Scan on ledger_lines  (cost=0.00..16250.00 rows=4975 width=48)
  Filter: (amount_minor > 900000)
```

Every node in `EXPLAIN` prints `cost=`*startup*`..`*total*:

- **Startup cost** is the work done before the node can return its *first* row. A sequential scan can return a row as soon as it reads page 0, so its startup cost is 0. A `Sort` must read its entire input before emitting anything, so its startup cost is nearly its total cost.
- **Total cost** is the work to return *all* rows.
- `rows` is the estimated output row count and `width` is the estimated average row size in bytes.

The distinction matters because of `LIMIT`. If a plan's total cost is 50,000 for 100,000 rows but you only want 10 rows, the planner charges roughly startup + (total − startup) × $\frac{10}{100{,}000}$. A plan with near-zero startup cost (such as walking an index that is already in the requested `ORDER BY` order) can beat a plan with a lower total cost but a large startup cost (such as sorting everything first). This is the reason `ORDER BY created_at DESC LIMIT 20` on a table with an index on `created_at` returns in a millisecond.

7.4 Four Ways to Read One Table

For a single table with a predicate such as `WHERE merchant_id = 4242`, PostgreSQL considers up to four *access paths*.

7.4.1 Seq Scan

Read every page in physical order and test every row. The cost is almost independent of how many rows match. Sequential reads are the cheapest kind of I/O, and the operating system's read-ahead makes them faster still. For a predicate that keeps a large fraction of rows, nothing beats it.

7.4.2 Index Scan

Descend the B-tree (Chapter 3) to the first matching key, then walk the leaf pages. For each index entry, follow its `ctid` to the heap page, fetch the row, and check visibility against the snapshot (Chapter 5). Rows come out in index order, which can satisfy an `ORDER BY` for free.

The catch is the heap fetches. Each matching row may live on a different heap page, and each of those page reads may be a random read. How many distinct heap pages does an index scan touch? If matching rows sit next to each other on disk, 5,000 rows at 50 rows per page fit on just 100 consecutive pages. If matching rows are scattered at random across a 10,000-page table, almost every fetch lands on a different page, though some pages will be hit twice by chance. The planner computes both extremes—completely scattered versus perfectly clustered—and blends between them using the column's squared `correlation` statistic.

Formal

How PostgreSQL prices the heap side of an Index Scan (`cost_index` in `costsize.c`). Let the table have T pages, and let the scan fetch N rows.

- **Worst case (correlation 0, scattered rows):** the expected number of distinct pages visited when drawing N rows across T pages with caching is approximated by the Mackert–Lohman formula. When the table fits inside `effective_cache_size`, that count is
$$P_{\max} = \min\Big(\frac{2TN}{2T+N},\ T\Big),$$
and each page costs `random_page_cost`: $\mathrm{IO}_{\max} = P_{\max} \times 4.0$.
- **Best case (correlation ± 1, clustered rows):** the matching rows sit on consecutive pages. That touches $P_{\min} = \lceil \mathrm{sel} \times T \rceil$ pages, with the first a random read and the rest sequential: $\mathrm{IO}_{\min} = 4.0 + (P_{\min} - 1) \times 1.0$.
- **Interpolate with the square of the correlation c:**
$$\mathrm{IO} = \mathrm{IO}_{\max} + c^2\,(\mathrm{IO}_{\min} - \mathrm{IO}_{\max}).$$

CPU cost is added on top: `cpu_index_tuple_cost` plus one `cpu_operator_cost` per index entry, and `cpu_tuple_cost` per heap row.

Example 7.4 (The same 1% query on two different columns)**.** Use the `ledger_lines` table again: $T = 10{,}000$ pages, 500,000 rows. A predicate keeps 1%, so $N = 5{,}000$ rows.

Uncorrelated column ($c = 0.02$, effectively 0). Mackert–Lohman gives

$$P_{\max} = \frac{2 \times 10{,}000 \times 5{,}000}{20{,}000 + 5{,}000} = \frac{10^8}{25{,}000} = 4{,}000 \text{ pages}, \qquad \text{IO} \approx 4{,}000 \times 4.0 = 16{,}000.$$

Add about $5{,}000 \times (0.005 + 0.0025) = 37.5$ for index entries, $5{,}000 \times 0.01 = 50$ for heap rows, and a few dozen for index pages, and the total lands near 16,150. The sequential scan costs 16,250. At 1%, on an uncorrelated column, the two plans are essentially tied.

Correlated column ($c = 0.99$, such as `created_at` on an append-only table). Now $P_{\min} = 100$ pages and $\text{IO}_{\min} = 4 + 99 = 103$. With $c^2 = 0.98$:

$$\text{IO} = 16{,}000 + 0.98 \times (103 - 16{,}000) \approx 420.$$

The total is about 520, *thirty times* cheaper than the sequential scan. Same selectivity, same table size; the only difference is whether the matching rows sit together on disk.

That example explains a pattern every PostgreSQL user eventually notices: range queries on `id` or `created_at` of an append-only table use indexes even for large ranges, while the same range size on a randomly distributed column flips to a sequential scan early.

7.4.3 Bitmap Index Scan plus Bitmap Heap Scan

Between "a handful of rows" (Index Scan) and "a large fraction of the table" (Seq Scan) there is a wide middle zone where neither is good. Suppose 2% of `ledger_lines` match an uncorrelated predicate: $N = 10{,}000$ rows. An Index Scan would visit about 6,667 distinct pages (Mackert–Lohman again), in index-key order, which means jumping randomly back and forth across the file and possibly re-reading pages that fell out of cache. That costs about 26,700, well above the sequential scan.

PostgreSQL's answer is to split the work into two nodes:

1. **Bitmap Index Scan.** Walk the index and collect every matching 6-byte tuple identifier (`ctid`, or TID), but do *not* visit the heap yet. Instead, record each TID in an in-memory bitmap keyed by heap page number: "page 17, tuples 1 and 9; page 812, tuples 3 and 7."

2. **Bitmap Heap Scan.** Visit the heap pages listed in the bitmap *in increasing page order*. Each page is read once, no matter how many matching tuples it holds, and the reads move steadily forward through the file, so the operating system's read-ahead helps and the drive sees something close to a sequential pattern.

The planner prices the per-page cost on a sliding scale between `random_page_cost` and `seq_page_cost`: the larger the fraction of the table being read, the closer to sequential each page is. For our 2% case, about two thirds of the pages are touched, the per-page cost comes out near 1.55, and the whole plan costs about 10,600, comfortably below both the Index Scan (26,700) and the Seq Scan (16,250). This is why you so often see bitmap scans for predicates matching a few percent of a table.

Exact versus lossy bitmaps. The bitmap must fit in `work_mem`. While it does, every page entry records exactly which tuple offsets matched: an *exact* page. If the bitmap would grow past `work_mem`, PostgreSQL starts converting entries to *lossy* form: it keeps only "some tuple on this page matched" and forgets which ones. A lossy page costs one bit of information instead of a list of offsets, so memory stays bounded no matter how many rows match. The price is paid at the heap: for a lossy page, the Bitmap Heap Scan must re-evaluate the original condition against *every* tuple on that page. That re-evaluation is the `Recheck Cond` line you see in the plan, and `EXPLAIN ANALYZE` tells you how much it cost:

```
Bitmap Heap Scan on postings
    (cost=4120.55..98211.31 rows=201004 width=40)
    (actual time=38.412..402.118 rows=198734 loops=1)
  Recheck Cond: (merchant_id = 4242)
  Rows Removed by Index Recheck: 1210554
  Heap Blocks: exact=21004 lossy=61311
  Buffers: shared hit=1080 read=81785
  ->  Bitmap Index Scan on postings_merchant_idx
        (cost=0.00..4070.30 rows=201004 width=0)
        (actual time=31.552..31.552 rows=198734 loops=1)
        Index Cond: (merchant_id = 4242)
        Buffers: shared hit=550
```

(PostgreSQL prints each node's estimate and actuals on one long line; we wrap lines in this chapter so they fit the page.) Read the middle lines. Of the 82,315 heap pages visited, 61,311 were lossy, and rechecking every tuple on them discarded 1.2 million non-matching rows. Raising `work_mem` for this query (for example, `SET LOCAL work_mem = '64MB'`) keeps every page exact; the `Rows Removed by Index Recheck` line disappears and the heap scan does far less CPU work.

Combining indexes. Bitmaps can be combined before touching the heap. Consider `WHERE merchant_id = 4242 AND status = 'failed'` when each column has its own separate B-tree index. PostgreSQL can run two Bitmap Index Scans, intersect their page bitmaps with a `BitmapAnd` node (or union them with `BitmapOr` for `OR`), and read only the heap pages present in both. A plain Index Scan can use only one index per table.

7.4.4 Index-Only Scan

If every column the query needs is stored in the index, the heap fetch could be skipped entirely. The obstacle is visibility: index entries carry no `xmin`/`xmax`, so the index alone cannot tell whether an entry points to a row your snapshot should see.

PostgreSQL solves this with the *visibility map* (Chapters 3 and 5). The visibility map stores one bit per heap page meaning "every tuple on this page is visible to all current and future transactions." For each index entry, an Index-Only Scan checks the visibility-map bit for its heap page. If the bit is set, the index value is returned directly. If not, the scan falls back to fetching the heap tuple. `EXPLAIN ANALYZE` counts those fallbacks:

```
Index Only Scan using postings_acct_amt_idx on postings
    (actual time=0.031..9.880 rows=48200 loops=1)
  Index Cond: (account_id = 77)
  Heap Fetches: 48200
```

`Heap Fetches: 48200` out of `rows=48200` means the visibility map was useless and this "index-only" scan visited the heap for every row. The usual cause is a recently bulk-loaded or heavily updated table that `VACUUM` has not yet processed. After `VACUUM postings;` the same query reports `Heap Fetches: 0`. Covering indexes with `INCLUDE` columns, such as `CREATE INDEX ON postings (account_id) INCLUDE (amount_minor)`, exist to make Index-Only Scans possible; vacuum is what makes them actually index-only.

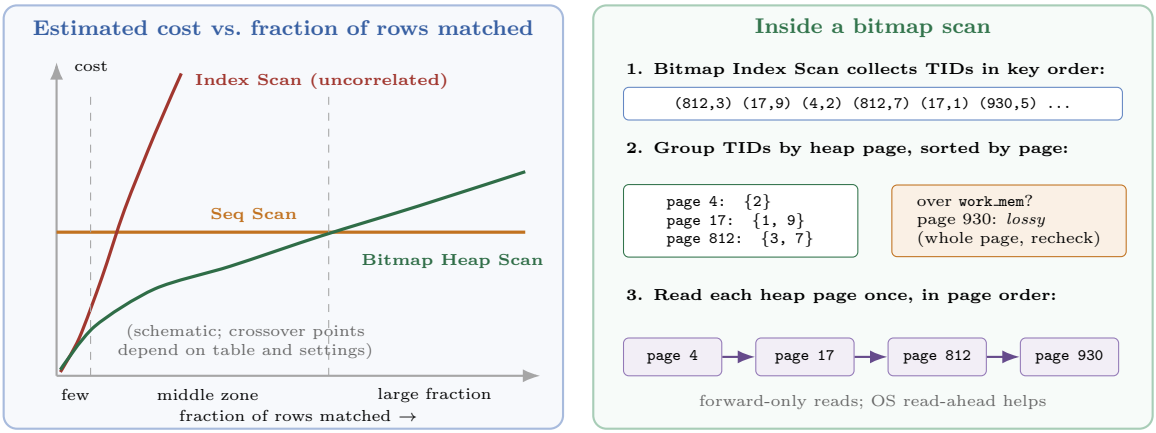


Figure 7.2: Left: as the fraction of matching rows grows, a plain Index Scan on an uncorrelated column becomes expensive quickly because each row may cost a random page read; a sequential scan costs the same regardless; a bitmap scan fills the middle zone. Right: a bitmap scan first collects all matching TIDs, groups them by heap page, and then visits each page once in physical order. If the bitmap outgrows `work_mem`, some pages become lossy and every tuple on them must be rechecked.

7.5 Three Ways to Join Two Tables

A join of two inputs, call them *outer* (left) and *inner* (right), can be computed by three physical algorithms. Each wins in a different situation, and the planner prices all three for every pair of inputs it considers.

You have written all three in Java. Given two `ArrayLists` to match up, you can write a double `for` loop (nested loop join), or replace the inner loop with a lookup in a `TreeMap` you already have (nested loop with an index probe). You can build a `HashMap` from the smaller list and look up each element of the larger one (hash join). Or, if both lists are sorted, you can walk them together with two indexes, as in the merge step of mergesort (merge join). The database versions are the same algorithms, with two extra concerns: the data may not fit in memory, and the planner must choose *before* it knows the real sizes.

7.5.1 Nested Loop

The simplest algorithm: for each outer row, scan the inner input for matches.

```
for each row o in outer:
    for each row i in inner where i.key = o.key:
        emit (o, i)
```

If the inner side is a full scan, the cost is proportional to $N_{\text{outer}} \times N_{\text{inner}}$, which is disastrous for large inputs: a million outer rows against a million inner rows is 10^{12} comparisons. But if the inner side is an *index lookup parameterized by the outer row* (an Index Scan with `Index Cond: (account_id = a.id)`), each inner probe costs one B-tree descent, about $\log M$ page touches, often all cache hits. Then a nested loop costs about $N_{\text{outer}} \times \log N_{\text{inner}}$, and it has a property neither of the other algorithms has: **zero startup cost**. It can return its first joined row after one outer row and one probe. Paired with `LIMIT`, that makes it unbeatable.

The rule of thumb: *a nested loop is the best join when the outer side is small*, and the worst when the outer side is large and the planner thought it was small. That second case is the most common cause of a sudden, catastrophic plan regression.

Since PostgreSQL 14, the planner can insert a `Memoize` node between a nested loop and its inner side. If the same outer key value repeats (many postings for the same account, for example), `Memoize` caches the inner results for that key, so a repeat costs a hash lookup instead of an index probe. `EXPLAIN ANALYZE` reports `Hits`, `Misses`, and `Evictions` for it.

7.5.2 Hash Join

A hash join works in two phases:

1. **Build.** Read the *inner* input (the planner chooses the smaller side) and insert every row into an in-memory hash table keyed on the join column.
2. **Probe.** Stream the *outer* input. For each row, hash its join key and look in one bucket for matches.

Each input is read once, so the cost is proportional to $N_{\text{outer}} + N_{\text{inner}}$. There is no need for any index or any sort order. The cost is memory and startup: no row can be emitted until the entire build side is hashed. The hash join only works for equality conditions (`=`), since hashing destroys order.

The hash table must fit in `work_mem` × `hash_mem_multiplier` (defaults `4MB` and `2.0`, so 8 MB). What if the build side is 400 MB?

Multi-batch (Grace) hash join. PostgreSQL splits the problem into *batches* using some bits of the hash value. Suppose it needs 64 batches. During the build phase, rows whose hash falls in batch 0 go into the in-memory table, and rows belonging to batches 1 through 63 are written to 63 temporary files. During the probe phase, outer rows for batch 0 are probed immediately; outer rows for other batches are written to 63 more temporary files. Then, for each batch $k = 1, \ldots, 63$, PostgreSQL loads inner batch k into memory, streams outer batch k past it, and moves on. Because matching rows have equal hashes, they always land in the same batch, so no match is missed.

The result is still linear in the input sizes, but almost all of both inputs gets written to disk and read back once. That roughly triples the I/O compared with an in-memory hash join, and you can see it in the plan:

```
Hash Join  (actual time=812.4..6411.9 rows=9840211 loops=1)
  Hash Cond: (p.account_id = a.id)
  Buffers: shared hit=2210 read=310118, temp read=96120 written=96120
  ->  Seq Scan on postings p  (actual rows=20000000 loops=1)
  ->  Hash  (actual time=811.9..811.9 rows=3000000 loops=1)
```

```
        Buckets: 131072  Batches: 64  Memory Usage: 7937kB
        ->  Seq Scan on accounts a  (actual rows=3000000 loops=1)
```

`Batches: 64` and the matching `temp read/written` counts tell you the hash spilled. If the planner *expected* one batch but the build side turned out much larger, you will also see `(originally 1)` next to the batch count, which is a sign of an underestimated row count. Figure 7.3 traces how one hash function routes Batch 0 straight through RAM while pairing spilled batches $1 \ldots 63$ on disk.

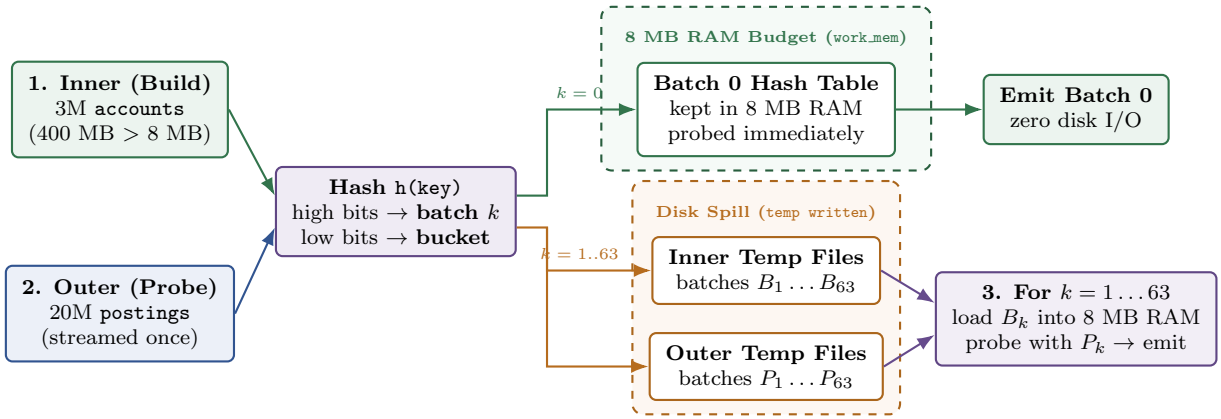


Figure 7.3: Multi-batch (Grace) hash join when the build input exceeds memory. High bits of `h(key)` split both inputs into 64 matching batches. Batch 0 joins in RAM immediately, while batches $1 \ldots 63$ spill to temporary files and replay one batch at a time.

7.5.3 Merge Join

If both inputs arrive sorted on the join key, they can be joined like a zipper. Keep a cursor on each input. Compare the two current keys; advance whichever is smaller; when the keys are equal, emit the matching rows (handling runs of duplicates on both sides) and advance. Each input is read once, so the cost is proportional to $N_{\text{outer}} + N_{\text{inner}}$, with very little memory.

Merge join is attractive when sorted input is free: both sides come from B-tree indexes on the join key, or the query needs the result ordered by that key anyway. Otherwise each side needs a `Sort` node. Sorts report how they ran:

- `Sort Method: quicksort Memory: 3120kB`: the input fit in `work_mem`.
- `Sort Method: top-N heapsort Memory: 26kB`: a `LIMIT` above the sort needs only the best N rows, so PostgreSQL keeps a heap of size N instead of sorting everything.
- `Sort Method: external merge Disk: 412000kB`: the input did not fit; PostgreSQL sorted chunks that fit in memory, wrote them to temporary files as sorted runs, and merged the runs.

Merge join also handles range and inequality join conditions that a hash join cannot, though in practice those are rare.

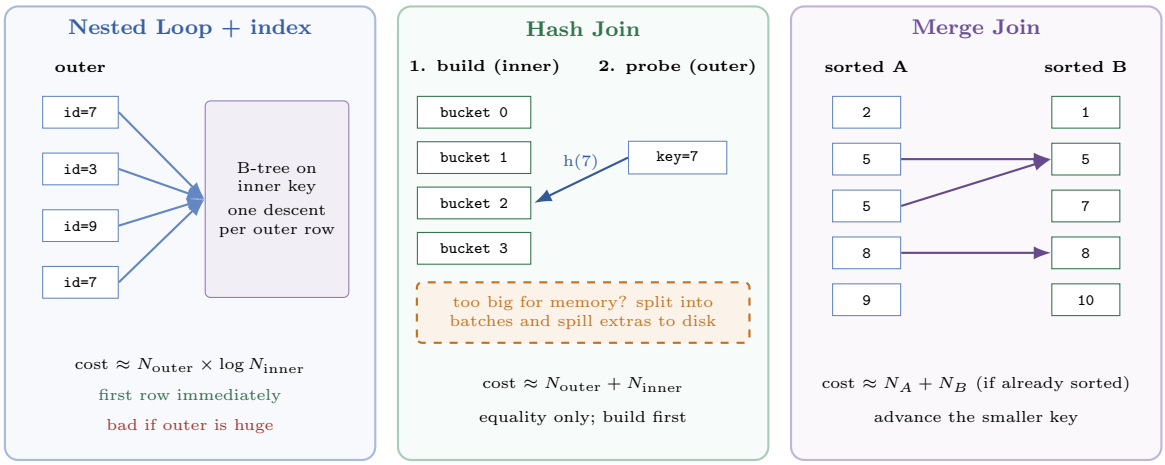


Figure 7.4: The three physical join algorithms. A nested loop with an index probe is ideal for a small outer input and returns rows immediately. A hash join reads each input once and needs memory for the build side, spilling to batches on disk when that memory runs out. A merge join walks two sorted inputs in step and is cheapest when an index or the query's `ORDER BY` already provides the sort.

Focus

Which join will the planner pick? A rough guide that matches most real plans:

- Small outer side (up to a few thousand rows) and an index on the inner join key: **Nested Loop**.
- Two large inputs, equality join, no useful order: **Hash Join**.
- Both inputs already ordered on the join key, or the output must be ordered by it: **Merge Join**.

Every one of these decisions depends on the estimated row counts. If the estimate for the outer side is 10 and the truth is 500,000, the planner will choose a nested loop that does 500,000 index probes. That is why reading a plan starts with comparing `rows` estimates against actual rows.

7.6 Searching Join Orders

A query that joins n tables can be evaluated in many orders. How many? Count the binary join trees whose leaves are the n tables. The shapes of binary trees with n leaves are counted by the Catalan number C_{n-1}, and the tables can be assigned to the leaves in $n!$ ways, so there are $n! \cdot C_{n-1}$ trees:

n (tables)	$n! \cdot C_{n-1}$ (join trees)	2^n (table subsets)
3	12	8
4	120	16
8	17,297,280	256
12	about 2.8×10^{13}	4,096

Each tree also needs a join algorithm at every node and an access path at every leaf. Enumerating trees one at a time is hopeless past a handful of tables. The third column shows the way out.

7.6.1 Dynamic Programming over Subsets

PostgreSQL's standard join search, inherited from IBM's System R optimizer (Selinger et al., 1979), builds plans bottom-up over *sets of tables*:

1. **Level 1:** for each single table, find the cheapest access path (Section 7.4).

2. **Level 2:** for each pair of tables $\{A, B\}$ that are connected by a join condition, consider joining the best plan for $\{A\}$ with the best plan for $\{B\}$, in both orders and with all three join algorithms. Keep the cheapest plan for the set $\{A, B\}$.

3. **Level k:** for each set S of k tables, consider every way to split S into two smaller sets $S_1 \cup S_2$ whose best plans are already known. Keep the cheapest plan for S.

The key insight is that the best plan for $\{A, B, C\}$ only needs the best plans for its subsets, not every tree that produces them. So the work grows with the number of subsets, 2^n, instead of the number of trees. (A refinement: the planner keeps more than one plan per set when a more expensive plan produces a useful sort order, because a sorted result might save a `Sort` or enable a merge join higher up. These are called *interesting orders*.)

7.6.2 Limits and the Genetic Optimizer

2^n is still exponential. Two settings keep planning time in check:

- `join_collapse_limit` and `from_collapse_limit` (default 8): the planner only reorders up to 8 items freely. Beyond that, explicit `JOIN` syntax order and subquery boundaries are partially respected as written.

- `geqo_threshold` (default 12): when a query has 12 or more items in `FROM`, PostgreSQL switches to the *Genetic Query Optimizer* (GEQO). GEQO treats a join order as a chromosome, generates a population of random orders, and repeatedly combines the cheaper ones. It finds a good plan quickly but not necessarily the best one, and because it is randomized, plans for very large joins can vary between runs (set `geqo_seed` to make them reproducible).

In production

ORMs and reporting tools sometimes generate queries joining 15 or 20 tables, many of them small lookup tables. If such a query plans slowly or chooses erratic plans, check the `Planning Time` line in `EXPLAIN ANALYZE`. Splitting the query, or writing explicit `JOIN`s in a sensible order and lowering `join_collapse_limit` for that session, gives you a stable plan and a planning time measured in milliseconds rather than seconds.

7.7 Reading `EXPLAIN (ANALYZE, BUFFERS)` Line by Line

Plain `EXPLAIN` shows the plan the planner chose and its predictions. `EXPLAIN ANALYZE` *runs* the query and adds what actually happened at every node. `BUFFERS` adds how many pages each node touched and where they came from. Use all three together.

Common trap

`EXPLAIN ANALYZE` really executes the statement. `EXPLAIN ANALYZE DELETE FROM postings WHERE ...` deletes the rows. Wrap data-modifying statements in a transaction you roll back:

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS) DELETE FROM postings WHERE ...;
ROLLBACK;
```

7.7.1 A Complete Plan

Here is the plan for the dashboard query from the start of the chapter:

```
Limit  (cost=48211.20..48211.23 rows=10 width=48)
       (actual time=212.441..212.446 rows=10 loops=1)
  Buffers: shared hit=40213 read=6120
  ->  Sort  (cost=48211.20..48236.20 rows=10000 width=48)
            (actual time=212.439..212.442 rows=10 loops=1)
        Sort Key: (sum(p.amount_minor)) DESC
        Sort Method: top-N heapsort  Memory: 26kB
        Buffers: shared hit=40213 read=6120
        ->  HashAggregate
              (cost=47745.10..47995.10 rows=10000 width=48)
              (actual time=205.118..209.870 rows=11842 loops=1)
              Group Key: a.id
              Batches: 1  Memory Usage: 2577kB
              Buffers: shared hit=40213 read=6120
              ->  Nested Loop
                    (cost=0.86..46120.40 rows=320000 width=24)
                    (actual time=0.071..151.904 rows=296050 loops=1)
                    Buffers: shared hit=40213 read=6120
                    ->  Index Scan using accounts_region_idx on accounts a
                          (cost=0.42..1210.40 rows=10000 width=24)
                          (actual time=0.031..6.211 rows=11842 loops=1)
                          Index Cond: (region = 'EU'::text)
                          Buffers: shared hit=310 read=95
                    ->  Index Scan using postings_acct_created_idx
                          on postings p
                          (cost=0.44..4.17 rows=32 width=16)
                          (actual time=0.004..0.011 rows=25 loops=11842)
                          Index Cond: ((account_id = a.id) AND
                                       (created_at >= '2026-10-01'))
                          Buffers: shared hit=39903 read=6025
Planning Time: 0.912 ms
Execution Time: 212.533 ms
```

Read from the inside out. Execution starts at the most indented nodes and flows upward. The two `Index Scan` nodes are the leaves; the `Nested Loop` joins them; `HashAggregate` groups; `Sort` orders; `Limit` stops after 10 rows. Figure 7.5 draws the same tree.

Estimated versus actual. Every node prints two parentheses. The first, `cost=... rows=... width=...`, is the planner's prediction. The second, `actual time=... rows=... loops=...`, is the measurement. Compare `rows` in each pair. For the EU accounts the planner predicted 10,000 and got 11,842: an error of 18%, which is excellent. For the postings per account it predicted 32 and got 25. These estimates are good, so the plan is probably good.

`actual time` is in milliseconds, per loop. `actual time=0.031..6.211` means the node returned its first row after 0.031 ms and its last row after 6.211 ms. Unlike `cost`, these are real milliseconds.

The `loops` multiplier. Look at the inner index scan on `postings`: `actual time=0.004..0.011 rows=25 loops=11842`. This node ran 11,842 times, once for each EU account from the outer side. **Both `actual time` and `rows` are averages per loop**, so the totals are

$$\text{rows} = 25 \times 11{,}842 = 296{,}050, \qquad \text{time} \approx 0.011 \text{ ms} \times 11{,}842 \approx 130 \text{ ms}.$$

That 130 ms is most of the query's 212 ms. If you read only the per-loop number, you would conclude this node costs 0.011 ms and is irrelevant. It is actually the most expensive node in the plan. Whenever you see `loops` greater than 1, multiply.

Buffers. `Buffers: shared hit=39903 read=6025` on the inner scan means it touched 45,928 pages in total: 39,903 were already in `shared_buffers` (`hit`), and 6,025 had to be requested from the operating system (`read`). A `read` may still be served from the OS page cache rather than the disk; `BUFFERS` cannot tell the difference, but `track_io_timing = on` adds an `I/O Timings` line that can. Other counters you will meet:

- `dirtied`: pages this node modified in shared buffers. On a `SELECT`, this usually means hint bits were set (Chapter 5).
- `written`: dirty pages this backend had to write out itself to make room, a sign that `shared_buffers` or the background writer is struggling.
- `temp read` / `temp written`: pages spilled to temporary files by sorts, hashes, and materialization. Any non-zero value here is worth a look at `work_mem`.

Buffer counts are cumulative: a parent's count includes its children's. Here every node above the nested loop shows the same totals because none of them reads any pages itself.

Planning and execution time. `Execution Time` covers running the plan, but not sending rows to the client. `Planning Time` covers the planner's search. For most OLTP queries, planning should be well under a millisecond; if it rivals execution time, prepared statements (Chapter 8) or simpler queries will help.

Pause

Pause and check: the 30-second `EXPLAIN (ANALYZE, BUFFERS)` checklist. Whenever you stare at an unfamiliar plan, check four things in order:

1. **Multiply by `loops`.** Any node with `loops > 1` reports per-iteration averages; multiply both `actual time` and `rows` by `loops` before deciding whether a node is cheap or expensive.
2. **Spot row-estimate gaps.** Find the deepest node where estimated `rows` and actual `rows` disagree by 10× or more. That node is where the planner's mental map diverged from reality.
3. **Check for memory spills.** Look for `Batches: > 1`, `Sort Method: external merge`, `lossy=` blocks, or `temp read/written`.
4. **Separate cache hits from reads.** High `shared read` with `track_io_timing = on` tells you the query waited on storage rather than CPU.

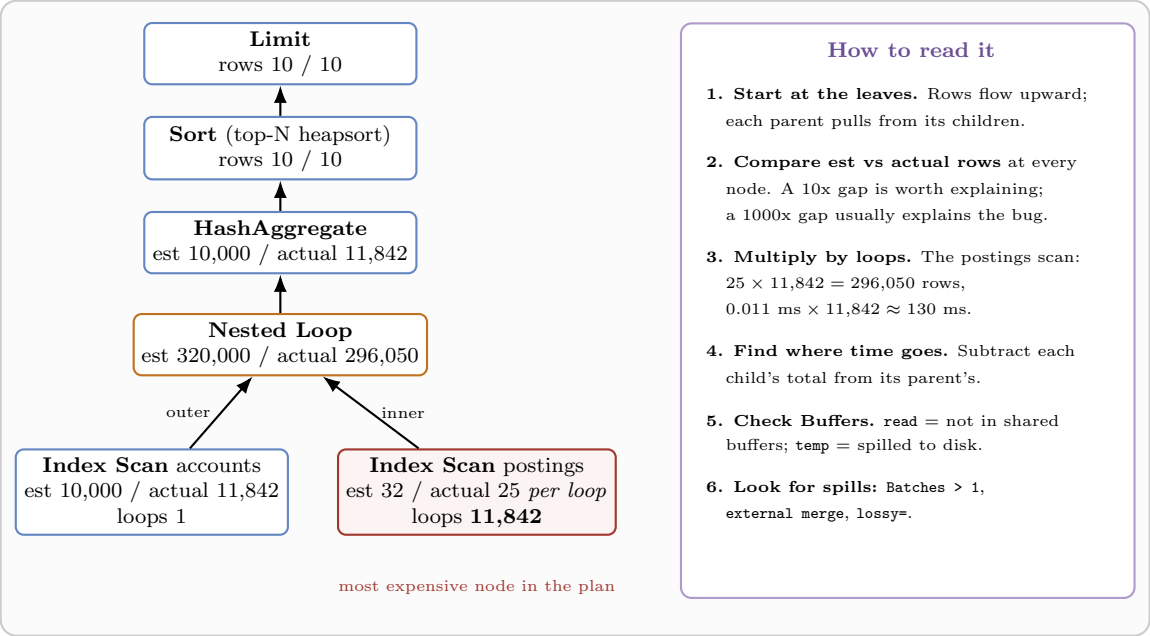


Figure 7.5: The dashboard plan as a tree. Each box shows estimated and actual rows. The inner index scan on `postings` looks cheap per loop but runs 11,842 times and accounts for most of the execution time.

7.7.2 Diagnosing a Bad Plan

Almost every slow plan comes down to one of a few causes. Here is how each shows up and how to fix it.

1. Stale statistics. A table received a large batch of rows, but autovacuum has not run `ANALYZE` yet. Typical symptom: a node estimates `rows=1` or a few hundred and actually produces hundreds

of thousands, often on a predicate over recent dates that lie beyond the last histogram bound. Fix: run `ANALYZE table;` immediately after bulk loads, and lower `autovacuum_analyze_scale_factor` for large, fast-growing tables.

2. Correlated predicates. Two predicates on related columns are multiplied as if independent, and the estimate comes out orders of magnitude too low (Section 7.2). Symptom: the estimate is far too small exactly at the node that applies both filters. Fix: `CREATE STATISTICS ... (dependencies, mcv)` on those columns, then `ANALYZE`.

3. Opaque expressions. A function wraps an indexed column (`date_trunc`, `lower`, a cast), so the planner falls back to default selectivities and cannot use a plain index. Symptom: `Seq Scan` with a `Filter:` line containing the function, and a round-number estimate such as 0.5% of the table. Fix: rewrite as a range on the raw column, or add an expression index.

4. Too little memory. Sorts, hashes, and bitmaps spill. Symptom: `Sort Method: external merge`, `Batches:` above 1, `lossy=` in `Heap Blocks`, and non-zero `temp read/written`. Fix: raise `work_mem` for that query or role with `SET LOCAL`. Be careful raising it globally: each sort or hash node in each concurrent query may use that much, so 200 connections times 3 nodes times 256 MB is 150 GB.

5. Wrong cost constants for the hardware. The planner avoids index scans on NVMe storage that would serve random reads nearly as fast as sequential ones. Symptom: a sequential scan is chosen, but `SET enable_seqscan = off` in a test session produces a much faster index plan. Fix: set `random_page_cost = 1.1` and `effective_cache_size` to roughly the memory available for caching (often 50–75% of RAM). Use `enable_*` settings only for diagnosis, never as a permanent fix.

Example 7.5 (A thousand-fold misestimate)**.** A fraud report filters `transfers` by `city = 'Lyon' AND postal_code = '69001'` and joins to `accounts`:

```
Nested Loop  (cost=0.85..412.66 rows=20 width=64)
             (actual time=0.090..48711.302 rows=19874 loops=1)
  ->  Seq Scan on transfers t  (cost=0.00..250.10 rows=20 width=40)
        (actual time=0.041..911.402 rows=19874 loops=1)
        Filter: ((city = 'Lyon') AND (postal_code = '69001'))
        Rows Removed by Filter: 19980126
  ->  Index Scan using accounts_pkey on accounts a
        (cost=0.43..8.12 rows=1 width=24)
        (actual time=2.401..2.405 rows=1 loops=19874)
        Index Cond: (id = t.account_id)
```

The scan on `transfers` estimated 20 rows and produced 19,874: almost exactly the factor of 1,000 that independence predicts when one predicate implies the other. On a 20-row estimate, a nested loop with 20 index probes is a sensible choice. With 19,874 probes, each a cold random read of about 2.4 ms (multiply by `loops`: $2.4 \times 19{,}874 \approx 47.7$ seconds), it is not. After `CREATE STATISTICS addr_stats (dependencies) ON city, postal_code FROM transfers; ANALYZE transfers;` the estimate becomes about 20,000, the planner switches to a hash join over `accounts`, and the query finishes in under two seconds.

Try it in PostgreSQL

Build intuition with a table you control:

```
CREATE TABLE t AS
SELECT g AS id,
       (random() * 1000)::int AS rnd,
       g / 1000 AS seq
  FROM generate_series(1, 2000000) g;
CREATE INDEX ON t (rnd);
CREATE INDEX ON t (seq);
ANALYZE t;

SELECT attname, n_distinct, correlation
  FROM pg_stats WHERE tablename = 't';

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE rnd < 5;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE seq < 10;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE rnd < 300;
```

Both of the first two predicates match about 10,000 rows. Notice how differently they are planned and how many buffers each touches, and connect what you see to the `correlation` values. Then set `work_mem = '64kB'` and rerun the `rnd < 300` query to make the bitmap go lossy.

The planner turns SQL into a plan; MVCC and locking keep concurrent plans correct; WAL keeps their results durable. In Chapter 8 we put all of it to work: designing financial schemas whose constraints make invalid states unrepresentable, replicating them across machines, and serving them to thousands of application tasks through a connection pooler.

7.8 Exercises

Exercise 7.1 (Selectivity from `pg_stats`)**.** A column `country` on a 4,000,000-row table has `null_frac = 0.01`, `n_distinct = 60`, `most_common_vals = {US,GB,DE,FR}`, and `most_common_freqs = {0.40,0.15,0.10,0.08}`. Estimate the rows returned by (a) `country = 'DE'`, (b) `country = 'NZ'`, and (c) `country IN ('GB','NZ')`. Show each step.

Exercise 7.2 (Histogram interpolation)**.** A column has `null_frac = 0`, no MCV list, and `histogram_bounds = {10,20,40,80,160,320}` on a 1,000,000-row table. Estimate the rows for (a) `x < 60`, (b) `x > 200`, and (c) `x BETWEEN 15 AND 100`. Why does an equi-depth histogram give better estimates than an equi-width one when the data is skewed?

Exercise 7.3 (Pricing scans by hand)**.** A table occupies 25,000 pages and holds 2,000,000 rows. Using the default cost constants: (a) Compute the total cost of a sequential scan with a single `WHERE` predicate. (b) A predicate keeps 0.5% of rows on a column with correlation 0. Use the Mackert–Lohman formula to estimate the heap I/O cost of an Index Scan, and compare it with (a). (c) Repeat (b) for correlation 0.95. (d) Repeat (b) and (c) with `random_page_cost = 1.1`. Which decisions change?

Exercise 7.4 (Startup cost and `LIMIT`)**.** Plan A for a query has `cost=0.43..90000.00 rows=1000000`. Plan B has `cost=21000.00..23500.00 rows=1000000`. Which plan does the

planner prefer with no `LIMIT`? With `LIMIT 50`? Find the `LIMIT` value at which the two plans cost the same, and describe what kind of physical operators would produce each cost profile.

Exercise 7.5 (Reading a lossy bitmap)**.** A plan reports `Heap Blocks: exact=4000 lossy=36000` and `Rows Removed by Index Recheck: 2100000` for a query that returns 90,000 rows. (a) Explain what happened in memory. (b) Roughly how many tuples per page does this table hold? (c) Predict how the `Heap Blocks` line and the `Rows Removed` line change after `SET LOCAL work_mem = '256MB'`, and explain why the number of heap pages read does not change.

Exercise 7.6 (Choosing a join algorithm)**.** For each case below, state which join algorithm you expect the planner to pick and explain why. (a) Joining 50 rows from `orders` to `customers` on its primary key. (b) Joining 30 million `postings` to 3 million `accounts` for a monthly report with `work_mem = 64MB`. (c) The same query as (b), now with `ORDER BY account_id` and B-tree indexes on both join columns. (d) The same query as (a), except the planner estimated 50 outer rows when the true count is 2 million.

Exercise 7.7 (Multi-batch hash join arithmetic)**.** The build side of a hash join has 6,000,000 rows of 56 bytes each, and the hash memory limit is `work_mem` $\times$ `hash_mem_multiplier` $=$ 16 MB. (a) Estimate the number of batches (a power of two). (b) What fraction of the build and probe inputs is written to temporary files? (c) If the probe side is 40,000,000 rows of 48 bytes, estimate the total temporary-file bytes written. (d) How large must `work_mem` be for a single batch?

Exercise 7.8 (Counting join orders)**.** (a) Verify the table in Section 7.6 for $n = 3$ by listing all 12 join trees of tables A, B, C. (b) If only *left-deep* trees are allowed (every right child is a single table), how many trees are there for n tables? (c) Explain why dynamic programming over subsets needs to examine only 2^n sets, and estimate the number of (set, split) pairs it considers for $n = 10$.

Exercise 7.9 (Find the slow node)**.** In the plan below, compute the total rows and total time contributed by the inner node, identify which node is responsible for most of the runtime, and propose two different fixes.

```
Nested Loop  (actual time=0.05..8120.4 rows=420000 loops=1)
  ->  Index Scan using invoices_due_idx on invoices i
        (cost=0.43..95.12 rows=40 width=16)
        (actual time=0.02..58.1 rows=60000 loops=1)
        Index Cond: (due_date < '2026-11-01')
  ->  Index Scan using lines_invoice_idx on invoice_lines l
        (cost=0.43..12.30 rows=7 width=24)
        (actual time=0.010..0.133 rows=7 loops=60000)
        Index Cond: (invoice_id = i.id)
```

Exercise 7.10 (Index-only scans that are not)**.** A query on `(account_id) INCLUDE (amount_minor)` reports `Index Only Scan ... rows=12000` with `Heap Fetches: 11800`. (a) Explain what the visibility map has to do with this. (b) Give two operational changes that would bring `Heap Fetches` near zero. (c) Why can an Index-Only Scan never skip the visibility check entirely, even when the index contains every column the query needs?

Chapter 8

Financial Schemas, Replication, and Connection Pooling

For seven chapters we have worked inside the PostgreSQL engine room. Chapter 2 walked 8 KB slotted heap pages and shared buffer descriptors. Chapter 3 split B-trees and generalized search trees. Chapter 4 flushed write-ahead log records and full-page images to survive power loss. Chapter 5 traced multiversion snapshots and `VACUUM`. Chapter 6 separated concurrent transactions with row-level locks and serializable conflict tracking, and Chapter 7 priced execution plans from column statistics. Now we sit at the architect's desk and put all seven layers to work on a system where every bug has a price tag: a production double-entry financial ledger served by an asynchronous Rust (`Axum` and `SQLx`) API over PostgreSQL.

Financial systems punish vague thinking because every shortcut fails in a predictable, expensive way. Store balances in binary floating point and sums of fee postings stop matching the books to the cent. Store wall-clock timestamps without a UTC reference and the autumn daylight-saving fallback folds two distinct hours onto the same nominal minute. Choose random UUID primary keys and a sixty-gigabyte B-tree turns tiny inserts into a storm of 8 KB full-page writes across cold NVMe blocks. Check balances only in Rust handlers and a single concurrent retry creates money out of thin air. Leave replication asynchronous and a primary disk failure erases transfers you already confirmed to customers. Point five thousand Tokio tasks straight at PostgreSQL and shared-memory snapshot scans bring a thirty-two-core server to its knees.

In this final chapter, we motivate every design rule with the exact production failure it prevents. Then we trace how PostgreSQL's storage, indexing, WAL, locking, and wire-protocol internals enforce the guarantee.

Focus

By the end of this chapter you will be able to answer five practical engineering questions:

- Why does `0.1 + 0.2 = 0.3` evaluate to `false` in floating-point columns, and how do banks store money and timestamps without ever losing a cent or repeating an hour?
- Why do random UUID primary keys run fast at five million rows and then choke disk bandwidth at two billion rows?
- How can the database itself refuse any payment transfer whose debits and credits fail to

sum to zero, even if a bug slips into application code?

- How do you store ten billion historical ledger rows while keeping queries fast and archiving old months in milliseconds?
- How can a PostgreSQL cluster survive a data-center outage without losing confirmed transfers, and serve five thousand concurrent Rust tasks using only forty-eight database processes?

8.1 Data Types That Never Lie About Money, Time, or Identity

A relational schema is a physical contract about the bit patterns allowed onto 8 KB heap pages. If a column's type rounds fractions, discards UTC context, or scatters keys uniformly across disk, no amount of application-layer discipline can undo the damage downstream.

8.1.1 Money: Why Binary Floats Fail, and `BIGINT` vs. `NUMERIC`

Suppose a payment platform stores account adjustments in PostgreSQL's 8-byte `DOUBLE PRECISION` type (`float8`, matching `f64` in Rust and `double` in Java or C++). A merchant collects a 0.10 USD platform fee and a 0.20 USD network fee, expecting a 0.30 USD total. Ask PostgreSQL whether the two sides match:

```
SELECT (0.1::float8 + 0.2::float8) = 0.3::float8 AS exact_match,
       (0.1::float8 + 0.2::float8) - 0.3::float8 AS drift;
-- exact_match: false | drift: 5.551115123125783e-17
```

The comparison returns `false`. Why? IEEE 754 binary floating point writes every number in base 2 as $(-1)^s \times m \times 2^e$ using a 53-bit binary significand m. Just as $1/3 = 0.333\ldots_{10}$ never terminates in base 10, one tenth never terminates in base 2: $0.1_{10} = 0.0001100110011\overline{0011}_2$. Chopping that infinite bit pattern at bit 53 introduces a tiny rounding error of roughly 10^{-17} on a single addition. Worse, parallel `HashAggregate` workers (Chapter 7) sum rows in whatever order chunks arrive. Across ten million fee postings, two runs of the exact same reconciliation query can disagree by whole cents. Binary floats (`REAL` and `DOUBLE PRECISION`) never belong in monetary columns.

PostgreSQL gives you two exact alternatives, each suited to a clear job:

1. **`BIGINT` (`int8`) minor units.** Instead of storing currency as decimal fractions, store integer counts of the asset's smallest indivisible unit. Use cents (10^{-2} USD) for retail payments, micro-dollars (10^{-6} USD) for sub-cent API billing, or satoshis (10^{-8} BTC). On disk, `BIGINT` takes a fixed 8 bytes on the heap page, maps directly to `i64` in Rust or `long` in Java, and adds in a single CPU integer instruction. Its signed range reaches $\pm(2^{63} - 1) \approx \pm 9.22 \times 10^{18}$. In USD cents that exceeds 92 quadrillion USD; in micro-dollars it exceeds 9.22 trillion USD. Always store the asset's ISO currency code or scale alongside the integer so application code never adds Japanese yen (10^0) to US cents (10^{-2}) or Kuwaiti dinar fils (10^{-3}).

2. **`NUMERIC(p, s)` (equivalently `DECIMAL(p, s)`).** Foreign-exchange (FX) rates (`1.08437525`), fractional equity shares (`0.00418275` shares), and bond accruals mix many decimal scales

s with total precision p past twenty digits. Multiplying two such figures can also overflow a 64-bit integer intermediate. To store decimal fractions without binary rounding, PostgreSQL packs *four base-10 digits into each 16-bit integer word* (working in base 10,000), preceded by a short variable-length header (`NumericVar` in `numeric.c`) recording the weight, sign, and display scale. Because every decimal fraction terminates cleanly in base 10,000, `0.1::numeric + 0.2::numeric = 0.3::numeric` is exact to every declared digit. The tradeoff is CPU and space. `NUMERIC` uses a variable-length (`varlena`) header—typically 8–16 bytes on disk—and loops over digit words in software rather than running one hardware instruction. Heavy `SUM(amount)` scans therefore spend roughly 5× to 15× more CPU than `SUM(amount_minor)` on `BIGINT`. In Rust `SQLx`, `NUMERIC` maps cleanly to `rust_decimal::Decimal` or `bigdecimal::BigDecimal`.

The rule of thumb is straightforward. Use `BIGINT` minor units for single-scale ledger postings and account balances. Use `NUMERIC(p, s)` for FX rates, unit prices, and interest multipliers where mixed scales or wide intermediates arise.

8.1.2 Time: `TIMESTAMPTZ` and the Daylight-Saving Trap

At 01:59:59 Eastern Daylight Time on the first Sunday of November, clocks in New York jump backward to 01:00:00 Eastern Standard Time. Suppose a ledger table uses `TIMESTAMP WITHOUT TIME ZONE` (which SQL abbreviates as plain `TIMESTAMP`). A customer wires funds at 01:15 EDT (`05:15 UTC`). Fifty minutes later, she wires a second payment at 01:05 EST (`06:05 UTC`). Stored as wall-clock New York `TIMESTAMP` values, the second transfer appears to happen *ten minutes before* the first: `'2026-11-01 01:05:00'` sorts ahead of `'2026-11-01 01:15:00'`. Worse still, `TIMESTAMP WITHOUT TIME ZONE` silently discards timezone offsets on input. If one API pod connects with `TimeZone = 'UTC'` and an admin script connects with `TimeZone = 'America/Chicago'`, both write incompatible local clocks into the same column with zero warning.

Intuition

Despite its verbose SQL name, `TIMESTAMP WITH TIME ZONE` (`TIMESTAMPTZ`) stores **zero time-zone string or UTC offset on disk**. Both `TIMESTAMP` and `TIMESTAMPTZ` occupy the exact same 8 bytes in a heap tuple: a signed 64-bit integer counting microseconds since PostgreSQL's epoch, `2000-01-01 00:00:00 UTC`. The sole difference happens at the input and output boundary. When you insert `'2026-11-01 01:15:00-04'` into a `TIMESTAMPTZ` column, PostgreSQL immediately converts that instant into UTC microseconds and stores that single universal point on the physical timeline. On `SELECT`, it formats those UTC microseconds into whatever session `TimeZone` the reader requested.

The rule for financial schemas has only two cases:

- Use `TIMESTAMPTZ` for every physical instant when an event occurred (`created_at`, `posted_at`, `expires_at`). In Rust `SQLx`, bind it to `chrono::DateTime<Utc>` or `time::OffsetDateTime` with the session `TimeZone` pinned to `'UTC'`.
- Use 4-byte `DATE` for calendar dates defined by legal contract, such as a wire's settlement `value_date = '2026-11-02'` or a monthly statement date. Never store a contract date as midnight timestamp, which shifts to the previous day whenever a reader views it from a western timezone.

8.1.3 Primary Keys at Scale: Why `UUIDv4` Keys Thrash B-Trees and `UUIDv7` Flies

An 8-byte `BIGINT GENERATED ALWAYS AS IDENTITY` sequence appends strictly to the right edge of its primary-key B-tree, keeping inserts fast and compact. Yet distributed financial systems rarely expose sequential integers as public IDs. Sequential IDs leak transfer volume to competitors (`entry_id = 1042` versus `1089` an hour later). They also force Rust services or mobile apps to make a database round-trip before knowing a new resource's ID. That pushes teams toward 128-bit `UUID` keys, stored natively in PostgreSQL's 16-byte `uuid` type rather than 36-byte ASCII `TEXT`.

Here is the trap that catches growing ledgers six months after launch. You pick random `UUIDv4`, which fills 122 of its 128 bits from a cryptographic random generator. At five million rows, the primary-key B-tree is roughly 150 MB. It fits easily inside an 8 GB `shared_buffers` pool (Chapter 2) and sustains 35,000 inserts per second. A year later, `postings` reaches two billion rows and its primary B-tree spans 65 GB on a server with 32 GB of RAM. Insert throughput drops tenfold while NVMe write bandwidth pins at 100%, even though each new row is barely 100 bytes. Three mechanisms from Chapters 2–4 combine to cause this collapse:

1. **Buffer-pool cache misses.** Every `UUIDv4` lands at a uniform random spot across $[0, 2^{128})$. Once the 65 GB leaf layer outgrows `shared_buffers`, almost every `INSERT` targets a cold leaf page on disk, stalling on a random 8 KB read and evicting another cached page.

2. **Midpoint page-split bloat.** As Chapter 3 showed, random B-tree inserts split full leaf pages 50/50, leaving average occupancy near 70%. Monotonically increasing right-edge inserts split at the insertion point and keep leaves 90% full.

3. **WAL `full_page_writes` amplification.** Chapter 4 showed that the first change to *any* 8 KB page after a `CHECKPOINT` must log the entire 8 KB Full Page Image (FPI) into WAL to survive torn writes. Right after a checkpoint, ten thousand random `UUIDv4` inserts dirty roughly ten thousand distinct leaf pages. A few hundred kilobytes of new keys explode into $10{,}000 \times 8$ KB ≈ 80 MB of WAL.

Time-ordered `UUIDv7` (standardized in RFC 9562, built into PostgreSQL 18 as `uuidv7()`, and available in Rust via `uuid::Uuid::now_v7()`) eliminates all three penalties without central sequence locks. How? It places a 48-bit big-endian Unix millisecond timestamp at the very front of the 128-bit value. Next come a 4-bit version tag (`0111`), 12 bits of sub-millisecond counter or random data (`rand_a`), a 2-bit variant (`10`), and 62 bits of cryptographic randomness (`rand_b`). PostgreSQL compares 16-byte `uuid` values from most-significant byte to least. Every `UUIDv7` minted during the current millisecond therefore sorts to the *far right* of the entire historical B-tree (Figure 8.1). Concurrent Rust pods still generate collision-free IDs offline, yet all inserts land on a few hot rightmost leaf pages pinned in `shared_buffers`.

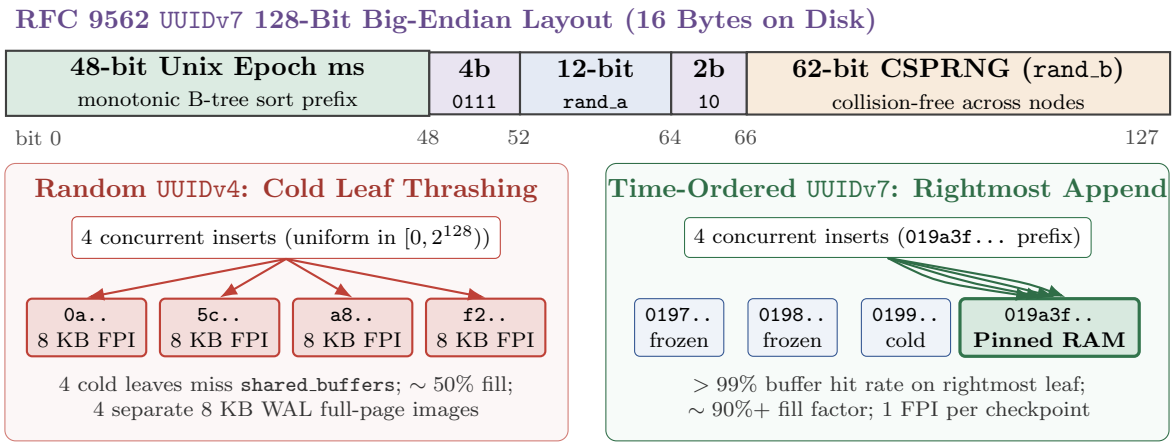


Figure 8.1: Bit layout of RFC 9562 `UUIDv7` (top) and its effect on a multi-gigabyte B-tree (bottom). Random `UUIDv4` keys scatter writes across cold disk pages and amplify post-checkpoint WAL full-page images, whereas `UUIDv7` funnels concurrent inserts onto the hot rightmost leaf in `shared_buffers`.

8.2 Encoding Domain Invariants Directly in the Schema

"Our Rust service validates every request struct before calling SQL, so the database only needs plain columns." That sentence precedes nearly every silent data-corruption postmortem. Application code is only one doorway into a database. Settlement batch jobs, emergency `psql` backfills, schema migrations, and concurrent network retries all write to the same tables. Worse, under `READ COMMITTED` isolation (Chapter 6), two Rust handlers can each read a valid snapshot and still commit a combined state that breaks business rules. Pushing invariants into PostgreSQL turns application bugs from silent ledger corruption into loud, rolled-back `ERROR` responses.

8.2.1 `NOT NULL`, `CHECK`, Domains, and Atomic Idempotency Keys

Recall from Chapter 1 how three-valued logic treats `NULL` inside a `CHECK` constraint. A row passes `CHECK (expr)` whenever `expr` evaluates to `TRUE` *or* `UNKNOWN`. If you declare `amount_minor BIGINT CHECK (amount_minor > 0)` without `NOT NULL`, inserting `NULL` evaluates `NULL > 0` to `UNKNOWN` and lands cleanly on the heap page. Always pair `NOT NULL` with `CHECK`.

For constrained text categories, choose deliberately among three tools:

- `CREATE TYPE ... AS ENUM` stores each label as a compact 4-byte object identifier (OID) on disk. It fits closed state machines whose values match fixed code branches (`'pending'`, `'posted'`, `'voided'`). Keep in mind that PostgreSQL enums cannot drop old labels without recreating the type.
- **`DOMAIN`s or `CHECK`-constrained `TEXT`** validate text format across dozens of tables while letting you add new values without DDL locks. For example, `CHAR(3)` checked against a three-letter uppercase pattern enforces ISO currency codes cleanly. When each currency needs metadata such as its minor-unit scale (2 for `USD`, 0 for `JPY`), point a foreign key at a tiny `assets` table; its single 8 KB page stays permanently cached in `shared_buffers`.
- **Storage-enforced idempotency keys.** Suppose a mobile app sends `POST /v1/transfers`

and its network socket times out before the HTTP reply arrives. The client cannot tell whether the commit succeeded, so it must retry with the same client-generated `idempotency_key`. Running a `SELECT` check in Rust before `INSERT` creates a classic time-of-check to time-of-use race if two retries arrive together. Declaring `UNIQUE (client_id, idempotency_key)` closes that race inside the storage engine. PostgreSQL's B-tree uniqueness check (`_bt_check_unique`) inspects even uncommitted in-flight index entries, blocks the second retry on the first transaction's lock, and resolves cleanly via `ON CONFLICT DO NOTHING`.

8.2.2 Temporal Non-Overlapping Invariants with `EXCLUDE USING gist`

Suppose each savings account has a tiered interest-rate schedule valid over a time window. Two admin jobs concurrently insert schedules for account A: one sets 4.50% over $[\text{Jun } 1, \text{Dec } 1)$ and the other sets 5.00% over $[\text{Sep } 1, \text{Mar } 1)$. A nightly accrual query joins `accounts` to `account_rate_schedules` `WHERE now() <@ effective_during`. Throughout autumn it matches *both* rows and pays 9.50% interest. No `UNIQUE` constraint on `(account_id, effective_from)` can catch partial overlap because the two start timestamps differ.

Chapter 3 introduced Generalized Search Trees (GiST) as balanced trees over bounding predicates such as box or range overlap (`&&`). Pair GiST with the bundled `btree_gist` extension, which adds plain scalar equality (`=`) to GiST keys. PostgreSQL's **exclusion constraint** then rejects any two rows where *all* listed operator comparisons return true at once:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE account_rate_schedules (
    schedule_id      UUID PRIMARY KEY DEFAULT uuidv7(),
    account_id       UUID NOT NULL,
    annual_bps       INTEGER NOT NULL CHECK (annual_bps BETWEEN 0 AND 10000),
    effective_during TSTZRANGE NOT NULL,
    CONSTRAINT non_empty_window CHECK (NOT isempty(effective_during)),
    CONSTRAINT no_overlapping_rates EXCLUDE USING gist (
        account_id       WITH =,
        effective_during WITH &&
    )
);
```

Notice the range type `TSTZRANGE` and PostgreSQL's half-open convention `'[)'` (closed lower bound, open upper bound). Suppose an old rate schedule `[2026-01-01Z, 2026-06-01Z)` ends at midnight June 1 and its replacement `[2026-06-01Z, infinity)` starts at that exact microsecond. The two windows touch with zero gap yet share zero instants, so `&&` evaluates to `false`. Even under `READ COMMITTED`, two concurrent transactions inserting overlapping windows for the same `account_id` collide inside the GiST index just like a duplicate primary key. One commits; the other aborts with SQLSTATE `23P01` (`exclusion_violation`).

8.3 A Double-Entry Financial Ledger in PostgreSQL and Rust

Beginners often model money with a single mutable column: subtract 5,000 from Alice's `balance` row with one `UPDATE`, then add 5,000 to Bob's row with a second `UPDATE`. Three production hazards make that design unacceptable for a real ledger:

1. **Destroyed history.** Overwriting `balance` in place erases *why* the balance changed. Once `VACUUM` reclaims old tuple versions (Chapter 5), regulatory audits, dispute reversals, and historical reconstructions become impossible.
2. **Unchecked money creation.** Nothing in the schema ties the two `UPDATE` statements together. A bug in a new promo-credit handler that runs one `UPDATE` without its counterparty silently alters total customer funds.
3. **No separation between immutable facts and fast balance reads.** Recording append-only ledger legs separately from cached account balances lets the database enforce conservation of money while keeping balance lookups $O(1)$.

8.3.1 Conservation of Value and `DEFERRABLE INITIALLY DEFERRED` Triggers

Luca Pacioli codified the cure in Venice in 1494: **double-entry bookkeeping**. Every financial event is recorded as an immutable `journal_entry` header paired with two or more immutable `postings` (legs) in the same currency whose signed minor-unit amounts sum to zero:

$$\forall\, e \in \text{journal_entries}: \qquad |P(e)| \geq 2 \quad \text{and} \quad \sum_{p \in P(e)} p.\texttt{signed_amount_minor} = 0.$$

When Alice sends 50.00 USD (5,000 cents) to Bob, we never invent 5,000 cents in Bob's row. Instead we append one leg of -5000 against Alice's account and one leg of $+5000$ against Bob's account under the same `entry_id`. If a 1% platform fee applies, the entry has three legs: -5000 (Alice), $+4950$ (Bob), and $+50$ (Platform Fee Revenue), summing cleanly to $-5000 + 4950 + 50 = 0$.

How do we force PostgreSQL itself to reject any transaction that breaks this zero-sum balance law? A table-level `CHECK` constraint can inspect only a *single* row. And an ordinary `AFTER INSERT` trigger fires too early: the moment you insert Alice's -5000 leg (before inserting Bob's $+5000$ leg), the running sum for `entry_id` is $-5000 \neq 0$.

What we want is a check that waits until all legs of the transfer are in place, then runs right as the transaction asks to commit. PostgreSQL provides exactly that mechanism: a `CONSTRAINT TRIGGER` declared `DEFERRABLE INITIALLY DEFERRED`. Instead of firing after each `INSERT`, PostgreSQL queues a small trigger event in backend memory and runs the verification function *when the client issues `COMMIT`*, right before flushing the commit record to WAL (Figure 8.2). Let us write out the complete ledger schema, including an immutability trigger on `postings` and a cached balance on `accounts` guarded against unauthorized overdrafts:

```
CREATE TABLE accounts (
    account_id           UUID PRIMARY KEY DEFAULT uuidv7(),
    currency             CHAR(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),
    cached_balance_minor BIGINT NOT NULL DEFAULT 0,
    allow_overdraft      BOOLEAN NOT NULL DEFAULT FALSE,
    created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT non_negative_unless_overdraft
        CHECK (allow_overdraft OR cached_balance_minor >= 0)
);

CREATE TABLE journal_entries (
    entry_id        UUID PRIMARY KEY DEFAULT uuidv7(),
    client_id       UUID NOT NULL,
```

```
    idempotency_key TEXT NOT NULL CHECK (length(idempotency_key) <= 128),
    currency        CHAR(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),
    description     TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_client_idempotency UNIQUE (client_id, idempotency_key)
);

CREATE TABLE postings (
    posting_id          UUID PRIMARY KEY DEFAULT uuidv7(),
    entry_id            UUID NOT NULL REFERENCES journal_entries(entry_id),
    account_id          UUID NOT NULL REFERENCES accounts(account_id),
    signed_amount_minor BIGINT NOT NULL CHECK (signed_amount_minor <> 0),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_postings_entry   ON postings (entry_id);
CREATE INDEX idx_postings_account ON postings (account_id, created_at DESC);

CREATE OR REPLACE FUNCTION forbid_posting_mutation()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'postings are append-only; post a reversing entry';
END;
$$;

CREATE TRIGGER trg_postings_immutable
BEFORE UPDATE OR DELETE ON postings
FOR EACH ROW EXECUTE FUNCTION forbid_posting_mutation();

CREATE OR REPLACE FUNCTION verify_entry_zero_sum()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_entry_id UUID := COALESCE(NEW.entry_id, OLD.entry_id);
    v_legs     BIGINT;
    v_net_sum  BIGINT;
    v_bad_ccy  BIGINT;
BEGIN
    SELECT COUNT(*), COALESCE(SUM(p.signed_amount_minor), 0),
           COUNT(*) FILTER (WHERE a.currency <> je.currency)
      INTO v_legs, v_net_sum, v_bad_ccy
      FROM journal_entries je
      LEFT JOIN postings p ON p.entry_id = je.entry_id
      LEFT JOIN accounts a ON a.account_id = p.account_id
     WHERE je.entry_id = v_entry_id;

    IF v_legs < 2 OR v_net_sum <> 0 OR v_bad_ccy > 0 THEN
        RAISE EXCEPTION USING
            ERRCODE = '23514',
            MESSAGE = 'unbalanced entry ' || v_entry_id
                   || ': legs=' || v_legs || ', sum=' || v_net_sum
                   || ', ccy_mismatch=' || v_bad_ccy;
    END IF;
    RETURN NULL;
END;
```

```
$$;

CREATE CONSTRAINT TRIGGER trg_verify_entry_zero_sum
AFTER INSERT OR UPDATE OR DELETE ON postings
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION verify_entry_zero_sum();
```

Common trap

`verify_entry_zero_sum` checks *three* things together: `v_legs < 2`, `v_net_sum <> 0`, and `v_bad_ccy > 0`. Without `LEFT JOIN` and `v_legs < 2`, an entry whose postings were somehow deleted (if someone disabled the immutability trigger during maintenance) would return `SUM(...) = NULL`, which naive `IF v_net_sum <> 0` evaluates to `UNKNOWN` and lets slip past. And without checking `a.currency <> je.currency`, a buggy handler could debit 5000 JPY from a yen account and credit 5000 USD cents to a dollar account while still summing to zero numerically. Finally, attach a second deferred check on `journal_entries` (or always insert postings in the same helper) so an empty header with zero postings cannot commit alone.

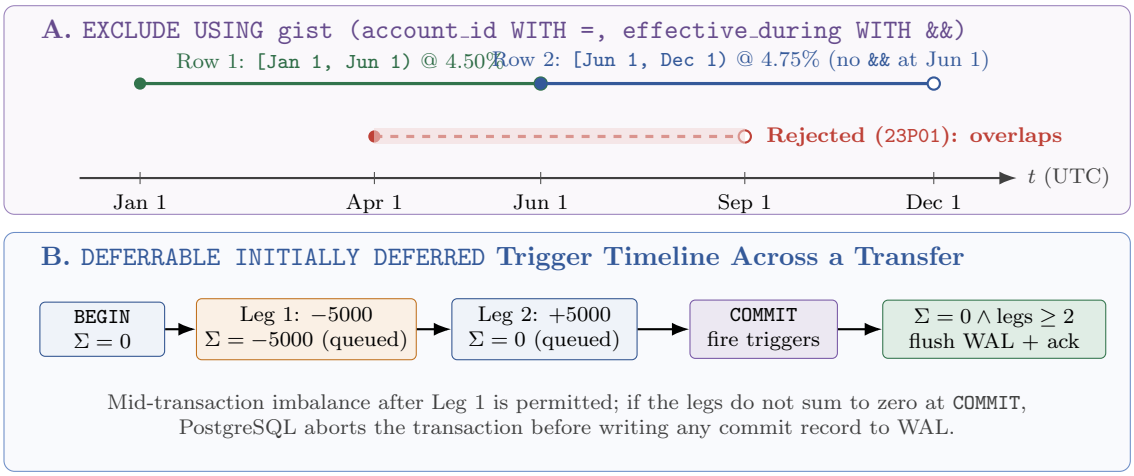


Figure 8.2: Two declarative PostgreSQL invariants that replace fragile application-side checks: (A) half-open `TSTZRANGE` intervals enforced by `EXCLUDE USING gist`, and (B) commit-time conservation verification via a deferred constraint trigger.

8.3.2 Deadlock-Free Transfers in Rust (`Axum` + `SQLx`)

Now let us wire our schema to a Rust `Axum` service using `SQLx`. Two concurrency traps await careless transfer code even when the schema is sound:

1. **Opposite-order deadlocks (Chapter 6).** Suppose Task 1 transfers 10 USD from Alice to Bob while Task 2 transfers 5 USD from Bob to Alice. If each handler locks `from_account` first and `to_account` second, Task 1 holds Alice's lock and waits on Bob, while Task 2 holds Bob's lock and waits on Alice. Neither can move until PostgreSQL's `deadlock_timeout` aborts one of them. Sorting the account UUIDs in ascending order (`ORDER BY account_id`) before locking makes two-account deadlocks impossible, because a strict total order contains no cycles.

2. **Over-strong row locks blocking foreign keys.** Recall from Chapter 6 that `INSERT INTO postings` takes a lightweight `FOR KEY SHARE` lock on the referenced `accounts` row so nobody deletes that account mid-transfer. Locking `accounts` with `SELECT ... FOR UPDATE` tells PostgreSQL you might change the primary key `account_id` itself, which blocks `FOR KEY SHARE`. Use `FOR NO KEY UPDATE` instead. It still serializes concurrent balance updates on the same account, while staying compatible with foreign-key checks.

Listing 8.1 puts every piece together: input checks, atomic idempotency claim, sorted `FOR NO KEY UPDATE` locking, double-entry leg insertion, cached balance updates, and commit-time verification.

Listing 8.1: Idempotent, deadlock-free double-entry transfer in Rust with `SQLx`.

```
use sqlx::{PgPool, Postgres, Transaction};
use uuid::Uuid;

#[derive(Debug)]
pub struct TransferCmd {
    pub client_id: Uuid,
    pub idempotency_key: String,
    pub from_account: Uuid,
    pub to_account: Uuid,
    pub currency: String,
    pub amount_minor: i64,
    pub description: String,
}

#[derive(Debug, thiserror::Error)]
pub enum LedgerError {
    #[error("invalid transfer parameters")]
    InvalidInput,
    #[error("account missing or currency mismatch")]
    AccountMismatch,
    #[error("insufficient funds")]
    InsufficientFunds,
    #[error("database error: {0}")]
    Db(#[from] sqlx::Error),
}

#[derive(sqlx::FromRow)]
struct LockedAccount {
    account_id: Uuid,
    currency: String,
    cached_balance_minor: i64,
    allow_overdraft: bool,
}

pub async fn execute_transfer(
    pool: &PgPool,
    cmd: &TransferCmd,
) -> Result<Uuid, LedgerError> {
    if cmd.amount_minor <= 0 || cmd.from_account == cmd.to_account {
        return Err(LedgerError::InvalidInput);
    }

    let mut tx: Transaction<'_, Postgres> = pool.begin().await?;
```

```
44
45      // 1. Atomic idempotency claim via UNIQUE (client_id, idempotency_key).
46      let claimed: Option<Uuid> = sqlx::query_scalar(
47          "INSERT INTO journal_entries \
48              (client_id, idempotency_key, currency, description) \
49           VALUES ($1, $2, $3, $4) \
50           ON CONFLICT (client_id, idempotency_key) DO NOTHING \
51           RETURNING entry_id",
52      )
53      .bind(cmd.client_id)
54      .bind(&cmd.idempotency_key)
55      .bind(&cmd.currency)
56      .bind(&cmd.description)
57      .fetch_optional(&mut *tx)
58      .await?;
59
60      let Some(entry_id) = claimed else {
61          // Retry detected: return the committed entry_id cleanly.
62          let existing: Uuid = sqlx::query_scalar(
63              "SELECT entry_id FROM journal_entries \
64               WHERE client_id = $1 AND idempotency_key = $2",
65          )
66          .bind(cmd.client_id)
67          .bind(&cmd.idempotency_key)
68          .fetch_one(&mut *tx)
69          .await?;
70          tx.rollback().await?;
71          return Ok(existing);
72      };
73
74      // 2. Canonical lock order prevents A->B vs B->A deadlocks.
75      let mut ids = [cmd.from_account, cmd.to_account];
76      ids.sort_unstable();
77
78      let locked: Vec<LockedAccount> = sqlx::query_as(
79          "SELECT account_id, currency, cached_balance_minor, \
80                  allow_overdraft \
81           FROM accounts \
82           WHERE account_id = ANY($1) \
83           ORDER BY account_id \
84           FOR NO KEY UPDATE",
85      )
86      .bind(&ids[..])
87      .fetch_all(&mut *tx)
88      .await?;
89
90      if locked.len() != 2
91          || locked.iter().any(|a| a.currency.trim() != cmd.currency)
92      {
93          return Err(LedgerError::AccountMismatch);
94      }
95
96      let sender = locked
97          .iter()
```

```
        .find(|a| a.account_id == cmd.from_account)
        .ok_or(LedgerError::AccountMismatch)?;

    if !sender.allow_overdraft
        && sender.cached_balance_minor < cmd.amount_minor
    {
        return Err(LedgerError::InsufficientFunds);
    }

    // 3. Append both immutable postings in a single multi-row INSERT.
    sqlx::query(
        "INSERT INTO postings \
            (entry_id, account_id, signed_amount_minor) \
         VALUES ($1, $2, $3), ($1, $4, $5)",
    )
    .bind(entry_id)
    .bind(cmd.from_account)
    .bind(-cmd.amount_minor)
    .bind(cmd.to_account)
    .bind(cmd.amount_minor)
    .execute(&mut *tx)
    .await?;

    // 4. Update cached balances while holding FOR NO KEY UPDATE locks.
    sqlx::query(
        "UPDATE accounts \
         SET cached_balance_minor = cached_balance_minor + CASE \
             WHEN account_id = $1 THEN -$3 \
             WHEN account_id = $2 THEN  $3 \
         END \
         WHERE account_id IN ($1, $2)",
    )
    .bind(cmd.from_account)
    .bind(cmd.to_account)
    .bind(cmd.amount_minor)
    .execute(&mut *tx)
    .await?;

    // 5. Commit fires trg_verify_entry_zero_sum inside PostgreSQL!
    tx.commit().await?;
    Ok(entry_id)
}
```

Pause

Pause and check: four independent safety nets on every transfer. Trace what happens if a bug or network glitch hits Listing 8.1:

1. **Duplicate retry:** `UNIQUE (client_id, idempotency_key)` catches the retry atomically and returns the original `entry_id`.
2. **Opposite-direction race:** sorting `ids` before `FOR NO KEY UPDATE` prevents $A \to B$ versus $B \to A$ deadlocks while serializing concurrent debits on the same sender.

3. **Overdraft race:** `CHECK (allow_overdraft OR cached_balance_minor >= 0)` rejects negative balances on the heap tuple itself.
4. **Unbalanced legs:** `trg_verify_entry_zero_sum` runs at `COMMIT` and aborts the transaction before WAL flush if the postings do not sum to zero in a single currency.

8.4 Declarative Partitioning for Billion-Row Ledgers

Imagine our ledger sustains 150 transfers (300 postings) per second around the clock. That adds $300 \times 86{,}400 \times 365 \approx 9.5$ billion new `postings` rows a year. At roughly 100 bytes per row, the heap alone grows by nearly a terabyte a year before indexes. Keeping years of history inside one giant physical table causes three operational headaches:

1. **Cold secondary indexes.** Even with `UUIDv7` keeping primary key inserts hot, secondary indexes such as `(account_id, created_at DESC)` span years of history across hundreds of gigabytes and push active pages out of `shared_buffers`.
2. **Multi-hour `VACUUM` freeze passes.** Chapter 5 showed that even an append-only table needs periodic `autovacuum` freeze scans to mark pages all-frozen in the Visibility Map before the 32-bit transaction ID horizon wraps around. Scanning a half-terabyte single heap file saturates disk reads for hours.
3. **Retention archival via `DELETE` bloats disk.** Suppose compliance policy moves postings older than seven years to cold object storage. Running `DELETE` on 250 million old rows does *not* shrink the file on disk: it marks 250 million dead tuples, writes tens of gigabytes of WAL, and leaves half-empty pages behind.

PostgreSQL's **declarative range partitioning** solves all three problems. SQL queries still address one logical parent table (`postings`), while storage splits cleanly into monthly physical child tables (`postings_2026_11`, `postings_2026_12`, and so on). Each child owns its own heap files, Visibility Map, and local B-tree indexes:

```
CREATE TABLE postings (
    posting_id          UUID NOT NULL DEFAULT uuidv7(),
    entry_id            UUID NOT NULL,
    account_id          UUID NOT NULL,
    signed_amount_minor BIGINT NOT NULL CHECK (signed_amount_minor <> 0),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (posting_id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE postings_2026_11 PARTITION OF postings
    FOR VALUES FROM ('2026-11-01 00:00:00+00')
                TO ('2026-12-01 00:00:00+00');

CREATE TABLE postings_2026_12 PARTITION OF postings
    FOR VALUES FROM ('2026-12-01 00:00:00+00')
                TO ('2027-01-01 00:00:00+00');

CREATE INDEX idx_postings_acct_time
```

```
    ON postings (account_id, created_at DESC);
```

Look at what changes inside the engine:

- **Plan-time and runtime partition pruning.** When a customer loads a November statement (filtering `account_id` and November `created_at` bounds), the planner (Chapter 7) checks each child's `FOR VALUES` range and skips 83 out of 84 monthly partitions before reading a single index page. Even when date bounds come from join keys or prepared parameters at runtime, the executor's `Append` node prunes non-matching partitions on the fly. Only the current month's indexes need to fit in `shared_buffers`.

- **Localized `autovacuum`.** Once November ends, `VACUUM` freezes `postings_2026_11` once in its Visibility Map and never scans those pages again.

- **Millisecond, zero-bloat archival.** Retiring the oldest month never runs `DELETE`:

```
ALTER TABLE postings DETACH PARTITION postings_2019_10 CONCURRENTLY;
-- Export standalone table postings_2019_10 to Parquet/archive, then:
DROP TABLE postings_2019_10;
```

  `DETACH PARTITION CONCURRENTLY` unhooks the child from the parent catalog without blocking live reads or inserts. After archiving, `DROP TABLE` unlinks its 1 GB OS segment files in milliseconds with zero dead tuples.

Common trap

Notice that our partitioned table declared `PRIMARY KEY (posting_id, created_at)` rather than `PRIMARY KEY (posting_id)`. PostgreSQL rejects any `UNIQUE` or `PRIMARY KEY` constraint on a partitioned table unless *every column of the partition key appears inside the constraint.* From Chapter 3 you can see why: each monthly partition owns a separate local B-tree, and enforcing global uniqueness on `posting_id` alone would force every single `INSERT` to probe all 84 monthly B-trees. Keep `journal_entries` unpartitioned (or partitioned with `created_at` included in the idempotency window) if you need a single global B-tree on `(client_id, idempotency_key)`.

8.5 High Availability: Physical Streaming and Logical Replication

A RAID array or cloud block volume protects one server against a single drive failure. It cannot save you from a kernel panic, a motherboard fault, or a data-center power loss. Yet standing up a standby server is only half the design. Every high-availability cluster is judged by two targets: its **Recovery Point Objective (RPO)**, the maximum window of committed data a failover may lose, and its **Recovery Time Objective (RTO)**, how long traffic stays offline while promoting a standby. Depending on two configuration settings, an automatic failover that meets a ten-second RTO can still break RPO = 0 by erasing confirmed deposits, or serve stale balances right after a transfer succeeds.

8.5.1 Physical Streaming WAL Replication and Standby Conflicts

Recall from Chapter 4 that PostgreSQL's Write-Ahead Log records physical page edits ordered by a rising 64-bit byte offset called the Log Sequence Number (LSN, represented internally as `XLogRecPtr`). **Physical streaming replication** turns that local crash-recovery log into a live network stream:

1. On the primary, a background sender process (`walsender`) reads newly written WAL bytes straight from memory or `pg_wal` and streams them over TCP to each standby. Attaching a **physical replication slot** tracks how far each standby has caught up so the primary never deletes a 16 MB WAL file that a temporarily disconnected replica still needs. Always alert on slot lag in `pg_replication_slots`: an abandoned slot for a dead standby holds onto WAL forever until the primary's disk fills 100%.
2. On the standby, a receiver process (`walreceiver`) writes incoming WAL bytes to the standby's own `pg_wal` directory and flushes them to disk. Alongside it, the standby's `startup` process continuously replays those records onto the standby's 8 KB heap and index pages.

Because physical replication ships exact page changes, a physical standby is a **byte-for-byte clone** of the primary. Every 8 KB slotted page, every `xmin`/`xmax` stamp, and every B-tree leaf matches across the cluster. With `hot_standby = on`, read-only queries can run directly on the standby while replay continues underneath them.

That concurrent reading creates a subtle clash with Chapter 5's `VACUUM`. Suppose a slow reporting query runs for forty seconds on the standby using snapshot `xmin = 50000`. Meanwhile on the primary, no old query is active, so `VACUUM` reclaims dead rows from transaction `50010` and logs a cleanup record (`HEAP2_PRUNE`). When the standby's replay process reaches that record, applying it right away would erase rows that the standby's running query still needs to see. PostgreSQL resolves this **recovery conflict** with `max_standby_streaming_delay` (default `30s`). The standby pauses WAL replay up to that limit so the query can finish. If the timer expires first, PostgreSQL cancels the standby query with `ERROR: canceling statement due to conflict with recovery`. Turning on `hot_standby_feedback = on` tells the standby to report its oldest active `xmin` back to the primary's snapshot table (`ProcArray`). That prevents standby query cancellations, at the price of holding back dead-row cleanup on the primary while long replica queries run.

8.5.2 The `synchronous_commit` Durability and Visibility Ladder

When a Rust handler calls `tx.commit().await`, how far through the replication pipeline must the commit LSN travel before the primary tells the client "success"? PostgreSQL lets you pick the exact rung on the latency-versus-safety ladder, either globally in `postgresql.conf` or per transaction with `SET LOCAL`. Two settings work together: `synchronous_commit` chooses the rung, and `synchronous_standby_names` names which standbys count—for example, `ANY 1 (az2_standby, az3_standby)` across three availability zones.

Each standby continuously reports three LSN watermarks back to the primary, all visible in `pg_stat_replication`:

- `write_lsn`: WAL bytes copied into the standby's OS page cache;
- `flush_lsn`: WAL bytes flushed with `fsync` onto the standby's NVMe drive;

- `replay_lsn`: WAL bytes applied to the standby's shared buffers, making the commit visible to standby readers.

Figure 8.3 and Table 8.1 compare all five levels.

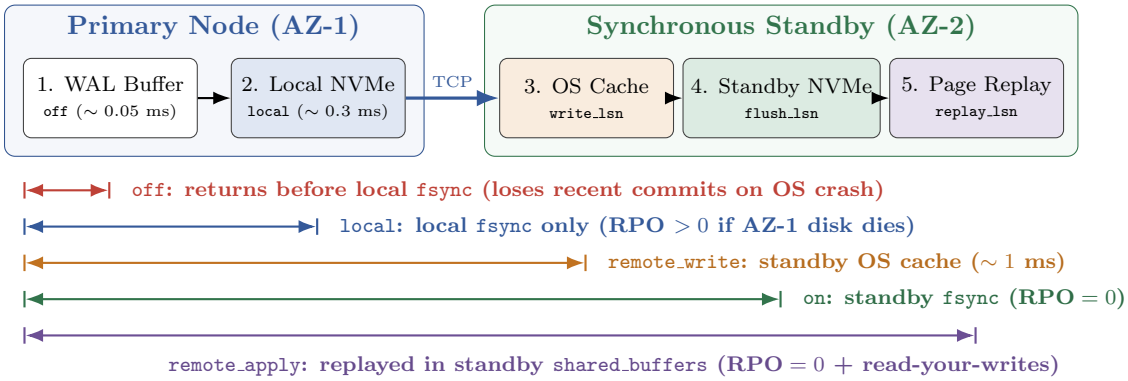


Figure 8.3: The five commit-wait watermarks of `synchronous_commit` when `synchronous_standby_names` is active.

Mode	Wait Condition	Commit Latency	AZ-1 Loss RPO / Standby Reads
`off`	WAL buffer copy	∼ 0.05 ms	Loses ∼ 200–600 ms on reboot
`local`	Primary `fsync`	∼ 0.3 ms	RPO > 0 on AZ loss; stale reads
`remote_write`	Standby `write()`	∼ 1.0 ms	RPO = 0 unless dual crash; brief lag
`on`	Standby `fsync`	∼ 1.8 ms	**RPO = 0 on AZ loss**; brief lag
`remote_apply`	Standby WAL replay	∼ 2.4 ms	**RPO = 0 + causal standby reads**

Table 8.1: Durability (Recovery Point Objective, RPO) and intra-region cross-AZ latency across `synchronous_commit` modes.

Two production lessons follow from Table 8.1. First, if you leave `synchronous_standby_names` empty (the default), `synchronous_commit = on` quietly falls back to `local`: PostgreSQL flushes only the primary's disk and ships WAL asynchronously. Second, consider a three-AZ cluster configured with `ANY 1 (az2_standby, az3_standby)`. Setting `synchronous_commit = on` guarantees zero data loss (RPO = 0) across a full availability-zone outage, while still tolerating one slow or rebooting standby without stalling writes. If an API endpoint immediately redirects a customer to read from a replica, that single handler can ask for read-your-writes visibility with `SET LOCAL synchronous_commit = 'remote_apply'`.

8.5.3 Logical Replication for Zero-Downtime Upgrades and Event Streaming

Physical replication requires the primary and standby to run the *exact* same major PostgreSQL version and CPU architecture, because 8 KB page layouts change across major releases. How, then, do you upgrade a live financial ledger from PostgreSQL 16 to PostgreSQL 18 without downtime, or stream committed `postings` rows into an analytics warehouse?

Setting `wal_level = logical` turns on **logical decoding**. Instead of shipping raw 8 KB page diffs, the primary decodes WAL back into logical row changes. Because concurrent transactions

write interleaved WAL records, a per-slot staging area called the **Reorder Buffer** gathers each transaction's records by transaction ID (spilling huge transactions to disk if needed). Rolled-back transactions are discarded. The moment a transaction commits, an output plugin (`pgoutput`) emits structured `INSERT`, `UPDATE`, and `DELETE` row events:

```
-- On the primary (or PG 16 source cluster):
CREATE PUBLICATION ledger_pub
    FOR TABLE accounts, journal_entries, postings
    WITH (publish_via_partition_root = true);

-- On the PG 18 target cluster:
CREATE SUBSCRIPTION ledger_sub
    CONNECTION 'host=pg16-primary dbname=ledger user=repl_role'
    PUBLICATION ledger_pub;
```

Notice `publish_via_partition_root = true`. Subscribers receive row events tagged under the parent table `postings`, so the target cluster can use a fresh partitioning layout or new indexes. Keep two limits in mind during logical upgrades: schema changes (`ALTER TABLE`) do not replicate automatically, and `SEQUENCE` counters are not copied in real time. Using time-ordered `UUIDv7` keys avoids the sequence-cutover headache altogether.

8.6 Connection Pooling at Scale: Process Architecture and PgBouncer

Our Rust `Axum` service runs on Tokio's async runtime, where one pod juggles ten thousand open HTTP sockets in a few dozen megabytes of RAM. Suppose you scale out to 50 Kubernetes pods, set each pod's `sqlx::PgPool` limit to 100, and raise PostgreSQL's `max_connections = 5500` on a 32-core server so all 5,000 sockets connect directly. Everything looks calm in staging. Then Monday morning traffic arrives. Database CPU hits 100% inside kernel spinlocks, free RAM disappears, and 99th-percentile transfer latency jumps from 3 ms to four seconds.

8.6.1 Why Five Thousand Backend Processes Melt PostgreSQL

Unlike Tokio's lightweight tasks, PostgreSQL uses a **one-OS-process-per-connection** model. Whenever a client opens a TCP connection, the main supervisor process (`postmaster`) forks a dedicated `postgres` OS process for that client. Keeping 5,000 backend processes open hurts throughput in three physical ways:

1. **Per-backend private memory.** Although `shared_buffers` is shared, every `postgres` process keeps its own private table and catalog caches (`relcache` and `catcache`, which grow large on partitioned schemas), cached plans, and `work_mem` buffers (Chapter 7). At 10–16 MB of resident RAM per active process, 5,000 backends eat 50–80 GB before caching a single table page.

2. **Snapshot scans across `ProcArray`.** Chapter 5 showed that every `READ COMMITTED` query builds a fresh snapshot (`GetSnapshotData()`) by scanning the shared-memory array of connected backends (`ProcArray`) to list active transaction IDs. Scanning 5,000 slots hundreds of thousands of times per second causes fierce CPU cache-line contention around `ProcArrayLock`.

3. **OS scheduler preemption inside locks.** When 600 processes wake up at once on 32 CPU cores, the Linux scheduler constantly preempts them, flushing CPU caches. Worse, whenever the OS deschedules a backend *while it holds a lightweight buffer latch or spinlock*, dozens of other backends pile up waiting behind that sleeping process.

Always benchmark your own hardware, but a reliable starting rule on NVMe servers with N_{cores} cores is to keep active `postgres` processes near $1.5 \times N_{\text{cores}}$ to $3 \times N_{\text{cores}}$. On a 32-core primary, that is 48 **to** 96 **backends**. Adding more connections past that point adds wait time, not throughput.

8.6.2 `PgBouncer` Transaction Multiplexing and the Prepared Statement Trap

How can 50 Rust pods holding 2,500 client sockets share just 48 physical `postgres` processes? Place `PgBouncer` in front of the database (Figure 8.4). `PgBouncer` is a small, event-driven C proxy that speaks the PostgreSQL wire protocol using roughly 2 KB of RAM per client socket. It offers three pooling modes:

- `session` **mode** ties one `postgres` process to a client socket from connect until disconnect. Because `sqlx::PgPool` keeps long-lived TCP sockets open, `session` mode provides *zero* multiplexing: 2,500 client sockets still require 2,500 backends.
- `transaction` **mode** (the production standard) lends a `postgres` process to a client *only from `BEGIN` until `COMMIT` or `ROLLBACK`* (or for a single standalone query). The instant `COMMIT` finishes, `PgBouncer` hands that warm backend to the next waiting request. While an `Axum` task parses JSON, verifies signatures, or writes HTTP replies, it holds *zero* database processes.
- `statement` **mode** reclaims the backend after every single SQL command and rejects multi-statement `BEGIN ... COMMIT` blocks, so ledger transactions cannot use it.

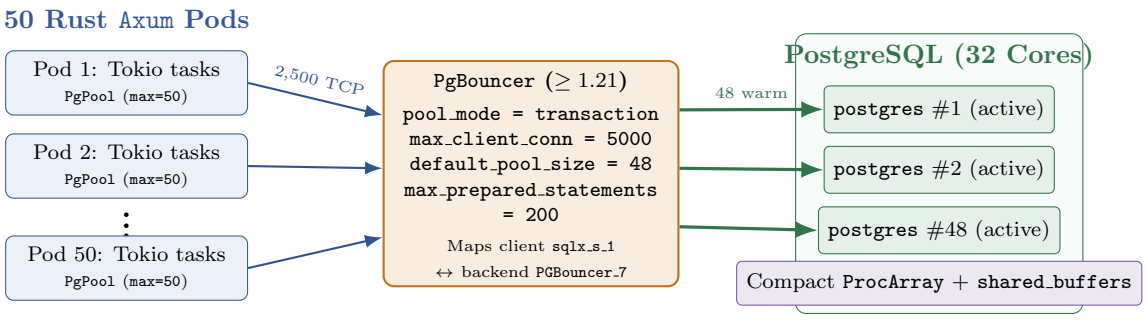


Figure 8.4: Multiplexing 2,500 `SQLx` client connections across 48 warm `postgres` backends via `PgBouncer` in `transaction` mode, keeping `ProcArray` compact and CPU caches hot.

Flipping `pool_mode = transaction` without knowing how PostgreSQL's wire protocol prepares queries triggers an outage within seconds: `ERROR: prepared statement "sqlx_s_1" does not exist` (or `already exists`). Here is why that happens on the wire:

1. To avoid re-parsing SQL text on every call, `SQLx` uses PostgreSQL's two-step extended query protocol and caches up to 100 named prepared statements per TCP socket. In Transaction A, Client 1 lands on Backend #4. It sends a `Parse` message saving statement `sqlx_s_1` inside Backend #4's private memory, runs it, and commits.

2. A millisecond later, Client 1 begins Transaction B, and `PgBouncer` assigns Backend #19. Because Client 1 is still using the same TCP socket to `PgBouncer`, `SQLx` assumes `sqlx_s_1` already exists. It skips `Parse` and sends a `Bind` packet asking for `sqlx_s_1`. Backend #19 has never heard of `sqlx_s_1` and aborts the transaction.

The same backend-hopping behavior breaks any feature that stores state for an entire *session* rather than a single *transaction*:

- Never run bare `SET synchronous_commit = 'remote_apply'` or `SET statement_timeout = '2s'` in transaction mode; that setting leaks onto whichever unrelated request borrows the backend next. Always write `SET LOCAL ...`, which resets automatically at `COMMIT` or `ROLLBACK`.
- Never call session-scoped `pg_advisory_lock`, which stays locked after your transaction finishes. Always call the transaction-scoped `pg_advisory_xact_lock` instead.
- `LISTEN / NOTIFY` channels need a separate direct or `session`-mode connection outside the transaction pool.

For prepared statements, you have two clean fixes. Modern `PgBouncer` ($\geq$ 1.21) supports protocol-level prepared-statement multiplexing when you set `max_prepared_statements = 200` in `pgbouncer.ini`. `PgBouncer` remembers each client's `Parse` query, maps client names (`sqlx_s_1`) onto shared server names (`PGBOUNCER_7`), and automatically prepares missing statements on whichever backend runs the transaction. That keeps Chapter 7's plan-cache speedups intact. If you run behind an older pooler that lacks protocol tracking, set `statement_cache_capacity(0)` on `PgConnectOptions` so `SQLx` sends unnamed single-use statements:

```rust
use sqlx::postgres::{PgConnectOptions, PgPoolOptions};
use std::str::FromStr;
use std::time::Duration;

pub async fn build_ledger_pool(
    db_url: &str,
    pgbouncer_tracks_stmts: bool,
) -> Result<sqlx::PgPool, sqlx::Error> {
    let mut opts = PgConnectOptions::from_str(db_url)?;
    if !pgbouncer_tracks_stmts {
        // Fallback for stateless transaction poolers: unnamed statements.
        opts = opts.statement_cache_capacity(0);
    }

    PgPoolOptions::new()
        .max_connections(50)
        .acquire_timeout(Duration::from_secs(2))
        .idle_timeout(Duration::from_secs(300))
        .connect_with(opts)
        .await
}
```

Pause

Pause and check: three rules for safe `PgBouncer` transaction pooling. Before switching any service to `pool_mode = transaction`, verify three items:

1. Enable `max_prepared_statements = 200` on `PgBouncer` ($\geq$ 1.21), or disable client statement caching (`statement_cache_capacity(0)`).
2. Replace every session `SET` with `SET LOCAL` inside a transaction block.
3. Replace every `pg_advisory_lock` with `pg_advisory_xact_lock`, and move `LISTEN / NOTIFY` onto a dedicated unpooled connection.

Here is the whole path of one 50.00 USD transfer through the machinery of this book. An `Axum` task borrows a pooled connection through `PgBouncer` and lands on one of 48 warm backends, keeping `ProcArray` snapshot scans tiny (Chapter 5). It generates `UUIDv7` keys, which append cleanly to the rightmost B-tree leaf already cached in `shared_buffers` (Chapters 2 and 3). It claims its idempotency key via a `UNIQUE` index, locks both accounts in sorted order with `FOR NO KEY UPDATE` (Chapter 6), and appends two `BIGINT` postings into the current month's partition. At `COMMIT`, the deferred trigger confirms that the legs sum to zero. With `synchronous_commit = on`, WAL flushes onto NVMe drives in two availability zones before the client hears success (Chapter 4).

8.7 Exercises

Exercise 8.1 (IEEE 754 Drift vs. Exact Ledger Representations)**.** Execute the following query in PostgreSQL:

```
SELECT SUM(val::float8) AS float_sum, SUM(val::numeric) AS num_sum
FROM generate_series(1, 1000000) AS g(i),
     LATERAL (VALUES (0.10), (0.20), (-0.30)) AS v(val);
```

Explain why `float_sum` is nonzero while `num_sum` is exact. Next, design the columns for an FX conversion table that converts `JPY` (scale 0), `USD` (scale 2), and `BTC` (scale 8). Use `BIGINT` minor units for posted legs and `NUMERIC(24, 12)` for the exchange rate. Specify where rounding happens and which rule it uses so each currency's legs still sum to zero. (Note that PostgreSQL's `round()` on `NUMERIC` rounds halves away from zero rather than to even; banker's rounding requires an explicit expression.)

Exercise 8.2 (The Autumn Daylight-Saving Collision)**.** In a `psql` session with `TimeZone` set to `'America/New_York'`, store `'2026-11-01 01:30:00-04'` and `'2026-11-01 01:30:00-05'` in a `TIMESTAMP` column and in a `TIMESTAMPTZ` column. Compute the exact difference between the two 64-bit integer values stored on the 8 KB heap page for `TIMESTAMPTZ` (in microseconds since `2000-01-01 00:00:00 UTC`). Show how plain `TIMESTAMP` corrupts an hourly time-weighted average price (TWAP) query across that fallback hour.

Exercise 8.3 (Quantifying `UUIDv4` WAL Amplification vs. `UUIDv7`)**.** Suppose a primary-key B-tree on `postings` has 8,000,000 leaf pages ($\approx$ 64 GB). Right after a `CHECKPOINT` completes (with `full_page_writes = on`), a burst inserts 40,000 new rows within two seconds. Estimate how many distinct 8 KB leaf pages are dirtied under random `UUIDv4` versus time-ordered `UUIDv7`, assuming 120 index entries fit per new rightmost leaf page. How many megabytes of 8 KB Full Page Images does each key type write to WAL?

Exercise 8.4 (Non-Overlapping Credit Limits with `EXCLUDE USING gist`)**.** Write the DDL for a `credit_limits` table where each `account_id` has an overdraft limit valid over an `effective_during`

column of type `TSTZRANGE`. Active schedules (`status = 'active'`) for the same account must never overlap in time, while historical `status = 'superseded'` rows may remain in the table for audits. Use PostgreSQL's partial exclusion syntax `EXCLUDE USING gist (col WITH op) WHERE (predicate)`. Finally, test what happens at the boundary when two intervals use closed bounds `'[]'` instead of half-open `'[)'`.

Exercise 8.5 (Auditing and Hardening the Double-Entry Deferred Trigger)**.** Suppose a transaction inserts four postings for the same `entry_id`: sender debit, recipient credit, platform fee, and partner rebate. How many times does a `FOR EACH ROW` deferred constraint trigger fire at `COMMIT`? Show how to skip redundant checks within the same transaction once `v_entry_id` has already been verified at the final command counter. Then write the companion deferred constraint trigger on `journal_entries` that rejects an entry header committed with zero rows in `postings`.

Exercise 8.6 (Deadlock Cycles and `FOR NO KEY UPDATE` Internals)**.** Construct a two-session `psql` schedule (T_1 transferring $A \to B$ and T_2 transferring $B \to A$) that deterministically triggers `ERROR: deadlock detected` when UUID sorting is omitted from Listing 8.1. Next, open two `psql` sessions and compare tuple lock modes from Chapter 6. Show why locking `accounts` with `FOR UPDATE` blocks a concurrent `INSERT INTO postings` foreign-key check, whereas `FOR NO KEY UPDATE` lets the foreign-key check proceed until the cached-balance `UPDATE` step.

Exercise 8.7 (Partitioning and Retention on a Multi-Billion-Row Table)**.** Write a PL/pgSQL helper procedure `ensure_monthly_partition(target_month DATE)` that safely creates next month's `postings_YYYY_MM` range partition ahead of time. Explain what happens to live `INSERT` traffic at `2027-01-01 00:00:00Z` if the cron job fails and no `DEFAULT` partition exists. Conversely, explain why letting rows pile up in a `DEFAULT` partition can turn a future `ATTACH PARTITION` into a long table-scan lock.

Exercise 8.8 (Sizing `synchronous_commit` Across Three Availability Zones)**.** A ledger cluster spans `az1` (primary), `az2` (standby), and `az3` (standby). Local NVMe `fsync` takes 0.25 ms, network round-trip time is 1.1 ms (`az1`–`az2`) and 1.6 ms (`az1`–`az3`), and standby WAL replay adds 0.35 ms. With `synchronous_standby_names = 'ANY 1 (az2, az3)'`, calculate expected commit latency during normal operation and during an `az2` outage for `local`, `remote_write`, `on`, and `remote_apply`. Which mode should `POST /v1/transfers` use if `GET /v1/accounts/:id/balance` is load-balanced across `az2` and `az3`?

Exercise 8.9 (Fixing a `PgBouncer` Transaction-Mode Rollout)**.** An engineering team flips `PgBouncer` from `pool_mode = session` to `pool_mode = transaction` in front of 40 Rust `Axum` pods. Within two minutes, monitoring reports three distinct failures. First, handlers log `prepared statement "sqlx_s_4" does not exist`. Second, fast transfers time out at 150 ms after an admin endpoint ran `SET statement_timeout = '150ms'`. Third, a nightly job calls `pg_advisory_lock(99)` and leaves the lock held after commit. Trace the backend-multiplexing cause of all three bugs and write the configuration, SQL, and Rust fixes.